

CEN207 Term Project — Project Guide

CEN207 Data Structures (formerly CE205) · Fall 2026–2027

This term you will build **one project**: an application you choose from the list, implemented **twice** with the data structures and algorithms you learn in class — first in **C** (midterm check), then extended in **Java** (final check). The goal is to use every data structure inside a real problem and to answer "why did I choose this structure?" with numbers: complexity and measurements, not opinions.

Language and tools

C (CMake, GoogleTest, Doxygen) for the midterm check; Java (JDK 21, Maven, JUnit 5) for the final check; Git and GitHub throughout.

Team

At most 3 students (working alone is also allowed). Each topic is taken by one team only. No team changes after week 3.

Midterm check

C implementation, report and demo — week 7, 30.10.2026. 60% of the midterm grade.

Final check

Java implementation, report and demo — week 15, 25.12.2026. 70% of the final grade.

At a glance

- **Team:** at most 3 students (you may also work alone). Each topic can be taken by one team only. Teams cannot change after week 3 (04.10.2026).
- **Midterm check (RAP1):** C implementation, report and demo — **week 7, 30.10.2026**. 60% of the midterm grade.
- **Final check (RAP2):** Java implementation, report and demo — **week 15, 25.12.2026**. 70% of the final grade.
- **Tools:** CMake + GoogleTest + Doxygen for C; JDK 21 + Maven + JUnit 5 for Java; Git and GitHub.
- **Scope:** every requirement code (V1–V6, F1–F9) is mandatory for every topic; the topic box tells you what each code represents in that application.
- **Deadlines:** topic and team selection by the end of week 3 (04.10.2026); project plan approval in week 4 (09.10.2026).

1. Calendar

Milestone	Week	Date
Topic and team selection	end of week 3	04.10.2026
Project plan approval	week 4	09.10.2026
Midterm demo + interim report (RAP1)	week 7	30.10.2026
Quiz-1	week 8	31.10–08.11.2026
Final demo + final report (RAP2)	week 15	25.12.2026
Quiz-2	week 16	04–17.01.2027

The full weekly schedule is in the [syllabus](#).

2. Assessment structure

$\text{Grade}_{\text{Midterm}} = 0.6 \cdot \text{RAP1} + 0.4 \cdot \text{Quiz-1}$ · $\text{Grade}_{\text{Final}} = 0.7 \cdot \text{RAP2} + 0.3 \cdot \text{Quiz-2}$ · $\text{Course grade} = 0.4 \cdot \text{Midterm} + 0.6 \cdot \text{Final}$.

LO	Definition	Bloom level	Knowledge/Skill	RAP1	RAP2
LO.1	Explains the definitions, representations and basic operations of linear and non-linear data structures	Understand	Knowledge	✓	✓
LO.2	Analyzes the time and space complexity of algorithms with asymptotic notation (Big-O)	Analyze	Skill	✓	✓
LO.3	Implements fundamental sorting and searching algorithms	Apply	Skill	✓	✓
LO.4	Implements tree structures and hash tables	Apply	Skill	✓	✓
LO.5	Implements the graph data structure and fundamental graph algorithms	Apply	Skill	✓	✓
LO.6	Evaluates sequential, direct and indexed file organization techniques	Evaluate	Skill	—	✓
LO.7	Selects the data structures and algorithms best suited to a problem and develops an efficient solution	Synthesize	Skill	✓	✓

3. Tools and setup

Tool	What for	Output	Submission condition
GCC / Clang / MSVC + CMake	Build the C application	Executable (not submitted)	Builds with zero errors on Windows and WSL/Linux
GoogleTest + gcov/lcov	Unit tests and coverage (C)	HTML coverage report	100% statement coverage; report inside the archive
Doxygen	Source documentation (C)	PDF	100% coverage; PDF only, no HTML folder
JDK 21 + Maven	Build the Java application	Release build	Builds with <code>mvn clean verify</code> , zero errors
JUnit 5 + JaCoCo	Unit tests and coverage (Java)	HTML coverage report	100% statement coverage; report inside the archive
Javadoc / Doxygen	Source documentation (Java)	PDF	100% coverage; PDF only, no HTML folder
Git + GitHub	Version control, collaboration	Private repository	Meaningful commits from both members, proper <code>.gitignore</code>
GitHub Actions	Continuous integration (optional)	CI status	If enabled, build and tests must be green before merging

Fork the templates, name them with the course code, make the repository **private** and add the instructor and your teammate as **collaborators**.

Check	Template	Repository name
Midterm (C)	https://github.com/ucoruh/cpp-cmake-ctest-template	cen207-project-name-surname-c
Final (Java)	https://github.com/ucoruh/eclipse-java-maven-template	cen207-project-name-surname-java



Use the template fully

The templates already provide building, unit testing, documentation generation, test and documentation coverage measurement and packaging. The project follows a **lib / app / test** layout: data structures and algorithms live in `lib`, menus and user interaction in `app`, unit tests in `test`. Both `app` and `test` use `lib`.

Before you start, make sure your development environment is ready: see the [prerequisites page](#).

4. Choosing a topic

Topics are listed in the **Appendix — Project topic list** at the end of this page (**200 topics** in eight groups). Each topic box starts with a short summary, then states in one sentence **what each requirement code stores and which operation it performs** in that application.

How to choose

1. Browse the list and pick a topic.
 2. Write your choice in the **team and topic table** on Microsoft Teams. Each topic goes to one team only; the first team to write it gets it.
 3. Get it approved together with your project plan; the topic cannot change after approval.
- The mappings in a box are a **starting suggestion**; if a more natural feature meets the same requirement, you may change it and justify the change in your report. The requirement itself (which structure, which operation) stays.
 - If you have an idea that is not on the list, you may choose it with the instructor's approval, provided it meets every requirement meaningfully.
 - If you are retaking the course, choose a topic **different from your previous project**.

5. Requirements

Each code lists the **week it is taught** and the **learning outcome (LO)** it measures. You implement the basic operations of every structure (insert, delete, search, list...) **yourself**; ready-made library structures (e.g. Java `java.util` collections) do not count towards these requirements.

5.1 Midterm scope — C implementation (weeks 1–6)

Code	Requirement	Week	LO
V1	Linked list: doubly linked list + XOR linked list or circular list; insert, delete, search, traverse both ways	2	LO.1
V2	Sparse matrix: a structure that stores only the filled cells of mostly empty 2-D data; read, write, walk rows/columns	2	LO.1
V3	Stack and queue: a stack (undo, expression evaluation or backtracking) and a queue (waiting line, buffer); with an array or a linked list	3	LO.1
V4	Tree and heap: a binary tree with its three traversals; a heap-based priority queue; heap sort	4	LO.1, LO.4
V5	Graph and traversal: adjacency list or matrix; BFS and DFS	5	LO.1, LO.5
V6	Search and hashing: binary search; a hash table with collision resolution (chaining or open addressing)	6	LO.3, LO.4

5.2 Final scope — Java implementation (weeks 9–14)

Code	Requirement	Week	LO
F1	Port to Java: Java versions of V1–V6 using generics; all features in a single menu	9–14	LO.1, LO.7
F2	Graph algorithms: at least two of — minimum spanning tree (Prim/Kruskal), shortest path (Dijkstra/Bellman–Ford), topological sort, strongly connected components, cycle detection, maximum flow	9	LO.5
F3	Sorting: at least three algorithms (insertion, selection, quick, merge, heap...) compared by timing on different data sizes	10	LO.2, LO.3
F4	BST and AVL: binary search tree and AVL (balancing rotations); insert, delete, search, range query	11	LO.4
F5	String algorithms: search with KMP or Boyer–Moore + edit distance or LCS	12	LO.1, LO.3
F6	Trie and disjoint sets: prefix tree (trie) + union-find	9, 12	LO.4
F7	File organisation: a sequential file + a direct-access (hashed) file; collision resolution in the file (progressive overflow, linear quotient or Brent's method)	13	LO.6
F8	B+ tree index: a B-tree or B+ tree secondary-key index over the records in a file	14	LO.4, LO.6
F9	Growing files: extendible hashing or external merge sort of a file that does not fit in memory	14	LO.3, LO.6

5.3 Rules for both checks

- **Console application with keyboard-navigable menus:** arrow keys or Tab; number entry alone is not enough.
- **Persistent data in binary files** (`.bin` / `.dat`): records come back after the program is closed and reopened.
- **Complexity:** the time and space complexity (Big-O) of every operation is written in the code (Doxygen/Javadoc comments) and in the report; give a **measurement table** over different data sizes for at least two structures.
- **Tests and documentation:** unit-test coverage **100%**, documentation coverage **100%** (C: GoogleTest + gcov/lcov, Doxygen; Java: JUnit 5 + JaCoCo, Javadoc/Doxygen).
- **Platforms:** the application builds and runs on both **Windows** and **WSL/Linux**.
- **GitHub:** private repository, meaningful commits, branches, a proper `.gitignore`; **no** compiled files (`.exe` , `.o` , `.class` , `.jar`) in the repository. The Java repository produces a **release**.
- **Data:** all data is synthetic (sample data you generate); no real personal data.

6. Deliverables

Upload a **single archive** to the Microsoft Teams assignment; put the repository link on the report cover.

Deliverable	Midterm (RAP1)	Final (RAP2)
Source code archive (no compiled files)	cen207-midterm-name-surname.zip	cen207-final-name-surname.zip
Report (<code>.docx</code>): design, why each structure was chosen, complexity and measurement tables, test and coverage screenshots	✓	✓ (midterm part updated)
Doxygen/Javadoc output — PDF only	✓	✓
Test coverage report (HTML folder inside the archive)	✓	✓
Presentation (at most 10 slides)	—	✓
Video (each member explains their own contribution; at most 4 min per person)	—	✓
Live demo and questions (~10 min per team)	week 7	week 15

Archive structure

Midterm archive (C):

```

cen207-midterm-name-surname.zip
├── cen207-project-name-surname-c/      # GitHub repo clone (gitignore-filtered)
│   ├── lib/                          # data structures and algorithms (V1-V6)
│   ├── app/                          # console menus, uses lib
│   ├── test/                         # GoogleTest unit tests, uses lib
│   ├── CMakeLists.txt
│   ├── report/
│   │   └── cen207-midterm-name-surname.docx
│   ├── docs/                         # Doxygen output – PDF only
│   │   └── cen207-midterm-name-surname-doxygen.pdf
│   ├── coverage/                     # gcov/lcov HTML coverage report
│   ├── .gitignore
│   └── README.md

```

Final archive (Java):

```

cen207-final-name-surname.zip
├── cen207-project-name-surname-java/ # GitHub repo clone (gitignore-filtered)
│   ├── src/main/java/...           # lib + app packages: F1 port of V1-V6, plus F2-F9
│   ├── src/test/java/...          # JUnit 5 unit tests
│   ├── pom.xml
│   ├── report/
│   │   └── cen207-final-name-surname.docx
│   ├── docs/                      # Javadoc/Doxygen output – PDF only
│   │   └── cen207-final-name-surname-doxygen.pdf
│   ├── coverage/                  # JaCoCo HTML coverage report
│   ├── presentation/
│   │   └── cen207-final-name-surname.pptx
│   ├── video/                     # not committed to GitHub; add before zipping
│   │   ├── video-name1-surname1.mp4
│   │   └── video-name2-surname2.mp4
│   ├── .gitignore
│   └── README.md

```

Naming

Name every file with the course code, the checkpoint and your name-surname (e.g. `cen207-midterm-name-surname.zip`, `cen207-final-name-surname.zip`, `cen207-midterm-name-surname.docx`); repository names follow the pattern in §3.

7. Team workflow and engineering practices

- **GitHub Flow:** `main` is protected; do all work on feature branches (`feature/hash-table`, `fix/avl-rotation`) and merge them through pull requests.
- **Commit messages (Conventional Commits):** `feat(lib): implement AVL rotation`, `fix(app): correct menu navigation`, `test(hash): add collision unit tests`, `docs(doxygen): document graph module`.
- **Pull requests and review:** open a PR from your branch to `main`; your teammate reviews it — what changed, how it was tested — and approves before it is merged.
- **Issues and board:** open a GitHub Issue for every requirement code (V1–V6, F1–F9) and track it on a Projects board (Backlog → In progress → In review → Done).
- **Continuous integration:** if GitHub Actions is enabled for the repository, never merge while the build or the tests are red.
- **Version tags:** tag the state you submit for grading — `midterm-v1.0` for the midterm check, `final-v1.0` for the final check.
- **Definition of Done** — a requirement code is done when it builds without warnings, its unit tests pass with full coverage, its complexity is documented, and your teammate has reviewed the code.

8. Rubrics

Each criterion is scored on a **1–5** scale and multiplied by its weight: $points = (level \div 5) \times criterion\ points$.

Achievement levels (for every criterion):

Level	Meaning
5 — Excellent	Everything in scope works, is tested and documented; can be explained step by step in the demo
4 — Good	Minor gaps or edge-case bugs; complete and tested overall
3 — Adequate	Basic operations work; clear gaps in tests, documentation or measurements
2 — Poor	Compiles, but most operations are wrong or missing; weak explanation
1 — No evidence	Not submitted or not working

8.1 Midterm check rubric — RAP1 (C, 100 points)

#	Criterion	Scope	LO	Points
1	Linear structures	V1 linked lists, V2 sparse matrix, V3 stack and queue	LO.1	20
2	Tree and heap	V4 binary tree traversals, heap, priority queue, heap sort	LO.1, LO.4	15
3	Graph and traversal	V5 representation, BFS and DFS; answering the application's question	LO.5	15
4	Search and hashing	V6 binary search, hash function, collision resolution, load factor	LO.3, LO.4	10
5	Complexity analysis	Big-O of every operation; measurement table for at least two structures	LO.2	10
6	Problem analysis and structure choice	"Why this structure?" in the report: alternatives and trade-offs	LO.7	10
7	Software engineering	CMake, GoogleTest 100%, Doxygen 100%, binary files, Windows + WSL, GitHub usage	LO.7	20

8.2 Final check rubric — RAP2 (Java, 100 points)

#	Criterion	Scope	LO	Points
1	Port to Java and integrated application	F1 generics, all features in one menu, binary files	LO.1, LO.7	10
2	Graph algorithms	F2 two algorithms, correct results and explanation	LO.5	10
3	Sorting	F3 three algorithms, measurement and comparison	LO.2, LO.3	10
4	Balanced trees	F4 BST and AVL, rotations, range query	LO.4	15
5	String structures and algorithms	F5 search and alignment, F6 trie and union-find	LO.1, LO.3, LO.4	15
6	File organisation	F7 sequential and direct files, F8 B+ index, F9 extendible hashing or external sort	LO.6	20
7	Software engineering and presentation	Maven, JUnit 5 100%, documentation 100%, release, presentation and video, demo questions	LO.7	20

9. Acceptance conditions

Submissions are not accepted if...

- there is no GitHub repository, it is not private, or team members have no commits,
- unit-test or documentation coverage is below 100%,
- the repository or archive contains compiled files, or the Java repository has no release,
- the application does not build and run on Windows or WSL/Linux,
- plagiarism is detected.

10. Questions asked in the demo

- **Git and GitHub:** Did you fork the template with the correct name? Do both members have commits, were branches used? How were merges and conflicts resolved? Is `.gitignore` correct?
- **Setup and build:** Build and run the application on Windows and in WSL; show the `lib`, `app`, `test` split and their dependencies.
- **Data structures:** Explain a chosen operation (e.g. deleting from a doubly linked list, an AVL rotation, a hash collision) line by line in the code and with a box-and-arrow drawing of memory. What is its complexity, and why?

- **Tests and documentation:** Open the test and documentation coverage reports; show an edge-case test.
- **File operations:** Add a record, close and reopen the program, show that the record comes back from the binary file.
- **Programming:** pointers and arrays, `struct`, `malloc / free`, file reading and writing, watching the call stack and variables in the debugger (C); generics, interfaces, exceptions (Java).

11. Professional responsibility and academic integrity

- **Ethics:** the IEEE and ACM codes of ethics both ask you to be honest about what your code does and does not do, and to give and accept fair technical criticism — this is exactly what code review and the demo check.
- **License and attribution:** cite the external sources (books, papers, code snippets, links) you used in the report; if you reuse a small piece of someone else's code, mark its source and license in a comment.
- **Synthetic data:** all data you use is synthetic — sample data you generate yourself (see §5.3); never use real personal data.
- **Plagiarism:** the code and report belong to your team. Copying code from another team, from previous terms or from a project on the internet is plagiarism; similarity checks are run and plagiarism gives the project zero points.
- **Shared understanding:** every member must be able to explain **all** of the team's code; code that cannot be explained in the demo is not graded.

12. Frequently asked questions

? Can I work alone instead of in a team of two?



Yes. Team size is at most 3 students, and working alone is allowed. Teams are fixed at the end of week 3 (04.10.2026) and cannot change afterwards.

? What happens if two teams want the same topic?



The first team to write the topic in the Microsoft Teams table gets it; the other team chooses a different one.

? Can I change my topic after it has been approved?



No. The topic is approved together with your project plan and cannot change afterward.

? What if my unit-test or documentation coverage is below 100%?



The submission is not accepted (see §9); reach 100% coverage before the deadline.

? Do ready-made library collections (e.g. Java's `java.util`) count towards the requirement codes?



No. You implement the basic operations of every structure yourself (see §5).

? Is a late submission possible?

The syllabus rules on late submission apply; this guide does not add separate exceptions.

? Can I submit compiled files (.exe, .o, .class, .jar) inside the repository or the archive?

No. Remove them with `.gitignore` ; their presence is grounds for rejection (see §9).

Appendix — Project topic list

Choose one of the topics below (see 4. Choosing a topic). Topics are in eight groups; all data is synthetic and the applications use no network connection. The codes in the boxes are the requirements in section 5.

001–025 · Transport, maps and routing

001 — Metro Network Route and Transfer Planner



Summary: A console application that merges a city's metro, tram and funicular lines into one network and suggests the fastest or fewest-transfer route to a passenger. It works on a synthetic network of about 12 lines and 250 stations; stations closed for faults or maintenance are taken into account immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each line is a doubly linked list of stations (both directions); circle lines use a circular list.
- **V2 Sparse matrix:** In the time-slot × station passenger-density table, only measured cells are stored.
- **V3 Stack and queue:** A passenger's route adjustments are undone with a stack; turnstile entries are simulated with a queue.
- **V4 Tree and heap:** Fault reports are kept in a heap by severity, so the maintenance crew gets the most urgent one first.
- **V5 Graph and BFS/DFS:** Stations are nodes and connections are edges; BFS finds the fewest-stop route, DFS finds regions cut off after a station closes.
- **V6 Search and hashing:** Station name → station record is kept in a hash table; sorted station codes are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest route in minutes; Kruskal computes the cheapest connecting network for a new line investment.
- **F3 Sorting:** Route options are sorted by duration, number of transfers, and walking distance with three different algorithms, comparing their timing.
- **F4 BST and AVL:** Departure times for each station are kept in an AVL tree; the query "next train after 15:42" is answered in the balanced tree.
- **F5 String algorithms:** A passenger-typed station name is searched with KMP; a misspelled name gets the closest suggestion via edit distance.
- **F6 Trie and disjoint sets:** Station names autocomplete through a trie; union-find groups regions that remain mutually reachable after stations close.
- **F7 File organisation:** Trip records are kept in a sequential file; station cards are kept in a direct-access file, resolving collisions with Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on card number is kept in the travel-history file.
- **F9 Extendible hashing / external sort:** Millions of monthly turnstile records don't fit in memory, so they are sorted by hour with an external merge sort.

Extension: Recalculating the route during the journey from real-time delay notices.

002 — 🚌 City Bus Stop Arrival Estimator



Summary: A console application that tells waiting passengers how many minutes until the next vehicle reaches a given stop, blending the timetable with simulated live position pings. It runs on a synthetic network of about 45 routes and 600 stops; vehicles report their position every 30 seconds and delays feed into the estimate immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each route's stop order is kept as a doubly linked list (outbound/inbound direction); loop routes are modeled with a circular list.
- **V2 Sparse matrix:** In the stop × 15-minute time-slot boarding-count table, only cells with an actual boarding recorded are stored.
- **V3 Stack and queue:** The dispatcher's hold-vehicle interventions are undone with a stack; vehicles at the terminal wait in a queue in arrival order.
- **V4 Tree and heap:** Vehicles running behind schedule are kept in a heap keyed by delay minutes, so the most delayed one is rerouted first.
- **V5 Graph and BFS/DFS:** Stops are nodes and street segments are edges; BFS finds the nearest stop reachable on foot within N minutes, DFS finds stops cut off after a street closes.
- **V6 Search and hashing:** Stop code → stop record is kept in a hash table; sorted route numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest transfer route between two stops in minutes; Kruskal proposes the cheapest new street connection to link an isolated cluster of stops.
- **F3 Sorting:** Upcoming vehicles at a stop are sorted by estimated wait time with three different algorithms, comparing their timing across 600 stops.
- **F4 BST and AVL:** Each stop's scheduled departure times are kept in an AVL tree, so "next vehicle after 14:10" queries stay fast through frequent timetable edits.
- **F5 String algorithms:** A passenger-typed stop name is matched with KMP search; a misspelled name gets the closest suggestion via edit distance.
- **F6 Trie and disjoint sets:** Stop names autocomplete through a trie; union-find groups stops that remain mutually reachable after several streets close.
- **F7 File organisation:** Daily trip logs are kept in a sequential file; stop records are kept in a direct-access file, resolving collisions with the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on route number speeds up lookups in the trip-log file.
- **F9 Extendible hashing / external sort:** Monthly boarding records exceed memory, so they are sorted by timestamp with an external merge sort.

Extension: Adding a real-time GPS feed so estimates adjust instantly to traffic delays.

003 — Airport Runway Departure Sequencer



Summary: A console application that sequences aircraft departures on a single runway, factoring in ground-handling delays and separation rules between aircraft categories. It works on a synthetic schedule of about 200 flights a day; gate delays and runway faults affect the sequence immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The pushback queue for each gate area is a doubly linked list so ground control can insert or remove a flight from either end; the shared tow-truck rotation uses a circular list.
- **V2 Sparse matrix:** In the gate × 10-minute time-slot occupancy table, only cells with an aircraft actually parked are stored.
- **V3 Stack and queue:** Cancelled clearance instructions are undone with a stack; aircraft at the holding point wait in a queue in arrival order.
- **V4 Tree and heap:** Aircraft are kept in a heap keyed by scheduled departure time, so the tower always sees the next flight due for runway clearance.
- **V5 Graph and BFS/DFS:** Taxiway junctions are nodes and taxiway segments are edges; BFS finds the shortest taxi route from gate to runway, DFS finds areas cut off after a segment closes.
- **V6 Search and hashing:** Flight number → flight record is kept in a hash table; sorted scheduled times are searched with binary search for the nearest free slot.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest taxi time to the runway threshold in minutes; topological sort orders flights sharing a taxiway conflict without collisions.
- **F3 Sorting:** Flights waiting for clearance are sorted by scheduled time and aircraft category with three algorithms, comparing their timing over 200 flights.
- **F4 BST and AVL:** Runway slot reservations are kept in an AVL tree keyed by time, so "next free slot after 09:35" queries stay fast as reservations change.
- **F5 String algorithms:** A controller-typed flight number is matched with Boyer-Moore search; a typo is corrected using edit distance against the day's flight list.
- **F6 Trie and disjoint sets:** Airline call signs autocomplete through a trie; union-find groups gates affected by a shared ground-power-unit outage.
- **F7 File organisation:** Daily departure logs are kept in a sequential file; aircraft records are kept in a direct-access file, resolving collisions with the progressive overflow method.
- **F8 B+ tree index:** A B+ tree secondary index on airline code speeds up lookups in the departure-log file.
- **F9 Extendible hashing / external sort:** A year of departure logs won't fit in memory, so they are sorted by date with an external merge sort.

Extension: Dynamically re-sequencing the queue when wind-dependent separation minima change.

004 — Ferry Trip and Vehicle Loading Planner



Summary: A console application that plans ferry crossings and vehicle deck placement between two ports, tracking vehicle weight and lane capacity. It works on a synthetic fleet of 6 ferries and about 80 crossings a day; last-minute cancellations and weight changes affect the loading plan immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Vehicles booked on a crossing form a doubly linked list in check-in order; a ferry's round trip between two ports is modeled as a circular list.
- **V2 Sparse matrix:** In the deck-lane × crossing occupancy table, only cells with a vehicle actually assigned are stored.
- **V3 Stack and queue:** A driver's last-minute lane change is undone with a stack; vehicles waiting at the ramp queue up in boarding order.
- **V4 Tree and heap:** Standby vehicles are kept in a heap keyed by priority (freight ahead of passenger cars), so the loading crew boards the most urgent one first.
- **V5 Graph and BFS/DFS:** Ports are nodes and ferry routes are edges; BFS finds the fewest-transfer path between two ports, DFS finds ports isolated after a route is cancelled.
- **V6 Search and hashing:** License plate → booking record is kept in a hash table; sorted departure times are searched with binary search for the next available crossing.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the shortest total travel time between two ports across connecting routes; cycle detection flags conflicting scheduling loops in the multi-leg timetable.
- **F3 Sorting:** Vehicles waiting to board are sorted by weight and check-in time with three algorithms, comparing their timing over 300 synthetic bookings.
- **F4 BST and AVL:** Each route's crossing schedule is kept in an AVL tree, so "next crossing after 13:00" queries stay fast as the timetable changes.
- **F5 String algorithms:** A driver-entered plate number is matched with KMP search; a misread plate is corrected against the booking list using edit distance.
- **F6 Trie and disjoint sets:** Vehicle categories autocomplete through a trie; union-find groups ports that remain mutually reachable after a route is suspended.
- **F7 File organisation:** Completed crossing logs are kept in a sequential file; vehicle records are kept in a direct-access file, resolving collisions with Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on route code speeds up lookups in the crossing-log file.
- **F9 Extendible hashing / external sort:** A season's worth of booking records exceeds memory, so they are sorted by date with an external merge sort.

Extension: Automatically reshuffling deck-lane assignments when a vehicle's declared weight changes at check-in.

005 — Bike-Share Station Rebalancer



Summary: A console application that tracks bike counts across a synthetic network of about 120 docking stations and plans routes for vans that move bikes between stations running empty or full. Rental and return events are processed as they happen; stations drifting from their target count are flagged immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each rebalancing van's stop order is a doubly linked list so a stop can be inserted mid-route; the depot's return loop is kept as a circular list.
- **V2 Sparse matrix:** In the station × hour rental-count table, only hours with an actual rental recorded are stored.
- **V3 Stack and queue:** A dispatcher's manual route edit is undone with a stack; bikes reported broken wait in a queue for pickup in report order.
- **V4 Tree and heap:** Stations are kept in a heap keyed by how far their bike count deviates from target, so the van visits the most imbalanced station first.
- **V5 Graph and BFS/DFS:** Stations are nodes and streets are edges; BFS finds the nearest station with an available bike within N stops, DFS finds stations cut off after a street closes.
- **V6 Search and hashing:** Station code → station record is kept in a hash table; sorted station IDs are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm builds the cheapest set of streets connecting all stations for a new maintenance route; Dijkstra finds the shortest rebalancing path between two stations.
- **F3 Sorting:** Stations needing a rebalancing visit are sorted by how imbalanced they are with three algorithms, comparing their timing across 120 stations.
- **F4 BST and AVL:** Each station's hourly rental history is kept in an AVL tree, so "busiest hour after 08:00" queries stay fast as new data arrives.
- **F5 String algorithms:** A user-typed station name is matched with KMP search; a misspelled name is corrected with edit distance.
- **F6 Trie and disjoint sets:** Station names autocomplete through a trie; union-find groups stations that remain mutually reachable by bike lane after a closure.
- **F7 File organisation:** Daily rental logs are kept in a sequential file; station records are kept in a direct-access file, resolving collisions with the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on station code speeds up lookups in the rental-log file.
- **F9 Extendible hashing / external sort:** A year of rental logs exceeds memory, so they are sorted by timestamp with an external merge sort.

Extension: Weather-adjusted demand prediction that shifts rebalancing priority ahead of rain or a heat spike.

006 — 🚗 Taxi Dispatch Matching Simulator



Summary: A console application simulating a dispatch center that matches incoming ride requests with nearby available drivers. It runs on a synthetic city grid of about 300 taxis and 1000 simulated ride requests; driver availability updates within seconds.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each driver's ride history is a doubly linked list so past trips can be browsed forward and back; a driver's break rotation is kept as a circular list.
- **V2 Sparse matrix:** In the city-zone × hour request-count table, only cells with an actual recorded request are stored.
- **V3 Stack and queue:** A rider's cancelled request is undone with a stack; incoming requests wait in a queue for matching in arrival order.
- **V4 Tree and heap:** Available drivers near a pickup point are kept in a heap keyed by distance, so the closest driver is matched first.
- **V5 Graph and BFS/DFS:** Intersections are nodes and streets are edges; BFS finds the nearest available driver within N intersections, DFS finds zones isolated after a road closes.
- **V6 Search and hashing:** Driver ID → driver record is kept in a hash table; sorted fare amounts are searched with binary search for fare-band reports.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest time from driver to pickup point; strongly connected components identify zones still mutually reachable after closures.
- **F3 Sorting:** Completed trips are sorted by fare, distance, and duration with three algorithms, comparing their timing over 1000 trips.
- **F4 BST and AVL:** Each driver's completed-trip timestamps are kept in an AVL tree, so "trips after 18:00" queries stay balanced as new trips are added.
- **F5 String algorithms:** A rider-typed pickup address is matched against the street list with Boyer-Moore search; a typo is corrected with edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete through a trie; union-find groups zones that remain mutually reachable after several roads close.
- **F7 File organisation:** Daily trip logs are kept in a sequential file; driver records are kept in a direct-access file, resolving collisions with the progressive overflow method.
- **F8 B+ tree index:** A B+ tree secondary index on driver ID speeds up lookups in the trip-log file.
- **F9 Extendible hashing / external sort:** As new drivers register through the year, the driver file grows bucket by bucket with extendible hashing instead of a full reorganization.

Extension: A surge-pricing simulation that reprioritizes matching when request density in a zone spikes.

007 — 🚂 Railway Switch and Platform Manager



Summary: A console application that manages switch settings and platform assignments at a station to avoid routing conflicts between trains. It works on a synthetic station with 10 platforms and about 150 train movements a day; delays and switch faults affect assignments immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each train's scheduled stop sequence is a doubly linked list of stations; a shuttle service looping between two stations is modeled as a circular list.
- **V2 Sparse matrix:** In the platform × 15-minute time-slot occupancy table, only cells with a train actually assigned are stored.
- **V3 Stack and queue:** A dispatcher's cancelled platform reassignment is undone with a stack; trains waiting for a free platform queue up in arrival order.
- **V4 Tree and heap:** Delayed trains are kept in a heap keyed by delay minutes, so the dispatcher reassigns the most delayed train's platform first.
- **V5 Graph and BFS/DFS:** Track junctions are nodes and switch-equipped track segments are edges; BFS finds the shortest routing path from approach track to platform, DFS finds platforms cut off after a switch fault.
- **V6 Search and hashing:** Train number → train record is kept in a hash table; sorted scheduled arrival times are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest routing path in minutes; cycle detection flags conflicting switch settings that would create a routing loop.
- **F3 Sorting:** Trains waiting for a platform are sorted by priority (express ahead of local) and scheduled time with three algorithms, comparing their timing over 150 movements.
- **F4 BST and AVL:** Each platform's reservation times are kept in an AVL tree, so "next free slot after 10:20" queries stay fast as reservations change.
- **F5 String algorithms:** A dispatcher-typed train number is matched with KMP search; a typo is corrected using edit distance against the day's schedule.
- **F6 Trie and disjoint sets:** Station names autocomplete through a trie; union-find groups track sections that remain mutually reachable after a switch failure.
- **F7 File organisation:** Daily movement logs are kept in a sequential file; train records are kept in a direct-access file, resolving collisions with Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on train number speeds up lookups in the movement-log file.
- **F9 Extendible hashing / external sort:** A year of movement logs exceeds memory, so they are sorted by date with an external merge sort.

Extension: A fault-injection mode that simulates a switch failure and reroutes affected trains live.

008 — Highway Toll Queue Simulator



Summary: A console application simulating vehicle queues at a toll plaza with 12 booths mixing cash and automatic-pass lanes. It processes about 5000 simulated vehicles a day; when a booth closes, traffic is redirected to other lanes immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The queue of vehicles waiting behind each booth is a doubly linked list so a vehicle can be inserted when a faster lane opens; the plaza's booth-rotation for maintenance breaks uses a circular list.
- **V2 Sparse matrix:** In the booth × 10-minute time-slot vehicle-count table, only cells with recorded traffic are stored.
- **V3 Stack and queue:** An operator's cancelled lane-closure action is undone with a stack; vehicles approaching each booth queue up and are served in arrival order.
- **V4 Tree and heap:** Booths are kept in a heap keyed by current queue length, so incoming traffic is directed to the shortest queue first.
- **V5 Graph and BFS/DFS:** Lane merge points are nodes and lane segments are edges; BFS finds the fewest-merge path to an open booth, DFS finds lanes cut off after a booth closes.
- **V6 Search and hashing:** License plate → toll-account record is kept in a hash table; sorted booth IDs are searched with binary search for maintenance scheduling.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest expected path to a booth in seconds given current queue lengths; max flow estimates the plaza's total vehicle throughput per minute.
- **F3 Sorting:** Vehicles in the automatic-pass log are sorted by toll amount and processing time with three algorithms, comparing their timing over 5000 records.
- **F4 BST and AVL:** Each booth's hourly throughput counts are kept in an AVL tree, so "busiest hour after 07:00" queries stay fast as counts update.
- **F5 String algorithms:** An operator-entered plate number is matched with KMP search; a misread plate is corrected against the account list using edit distance.
- **F6 Trie and disjoint sets:** Vehicle class codes autocomplete through a trie; union-find groups lanes that remain mutually reachable after several booths close.
- **F7 File organisation:** Daily transaction logs are kept in a sequential file; toll-account records are kept in a direct-access file, resolving collisions with the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on plate number speeds up lookups in the transaction-log file.
- **F9 Extendible hashing / external sort:** As new toll-tag accounts are registered, the account file grows bucket by bucket with extendible hashing instead of a full reorganization.

Extension: A dynamic booth-opening rule that automatically opens a closed booth once queue length crosses a threshold.

009 — Intersection Traffic Light Timer



Summary: A console application that computes traffic-light phase timing for a synthetic network of about 25 connected intersections from simulated vehicle-count sensors, aiming to cut total waiting time. Sensor readings update every few minutes; a spike in traffic density feeds into phase timing immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each intersection's phase sequence (the green-yellow-red cycle per direction) is a doubly linked list so a phase can be inserted for a temporary pedestrian crossing; the repeating daily cycle is modeled as a circular list.
- **V2 Sparse matrix:** In the intersection $\times$ 5-minute time-slot vehicle-count table, only cells with an actual sensor reading are stored.
- **V3 Stack and queue:** An operator's manual phase-timing override is undone with a stack; vehicles detected waiting at a red light queue up in detection order.
- **V4 Tree and heap:** Intersections are kept in a heap keyed by current queue length, so the control system retimes the most congested intersection first.
- **V5 Graph and BFS/DFS:** Intersections are nodes and connecting roads are edges; BFS finds the shortest green-wave path between two intersections, DFS finds intersections isolated after a road closes.
- **V6 Search and hashing:** Intersection ID $\rightarrow$ intersection record is kept in a hash table; sorted phase-duration values are searched with binary search for reporting.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest travel path across several intersections given current phase timings; topological sort orders intersections along a one-way corridor for coordinated green-wave timing.
- **F3 Sorting:** Intersections are sorted by average wait time with three algorithms, comparing their timing across 25 intersections and a day of sensor data.
- **F4 BST and AVL:** Each intersection's sensor readings are kept in an AVL tree keyed by timestamp, so "readings after 08:00" queries stay fast as new data streams in.
- **F5 String algorithms:** An operator-typed intersection name is matched with KMP search; a typo is corrected against the intersection list using edit distance.
- **F6 Trie and disjoint sets:** Intersection names autocomplete through a trie; union-find groups intersections that remain mutually reachable after a road closes.
- **F7 File organisation:** Daily sensor logs are kept in a sequential file; intersection records are kept in a direct-access file, resolving collisions with the progressive overflow method.
- **F8 B+ tree index:** A B+ tree secondary index on intersection ID speeds up lookups in the sensor-log file.
- **F9 Extendible hashing / external sort:** A year of sensor logs exceeds memory, so they are sorted by timestamp with an external merge sort.

Extension: Emergency-vehicle preemption that forces a green-wave path when a simulated ambulance signal is detected.

010 — Offline Map Place Search Engine



Summary: A console application for a navigation device that searches a synthetic city map (about 5000 points of interest) by name or category with no network access at all. All data is kept as a pre-downloaded copy on the device; nearby results are recomputed as the user moves.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Points of interest sharing a street are kept as a doubly linked list in address order; a walking tour that loops back to its start is modeled as a circular list.
- **V2 Sparse matrix:** In the map-tile $\times$ category presence table, only tile-category pairs that actually contain a point of interest are stored.
- **V3 Stack and queue:** A user's navigation-history back button is implemented with a stack; nearby search results queue up for display in distance order.
- **V4 Tree and heap:** Search results are kept in a heap keyed by distance from the user's position, so the nearest matches are shown first.
- **V5 Graph and BFS/DFS:** Map intersections are nodes and streets are edges; BFS finds the nearest point of interest within N street segments, DFS finds areas isolated after a road closes on the map.
- **V6 Search and hashing:** Place name $\rightarrow$ place record is kept in a hash table; sorted postal codes are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest walking route to a selected point of interest; Kruskal proposes the minimum set of streets needed to keep every point of interest reachable for a reduced offline map.
- **F3 Sorting:** Search results are sorted by distance, rating, and category relevance with three algorithms, comparing their timing over 5000 points of interest.
- **F4 BST and AVL:** Points of interest are indexed in an AVL tree keyed by street address, so "next address after 120 Main" queries stay balanced as the map updates.
- **F5 String algorithms:** A user-typed place name is matched with KMP search; a misspelled name gets the closest suggestion via edit distance.
- **F6 Trie and disjoint sets:** Place names autocomplete through a trie; union-find groups map regions that remain mutually reachable after roads are marked closed.
- **F7 File organisation:** The full point-of-interest catalog is kept in a sequential file; frequently accessed place records are kept in a direct-access file, resolving collisions with Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on category code speeds up lookups in the point-of-interest file.
- **F9 Extendible hashing / external sort:** The nationwide point-of-interest file exceeds memory, so it is sorted by postal code with an external merge sort.

Extension: Offline turn-by-turn voice guidance generated from the computed walking route.

011 — Campus Walking Pathfinder



Summary: A mobile application that suggests a pedestrian the shortest or barrier-free (wheelchair-accessible) route between buildings, stairs, ramps and indoor passages on a university campus. It works on a synthetic campus map of about 40 buildings and 300 walking segments; passages closed for construction are removed from routing immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each walking route is a doubly linked list of consecutive junction points; the circular path ringing the campus perimeter uses a circular list.
- **V2 Sparse matrix:** In the building × time-slot density table, only cells where a student count was taken are stored.
- **V3 Stack and queue:** The user's undo steps while adjusting a route are handled with a stack; turnstile entries at the campus gate are simulated with a queue.
- **V4 Tree and heap:** Reported ramp/stair faults are kept in an urgency heap, so the most dangerous fault is repaired first.
- **V5 Graph and BFS/DFS:** Junctions are nodes, walking segments are edges; BFS finds the fewest-step route, DFS finds buildings unreachable after a passage closes.
- **V6 Search and hashing:** Building name to building record is kept in a hash table; sorted room numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest route by walking time; Prim computes the cheapest cabling network for new campus lighting.
- **F3 Sorting:** Alternative routes are sorted and compared using three algorithms by walking time, number of stairs and accessibility suitability.
- **F4 BST and AVL:** Building opening and closing hours are kept in an AVL tree; the query "nearest library open right now" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the building name the user typed; on a typo, edit distance suggests the closest building name.
- **F6 Trie and disjoint sets:** Building and unit names are auto-completed with a trie; union-find groups campus areas that remain mutually walkable after construction closures.
- **F7 File organisation:** Daily foot-traffic count records are stored in a sequential file, building cards in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** The building records file keeps a B+ tree secondary index on building code.
- **F9 Extendible hashing / external sort:** Millions of sensor-crossing records accumulated over a semester do not fit in memory, so they are sorted by hour with an external merge sort.

Extension: Feeding real-time corridor-crowding sensor data into the route.

012 — ~~5~~ Parcel Delivery Route Optimiser



Summary: A desktop application that suggests a delivery company's trucks the stop order that completes daily parcel deliveries in the least time. It works on a synthetic region of about 25 trucks and 500 delivery addresses; traffic closures and cancelled deliveries are reflected in the route immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each truck's delivery order is a doubly linked list of stops (for reordering forward/backward); circular service loops that return to the depot use a circular list.
- **V2 Sparse matrix:** In the district × time-slot delivery-density table, only cells where a parcel was dropped off are stored.
- **V3 Stack and queue:** The driver's stop-order changes are undone with a stack; trucks waiting to leave the depot are simulated with a queue.
- **V4 Tree and heap:** Urgent (same-day) parcels are kept in a priority heap, so the parcel with the nearest deadline goes out first.
- **V5 Graph and BFS/DFS:** Junctions are nodes, roads are edges; BFS finds the fewest-stop delivery order, DFS finds neighbourhoods unreachable after a road closes.
- **V6 Search and hashing:** Recipient name to address record is kept in a hash table; sorted tracking numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest travel time between two addresses; Kruskal computes the cheapest road network for a new distribution centre.
- **F3 Sorting:** Pending parcels are sorted and compared using three algorithms by delivery deadline, weight and distance.
- **F4 BST and AVL:** Each address's past delivery times are kept in an AVL tree; the query "best delivery time for this address" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the street name typed; on a typo, edit distance suggests the closest matching address.
- **F6 Trie and disjoint sets:** Neighbourhood and street names are auto-completed with a trie; union-find groups delivery zones that remain mutually reachable after a road closes.
- **F7 File organisation:** Completed delivery records are stored in a sequential file, customer cards in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** The delivery-history file keeps a B+ tree secondary index on tracking number.
- **F9 Extendible hashing / external sort:** Millions of yearly delivery records do not fit in memory, so they are sorted by delivery date with an external merge sort.

Extension: Recalculating the route during the day using live traffic data.

013 — Ambulance Dispatch and Hospital Selector



Summary: A console application that assigns incoming emergency calls at a city's dispatch centre to the nearest available ambulance and directs the patient to the most suitable hospital. It works on a synthetic city model of about 30 ambulances and 15 hospitals; ICU occupancy and road closures are taken into account immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each ambulance's case history is a doubly linked list of records; the shift-rotation cycle uses a circular list.
- **V2 Sparse matrix:** In the district × time-slot call-density table, only cells where a call was received are stored.
- **V3 Stack and queue:** The dispatcher's case-priority corrections are undone with a stack; idle ambulances waiting for assignment are simulated with a queue.
- **V4 Tree and heap:** Pending cases are kept in a triage-urgency heap, so the most critical case is assigned an ambulance first.
- **V5 Graph and BFS/DFS:** Junctions are nodes, roads are edges; BFS finds the fewest-turn route, DFS finds neighbourhoods unreachable after a road closes.
- **V6 Search and hashing:** Patient ID number to case record is kept in a hash table; sorted hospital codes are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes an ambulance's arrival time to a case; Prim computes the cheapest road network for siting a new station.
- **F3 Sorting:** Eligible hospitals are sorted and compared using three algorithms by distance, free bed count and specialty match.
- **F4 BST and AVL:** Hospitals' past ICU-occupancy history is kept in an AVL tree; the query "nearest hospital with a free bed right now" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the district name the dispatcher typed; on a typo, edit distance suggests the closest matching address.
- **F6 Trie and disjoint sets:** District and street names are auto-completed with a trie; union-find groups districts that remain mutually reachable after a road closes.
- **F7 File organisation:** Closed case records are stored in a sequential file, hospital capacity cards in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** The case-records file keeps a B+ tree secondary index on patient ID number.
- **F9 Extendible hashing / external sort:** Millions of yearly call records do not fit in memory, so they are sorted by call time with an external merge sort.

Extension: Updating arrival-time estimates in real time using traffic-camera data.

014 — Tram Line Headway Simulator



Summary: A desktop application that simulates a single tram line's timetable to keep the headway between vehicles even. It works on a synthetic line of about 8 trams and 40 stops; a vehicle delayed by a fault immediately affects the headway of the following services.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The stops along the line form a sequential doubly linked list; a loop line whose start and end point coincide uses a circular list.
- **V2 Sparse matrix:** In the stop $\times$ time-slot boarding-alighting density table, only measured cells are stored.
- **V3 Stack and queue:** The schedule planner's headway corrections are undone with a stack; spare vehicles waiting in the depot are simulated with a queue.
- **V4 Tree and heap:** Fault reports are kept in a severity heap, so the fault most disruptive to the line is attended to first.
- **V5 Graph and BFS/DFS:** Stops are nodes, track links are edges; BFS finds the fewest-transfer path between two stops, DFS finds line segments cut off after a track fault.
- **V6 Search and hashing:** Stop name to stop record is kept in a hash table; sorted service numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest travel time between two stops; cycle detection flags unwanted track loops in the line diagram.
- **F3 Sorting:** The day's services are sorted and compared using three algorithms by departure time, occupancy rate and delay amount.
- **F4 BST and AVL:** Each stop's past arrival times are kept in an AVL tree; the query "first tram after 15:00" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the stop name a passenger typed; on a typo, edit distance suggests the closest matching stop.
- **F6 Trie and disjoint sets:** Stop names are auto-completed with a trie; union-find groups line segments that remain mutually reachable after a track fault.
- **F7 File organisation:** Completed service records are stored in a sequential file, vehicle maintenance cards in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** The service-records file keeps a B+ tree secondary index on service number.
- **F9 Extendible hashing / external sort:** Millions of yearly stop-crossing records do not fit in memory, so they are sorted by date with an external merge sort.

Extension: Automatically tightening the headway in real time based on live passenger crowding.

015 — 🚛 Long-Haul Rest and Fuel Stop Planner



Summary: A mobile application that, based on a vehicle's tank capacity and a driver's fatigue limit, suggests the order of rest and fuel stops on long road trips. It works on a synthetic motorway network of about 200 stations and 60 rest facilities; closed stations are removed from the route immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The stops along a route form a sequential doubly linked list; a ring road around a city centre uses a circular list.
- **V2 Sparse matrix:** In the motorway-section × time-slot occupancy table, only measured station-hour pairs are stored.
- **V3 Stack and queue:** The driver's stop-choice changes are undone with a stack; vehicles waiting at a busy station are simulated with a queue.
- **V4 Tree and heap:** Reported station faults (broken pump, tank empty) are kept in a severity heap.
- **V5 Graph and BFS/DFS:** Junctions are nodes, road sections are edges; BFS finds the fewest-stop route, DFS finds stations unreachable after road works.
- **V6 Search and hashing:** Station brand to station record is kept in a hash table; sorted kilometre markers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the nearest reachable station given tank-range constraints; Kruskal computes the cheapest road network for a new station investment.
- **F3 Sorting:** Alternative stops are sorted and compared using three algorithms by price, distance and occupancy.
- **F4 BST and AVL:** Each station's past fuel prices are kept in an AVL tree; the query "nearest cheapest-priced station" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the station brand typed; on a typo, edit distance suggests the closest matching brand name.
- **F6 Trie and disjoint sets:** Station brands are auto-completed with a trie; union-find groups motorway sections that remain mutually reachable after road works.
- **F7 File organisation:** Past rest-stop records are stored in a sequential file, station cards in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** The station-records file keeps a B+ tree secondary index on kilometre marker.
- **F9 Extendible hashing / external sort:** Millions of yearly price-update records do not fit in memory, so they are sorted by date with an external merge sort.

Extension: Recalculating the cheapest stop during the trip using live fuel-price notifications.

016 — Sailing Regatta Course Calculator



Summary: A desktop application that orders a sailing race's marker buoys by wind direction and computes the best leg sequence for racers. It works on a synthetic race course of about 10 buoys and 20 boats; the course is recalculated immediately whenever the wind direction changes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The legs of the course (buoy order) form a doubly linked list; lap courses whose start and finish point coincide use a circular list.
- **V2 Sparse matrix:** In the zone × minute wind-speed reading table, only cells with sensor data are stored.
- **V3 Stack and queue:** The race officer's course changes are undone with a stack; boats waiting at the start line are simulated with a queue.
- **V4 Tree and heap:** Umpire violation reports are kept in a severity heap, so the most serious violation is reviewed first.
- **V5 Graph and BFS/DFS:** Buoys are nodes, possible legs are edges; BFS finds the fewest-turn course, DFS finds zones left in wind shadow.
- **V6 Search and hashing:** Boat sail number to boat record is kept in a hash table; sorted finish times are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest leg sequence given the wind angle; an MST algorithm computes the shortest total mooring-line length for anchoring the buoys.
- **F3 Sorting:** Race results are sorted and compared using three algorithms by finish time, handicap score and penalty minutes.
- **F4 BST and AVL:** Each boat's past leg times are kept in an AVL tree; the query "fastest boat upwind" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the boat name typed; on a typo, edit distance suggests the closest matching boat name.
- **F6 Trie and disjoint sets:** Boat names are auto-completed with a trie; union-find identifies groups of boats moving close together after being caught in wind shadow.
- **F7 File organisation:** Past race results are stored in a sequential file, boat registration cards in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** The race-results file keeps a B+ tree secondary index on sail number.
- **F9 Extendible hashing / external sort:** Millions of wind-sensor records accumulated over a season do not fit in memory, so they are sorted by timestamp with an external merge sort.

Extension: Rearranging course legs mid-race using a live wind forecast.

017 — Skyscraper Elevator Call Scheduler



Summary: A simulation application that assigns passenger calls in a multi-storey building to multiple elevator cars with the shortest possible wait time. It works on a synthetic building model of about 60 floors and 6 elevator cars; a car under maintenance is removed from assignment immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each elevator's stop sequence is a doubly linked list of floors (for up/down direction); the service corridor ringing the ground floor uses a circular list.
- **V2 Sparse matrix:** In the floor × time-slot call-density table, only cells where a call was made are stored.
- **V3 Stack and queue:** A passenger's floor-selection cancellations are undone with a stack; passengers waiting in the lobby are simulated with a queue.
- **V4 Tree and heap:** Pending calls are kept in a wait-time heap, so the longest-waiting call is assigned a car first.
- **V5 Graph and BFS/DFS:** Floors are nodes, elevator-stair links are edges; BFS finds the fewest-transfer evacuation route in an emergency, DFS finds floors unreachable after a car goes into maintenance.
- **V6 Search and hashing:** Passenger card number to access-permission record is kept in a hash table; sorted floor numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes a passenger's total travel time from floor A to floor B; topological sorting computes the car visiting order during the morning rush.
- **F3 Sorting:** Pending calls are sorted and compared using three algorithms by wait time, destination floor and car occupancy.
- **F4 BST and AVL:** Each car's past stop statistics are kept in an AVL tree; the query "least-used car" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the office/company name a passenger typed; on a typo, edit distance suggests the closest matching directory entry.
- **F6 Trie and disjoint sets:** Company names in the floor directory are auto-completed with a trie; union-find identifies floor groups that remain mutually reachable during maintenance.
- **F7 File organisation:** Daily call records are stored in a sequential file, car maintenance cards in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** The call-records file keeps a B+ tree secondary index on passenger card number.
- **F9 Extendible hashing / external sort:** Millions of yearly call records do not fit in memory, so they are sorted by timestamp with an external merge sort.

Extension: Pre-positioning cars near the lobby using a model that predicts the morning rush.

018 — ✈ Flight Connection Finder



Summary: A console application that finds a passenger the fewest-transfer or shortest total-duration flight connection in an airline's schedule. It works on a synthetic schedule of about 35 airports and 180 flight routes; cancelled flights are removed from the connection search immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A passenger's multi-leg ticket is a doubly linked list of flight legs; shuttle flights (repeating between the same two airports) use a circular list.
- **V2 Sparse matrix:** In the airport × time-slot departure-density table, only cells with a scheduled flight are stored.
- **V3 Stack and queue:** A passenger's alternative-connection attempts are undone with a stack; passengers at the check-in counter are simulated with a queue.
- **V4 Tree and heap:** Delay notices are kept in an impact-size heap, so the delay disrupting the most connections is reported to operations first.
- **V5 Graph and BFS/DFS:** Airports are nodes, direct flights are edges; BFS finds the fewest-transfer connection, DFS finds destinations unreachable when an airport closes.
- **V6 Search and hashing:** Ticket PNR code to reservation record is kept in a hash table; sorted flight numbers are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest connection by total flight time; a maximum-flow algorithm computes the maximum passenger throughput an airport can handle at peak hour.
- **F3 Sorting:** Connection options are sorted and compared using three algorithms by total duration, number of transfers and layover time.
- **F4 BST and AVL:** Each airport's past departure times are kept in an AVL tree; the query "first flight after 18:00" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the city name a passenger typed; on a typo, edit distance suggests the closest matching airport.
- **F6 Trie and disjoint sets:** City and airport names are auto-completed with a trie; union-find identifies groups of airports that remain mutually reachable by flight after cancellations.
- **F7 File organisation:** Completed flight records are stored in a sequential file, passenger ticket cards in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** The reservation-records file keeps a B+ tree secondary index on PNR code.
- **F9 Extendible hashing / external sort:** Millions of yearly ticket records do not fit in memory, so they are sorted by flight date with an external merge sort.

Extension: Computing the risk of missing a connection in real time using live delay data.

019 — 🚧 Road Works Impact Analyser



Summary: A desktop application that predicts the impact of planned road works on a city's traffic flow in advance and suggests alternative routes. It works on a synthetic city road network of about 300 junctions and 50 planned work sites; the affected area is recalculated immediately once a work site closes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The junctions along a route form a doubly linked list; the ring road around the city centre uses a circular list.
- **V2 Sparse matrix:** In the road-section × time-slot traffic-density table, only measured cells are stored.
- **V3 Stack and queue:** A traffic engineer's work-plan changes are undone with a stack; vehicles approaching a work site are simulated with a queue.
- **V4 Tree and heap:** Planned works are kept in a city-wide-impact heap, so the most disruptive work is reviewed first.
- **V5 Graph and BFS/DFS:** Junctions are nodes, road sections are edges; BFS finds the least-detour alternative route after a closure, DFS finds neighbourhoods unreachable after a section closes.
- **V6 Search and hashing:** Work-site code to work record is kept in a hash table; sorted junction codes are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the travel-time difference before and after a closure; cut-edge (bridge) detection finds critical single-path road sections.
- **F3 Sorting:** Planned works are sorted and compared using three algorithms by affected vehicle count, duration and priority score.
- **F4 BST and AVL:** Each road section's past traffic density is kept in an AVL tree; the query "least congested alternative at this hour" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the street name typed; on a typo, edit distance suggests the closest matching street name.
- **F6 Trie and disjoint sets:** Street and neighbourhood names are auto-completed with a trie; union-find identifies road-network sections split apart by simultaneous closures.
- **F7 File organisation:** Completed work records are stored in a sequential file, work-site cards in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** The work-records file keeps a B+ tree secondary index on work-site code.
- **F9 Extendible hashing / external sort:** Millions of yearly traffic-sensor records do not fit in memory, so they are sorted by date with an external merge sort.

Extension: Updating the impact forecast during the works using live traffic-camera data.

020 — ❄️ Snowplough Route Planner



Summary: A desktop application that, after a snowfall, suggests a district's snowploughs a route covering priority streets first. It works on a synthetic district road network of about 15 vehicles and 400 street segments; segments reported with heavy snow buildup are added to the route immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each vehicle's plough order is a doubly linked list of street segments; patrol vehicles that repeat a loop route use a circular list.
- **V2 Sparse matrix:** In the street-segment × hour snow-depth reading table, only cells with sensor data are stored.
- **V3 Stack and queue:** The field supervisor's route-priority changes are undone with a stack; vehicles waiting at the depot to load salt/sand are simulated with a queue.
- **V4 Tree and heap:** Reported snow-buildup complaints are kept in an urgency heap, so a critical segment such as a school route is ploughed first.
- **V5 Graph and BFS/DFS:** Junctions are nodes, street segments are edges; BFS finds the fewest-pass plough round, DFS finds neighbourhoods unreachable while a segment stays unploughed.
- **V6 Search and hashing:** Street name to segment record is kept in a hash table; sorted segment codes are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest arrival time from the depot to the farthest segment; an MST algorithm computes the shortest total plough distance covering all priority streets.
- **F3 Sorting:** Pending segments are sorted and compared using three algorithms by priority level, snow depth and time since last plough.
- **F4 BST and AVL:** Each segment's past plough times are kept in an AVL tree; the query "priority street unploughed for 24 hours" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches the street name typed; on a typo, edit distance suggests the closest matching street name.
- **F6 Trie and disjoint sets:** Street names are auto-completed with a trie; union-find groups cleared areas that become mutually reachable as ploughing progresses.
- **F7 File organisation:** Completed plough records are stored in a sequential file, vehicle maintenance cards in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** The plough-records file keeps a B+ tree secondary index on segment code.
- **F9 Extendible hashing / external sort:** Millions of sensor records accumulated over a winter season do not fit in memory, so they are sorted by timestamp with an external merge sort.

Extension: Recalculating route priorities during the plough round using a live snowfall forecast.

021 — Sightseeing Tour Route Builder



Summary: A console application that selects points of interest — museums, parks, squares — within a single day's time budget and proposes a walking or transit route that maximizes the visitor's collected interest score. It works on a synthetic city of about 8 districts and 180 points of interest; sites that are closed or temporarily inaccessible are reflected in the plan immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each candidate tour route is a doubly linked list of points of interest in visit order; round trips that return to the starting point use a circular list.
- **V2 Sparse matrix:** In the time-slot × point-of-interest visitor-density table, only measured cells are stored.
- **V3 Stack and queue:** The tourist's manual edits to today's itinerary are undone with a stack, and visitor groups queuing to enter a ticketed site are simulated with a queue.
- **V4 Tree and heap:** Today's candidate points of interest are kept in a heap keyed by score per minute, and the planner always pops the best next stop within the remaining time budget.
- **V5 Graph and BFS/DFS:** Points of interest are nodes and walkable paths are edges; BFS finds the fewest-stops route between two points, and DFS finds every point still reachable in a district after a path closes.
- **V6 Search and hashing:** Point-of-interest name lookups go through a hash table of name-to-record mappings, and points sorted by opening time are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the shortest walking time between two stops, and topological sort orders points that require a gate visit before entry, such as buying a ticket before the exhibit.
- **F3 Sorting:** Daily tour options are sorted by total score, total walking distance and number of transfers using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Points of interest store their opening hours in an AVL tree, and the query "first point open after 14:00" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches the point-of-interest name the visitor typed, and edit distance suggests the closest matching name when there is a typo.
- **F6 Trie and disjoint sets:** A trie auto-completes point-of-interest names, and union-find groups the points that remain walkably reachable from each other once a path closes.
- **F7 File organisation:** Visit records are kept in a sequential file, and point-of-interest cards are kept in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The visit-history file keeps a B+ tree secondary index by date.
- **F9 Extendible hashing / external sort:** Millions of monthly GPS location records don't fit in memory, so they are sorted by timestamp with an external merge sort.

Extension: Recalculating the remaining route during the tour using live transit delay notices.

022 — 🚂 Freight Train Wagon Shunting Simulator



Summary: A shunting simulation that routes wagons in a classification yard to the correct sidings by destination and assembles the day's outbound trains. It works on a synthetic daily traffic of about 20 sidings and 400 wagons; faulty wagons and blocked sidings are reflected in the shunting plan immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each siding holds its wagons as a doubly linked list in coupling order, and loop tracks that return to the same switch use a circular list.
- **V2 Sparse matrix:** In the siding × hour-slot occupancy table, only the cells with a wagon present at that hour are stored.
- **V3 Stack and queue:** A dead-end siding behaves as a stack in its own right, since the last wagon pushed in is always the first one pulled out, and wagons waiting at the yard entry signal are held in a queue.
- **V4 Tree and heap:** Wagons carrying high-priority cargo are kept in a heap ordered by delivery urgency, and the shunting planner always pulls the most urgent wagon for the next move.
- **V5 Graph and BFS/DFS:** Switch points are nodes and track segments are edges; BFS finds the route with the fewest switch changes from a siding to a platform, and DFS finds every siding still reachable after a switch failure.
- **V6 Search and hashing:** Wagon-ID lookups go through a hash table of ID-to-record mappings, and wagons sorted by weight are searched with binary search for load balancing.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the most time-efficient wagon routing between two sidings weighted by shunting time, and cycle detection catches invalid closed routes created by a switch configuration.
- **F3 Sorting:** Wagons scheduled for the next outbound train are sorted by destination, weight and cargo priority using three different algorithms, and their build times are compared.
- **F4 BST and AVL:** Scheduled departures for each siding are stored in an AVL tree, and the query "first departure after 09:00" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches the cargo-type text in a manifest, and edit distance corrects a mistyped wagon code.
- **F6 Trie and disjoint sets:** A trie auto-completes station and destination codes, and union-find groups wagons that must travel together to the same destination into a single block.
- **F7 File organisation:** Shunting move records are kept in a sequential file, and wagon master records are kept in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The wagon movement-history file keeps a B+ tree secondary index by wagon ID.
- **F9 Extendible hashing / external sort:** Millions of yearly wagon movement records don't fit in memory, so they are sorted by timestamp with an external merge sort for the audit report.

Extension: Updating the shunting plan instantly against on-yard deviations using real-time wagon GPS positions.

023 — E-Scooter Battery Collection Planner



Summary: A planning application that tracks the battery levels of e-scooters scattered across a city and proposes the most efficient collection order for each van. It works on a synthetic fleet of about 15 zones and 600 scooters; scooters whose battery drops below the critical threshold are added to the plan immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each collection van's route is a doubly linked list of scooter stops in visit order, and vans that loop back to the depot use a circular list to close the route.
- **V2 Sparse matrix:** In the zone × hour-slot scooter-count table, only the cells with a scooter present at that hour are stored.
- **V3 Stack and queue:** The dispatcher's manual edits to a van's route are undone with a stack, and scooters waiting their turn at the depot for a battery swap are held in a queue.
- **V4 Tree and heap:** Scooters are kept in a heap ordered by battery percentage, and the dispatcher always pulls the lowest-battery scooter first.
- **V5 Graph and BFS/DFS:** Intersections are nodes and streets are edges; BFS finds the fewest-turn route to the nearest low-battery scooter, and DFS finds every scooter still reachable in a zone after a street closes.
- **V6 Search and hashing:** Scooter-ID lookups go through a hash table of ID-to-record mappings, and scooters sorted by battery percentage are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the shortest-time collection route covering several scooters, and Prim's algorithm finds the cheapest connection layout for a new charging-station network.
- **F3 Sorting:** Scooters are sorted by battery level, distance from the depot and revenue per ride using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Battery-swap appointments are stored in an AVL tree, and the query "first appointment after 10:00" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches fleet notes for the keyword "damaged", and edit distance matches a misread scooter QR code.
- **F6 Trie and disjoint sets:** A trie auto-completes zone names, and union-find groups the scooters that can be collected together in a single van run.
- **F7 File organisation:** Battery-swap records are kept in a sequential file, and scooter master records are kept in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The swap-history file keeps a B+ tree secondary index by scooter ID.
- **F9 Extendible hashing / external sort:** Millions of seasonal trip records don't fit in memory, so they are sorted by date with an external merge sort for the monthly billing report.

Extension: Reordering the collection sequence in advance with a battery-drain prediction model learned from historical usage data.

024 — 🚌 Bus Timetable Merger



Summary: A console application that merges weekday, weekend and holiday timetables prepared by different operators into one consistent schedule and reports conflicts and gaps. It works on a synthetic network of about 25 lines and 300 stops; when one line's timetable is updated, the merged schedule is recalculated.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each bus line's stop sequence is a doubly linked list of stops carrying scheduled departure offsets, and loop routes that return to their first stop are stored as a circular list.
- **V2 Sparse matrix:** In the stop × time-slot departure table, only the slots with a scheduled trip are stored.
- **V3 Stack and queue:** The scheduler's manual edits to the merged timetable are undone with a stack, and passengers boarding a stop in arrival order are simulated with a queue for a capacity check.
- **V4 Tree and heap:** Departures from the lines being merged are kept in a heap ordered by time, and the merger always pops the earliest unprocessed departure into the combined schedule.
- **V5 Graph and BFS/DFS:** Stops are nodes and direct line connections are edges; BFS finds the fewest-transfer route, and DFS finds every stop reachable from a hub within operating hours.
- **V6 Search and hashing:** Stop-name lookups go through a hash table of name-to-record mappings, and binary search over sorted departure times finds the next bus after a given time.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm treats the merged schedule as weighted edges to compute the fastest multi-line travel time between two stops, and topological sort orders timetable-update tasks that depend on each other, such as shift approval before publishing.
- **F3 Sorting:** Merged departures are sorted by time, line number and stop order using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Each stop's departures are stored in an AVL tree, and the query "first bus after 08:15" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches route description text, and edit distance corrects a stop name the passenger mistyped.
- **F6 Trie and disjoint sets:** A trie auto-completes stop names, and union-find groups stops into the same fare zone as zone boundaries merge.
- **F7 File organisation:** Daily departure records are kept in a sequential file, and stop master records are kept in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The timetable-change-history file keeps a B+ tree secondary index by effective date.
- **F9 Extendible hashing / external sort:** Millions of ridership records accumulated over years don't fit in memory, so they are sorted by stop and date with an external merge sort for the annual report.

Extension: Updating the merged schedule during service using delay information from buses' live GPS positions.

025 — 🚀 Satellite Ground Station Pass Scheduler



Summary: A scheduling application that plans which satellite pass each ground-station antenna tracks and when, resolving conflicts by priority. It works on a synthetic network of about 6 ground stations and 90 satellites; an antenna fault or an urgent-priority pass triggers immediate replanning.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each ground station's daily pass queue is a doubly linked list of passes ordered by start time, and antennas running a repeating calibration routine cycle through a circular list.
- **V2 Sparse matrix:** In the satellite × ground-station visibility table, only the pairs with an actual visibility window that day are stored.
- **V3 Stack and queue:** The scheduler's manual reassignments of passes to antennas are undone with a stack, and satellites waiting for a free antenna slot are held in a queue.
- **V4 Tree and heap:** Pending passes are kept in a heap ordered by mission priority, and the scheduler always pulls the highest-priority pass when an antenna slot conflicts.
- **V5 Graph and BFS/DFS:** Ground stations are nodes and data-relay links are edges; BFS finds the fewest-hop path to mission control, and DFS finds every station still reachable after one station fails.
- **V6 Search and hashing:** Satellite-NORAD-ID lookups go through a hash table of ID-to-record mappings, and binary search over sorted pass start times finds the next pass after a given time.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the minimum-latency relay path weighted by link delay, and cycle detection catches invalid circular relay configurations.
- **F3 Sorting:** Pending passes are sorted by elevation angle, priority and duration using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Each antenna's scheduled passes are stored in an AVL tree, and the query "first free slot after 14:00 UTC" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches mission log text for the keyword "anomaly", and edit distance corrects a mistyped satellite catalog name.
- **F6 Trie and disjoint sets:** A trie auto-completes satellite names, and union-find groups ground stations into the same relay-network partition as links go up or down.
- **F7 File organisation:** Pass telemetry records are kept in a sequential file, and satellite master records are kept in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The pass-history file keeps a B+ tree secondary index by satellite ID.
- **F9 Extendible hashing / external sort:** Millions of yearly telemetry records don't fit in memory, so they are sorted by timestamp with an external merge sort for the mission review report.

Extension: Automatically rescheduling upcoming passes using updated orbital element (TLE) data.

026–050 · Games and puzzles

026 — ★ Sudoku Solver and Hint Engine



Summary: A console application that solves 9x9 sudoku puzzles by backtracking and gives the player step-by-step hints during solving. It works on a synthetic library of about 500 puzzles; every move the player makes is reflected in the candidate lists immediately, and the hint engine suggests the most constrained cell.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each cell's list of remaining candidate digits is a doubly linked list that shrinks as constraints propagate and regrows on backtrack, and the hint engine cycles through the nine possible digits with a circular list.
- **V2 Sparse matrix:** In the cell × candidate-digit (1-9) table, only the digits still possible for that cell are stored.
- **V3 Stack and queue:** The solver's backtracking moves are pushed onto a stack so a dead end can be undone, and player hint requests are processed in order from a queue.
- **V4 Tree and heap:** Empty cells awaiting a hint are kept in a heap ordered by their remaining candidate count, and the engine always pops the most constrained cell to try first.
- **V5 Graph and BFS/DFS:** The 81 cells are nodes and cells sharing a row, column or box are edges; DFS runs the backtracking search over the graph, and BFS propagates a placed digit's eliminations to its directly connected neighbors.
- **V6 Search and hashing:** The puzzle library stores puzzle-ID-to-grid mappings in a hash table, and puzzles sorted by difficulty score are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection identifies contradictory constraint loops that make a puzzle unsolvable, and topological sort orders the chain of forced single-candidate placements during hint generation.
- **F3 Sorting:** The puzzle library is sorted by difficulty score, solve time and number of givens using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** The player's personal best times per difficulty level are stored in an AVL tree, and the query "best medium-level time after a given date" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches puzzle notes and tags for a keyword such as "diagonal variant", and edit distance corrects a mistyped tag.
- **F6 Trie and disjoint sets:** A trie auto-completes puzzle tags, and union-find groups merged cages into the same cell cluster for the killer-sudoku variant.
- **F7 File organisation:** Solve-attempt records are kept in a sequential file, and puzzle master records are kept in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The puzzle archive file keeps a B+ tree secondary index by difficulty score.
- **F9 Extendible hashing / external sort:** An archive of hundreds of thousands of generated puzzles doesn't fit in memory, so it is sorted by difficulty with an external merge sort for catalog export.

Extension: Generating new puzzles at a target difficulty with constraint-based generation and feeding them back into the hint engine.

027 — 1 Chess Move History and Opening Book



Summary: A console application that records played chess games move by move, supports undo/redo, compares them against known openings, and suggests the next book move. It works on a synthetic database of about 40 openings and 5,000 recorded games; once the player leaves the book, the suggestion stops immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The game's main line is a doubly linked list of moves supporting forward and back navigation, and side variations at a branch point are cycled through with a circular list during review.
- **V2 Sparse matrix:** In a position's square × piece-attack-count table, only the squares controlled by at least one piece are stored.
- **V3 Stack and queue:** Played moves are pushed onto an undo stack with a matching redo stack for step-by-step review, and pending engine move-suggestion requests are processed in order from a queue.
- **V4 Tree and heap:** Candidate replies from the opening book for the current position are kept in a heap ranked by win rate, and the engine pops the highest-scoring reply to suggest to the player.
- **V5 Graph and BFS/DFS:** Positions are nodes and legal moves are edges; BFS finds the shortest checkmate sequence from a position, and DFS scans the full game tree from a branch point for deep analysis.
- **V6 Search and hashing:** Position (FEN) lookups go through a hash table of position-to-book-entry mappings, and recorded games sorted by date are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection catches threefold repetition in the position graph, and Dijkstra's algorithm, weighted by move quality, computes the best move sequence to a target position.
- **F3 Sorting:** Opening-book replies are sorted by win rate, frequency played and average rating using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** The player's game records are stored in an AVL tree by date, and the query "games played after a given date" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches imported PGN text for a move sequence, and edit distance matches near-identical opening lines to catch transpositions or typos.
- **F6 Trie and disjoint sets:** A trie stores opening-book move sequences in SAN notation for fast prefix lookup, and union-find groups games into the same opening family as shared move prefixes merge.
- **F7 File organisation:** Move-by-move game logs are kept in a sequential file, and player master records are kept in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The game archive file keeps a B+ tree secondary index by player rating.
- **F9 Extendible hashing / external sort:** A master database of millions of recorded games doesn't fit in memory, so it is sorted by date with an external merge sort for tournament report generation.

Extension: Suggesting the best replies live during review using analysis from a connected chess engine.

028 — 🃏 Solitaire (Klondike) Card Game Engine



Summary: A console application that runs single-player Klondike solitaire under standard rules, validates each move, and offers hints through an optional auto-solver. It works on a synthetic archive of about 200 played deals; when an invalid move is attempted, the board reverts to its previous state immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each tableau pile is a doubly linked list of cards that lets a mid-pile run be spliced out and moved together, and the recycling stock/waste pile is a circular list that loops back to the start once exhausted.
- **V2 Sparse matrix:** In the tableau-column $\times$ row-position table, only the cells with a card present are stored, since column heights differ.
- **V3 Stack and queue:** Each completed move is pushed onto an undo stack so the most recent move can always be reversed, and queued auto-complete moves are played to the foundations one at a time from a queue.
- **V4 Tree and heap:** Legal next moves are kept in a heap ranked by how many face-down cards they would reveal, and the auto-solver always pops the move with the highest immediate payoff.
- **V5 Graph and BFS/DFS:** Board states are nodes and legal moves are edges; BFS finds the fewest-move solving sequence from the current state, and DFS follows a single move branch deep to check whether it is solvable.
- **V6 Search and hashing:** Seen board states are kept in a hash table so the auto-solver never revisits the same state, and saved games sorted by completion time are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection in the auto-solver's state search avoids infinite loops, and Dijkstra's algorithm, weighted by move cost, finds the minimum-move path to a fully solved state.
- **F3 Sorting:** Completed games are sorted by move count, elapsed time and score using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Player statistics per difficulty level (draw-one vs. draw-three) are stored in an AVL tree by date, and the query "best time after a given date" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches saved-game move logs for a keyword such as "foundation", and edit distance matches near-identical move notations when importing a replay with typos.
- **F6 Trie and disjoint sets:** A trie stores move-notation sequences for fast replay lookup and autocomplete, and union-find groups same-suit runs merged together as tableau sequences are built.
- **F7 File organisation:** Move logs are kept in a sequential file, and player statistics master records are kept in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The saved-game archive file keeps a B+ tree secondary index by completion date.
- **F9 Extendible hashing / external sort:** An archive of millions of played-game logs doesn't fit in memory, so it is sorted by score with an external merge sort for leaderboard generation.

Extension: Checking every newly dealt hand's solvability with the auto-solver before it is offered to the player.

029 — Minesweeper Flood-Reveal Engine



Summary: A minesweeper engine that flags hidden mines on a rectangular board, auto-reveals empty regions, and offers probability-based hints. It works on a synthetic expert-level board of about 16x30 cells with 99 mines; opening a cell chain-reveals its connected zero-adjacency region immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of currently flagged cells is a doubly linked list for quick insertion and removal as flags are toggled, and hint mode cycles through suggested safe cells with a circular list that wraps back to the first suggestion.
- **V2 Sparse matrix:** In the row × column table, only the state of cells that have been revealed or flagged so far is stored; most cells on a large board remain untouched.
- **V3 Stack and queue:** Every reveal and flag-toggle move is pushed onto a stack so the player can undo their last action, and hint requests are processed in order from a queue.
- **V4 Tree and heap:** Unrevealed cells eligible for a hint are kept in a heap keyed by computed mine probability, and the hint engine pops the safest cell to highlight for the player.
- **V5 Graph and BFS/DFS:** Cells are nodes and 8-neighbor adjacency is edges; BFS chain-reveals the connected empty region when a zero cell is opened, and DFS scans an isolated pocket to confirm it is fully surrounded before it is flagged.
- **V6 Search and hashing:** Cell-coordinate lookups go through a hash table of coordinate-to-state mappings, and saved game replays sorted by completion time are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm, treating flagged or uncertain cells as more costly to cross, finds the safest path from the revealed area to a target cell, and cycle detection catches constraint-dependency loops that require a guess.
- **F3 Sorting:** Leaderboard entries are sorted by completion time, board size and flags used, using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Personal best times per difficulty level are stored in an AVL tree by date, and the query "best time after a given date" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches replay-log text for a move-type keyword such as "flag", and edit distance matches near-identical player nicknames on the leaderboard.
- **F6 Trie and disjoint sets:** A trie enables fast leaderboard search by player-nickname prefix, and union-find groups revealed cells into connected empty regions as the flood fill expands.
- **F7 File organisation:** Move logs are kept in a sequential file, and leaderboard master records are kept in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The game archive file keeps a B+ tree secondary index by completion date.
- **F9 Extendible hashing / external sort:** An archive of millions of recorded games doesn't fit in memory, so it is sorted by completion time with an external merge sort for global leaderboard ranking.

Extension: A probability-based hint mode that highlights the statistically safest unrevealed cell using accumulated constraint data.

030 — 🧟 Maze Chase Game AI



Summary: A maze chase game in which the player collects pellets while evading ghosts that switch between behavior modes, each ghost chasing the player with its own pathfinding logic. It works on a synthetic maze of about 30x28 cells with 4 ghosts; eating a power pellet immediately switches the ghosts into flee mode.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each ghost's planned path to its target is a doubly linked list of cells, letting a new segment be spliced in when the path is recomputed, and its fixed patrol route when not chasing uses a circular list.
- **V2 Sparse matrix:** In the row × column table of pellet locations, only the cells that currently contain a pellet are stored.
- **V3 Stack and queue:** The player's queued direction presses are buffered in a queue so a turn executes at the next valid intersection, and a ghost retracing its path back to base after being eaten pops cells off a stack.
- **V4 Tree and heap:** Ghosts pending a target reassignment are kept in a heap keyed by distance to the player, and the nearest ghost is always popped first to receive the updated chase target.
- **V5 Graph and BFS/DFS:** Cells are nodes and open passages are edges; BFS computes each ghost's shortest path to the player's current cell every tick, and DFS confirms in the maze generator that every cell is reachable from the start.
- **V6 Search and hashing:** Maze-cell-coordinate lookups go through a hash table of coordinate-to-state (wall/pellet/power-pellet) mappings, and the saved high-score list is searched by score with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm carves the maze out of a grid as a random spanning tree, and Dijkstra's algorithm, with weighted edges for slow zones, computes a ghost's actual fastest chase path.
- **F3 Sorting:** The high-score table is sorted by score, level reached and survival time using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** High scores for each maze level are stored in an AVL tree, and the query "scores above a given threshold" is answered in the balanced tree.
- **F5 String algorithms:** KMP searches replay-log text for an event keyword such as "power pellet", and edit distance matches near-identical nicknames when merging duplicate leaderboard entries.
- **F6 Trie and disjoint sets:** A trie enables fast level-select search by maze level name or code, and union-find groups maze cells during generation and validation to prove the whole maze is one reachable component.
- **F7 File organisation:** Gameplay event logs are kept in a sequential file, and player profile master records are kept in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The replay archive file keeps a B+ tree secondary index by score.
- **F9 Extendible hashing / external sort:** An archive of millions of recorded play sessions doesn't fit in memory, so it is sorted by score with an external merge sort for global tournament ranking.

Extension: Adaptive difficulty that tunes ghost aggressiveness in real time based on the player's recent survival performance.



031 — ABC Word Grid (Boggle) Solver



Summary: A console application that scans a shared letter grid for every valid word that can be traced through adjacent cells and scores the results. It works on a synthetic 5x5 board against a dictionary of about 50,000 words; a new letter layout is generated each round, and found words are scored once time runs out.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The sequence of cells visited for the word currently being traced is kept as a doubly linked list; "undo" removes the last cell from the list.
- **V2 Sparse matrix:** On the 5x5 board, cells carrying a special letter multiplier (bonus tile) are stored in a sparse matrix; most cells carry no multiplier.
- **V3 Stack and queue:** The depth-first traversal that builds a candidate letter sequence is backtracked with a stack; found words are placed on a queue awaiting dictionary validation.
- **V4 Tree and heap:** Found words are kept in a heap ordered by score; at the end of a round, the top 5 highest-scoring words are pulled from the heap.
- **V5 Graph and BFS/DFS:** Board cells are nodes and the 8 neighboring cells are edges; DFS searches from a starting cell for the longest valid word paths.
- **V6 Search and hashing:** The dictionary's roughly 50,000 words are stored in a hash table so a candidate string's validity is checked in constant time; binary search over the sorted list of already-found words blocks duplicate entries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm finds the lowest-cost path from a starting cell to a target cell in a challenge mode where some cells add extra time cost; cycle detection rejects invalid grid links that create an infinite loop between two cells.
- **F3 Sorting:** At the end of a round, players are ranked by score with three different algorithms and the algorithms' running times are compared.
- **F4 BST and AVL:** Found words are kept in alphabetical order in an AVL tree; a "words starting with M" range query is answered from the balanced tree.
- **F5 String algorithms:** A word the player types by hand is scanned against dictionary entries with KMP; if no match is found, edit distance suggests the nearest valid word.
- **F6 Trie and disjoint sets:** During DFS, a trie checks prefixes so branches with an invalid prefix are pruned early; union-find groups connected clusters of bonus cells (colored regions) on the board.
- **F7 File organisation:** Round logs are stored in a sequential file, and player profiles are stored in a direct-access file keyed by player ID (collisions resolved with progressive overflow).
- **F8 B+ tree index:** The dictionary file keeps a B+ tree secondary index on word length, speeding up a "list all 7-letter words" query.
- **F9 Extendible hashing / external sort:** Millions of round records collected over months no longer fit in memory, so they are sorted by date with an external merge sort.

Extension: A timed multiplayer mode where the same board is shared and solved simultaneously.

032 — Board Game Turn and Dice Engine



Summary: An engine for 2-6 players who move around a synthetic property board on dice rolls, draw event cards, and buy and sell properties. The board has 40 squares, a synthetic property catalogue, and a 30-card event deck; a player's balance and holdings update at the end of every turn.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The board's 40 squares are kept as a circular linked list; on a dice roll, the player is advanced that many squares around the circular list.
- **V2 Sparse matrix:** In the player x square ownership table (player count x 40 squares), only owned squares are stored.
- **V3 Stack and queue:** Event cards a player draws in turn are placed on a queue awaiting resolution; chained effects such as "roll again" are held on a stack.
- **V4 Tree and heap:** Pending special effects (skip a turn, extra roll) are kept in a heap ordered by remaining duration; the effect with the least time left is applied first.
- **V5 Graph and BFS/DFS:** Shortcut links between squares (teleport tiles) are modeled as a graph; BFS computes how few dice rolls are needed to travel from one square to another.
- **V6 Search and hashing:** A player name maps to a balance record in a hash table; binary search over the property cards sorted by price computes rent.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the lowest-cost route between two squares in a mode where teleport tiles carry different costs; cycle detection finds closed debt chains among players (A owes B, B owes C, ...).
- **F3 Sorting:** At the end of the game, players are ranked by total net worth with three algorithms and their running times are compared.
- **F4 BST and AVL:** Property cards are kept in an AVL tree by price; a "properties priced 300-500" range query is answered from the balanced tree.
- **F5 String algorithms:** A property name the player types is searched with the Boyer-Moore algorithm; if nothing matches, edit distance suggests the nearest property name.
- **F6 Trie and disjoint sets:** Property names are auto-completed with a trie; union-find merges properties belonging to the same color set so the group bonus can be computed.
- **F7 File organisation:** Turn logs are stored in a sequential file, and player account summaries are stored in a direct-access file keyed by player ID (collisions resolved with linear quotient).
- **F8 B+ tree index:** The game history file keeps a B+ tree secondary index on player ID.
- **F9 Extendible hashing / external sort:** As dice-roll logs across thousands of games keep growing, they are stored with extendible hashing, doubling the directory as needed.

Extension: Server-side synchronized verification of dice rolls in an online multiplayer mode.

033 — ✕ Turn-Based Strategy Map Engine



Summary: A console engine in which two sides take turns moving units and launching attacks on a synthetic battlefield. The terrain is roughly 20x20 tiles and each player controls 8-10 units; terrain type (mountain, forest, water) changes movement and attack cost.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each unit's movement history (the tiles it has crossed) is kept as a linked list; a "rewind move" command removes the last move from the list.
- **V2 Sparse matrix:** In the map tile x unit ID influence table, only tiles within a unit's range are stored, since the terrain is large but a unit's range is small.
- **V3 Stack and queue:** A player's orders (move, attack) can be undone from a stack; the order in which units act is managed with a queue based on initiative value.
- **V4 Tree and heap:** The AI evaluates candidate moves in a decision tree; units' attack priority is kept in a heap so the lowest-health target is engaged first.
- **V5 Graph and BFS/DFS:** Map tiles are nodes and adjacency is edges; BFS finds every tile reachable within a unit's range, and DFS finds regions reachable without crossing a given terrain type.
- **V6 Search and hashing:** A unit ID maps to a unit record in a hash table; binary search over units sorted by power score selects targets.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm accounts for terrain-dependent movement cost to find the cheapest route between two units; the MST algorithm builds the AI's supply network connecting its base and resource points at the lowest total cost.
- **F3 Sorting:** At the end of a turn, units are ranked by power score with three algorithms and their running times are compared.
- **F4 BST and AVL:** Players' resources (gold/mana) are kept in an AVL tree; a "players with over 500 resources" range query is answered from the balanced tree.
- **F5 String algorithms:** A unit or ability name is searched with KMP; on a typo, edit distance suggests the nearest ability name.
- **F6 Trie and disjoint sets:** Ability names are auto-completed with a trie; union-find groups players/regions belonging to the same alliance so alliance territory can be computed.
- **F7 File organisation:** Turn logs are stored in a sequential file, and unit inventories are stored in a direct-access file keyed by unit ID (collisions resolved with Brent's method).
- **F8 B+ tree index:** The battle history file keeps a B+ tree secondary index on map coordinates.
- **F9 Extendible hashing / external sort:** Move logs from thousands of matches no longer fit in memory, so they are sorted by turn number with an external merge sort.

Extension: Automatically adjusting AI difficulty based on player performance.

034 — Falling Blocks Game Engine



Summary: An engine that drops seven-piece blocks in random order onto a standard 10x20 board, lets the player rotate and place them, and clears completed rows. Pieces are dealt using the "seven-bag" method; score is computed from cleared rows and fall speed.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The upcoming-piece queue (the seven-bag) is kept as a linked list; when the player holds a piece, it is removed from the list's head and placed in the hold slot.
- **V2 Sparse matrix:** On the 10x20 board, filled cells are stored in a sparse matrix, since almost every cell is empty at the start of a game.
- **V3 Stack and queue:** A piece's rotation attempts (wall kicks) are undone from a stack; upcoming pieces are held on a queue for the preview panel.
- **V4 Tree and heap:** An AI hint mode evaluates every possible placement of a piece in a decision tree; the placement that leaves the least empty space is picked from a heap.
- **V5 Graph and BFS/DFS:** Adjacencies among filled cells on the board are modeled as a graph; DFS detects whether the empty space above a column is sealed off by overhanging blocks (a buried hole).
- **V6 Search and hashing:** A tetromino shape maps to its rotation matrices in a hash table; binary search over the high-score table finds a new score's ranking position.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the order in which dependent special blocks resolve during a multi-row clear (combo); cycle detection verifies that transitions between tiles in the "portal" variant never form an infinite loop.
- **F3 Sorting:** The high-score table is sorted with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Players' top scores are kept in an AVL tree; an "scores above 500" range query is answered from the balanced tree.
- **F5 String algorithms:** Player nickname records are searched with the Boyer-Moore algorithm; on a typo, edit distance suggests the nearest username.
- **F6 Trie and disjoint sets:** Usernames are auto-completed with a trie; union-find groups adjacent same-colored blocks (in a special match-style mode) so they can be cleared together.
- **F7 File organisation:** Game session logs are stored in a sequential file, and player profiles are stored in a direct-access file keyed by player ID (collisions resolved with progressive overflow).
- **F8 B+ tree index:** The score history file keeps a B+ tree secondary index on the date field, speeding up a "last week's scores" query.
- **F9 Extendible hashing / external sort:** As the player base grows, the score records file is grown with extendible hashing, doubling the directory as needed.

Extension: Adding a garbage-line mechanic that sends cleared rows to an opponent in online multiplayer.

035 — 🐍 Snake Game with Replay



Summary: A console engine that steers a growing body around a 20x20 board to collect apples, recording every move frame by frame so a session can be replayed later. Synthetic obstacle cells sit on the board; when a game ends, all moves are written to a record file.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The body is kept as a doubly linked list; on every move a new head is prepended, and the tail is removed unless an apple was just eaten.
- **V2 Sparse matrix:** Obstacle/wall cells on the board are stored in a sparse matrix, since most cells are empty.
- **V3 Stack and queue:** Direction-key inputs are placed on a queue when the player presses more than one key within a frame; the replay viewer rewinds recorded moves with a stack.
- **V4 Tree and heap:** In AI mode, candidate paths are evaluated in a search tree; cells closest to the apple are kept in a heap.
- **V5 Graph and BFS/DFS:** Board cells are nodes and adjacency is edges; BFS finds the shortest path from the head to the apple, and DFS finds pockets sealed off by the body.
- **V6 Search and hashing:** Visited cells are kept in a hash table so a self-collision is checked in $O(1)$ time; binary search over the score table finds a new score's ranking position.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm finds the lowest-cost path from the head to the apple on a map containing special slow and fast tiles; cycle detection checks whether the body has sealed off a closed region around itself (a deadly trap).
- **F3 Sorting:** The replay list is sorted by score with three algorithms and their running times are compared.
- **F4 BST and AVL:** Saved replay files are kept in an AVL tree by score; a "replays scoring above 80" range query is answered from the tree.
- **F5 String algorithms:** Replay file names are searched with KMP; on a typo, edit distance suggests the nearest replay name.
- **F6 Trie and disjoint sets:** Player nicknames are auto-completed with a trie; union-find groups connected empty regions on the board (the areas the body can still reach).
- **F7 File organisation:** Move logs are stored in a sequential file, and replay metadata is stored in a direct-access file keyed by replay ID (collisions resolved with linear quotient).
- **F8 B+ tree index:** The replay file keeps a B+ tree secondary index on player name.
- **F9 Extendible hashing / external sort:** Move logs from thousands of replays no longer fit in memory, so they are sorted by timestamp with an external merge sort.

Extension: A viewer that lets saved replays be scrubbed forward and backward frame by frame.

036 — 🃏 Card Deck Shuffle and Deal Simulator



Summary: A simulator that shuffles a synthetic 52-card deck, deals it to 2-6 players, evaluates hand strength, and manages trades between players. The deck is reshuffled every hand; a player can offer to trade a card with another player.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The deck is a doubly linked list of cards; drawing a card removes it from the head of the list and adds it to a player's hand.
- **V2 Sparse matrix:** In the player x card-type holding table (player count x 52 cards), only cards currently in hand are stored.
- **V3 Stack and queue:** Discarded cards are kept on a stack; the deal order cycles through players with a queue.
- **V4 Tree and heap:** When evaluating hand strength, candidate combinations (flush, straight) are scanned in a decision tree; players' hand strengths are kept in a heap so the strongest hand can be found quickly.
- **V5 Graph and BFS/DFS:** In card-trade offers, players are nodes and offer relationships are edges; BFS finds how many steps it takes to build a trade chain from one player to another.
- **V6 Search and hashing:** A card code (e.g. "KS") maps to a card object in a hash table; binary search over a sorted hand-strength list finds a ranking position.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds closed trade chains (A to B to C back to A) where everyone comes out ahead; Dijkstra's algorithm computes the lowest-cost path for a card reaching a desired player in a network where each trade step carries a different cost.
- **F3 Sorting:** A player's cards in hand are sorted by value with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Hand results (points won) for the session are kept in an AVL tree; a "hands scoring above 150" range query is answered from the balanced tree.
- **F5 String algorithms:** Player names are searched with KMP; the common card pattern between two players' past hand sequences is aligned with LCS to measure play-style similarity.
- **F6 Trie and disjoint sets:** Player names are auto-completed with a trie; union-find groups players on the same team in partnership games (e.g. a 4-player partnership game) so team score can be computed.
- **F7 File organisation:** Shuffle logs are stored in a sequential file, and player statistics are stored in a direct-access file keyed by player ID (collisions resolved with Brent's method).
- **F8 B+ tree index:** The hand history file keeps a B+ tree secondary index on the date field, speeding up a "last month's hands" query.
- **F9 Extendible hashing / external sort:** As the player count grows, the hand records file is grown with extendible hashing, doubling the directory as needed.

Extension: A live analytics panel showing card-counting statistics during play.

037 — Tournament Bracket Manager



Summary: A management application that builds matchups for a synthetic 32- or 64-entrant competition, processes results, and updates the elimination tree. Entrants are placed by an initial seed rating; losers are eliminated at the end of each round and the bracket advances to the next stage.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each round's match list is kept as a linked list; when a match is postponed, its record is removed from the list and appended at the end.
- **V2 Sparse matrix:** In the entrant x entrant past-matchup table (64x64), only pairs that have already played each other are stored.
- **V3 Stack and queue:** Referee appeal requests are placed on a queue; corrections to a match result (undo) are kept on a stack.
- **V4 Tree and heap:** An elimination bracket is naturally a binary tree (leaves are first-round matches, the root is the final); pending matches are kept in a heap ordered by start time, and the next match is pulled from the heap.
- **V5 Graph and BFS/DFS:** Entrants are nodes and played matches are edges; BFS computes how many matches apart two entrants could meet in the tournament.
- **V6 Search and hashing:** An entrant ID maps to an entrant record in a hash table; binary search over entrants sorted by rating finds a seeding position.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected component analysis finds cyclic result groups during a round-robin stage (A beats B, B beats C, C beats A); topological sort determines the order in which the tournament moves from the group stage to the elimination stage.
- **F3 Sorting:** Entrants are ranked by their initial rating with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Entrants are kept in an AVL tree by rating; a "players rated above 1800" range query is answered from the tree.
- **F5 String algorithms:** An entrant name is searched with KMP; on a typo, edit distance suggests the nearest entrant name.
- **F6 Trie and disjoint sets:** Entrant names are auto-completed with a trie; union-find groups entrants from the same club so they can be kept apart in the first round.
- **F7 File organisation:** Match results are stored in a sequential file, and entrant profiles are stored in a direct-access file keyed by entrant ID (collisions resolved with progressive overflow).
- **F8 B+ tree index:** The tournament history file keeps a B+ tree secondary index on tournament date.
- **F9 Extendible hashing / external sort:** Match records from thousands of tournaments accumulated over years no longer fit in memory, so they are sorted with an external merge sort.

Extension: Automatically updating entrant ratings after each tournament with an Elo-style method.

038 — Game Leaderboard Engine



Summary: A ranking service that aggregates and ranks player scores across a multi-level synthetic game, offering seasonal and friend-based standings. It runs over thousands of synthetic player records; every score submission is screened for cheating.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A player's last 20 score records are kept as a doubly linked list; when a new score is added, the oldest record is removed from the list.
- **V2 Sparse matrix:** In the player x level best-score table (player count x level count), only completed levels are stored.
- **V3 Stack and queue:** Score submissions are placed on a queue awaiting cheat-screening; canceling an invalid score is undone from a stack.
- **V4 Tree and heap:** The top 10 scores per level are kept in a heap; when a new score arrives, the heap is updated and the current lowest score is evicted.
- **V5 Graph and BFS/DFS:** Friendship connections between players are modeled as a graph; BFS finds how many connections separate a player from the highest scorer in their friend circle.
- **V6 Search and hashing:** A player ID maps to a score record in a hash table; binary search over a sorted score list finds a player's rank.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** The MST algorithm builds a recommendation skeleton connecting a friend network at the lowest cost, using shared games played as the cost measure; Dijkstra's algorithm computes the lowest-cost connection from a player to the leaderboard's top scorer through the friendship chain.
- **F3 Sorting:** At the end of a season, all players are ranked by total score with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Player scores are kept in an AVL tree; a "players scoring 1000-2000" range query is answered from the balanced tree.
- **F5 String algorithms:** A player's username is searched with the Boyer-Moore algorithm; on a typo, edit distance suggests the nearest username.
- **F6 Trie and disjoint sets:** Usernames are auto-completed with a trie; union-find merges account groups flagged as cheat-suspects (accounts submitting scores from the same IP/device).
- **F7 File organisation:** Score submission logs are stored in a sequential file, and player profiles are stored in a direct-access file keyed by player ID (collisions resolved with linear quotient).
- **F8 B+ tree index:** The score history file keeps a B+ tree secondary index on season number, speeding up a "season 3 scores" query.
- **F9 Extendible hashing / external sort:** As the player base reaches the millions, the score submission file is grown with extendible hashing, doubling the directory as needed.

Extension: Computing separate regional (city/country) sub-leaderboards in real time.

039 — 🏰 Roguelike Dungeon Generator



Summary: A procedural map generator that lays out a fresh underground floor on every run, connects rooms with corridors, and places monsters and items. Each floor is a synthetic 50x50 grid; the player must reach the exit room before descending to the next floor.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A floor's rooms are kept as a linked list in generation order; when the player moves to the next floor, the previous floor's room list is freed from memory.
- **V2 Sparse matrix:** On the 50x50 floor grid, only room and corridor cells are stored in a sparse matrix; the rest is solid rock.
- **V3 Stack and queue:** The generation algorithm uses a stack while placing rooms with backtracking; monster spawn order is managed with a queue.
- **V4 Tree and heap:** Room placement is kept as a generation tree (a root room with branching corridors); monsters' threat level is kept in a heap so the strongest monster is placed in the room closest to the player first.
- **V5 Graph and BFS/DFS:** Rooms are nodes and corridors are edges; BFS finds the fewest-step path from the entrance room to the exit room, and DFS detects unreachable isolated rooms.
- **V6 Search and hashing:** A room coordinate maps to a room object in a hash table; binary search over items sorted by treasure value locates an item.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** The MST algorithm builds a floor skeleton connecting all rooms with the least total corridor length, after which a few extra links are added to create alternate routes; Dijkstra's algorithm lets monsters find the shortest corridor while chasing the player.
- **F3 Sorting:** Items found on a floor are sorted by value with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Monsters on each floor are kept in an AVL tree by threat level; a "monsters above level 5" range query is answered from the balanced tree.
- **F5 String algorithms:** An item or spell name the player types is searched with KMP; on a typo, edit distance suggests the nearest item name.
- **F6 Trie and disjoint sets:** Item and spell names are auto-completed with a trie; union-find merges room clusters as corridors connect them during generation to verify the floor is a single connected piece.
- **F7 File organisation:** Floor generation logs are stored in a sequential file, and player inventories are stored in a direct-access file keyed by player ID (collisions resolved with Brent's method).
- **F8 B+ tree index:** The dungeon history file keeps a B+ tree secondary index on floor number.
- **F9 Extendible hashing / external sort:** Map data from thousands of generated floors no longer fits in memory, so it is sorted by generation date with an external merge sort.

Extension: Dynamically scaling difficulty based on the player's death count on earlier floors.

040 — abc Hangman Word Guess Engine



Summary: A console engine that has a player guess a word letter by letter from a synthetic word pool, advancing a drawing step by step on each wrong guess. Words come from a pool of about 5,000 entries tagged by category and difficulty; the player is allowed 6 wrong guesses.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Letters the player has guessed are kept in order as a linked list; an "undo last guess" command removes the last node from the list.
- **V2 Sparse matrix:** In the word x letter guess table (word length x 29 letters), only letters the player has actually tried are stored.
- **V3 Stack and queue:** Wrong guesses are kept on a stack so the hangman drawing's steps can be rewound; in multiplayer mode, players' guessing order is managed with a queue.
- **V4 Tree and heap:** The hint system evaluates a word's possible letters against the remaining candidate word set in a decision tree; letters are kept in a heap ordered by usage frequency, so the most likely letter is suggested first.
- **V5 Graph and BFS/DFS:** Related links between word categories (animals, occupations) are modeled as a graph; BFS finds the shortest path from one category to another through related words.
- **V6 Search and hashing:** A word maps to its category and difficulty information in a hash table; binary search over words sorted by difficulty level speeds up random word selection.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** The MST algorithm builds a hint network connecting all categories at the lowest cost, using relationship strength between categories as the cost measure; Dijkstra's algorithm finds the lowest-cost relationship chain from one word to a target word in "word chain" hint mode.
- **F3 Sorting:** Players are ranked by correct-guess time with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The word pool is kept in an AVL tree by difficulty score; a "words with difficulty score 40-60" range query is answered from the balanced tree.
- **F5 String algorithms:** A player's full-word guess attempt is compared against the target word with KMP; on a wrong guess, edit distance shows the nearest valid word as a hint.
- **F6 Trie and disjoint sets:** The word pool is stored in a trie so words starting with a given letter can be listed quickly in hint mode; union-find merges related word groups (category clusters).
- **F7 File organisation:** Round logs are stored in a sequential file, and player statistics are stored in a direct-access file keyed by player ID (collisions resolved with progressive overflow).
- **F8 B+ tree index:** The word pool file keeps a B+ tree secondary index on the category field, speeding up an "animals category words" query.
- **F9 Extendible hashing / external sort:** Millions of round records accumulated over years no longer fit in memory, so they are sorted by player ID with an external merge sort.

Extension: Automatically adapting difficulty based on the player's performance in past sessions.

041 — Checkers Move Search Engine



Summary: A console-based checkers engine that plays as the computer opponent, searching for the best move on each turn and proposing it to the player. It runs on synthetic 8x8 games that start with 12 pieces per side, enforcing mandatory multi-jump captures and king promotion.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The move history is a doubly linked list of board snapshots that lets a player undo and redo moves; the chain of forced multi-jump captures for one turn is held in a circular list.
- **V2 Sparse matrix:** An 8x8 threat map counts, for every square, how many opposing pieces attack it; only squares actually under threat are stored.
- **V3 Stack and queue:** The search pushes each candidate in a forced-capture sequence onto a stack to backtrack when a branch turns out not to be the longest; hint moves offered to the player wait in a FIFO queue.
- **V4 Tree and heap:** A piece's next move candidates branch into a left-diagonal and right-diagonal binary tree, and a postorder traversal finds the capture chain that takes the most pieces; candidates are also ranked in a heap by evaluation score, so alpha-beta search tries the highest-scoring move first.
- **V5 Graph and BFS/DFS:** In the state graph where positions are nodes and legal moves are edges, DFS is the engine's core search algorithm; BFS computes the minimum number of moves for a man to reach the king row.
- **V6 Search and hashing:** Position-hash-to-evaluation pairs are stored in a hash table (a transposition table) with chaining, avoiding recomputation of positions seen before; the sorted opening-book entries are searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the lowest-risk-weighted path to victory in the state graph; DFS-based cycle detection catches a position repeating for the third time and declares the game a draw.
- **F3 Sorting:** 50,000 archived games are sorted by average decision time per move using insertion sort, quicksort and merge sort, and the three algorithms' running times are compared.
- **F4 BST and AVL:** Tournament players are kept in an AVL tree ordered by ELO rating; a "closest available opponent to 1500" query is answered on the balanced tree.
- **F5 String algorithms:** A specific opening sequence is searched for in recorded game notation (e.g. "e3-d4,f6-e5") with KMP; the similarity between two games' move sequences is measured with edit distance to suggest the closest past game.
- **F6 Trie and disjoint sets:** Saved opening names (e.g. "Edge Opening") are autocompleted with a trie; union-find merges pieces that defend each other along the same diagonal into shared defensive-block clusters.
- **F7 File organisation:** Finished games are stored in a sequential file; player profiles are kept in a direct-access file keyed by username, with collisions resolved by the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on the date field is kept over the game-record file; a "games played in the last month" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the player-profile file grows with new registrations, it is resized with extendible hashing to keep access performance stable.

Extension: Having the engine automatically update its evaluation weights from played games (simple reinforcement learning).

042 — 🔄 Sliding 15-Puzzle Solver



Summary: A console solver that finds the shortest sequence of slides to restore a shuffled 4x4 tile board to numeric order by moving the blank square. It uses A* and IDA* search and is tested against a synthetic archive of about 80 different scrambles.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A doubly linked list of board states stores the solution path so it can be replayed forward and backward one slide at a time; a circular list cycles through a bank of stored starting scrambles.
- **V2 Sparse matrix:** A pattern database keyed by the positions of tiles 1-4 stores precomputed heuristic distances for the solver; only combinations that are actually reachable are stored.
- **V3 Stack and queue:** The player's manual slides are pushed onto a stack so the last move can be undone; the solver's optimal move sequence is written to a queue and shown to the player one step at a time in FIFO order.
- **V4 Tree and heap:** Each state's possible moves branch into horizontal-slide and vertical-slide subtrees of a binary tree explored with a preorder traversal; the A* search frontier is kept in a heap ordered by total cost (g+h), expanding the lowest-cost state first.
- **V5 Graph and BFS/DFS:** In the state graph where board configurations are nodes and single slides are edges, BFS guarantees the minimum-move solution; depth-limited DFS (IDA*) searches larger scrambles without exceeding memory.
- **V6 Search and hashing:** Visited board states are marked in a hash table (open addressing) keyed by a hash of the tile arrangement, preventing duplicate expansion; the archive of scrambles, sorted by difficulty score, is searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** When moves are weighted by a per-tile difficulty penalty, Dijkstra finds the solution path with the lowest total difficulty; cycle detection prevents the search from re-expanding a board state it has already visited.
- **F3 Sorting:** 5,000 archived solved puzzles are sorted by move count using selection sort, quicksort and heapsort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Scramble records are kept in an AVL tree ordered by difficulty score; a "puzzle closest to difficulty 40" query is answered on the balanced tree.
- **F5 String algorithms:** The solution's move sequence is converted to a string of direction letters (U/D/L/R) and a given subpattern is searched for with KMP; the difference between a player's manual solution and the solver's optimal one is measured with edit distance.
- **F6 Trie and disjoint sets:** Saved scramble tags are autocompleted with a trie; union-find merges tiles that settle into their correct place into a shared "solved region" cluster to track progress.
- **F7 File organisation:** Solve records are stored in a sequential file; scramble definitions are kept in a direct-access file keyed by scramble ID, with collisions resolved by progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on move count is kept over the solve-record file; a "solutions under 30 moves" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** When millions of generated scrambles no longer fit in memory, they are sorted by difficulty score with an external merge sort.

Extension: Live comparison of the player's manual solving steps against the solver's optimal solution, with instant feedback.

043 — Tower Defence Wave Manager



Summary: The wave and economy manager for a console-based tower defense game in which a castle is defended against enemies arriving in waves by placing towers. It runs a 15-wave campaign on a synthetic 20x20 map, with enemy count and variety growing each wave.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The route enemies follow is a doubly linked list of waypoints; the targets within a tower's range are scanned with a circular list so targeting rotates round-robin among them.
- **V2 Sparse matrix:** On the 20x20 build grid, only cells that hold a tower (its type and level) are stored, and no memory is spent on empty cells.
- **V3 Stack and queue:** The player's tower placements and upgrades are pushed onto a stack so the last action can be undone; the enemies in a wave enter the path from a FIFO spawn queue.
- **V4 Tree and heap:** Each tower's upgrades branch into a damage path and a range path in an upgrade tree that tracks the player's choices; on-screen enemies are kept in a heap ranked by how far along the path they are, so towers target the furthest-advanced enemy first.
- **V5 Graph and BFS/DFS:** In the map graph where cells are nodes and adjacent open cells are edges, BFS verifies that a path from spawn to the castle still exists whenever the player places a new tower; DFS lists all possible detour routes.
- **V6 Search and hashing:** The tower catalogue maps tower name to stats (damage, range, cost) in a hash table with chaining; the sorted list of wave difficulty scores is searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** On the terrain-weighted cell graph, Dijkstra computes the fastest route enemies will take to the castle; topological sort on the wave dependency graph determines which prior waves must be cleared before a boss wave unlocks.
- **F3 Sorting:** 10,000 recorded matches are sorted by the wave reached using insertion sort, quicksort and merge sort, and the three running times are compared.
- **F4 BST and AVL:** The tower catalogue is kept in an AVL tree ordered by cost; a "tower closest to a 150 budget" query is answered on the balanced tree.
- **F5 String algorithms:** Wave spawn sequences are converted to strings of enemy-type letters (G, B, F...) and a given pattern is searched for with KMP; the similarity between two custom wave scripts is measured with edit distance.
- **F6 Trie and disjoint sets:** Tower and enemy names are autocompleted with a trie in the map editor; union-find merges adjacent open cells into shared "buildable region" clusters to quickly check the map isn't fully blocked.
- **F7 File organisation:** Match replay logs are stored in a sequential file; player profiles are kept in a direct-access file keyed by username, with collisions resolved by Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on score is kept over the match-record file; a "matches scoring over 5000" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the player-profile file grows with new registrations, it is resized with extendible hashing to keep lookup performance stable.

Extension: A map editor letting players draw and share their own enemy routes with the community.

044 — 🗂️ Memory Card Matching Game



Summary: A console memory-matching game with an AI opponent that builds its own memory model while the human player tries to recall symbols behind face-down cards to find their pairs. It is played on synthetic card themes from 4x4 up to 8x8 grids, logging the cards flipped and the time taken on every turn.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The pairs of cards a player flips are appended to a doubly linked move history that a replay can step through forward and backward; in multiplayer mode the next player is determined with a circular list.
- **V2 Sparse matrix:** A 16x16 co-occurrence table on a 4x4 card grid counts how often each pair of cells has been flipped together, and only pairs tried at least once are stored.
- **V3 Stack and queue:** The last two unmatched cards are held on a stack before being turned back face-down in LIFO order; in multiplayer mode the waiting players' names sit in a FIFO queue.
- **V4 Tree and heap:** The AI opponent branches its choice through a binary decision tree asking "is this card's match already remembered?"; remembered candidate pairs are kept in a heap ranked by confidence score, and the highest-scoring pair is tried first.
- **V5 Graph and BFS/DFS:** In the adjacency graph over the card grid, BFS reveals every cell within two steps of a flipped card when a "light" power-up is used; DFS flood-fills adjacent cards of the same theme (color/category) into a group.
- **V6 Search and hashing:** A card-symbol-to-grid-position mapping is kept in an open-addressing hash table so match checks run in constant time; the sorted list of best completion times is searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** When the game ends, Prim's algorithm builds a minimum spanning tree (MST) connecting the centres of all matched pairs for the victory animation; topological sort orders the unlock dependencies between power-ups.
- **F3 Sorting:** 8,000 completed game records are sorted by total move count using selection sort, quicksort and merge sort, and the running times are compared.
- **F4 BST and AVL:** Recorded games are kept in an AVL tree ordered by grid size (difficulty); a "record closest to a 6x6 grid" query is answered on the balanced tree.
- **F5 String algorithms:** A player's flip order is converted to a symbol string and a specific mistaken pattern is searched for with KMP; the similarity between two players' flip sequences is measured with edit distance.
- **F6 Trie and disjoint sets:** Card theme names (animals, fruits...) are autocompleted with a trie; union-find merges matched cards of the same theme into one cluster to compute a theme bonus.
- **F7 File organisation:** Game records are stored in a sequential file; player profiles are kept in a direct-access file keyed by username, with collisions resolved by progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on completion time is kept over the game-record file; a "games finished under 30 seconds" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the player-profile file grows with new registrations, it is resized with extendible hashing to keep lookup performance stable.

Extension: Automatically adjusting difficulty levels to match a card-reveal time that shortens over the session.

045 — ⑧ Billiards League Standings and Fixtures



Summary: A console application that generates the weekly fixtures of an amateur billiards league and keeps its standings table up to date. It builds a double round-robin fixture list for a synthetic 12-team league, and every reported match result immediately updates the standings and average.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The season fixture list is a doubly linked list of rounds that can be browsed forward and backward; round-robin pairings are generated by rotating the teams through a circular list each week.
- **V2 Sparse matrix:** In the team-by-team head-to-head results table, only the results of matches actually played are stored, leaving no cell for teams that haven't yet met.
- **V3 Stack and queue:** Corrections to wrongly entered match results are pushed onto a stack so the last correction can be undone; results submitted by referees wait in a FIFO approval queue before being posted to the standings.
- **V4 Tree and heap:** The playoff knockout stage is stored as a binary tree (bracket) where each match feeds two branches, and a postorder traversal prints the path to the championship; teams are kept in a heap ranked by points, and the playoff bracket is seeded starting from the highest-ranked team.
- **V5 Graph and BFS/DFS:** In the head-to-head graph where teams are nodes and a directed edge means "team A beat team B," BFS finds the shortest "beat chain" between two teams; DFS lists every team reachable through a chain of wins starting from the champion.
- **V6 Search and hashing:** A team-name-to-season-statistics mapping is kept in a hash table with chaining; a team's rank is found with binary search on the points-sorted standings list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection on the head-to-head graph finds rock-paper-scissors style win cycles among teams A, B and C to explain a points tie; when no cycle exists, topological sort attempts to rank the teams by their win relation.
- **F3 Sorting:** Match results from 15 seasons of archives are sorted by date using insertion sort, quicksort and merge sort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Players are kept in an AVL tree ordered by match average; an "opponent closest to a 65 average" query is answered on the balanced tree.
- **F5 String algorithms:** A specific shot term (e.g. "masse") is searched for in match note text with KMP; a misspelled team name is matched to the closest registered team with edit distance.
- **F6 Trie and disjoint sets:** Team and player names are autocompleted with a trie in the search box; union-find merges teams that have played each other at least once into a shared "connected fixture" cluster.
- **F7 File organisation:** Match history is stored in a sequential file; team records are kept in a direct-access file keyed by team ID, with collisions resolved by the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on the date field is kept over the match-record file; a "this month's fixtures" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** When a multi-season match archive no longer fits in memory, it is sorted by date with an external merge sort.

Extension: An end-of-season simulation that computes championship odds from the remaining fixtures.

046 — Crossword Puzzle Generator



Summary: A console tool that selects words from a given dictionary and places them in a grid so they intersect, generating a solvable crossword puzzle. It fills a synthetic 25x25 grid from a 40,000-word dictionary and computes a difficulty score from the intersection count and the ratio of empty cells.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of words still to be placed is a doubly linked list that can be traversed and reordered while backtracking when a word won't fit; candidates of the same length are cycled through with a circular list.
- **V2 Sparse matrix:** On a 25x25 puzzle grid, only cells that hold a placed letter are stored, and no memory is spent on empty/black squares.
- **V3 Stack and queue:** When a word doesn't fit, the backtracking algorithm pops and removes the most recently placed words from a stack; clues are presented to the solver in a FIFO queue following reading order.
- **V4 Tree and heap:** Each word's placement attempt branches into horizontal and vertical options in a binary tree explored with a preorder traversal; words are kept in a heap ranked by their number of possible intersections, and the most-intersecting word is placed first.
- **V5 Graph and BFS/DFS:** In the grid graph where filled cells are nodes and adjacency is an edge, BFS confirms that all words merge into a single connected region; DFS detects disconnected (isolated) word islands and flags the generator.
- **V6 Search and hashing:** The word bank maps word length to a list of candidate words in a hash table with chaining; a specific word is found with binary search in the alphabetically sorted dictionary.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort on the word-intersection graph produces a valid placement order specifying which word must be placed before which; cycle detection finds tightly interlocked clusters of words and factors them into the puzzle's difficulty score.
- **F3 Sorting:** A 40,000-word dictionary is sorted by word length using insertion sort, quicksort and merge sort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Dictionary words are kept in an AVL tree ordered by usage frequency; a "word closest to frequency rank 500" query is answered on the balanced tree.
- **F5 String algorithms:** Candidates matching the known letters of an empty grid slot are found in the dictionary with a KMP-based pattern search; when no exact match exists, the closest word is suggested with edit distance.
- **F6 Trie and disjoint sets:** The dictionary is kept in a trie, so candidates matching a known pattern such as "_A_LE" in the grid are found quickly; union-find groups words belonging to the same theme (e.g. animals) to support themed puzzle generation.
- **F7 File organisation:** Generated puzzles are archived in a sequential file; dictionary words are kept in a direct-access file keyed by the word itself, with collisions resolved by Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on difficulty score is kept over the puzzle-archive file; a "medium-difficulty puzzles" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the dictionary file grows with user contributions, it is resized with extendible hashing to keep lookup performance stable.

Extension: A themed puzzle-sharing mode where users add their own clue-word pairs.

047 — Domino Chain Game



Summary: The console engine for a classic domino game in which two players try to empty their hand by attaching matching tiles to either end of the chain on the table. It is played with a synthetic double-six set of 28 tiles, and a player who cannot move draws from the boneyard.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The tile chain on the table is itself a doubly linked list that can be extended at either end (head and tail); the players' turn order is determined by a circular list.
- **V2 Sparse matrix:** In a double-six tile set's pip-by-pip combination table, only the triangular half where $a \leq b$ is filled, leaving no cell for symmetric or invalid combinations.
- **V3 Stack and queue:** The last placed tile is kept on a stack so a wrong move can be undone; a player who cannot move draws from the boneyard, a FIFO queue of shuffled tiles.
- **V4 Tree and heap:** For every tile in hand, the AI player branches its "attach to the left end" / "attach to the right end" options into a binary tree searched with a preorder traversal for the best move; candidate moves are kept in a heap ranked by a blocking score, and the highest-scoring one is tried first.
- **V5 Graph and BFS/DFS:** In variants where doubles (e.g. 6-6) allow the chain to branch, the layout is kept as a graph whose nodes are tile ends; BFS finds the nearest open end from the hub, and DFS finds the longest branch.
- **V6 Search and hashing:** The tiles in hand are kept in a hash table keyed by pip pair (a,b), answering an "do I have a 4-5?" query in constant time; a turn's sorted score list is searched with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** On the graph of open ends weighted by pip totals, Dijkstra suggests which end lets a player empty their hand fastest; cycle detection prevents a tile from closing the chain back onto itself into an invalid loop.
- **F3 Sorting:** 20,000 recorded rounds are sorted by final score using selection sort, quicksort and merge sort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Players are kept in an AVL tree ordered by average round score; an "opponent closest to a score of 12" query is answered on the balanced tree.
- **F5 String algorithms:** A round's tile-play order is converted to a string of pip pairs and a frequently used opening pattern is searched for with KMP; the similarity between two players' play sequences is measured with edit distance.
- **F6 Trie and disjoint sets:** Tournament and player names are autocompleted with a trie in the search box; union-find merges tiles sharing a pip value (e.g. all the "sixes") into one group for statistics.
- **F7 File organisation:** Round history is stored in a sequential file; player profiles are kept in a direct-access file keyed by username, with collisions resolved by the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on the date field is kept over the round-record file; a "this week's rounds" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the player-profile file grows with new registrations, it is resized with extendible hashing to keep lookup performance stable.

Extension: A Mexican Train variant where several players can extend their own branch of the chain at the same time.

048 — Grid Robot Pathfinding Contest



Summary: A console contest system in which different pathfinding algorithms race across the same obstacle grid and have their performance compared. A*, Dijkstra-based and BFS-based robots are tested on synthetic 50x50 arenas, and each run's step count, time and area covered are logged.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A robot's found path is stored as a doubly linked list of grid cells that can be replayed forward and backward step by step; competing robots are tested in turn through a circular list.
- **V2 Sparse matrix:** On a 50x50 contest arena, only cells that hold an obstacle are stored, and no memory is spent on open cells.
- **V3 Stack and queue:** Obstacles placed by the arena designer are pushed onto a stack so the last one can be undone; the competing robot algorithms wait in a FIFO queue that determines their run order.
- **V4 Tree and heap:** Competing algorithms are eliminated in a single-elimination binary tree (bracket) where each match compares two rivals, and a postorder traversal finds the champion algorithm; the A* open list is kept in a heap ranked by total cost ($g+h$), expanding the lowest-cost cell first.
- **V5 Graph and BFS/DFS:** In the grid graph where cells are nodes and unobstructed adjacencies are edges, BFS finds the baseline fewest-steps path that sets the contest's base score; before a run starts, DFS checks whether the goal is reachable and computes the total area a robot could explore.
- **V6 Search and hashing:** Cells visited during search are marked in an open-addressing hash table keyed by coordinates to prevent revisiting; a robot's rank is found with binary search on the sorted contest leaderboard.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** On the weighted graph where some cells (e.g. mud) cost more to cross, Dijkstra finds the robot's true lowest-cost route used as the scoring reference; in multi-checkpoint contest variants, Prim's algorithm computes a minimum spanning tree connecting the checkpoints as a lower-bound reference score.
- **F3 Sorting:** 12,000 contest runs are sorted by step count to completion using insertion sort, quicksort and heapsort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Contest arena maps are kept in an AVL tree ordered by obstacle density; a "map closest to 30% density" query is answered on the balanced tree.
- **F5 String algorithms:** A robot's move sequence is converted to a string of direction letters (U/D/L/R) and a specific inefficient loop pattern is searched for with KMP; the difference between a robot's actual path and the optimal path is measured with edit distance to compute an efficiency score.
- **F6 Trie and disjoint sets:** Registered robot algorithm names are autocompleted with a trie; union-find merges adjacent open cells into shared clusters to verify the arena isn't split into unreachable islands before a contest run.
- **F7 File organisation:** Contest run logs are stored in a sequential file; arena map definitions are kept in a direct-access file keyed by map ID, with collisions resolved by progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on score is kept over the run-record file; a "best 100 runs" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the number of generated maps grows, the map file is resized with extendible hashing to keep lookup performance stable.

Extension: A plugin interface letting participants upload their own algorithms and have them auto-benchmarked on the same map set.

049 — 🎵 Rhythm Game Note Scheduler



Summary: A console-based rhythm game engine that schedules a song's note chart to millisecond precision and judges the player's hits against it. It works on synthetic 4-lane songs whose densest charts hold close to 3,000 notes, scoring every hit by its timing offset.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A song's note chart is kept as a doubly linked list ordered by timestamp, giving the editor note insertion/removal and forward/backward seeking through the playback position; in four-lane mode, lane order is cycled through a circular list.
- **V2 Sparse matrix:** On a 4-lane by 1/16-beat-subdivision table for a song, only cells where a note is actually placed are stored, and no memory is spent on empty time slots.
- **V3 Stack and queue:** Note placements in the chart editor are pushed onto a stack so the last note can be undone; during playback, upcoming notes are judged from a FIFO queue in the order they arrive at the hit line.
- **V4 Tree and heap:** A hit's timing offset is classified in a binary decision tree that branches on Perfect/Good/Miss thresholds; upcoming notes are kept in a min-heap ordered by exact hit timestamp, and the scheduler processes the nearest one first.
- **V5 Graph and BFS/DFS:** In a branching graph where song sections (intro, easy bridge, hard chorus...) are nodes and transitions based on player performance are edges, BFS finds the easiest completion path and DFS enumerates every possible playthrough variant for chart testing.
- **V6 Search and hashing:** A lane-and-timestamp-to-note mapping is kept in a hash table with chaining, found in constant time at the moment of a hit; the note nearest the current playback time is found with binary search on the timestamp-sorted note list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort on the section-transition graph guarantees a valid order specifying which section must be played before which; difficulty-weighted Dijkstra finds the easiest full playthrough path for a beginner-assist mode.
- **F3 Sorting:** A 3,000-note expert chart is re-sorted by timestamp after editing using insertion sort, quicksort and merge sort, and the algorithms' running times are compared.
- **F4 BST and AVL:** The song library is kept in an AVL tree ordered by BPM (tempo); a "song closest to 140 BPM" query is answered on the balanced tree.
- **F5 String algorithms:** A chart's lane sequence is converted to a number string and a recurring pattern (e.g. a jumpstream) is tagged with KMP; the similarity between two user-submitted charts is measured with edit distance to detect duplicate charts.
- **F6 Trie and disjoint sets:** Song and artist names are autocompleted with a trie on the song-select screen; union-find merges notes that must be struck simultaneously (a chord) into one cluster judged as a single hit.
- **F7 File organisation:** Note charts are stored in a sequential file ordered by time; song metadata is kept in a direct-access file keyed by song ID, with collisions resolved by the linear quotient method.
- **F8 B+ tree index:** A B+ tree secondary index on the song field is kept over the score-record file; a "this song's top 50 scores" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the song library file grows with community contributions, it is resized with extendible hashing to keep lookup performance stable.

Extension: Latency compensation that auto-calibrates note timing to microsecond precision from the player's past performance.

050 — 🏁 Racing Game Lap Times and Replay



Summary: A console replay system for a racing game that records lap times and plays back the previous best lap as a "ghost" car. It operates on an archive of thousands of recorded laps on synthetic tracks, storing each frame's car position, speed and inputs.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A replay's per-frame car position is kept as a doubly linked list, letting a viewer scrub forward and backward second by second; the track's checkpoints are kept as a circular list that loops back to the start.
- **V2 Sparse matrix:** In a track-segment by lap table, only cells where a collision or off-track excursion occurred are stored, leaving no cell for incident-free segment-lap pairs.
- **V3 Stack and queue:** Pieces added in the track editor are pushed onto a stack so the last piece can be undone; cars at the start line are sent off in FIFO order from a queue following the starting grid.
- **V4 Tree and heap:** At every corner, the AI driver branches its "inside line" / "outside line" options into a binary tree explored with a preorder traversal for the best route; cars in the race are kept in a heap ranked by distance covered, from which the live standings are updated.
- **V5 Graph and BFS/DFS:** On a track with alternate shortcuts, segments are nodes and connections are edges in a graph where BFS finds the route with the fewest segments; DFS scans every possible route variant in the track editor to confirm there are no dead ends.
- **V6 Search and hashing:** Best lap-time records keyed by (track, car) are kept in an open-addressing hash table; a specific time's rank is found with binary search on a track's sorted lap-time list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** On the track graph weighted by segment length, Dijkstra computes the fastest theoretical route among the shortcuts; cycle detection confirms that the pieces assembled in the editor form a single circuit closing back to the start/finish line.
- **F3 Sorting:** 25,000 recorded lap times are sorted by time using insertion sort, quicksort and merge sort, and the algorithms' running times are compared.
- **F4 BST and AVL:** Car setups are kept in an AVL tree ordered by best lap time; a "setup closest to 1:32.500" query is answered on the balanced tree.
- **F5 String algorithms:** A lap's steering/throttle inputs are converted to an event string and a frequently repeated braking mistake is detected with KMP; the difference between a player's lap and the ghost (best) lap is measured with edit distance to produce coaching feedback.
- **F6 Trie and disjoint sets:** Track and driver names are autocompleted with a trie in the search box; union-find merges adjacent track segments into the same timing sector (sector 1/2/3) cluster as they are defined in the track editor.
- **F7 File organisation:** Replay frames are stored in a sequential file ordered by time; lap-time records are kept in a direct-access file keyed by (track, car), with collisions resolved by progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on time is kept over the lap-record file; a "best 20 laps" query is answered quickly through this index.
- **F9 Extendible hashing / external sort:** As the number of laps played grows the record file, it is resized with extendible hashing to keep lookup performance stable.

Extension: Letting players upload their own ghost lap to the community for side-by-side comparison with others.

051–075 · Text, language and search

051 — Personal Document Search Engine

Summary: A console application that indexes a user's notes, course summaries and PDF-derived text into one collection and finds them instantly by keyword. It works over a synthetic collection of about 4,000 documents and 60,000 distinct terms, remembering recent searches and folder navigation.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of documents containing a given term (its postings) is kept as a doubly linked list; back-and-forward navigation through search history uses an XOR-linked list to save memory.
- **V2 Sparse matrix:** In the term × document matrix, only the cells where a term actually occurs (its term frequency) are stored, the rest stay empty.
- **V3 Stack and queue:** A stack undoes the sequence of search filters the user has applied; documents not yet indexed wait in a crawl queue.
- **V4 Tree and heap:** Category folders are kept in a binary tree whose in-order traversal lists documents alphabetically; a heap keeps the top-K most relevant results ranked by score.
- **V5 Graph and BFS/DFS:** Nodes are documents and edges are shared-term similarity; BFS finds documents within N hops of similarity to a seed document, DFS finds the connected cluster of related documents in a folder.
- **V6 Search and hashing:** A hash table maps each term to its posting list; binary search over a sorted array of document IDs intersects two terms' postings quickly.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the closest similarity path between two documents, and topological sort determines a valid indexing order for documents that reference one another.
- **F3 Sorting:** Search results are sorted by relevance score, modification date and file size using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Document records are kept in an AVL tree keyed by last-modified date; the query "documents modified between two dates" is answered on the balanced tree.
- **F5 String algorithms:** Boyer-Moore searches the document text for an exact phrase the user typed; on a typo, edit distance powers a "did you mean" suggestion.
- **F6 Trie and disjoint sets:** Terms typed into the search box are autocompleted with a trie; union-find groups near-duplicate documents into the same cluster.
- **F7 File organisation:** Document metadata is kept in a sequential log file, term-index entries in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** The document collection file keeps a secondary B+ tree index keyed by author name.
- **F9 Extendible hashing / external sort:** The inverted index, with millions of term-document entries, is too large for memory, so it is sorted by term with an external merge sort.

Extension: Adding a suggestion layer that auto-extracts tags or categories from document content.

052 — Spell Checker and Suggestion Engine



Summary: A console application that checks a student's typed essay text against a synthetic 50,000-word dictionary, flags misspelled words and offers corrections. Each session scans a document of a few thousand words, and the suggestions the user accepts or rejects are tracked through the session.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The words of the document being checked are kept as a doubly linked list for cursor movement; recently corrected words are kept in a circular list.
- **V2 Sparse matrix:** In the word $\times$ error-type matrix (omission, insertion, transposition, substitution), only the errors actually encountered are stored.
- **V3 Stack and queue:** Accepted corrections can be undone with a stack; flagged errors are presented to the user one at a time from a review queue.
- **V4 Tree and heap:** Document paragraphs are kept in a binary tree whose in-order traversal produces the corrected text; each error's top five suggestions are ranked in a heap by score.
- **V5 Graph and BFS/DFS:** Nodes are words, and edges connect word pairs one edit apart; BFS finds all suggestion candidates within k edits of a misspelled word, DFS finds a whole cluster of commonly confused words.
- **V6 Search and hashing:** A hash table maps each dictionary word to its validity/frequency; binary search over a sorted array of common words gives a fast fallback check.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the cheapest correction path using letter-substitution costs, and strongly connected components reveal clusters of frequently confused homophones.
- **F3 Sorting:** The suggestion list is sorted by edit distance, usage frequency and alphabetical order using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Dictionary words are kept alphabetically in an AVL tree; the query "first word after X" for prefix-based suggestions is answered on the balanced tree.
- **F5 String algorithms:** Every occurrence of a flagged word in the document is located with Boyer-Moore; the correction engine itself is built on edit distance/LCS.
- **F6 Trie and disjoint sets:** The dictionary is stored in a trie for fast prefix lookups; union-find groups words that share the same misspelling pattern.
- **F7 File organisation:** The document revision log is kept in a sequential file, dictionary entries in a direct-access file hashed by word (collision resolution with linear quotient).
- **F8 B+ tree index:** The error log file keeps a secondary B+ tree index keyed by session ID.
- **F9 Extendible hashing / external sort:** The user's personal dictionary file, which grows as new words are added, is managed with extendible hashing.

Extension: Going beyond spelling to check grammar rules such as subject-verb agreement.

053 — Smart Keyboard Autocomplete



Summary: A mobile-keyboard-style console application that predicts a typer's next word and completes it with a single keystroke. It runs on word-transition statistics learned from a synthetic corpus of about 20,000 sentences of everyday writing.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The words of the sentence currently being typed are kept as a doubly linked list; the three-suggestion carousel cycles through a circular list.
- **V2 Sparse matrix:** In the current-word $\times$ next-word (bigram) matrix, only word pairs that were actually observed are stored, so most of the matrix is empty.
- **V3 Stack and queue:** Accepted autocompletions can be undone with a stack; keystrokes are buffered in a queue for asynchronous suggestion computation.
- **V4 Tree and heap:** The user's past suggestion history is kept in a binary tree traversed in chronological order; next-word candidates are narrowed to the top K in a heap by frequency score.
- **V5 Graph and BFS/DFS:** Nodes are words and edges are observed "A followed by B" transitions; BFS finds all words reachable within N steps of a word, DFS traces a specific phrase's full chain.
- **V6 Search and hashing:** A hash table maps each word to its list of successors and counts; binary search over a sorted vocabulary array checks a word's validity.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by inverse frequency, finds the most likely word sequence, and cycle detection keeps the suggestion chain from looping forever.
- **F3 Sorting:** Candidate words are sorted by frequency, recency and length using three algorithms, and their running times are compared.
- **F4 BST and AVL:** The vocabulary is kept alphabetically in an AVL tree, supporting range queries such as all words starting with a given prefix.
- **F5 String algorithms:** KMP detects repeated phrase patterns in the user's typing history; edit distance corrects a mistyped partial word before it is completed.
- **F6 Trie and disjoint sets:** Prefix-based word completion, the core feature, is built on a trie; union-find groups words into synonym/style clusters for personalised suggestions.
- **F7 File organisation:** Every typing session is logged to a sequential file; word-frequency records are kept in a direct-access file hashed by word (collision resolution with Brent's method).
- **F8 B+ tree index:** The typing-history file keeps a secondary B+ tree index keyed by timestamp, for queries like "words typed in the last week."
- **F9 Extendible hashing / external sort:** A keystroke log of millions of entries is too large for memory, so it is sorted by word with an external merge sort to aggregate frequencies.

Extension: Improving suggestion quality with a personalised language model that learns the user's own writing style.

054 — Text Diff Tool



Summary: A console application that compares two text files, such as an old and new version of a report, line by line and reports insertions, deletions and changes. It works on synthetic file pairs of roughly 5,000 lines each.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each file version's lines are kept as a doubly linked list; the detected diff hunks are browsed through a circular list.
- **V2 Sparse matrix:** In the line $\times$ token matrix, only the words a line actually contains are stored, leaving most of the vocabulary's columns empty; this helps spot near-identical lines quickly.
- **V3 Stack and queue:** A stack lets the user step back and forward through diff hunks; large files are buffered in a queue for line-by-line comparison.
- **V4 Tree and heap:** The diff is broken into hunks kept in a binary tree whose in-order traversal produces the display order; the largest changes are shown first from a heap.
- **V5 Graph and BFS/DFS:** Nodes are lines from the two versions and edges are matching alignments between them; BFS finds the shortest edit path, DFS finds groups of files sharing a common ancestor version.
- **V6 Search and hashing:** A hash table maps a line's content hash to its line numbers to spot moved lines; binary search over a sorted array of line hashes quickly detects unchanged blocks.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the cheapest edit script using weighted edit operations, and topological sort orders a file's multiple saved revisions into a valid history chain.
- **F3 Sorting:** Diff hunks are sorted by size, line number and change type using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Unique lines are kept in an AVL tree keyed by content hash, so "does this line occur elsewhere" is answered quickly on the balanced tree.
- **F5 String algorithms:** A changed line's character-level diff is computed with edit distance/LCS; a specific string's history across all older versions is searched with KMP.
- **F6 Trie and disjoint sets:** Lines' common prefixes (indentation/structure) are grouped quickly with a trie; union-find groups duplicated or moved blocks across the file.
- **F7 File organisation:** The ordered comparison history is kept in a sequential file, line-hash records in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** The version-history file keeps a secondary B+ tree index keyed by save timestamp.
- **F9 Extendible hashing / external sort:** Two huge log files that are too large for memory are sorted with an external merge sort before being diffed.

Extension: Adding a three-way merge that combines two divergent edits made to the same original file.

055 — Dictionary and Thesaurus Engine



Summary: A console application over a synthetic 30,000-headword dictionary/thesaurus where a user looks up a word's meaning, synonyms and antonyms. The user can jump from one word to a related one and keep browsing.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A headword's senses are kept as a doubly linked list for browsing back and forward; the "word of the day" rotation cycles through a circular list.
- **V2 Sparse matrix:** In the word × word relation matrix, only pairs that are actually synonyms or antonyms are stored, along with the relation type.
- **V3 Stack and queue:** A chain of "jump to synonym" lookups can be backtracked with a stack; newly imported headwords wait in a queue to be validated.
- **V4 Tree and heap:** A word's sense hierarchy, from broad category to narrow meaning, is kept in a binary tree; synonym suggestions are ranked in a heap by similarity score.
- **V5 Graph and BFS/DFS:** Nodes are words and edges are synonym relations; BFS finds all synonyms within N degrees of a word, DFS finds a whole connected region of a semantic field.
- **V6 Search and hashing:** A hash table maps each headword to its definition record; binary search over a sorted array of headwords supports prefix-range lookups.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the "semantic distance" between two words along a synonym chain, and strongly connected components reveal tightly interlinked synonym clusters.
- **F3 Sorting:** Search results are sorted by usage frequency, alphabetical order and definition length using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Headwords are kept alphabetically in an AVL tree, so the "next word after X" query for flipping through the dictionary is answered on the balanced tree.
- **F5 String algorithms:** KMP searches definition text for a keyword; edit distance suggests the closest headword when a lookup is misspelled.
- **F6 Trie and disjoint sets:** Headwords are autocompleted with a trie; union-find groups words with the same meaning into one synonym class.
- **F7 File organisation:** Entries are kept in alphabetical order in a sequential file, headword records in a direct-access file hashed by word (collision resolution with linear quotient).
- **F8 B+ tree index:** The entries file keeps a secondary B+ tree index keyed by part of speech or usage-frequency rank.
- **F9 Extendible hashing / external sort:** The user-contributed word file, which keeps growing over time, is managed with extendible hashing.

Extension: Adding a corpus of real example sentences showing each word actually being used.

056 — Text Editor with Undo/Redo



Summary: A simple console text editor for note-taking, supporting undo/redo and find-and-replace on documents that can grow to a few thousand lines. Several document tabs can be open at once, each with its own edit history.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The lines that make up the document are kept as a doubly linked list; open document tabs are cycled through with Ctrl+Tab via a circular list.
- **V2 Sparse matrix:** In the line × formatting-attribute matrix (bold, italic, underline), only the character ranges that are actually formatted are stored.
- **V3 Stack and queue:** Undo and redo operations are each kept on their own stack; autosave snapshots wait in a queue to be written to disk.
- **V4 Tree and heap:** The document's heading/section structure is kept in a binary tree whose in-order traversal produces the table of contents; background tasks such as autosaving are scheduled in a heap by priority.
- **V5 Graph and BFS/DFS:** Nodes are document sections or bookmarks, and edges are cross-reference links between them; BFS finds sections within N links of the current position, DFS finds a whole cluster of cross-referenced sections.
- **V6 Search and hashing:** A hash table maps each bookmark/tag name to its line position; binary search over a sorted array of line-start offsets speeds up "go to line N."

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines a valid order when sections are reorganised under "before/after" constraints, and cycle detection catches circular cross-references before they break navigation.
- **F3 Sorting:** Open documents are sorted by name, last-modified time and size using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Bookmarks are kept in an AVL tree keyed by line number, so "next bookmark after the cursor" is answered on the balanced tree.
- **F5 String algorithms:** Find-and-replace across the document uses Boyer-Moore; edit distance powers a fuzzy find when an exact match fails.
- **F6 Trie and disjoint sets:** Bookmark/tag names are autocompleted with a trie; union-find groups lines belonging to the same paragraph after a reflow operation.
- **F7 File organisation:** The append-only autosave/version log is kept in a sequential file, bookmark records in a direct-access file hashed by tag name (collision resolution with Brent's method).
- **F8 B+ tree index:** The autosave log file keeps a secondary B+ tree index keyed by save timestamp.
- **F9 Extendible hashing / external sort:** The bookmark/tag file, which grows as it accumulates entries across many documents, is managed with extendible hashing.

Extension: Adding real-time collaborative editing with multiple cursors on the same document.

057 — Huffman File Compressor



Summary: A console application that compresses and decompresses text files with Huffman coding, working on a synthetic archive of about 100 sample files up to a few megabytes each. Files can be compressed individually or as a batch job.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The symbol-frequency table built while scanning a file is kept as a doubly linked list; the files in a batch job are processed in turn from a circular list.
- **V2 Sparse matrix:** In the byte-value × file matrix (256 possible values by N files in a batch), only the byte-value frequencies that actually occur in a given file are stored.
- **V3 Stack and queue:** A stack tracks the root-to-leaf path while decoding through the Huffman tree; the files in a batch compression job wait their turn in a queue.
- **V4 Tree and heap:** A min-heap directly builds the Huffman tree from symbol frequencies; the resulting binary tree's root-to-leaf traversal generates every symbol's code.
- **V5 Graph and BFS/DFS:** Nodes are folders and edges are subfolder/file containment; BFS schedules a directory tree for compression level by level, DFS finds all files under one subdirectory.
- **V6 Search and hashing:** A hash table maps each byte value to its Huffman code; canonical code assignment processes a sorted array of code lengths with binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree groups files with similar byte distributions to share a code table, and cycle detection stops symbolic-link loops while compressing a directory tree recursively.
- **F3 Sorting:** Symbols are sorted by frequency for canonical code generation, and batch files are sorted by size and achieved compression ratio, using three algorithms whose running times are compared.
- **F4 BST and AVL:** Archive entries are kept in an AVL tree keyed by filename, answering alphabetical range queries over the archive's contents on the balanced tree.
- **F5 String algorithms:** A given filename pattern is searched across a large archive's file list with KMP; edit distance suggests the closest matching name if the user mistypes an entry.
- **F6 Trie and disjoint sets:** Filenames are autocompleted with a trie when extracting from an archive; union-find groups similar files into batches that share a compression code table.
- **F7 File organisation:** The compressed byte stream itself is kept in a sequential file, the archive's file-entry index in a direct-access file hashed by filename (collision resolution with progressive overflow).
- **F8 B+ tree index:** A multi-archive catalogue keeps a secondary B+ tree index keyed by original file size or compression date.
- **F9 Extendible hashing / external sort:** Files too large for memory are compressed as a stream while the huge symbol-frequency log is processed with an external merge sort.

Extension: Replacing the static two-pass code table with adaptive (dynamic) Huffman coding that updates the tree as it streams data.

058 — " Academic Citation Network Analyser



Summary: A console application that analyses which papers influenced which in a synthetic corpus of about 2,000 academic papers linked by thousands of citation edges. A user can start from one paper and follow its citation chain step by step.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A paper's reference list is kept as a doubly linked list in its original order; an author's publication timeline is cycled through with a circular list.
- **V2 Sparse matrix:** In the paper × paper citation matrix, only the cells where one paper actually cites another are filled, the rest is empty.
- **V3 Stack and queue:** Manually chasing citations from paper to reference to that reference's own reference can be backtracked with a stack; newly imported papers wait in a queue to be linked into the network.
- **V4 Tree and heap:** The chain descending to a paper's most-cited reference is kept in a binary tree and traversed; papers are ranked into a "most cited" leaderboard using a heap.
- **V5 Graph and BFS/DFS:** Nodes are papers and edges are "A cites B" relations; BFS finds all papers within N citation-hops of a seed paper (its influence radius), DFS finds every paper it transitively influenced.
- **V6 Search and hashing:** A hash table maps each paper's title/DOI to its record; binary search over a sorted array of publication years supports range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort arranges papers into a valid order where each cites only earlier work, and Dijkstra finds the shortest citation chain between two papers.
- **F3 Sorting:** Papers are sorted by citation count, publication year and author name using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Papers are kept in an AVL tree keyed by publication year, answering range queries such as "papers published between 2015 and 2020."
- **F5 String algorithms:** A keyword is searched across abstracts with KMP; edit distance matches author names or titles recorded in slightly different formats, catching duplicate records.
- **F6 Trie and disjoint sets:** Author names and paper titles are autocompleted with a trie during search; union-find groups papers into connected research clusters based on citation connectivity.
- **F7 File organisation:** The raw imported citation log is kept in a sequential file, paper records in a direct-access file hashed by DOI (collision resolution with linear quotient).
- **F8 B+ tree index:** The papers file keeps a secondary B+ tree index keyed by citation count, for fast top-K lookups.
- **F9 Extendible hashing / external sort:** A citation-edge list of millions of entries is too large for memory, so it is sorted by source paper ID with an external merge sort.

Extension: Overlaying a co-authorship network alongside the citation network to visualise collaboration clusters.

059 — {..} JSON Parser and Query Tool



Summary: A console application that parses configuration or data JSON files and lets a user find specific fields with path-based queries. Synthetic JSON documents of tens of thousands of nodes are processed individually or in batches.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A JSON object's sibling key-value pairs are kept as a doubly linked list in their original order; open document tabs are cycled through with a circular list.
- **V2 Sparse matrix:** In the document $\times$ key-path matrix (each document in a batch against every distinct path encountered), only the paths a document actually contains are stored.
- **V3 Stack and queue:** A stack tracks nested object/array depth during parsing and backtracks on bracket-matching errors; streamed JSON tokens wait in a queue to be parsed.
- **V4 Tree and heap:** For fast field access, the keys at each level of a large object are kept in a binary search tree; query results across several documents are ranked in a heap by size or relevance.
- **V5 Graph and BFS/DFS:** Nodes are JSON values and edges are parent-child containment (or "\$ref" links between documents); BFS answers a depth-limited query within N levels, DFS fully traverses a document or resolves a chain of references.
- **V6 Search and hashing:** A hash table maps a flattened key path such as "user.address.city" to its value; binary search over a sorted array of numeric values in an array field supports range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders JSON files that reference each other through "\$ref" into a valid load order, and cycle detection catches circular references before they loop forever.
- **F3 Sorting:** An array's elements are sorted by a chosen field, and query results by key path and value, using three algorithms whose running times are compared.
- **F4 BST and AVL:** Key paths are kept alphabetically in an AVL tree, so "next key after X" for schema browsing is answered on the balanced tree.
- **F5 String algorithms:** A string value is searched across the whole document with Boyer-Moore; edit distance suggests the closest matching key name when a query key is misspelled.
- **F6 Trie and disjoint sets:** Key-path queries are autocompleted with a trie; union-find groups batch documents that share the same schema shape.
- **F7 File organisation:** The import log of parsed documents is kept in a sequential file, flattened key-path index records in a direct-access file hashed by path (collision resolution with Brent's method).
- **F8 B+ tree index:** A multi-document catalogue keeps a secondary B+ tree index keyed by a chosen numeric field, such as "id," for fast lookup.
- **F9 Extendible hashing / external sort:** As new JSON documents, each with tens of thousands of nodes, keep being added to the batch catalogue, the growing multi-file key-path index is managed with extendible hashing.

Extension: Validating a document's structure against a user-supplied JSON Schema and reporting errors.

060 — Scientific Calculator Expression Evaluator



Summary: A console calculator that parses and evaluates a user's nested, parenthesised mathematical expressions with variables and functions, keeping a history of past calculations. It is validated against a synthetic set of about 5,000 test expressions.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The tokens of an expression not yet evaluated are kept as a doubly linked list for cursor-based editing; memory registers M1 through M9 are cycled through with a circular list.
- **V2 Sparse matrix:** In the variable $\times$ saved-expression matrix, only the variables a given expression actually uses are stored, since most expressions use just a few of the defined variables.
- **V3 Stack and queue:** The shunting-yard algorithm, which converts infix to postfix and evaluates nested parentheses, uses an operator stack and an operand stack; expressions submitted as a batch wait in a queue.
- **V4 Tree and heap:** A parsed expression is built as a binary tree whose post-order traversal evaluates it; a batch of submitted expressions is prioritised in a heap so simpler ones are evaluated first.
- **V5 Graph and BFS/DFS:** Nodes are user-defined variables and edges are "A's definition uses B" dependencies; BFS finds all dependencies within N levels of a variable, DFS traces the full chain needed for recomputation.
- **V6 Search and hashing:** A hash table maps each variable name to its current value; binary search over a sorted array of built-in function names validates them quickly during parsing.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines a valid evaluation order for interdependent variable definitions, and cycle detection catches circular references such as "a = b+1, b = a+1" before evaluation.
- **F3 Sorting:** Expression history is sorted by evaluation time, result value and token count (complexity) using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Saved expressions are kept in an AVL tree keyed by timestamp, answering range queries such as "expressions evaluated in the last hour."
- **F5 String algorithms:** Expression history entries containing a given function name are found with KMP; edit distance suggests the closest valid function name when the user mistypes one, such as "sqrt" as "sqrtt."
- **F6 Trie and disjoint sets:** Function and variable names are autocompleted with a trie as the user types; union-find groups mutually dependent variables into connected components for batch recomputation.
- **F7 File organisation:** The ordered calculation history is kept in a sequential file, variable records in a direct-access file hashed by name (collision resolution with progressive overflow).
- **F8 B+ tree index:** The history file keeps a secondary B+ tree index keyed by result value, speeding up range queries such as "all results between 100 and 200."
- **F9 Extendible hashing / external sort:** A massive batch-evaluation log of millions of expressions is too large for memory, so it is sorted by timestamp with an external merge sort.

Extension: Adding symbolic differentiation and simplification over the parsed expression tree.

061 — <> HTML/XML Tag Validator



Summary: A console application that validates roughly 800 uploaded HTML/XML documents against tag-matching, nesting and required-attribute rules, then produces a structured error report.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each document's tags are chained in order in a doubly linked list; documents waiting in a batch validation run are cycled through with a circular list.
- **V2 Sparse matrix:** In the tag-instance by attribute-name table, only attributes that actually appear are stored, so most cells stay empty.
- **V3 Stack and queue:** Opening and closing tags are matched on a stack to catch nesting errors; the resulting error messages are buffered in a queue and printed in order.
- **V4 Tree and heap:** The document tree (DOM) is built as a binary-tree representation and walked with a preorder traversal to summarize its structure; errors are kept in a heap ordered by severity so the most critical one is shown first.
- **V5 Graph and BFS/DFS:** Elements are nodes, and parent-child relations plus id references are edges; BFS finds the shortest depth between the root and a given id, DFS finds elements left unreachable after an id-reference chain breaks.
- **V6 Search and hashing:** Tag name maps to its allowed child/attribute rule in a hash table; sorted error line numbers support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sorting flags entities or ids referenced before they are defined; cycle detection flags elements that reference each other in a loop.
- **F3 Sorting:** Errors are sorted by line number, severity and tag name with three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Error records are kept in an AVL tree keyed by line number; the query "errors between line 100 and 200" is answered on this tree.
- **F5 String algorithms:** KMP searches the document text for the tag/attribute pattern a user types; edit distance suggests the nearest valid tag name when one is misspelled.
- **F6 Trie and disjoint sets:** Known tag and attribute names are autocompleted with a trie; elements sharing the same CSS class are grouped with union-find.
- **F7 File organisation:** Validation records are kept in a sequential file, and tag rules in a direct-access file keyed by tag name (collision resolution with progressive overflow).
- **F8 B+ tree index:** A secondary B+ tree index keyed by tag name on the error-records file speeds up the query "all errors for tag X."
- **F9 Extendible hashing / external sort:** Millions of tag records gathered from thousands of documents, too large for memory, are sorted by severity with an external merge sort.

Extension: A live-validation mode that incrementally reparses the document and gives instant feedback as the user types.

062 — Multilingual Term Glossary



Summary: A console application that manages a term database together with its translations, synonyms and definitions across languages; it works on a synthetic glossary of about 5,000 terms in 6 languages.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each term's per-language variants are chained in a doubly linked list; navigation between related terms is supported with a circular list.
- **V2 Sparse matrix:** In the term by language table, only cells with an actual translation are filled, since most terms are not yet translated into every language.
- **V3 Stack and queue:** The last edit to a term's translation history is undone with a stack; translation requests awaiting approval sit in a queue.
- **V4 Tree and heap:** Terms are kept in a binary tree per language in alphabetical order and listed with an in-order traversal; terms with stale translations are kept in a heap so the most urgent one is shown to the translator first.
- **V5 Graph and BFS/DFS:** Terms are nodes, "related term" links are edges; BFS finds the shortest relation chain between two terms, DFS finds the whole concept cluster reachable from a term.
- **V6 Search and hashing:** Term name maps to its term record in a hash table; a sorted term array supports binary search for prefix listing.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm builds a concept map connecting all related terms with the fewest links; cycle detection flags a redundant synonym loop where terms reference each other circularly.
- **F3 Sorting:** Terms are sorted alphabetically, by number of translated languages, and by last-update date with three algorithms, comparing running times.
- **F4 BST and AVL:** For each language, terms are kept in an AVL tree by spelling; terms starting with a given prefix are listed with a range query.
- **F5 String algorithms:** KMP searches definitions and example sentences for a term a user types; edit distance suggests the closest known term when one is misspelled.
- **F6 Trie and disjoint sets:** Term names are autocompleted with a trie; terms with the same meaning across languages are merged into synonym clusters with union-find.
- **F7 File organisation:** Export records are kept in a sequential file, term records in a direct-access file keyed by term ID (collision resolution with linear quotient).
- **F8 B+ tree index:** A secondary B+ tree index keyed by language code on the term file speeds up "all terms with an English translation."
- **F9 Extendible hashing / external sort:** The ever-growing glossary file, which keeps gaining new terms, is organized with extendible hashing.

Extension: A crowd-sourced feature letting users propose and vote on new terms or translations.

063 — Log File Search and Pattern Matcher



Summary: A console tool that scans server log files and filters them by pattern, time range and severity; it works over a 30-day archive with roughly 200,000 synthetic log lines per day.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each log file's lines are kept in chronological order in a doubly linked list; a live-tail buffer showing the last N entries is implemented as a circular list.
- **V2 Sparse matrix:** In the hour-of-day by source-service table, only cells with actual events are filled, so the table is sparse since most services are silent most hours.
- **V3 Stack and queue:** START/END markers of nested transactions are matched on a stack to catch unclosed operations; incoming lines are buffered in a queue for batch processing.
- **V4 Tree and heap:** Entries are kept in a binary tree by module/submodule path and browsed with a traversal in hierarchical form; alerts are kept in a heap by severity so the most critical one shows first.
- **V5 Graph and BFS/DFS:** Services are nodes, request-trace links are edges; BFS finds the shortest call chain from a failure back to a given service, DFS finds all downstream components affected by a failing service.
- **V6 Search and hashing:** Request/session ID maps to its chain of entries in a hash table; entries sorted by timestamp support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sorting of the service-dependency graph determines restart order; cycle detection flags the circular dependency behind a cascading failure.
- **F3 Sorting:** Entries are sorted by timestamp, severity and source module with three algorithms, comparing running times.
- **F4 BST and AVL:** Entries are kept in an AVL tree by timestamp; "all entries between 14:00 and 14:05" is answered quickly on this tree.
- **F5 String algorithms:** KMP searches messages for a keyword pattern a user types; near-identical error messages are clustered with edit distance to deduplicate them.
- **F6 Trie and disjoint sets:** Known module/tag names are autocompleted with a trie; entries belonging to the same incident are merged into one cluster with union-find.
- **F7 File organisation:** Raw incoming lines are kept in a sequential file, indexed entries in a direct-access file keyed by request ID (collision resolution with Brent's method).
- **F8 B+ tree index:** A secondary B+ tree index keyed by severity level on the log file speeds up "all CRITICAL entries."
- **F9 Extendible hashing / external sort:** Millions of lines spanning several months of logs, too large for memory, are sorted chronologically with an external merge sort.

Extension: A live-tail mode that highlights matches instantly as new synthetic entries arrive.

064 — Plagiarism Similarity Checker



Summary: A console application that compares roughly 300 student submissions pairwise for textual similarity and flags suspicious pairs, producing about 45,000 comparisons in total.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each submission's paragraphs are kept in order in a doubly linked list; submissions waiting for a batch comparison run are cycled through with a circular list.
- **V2 Sparse matrix:** In the submission by submission similarity matrix, a score is stored only for pairs that overlap enough, so most cells stay empty since most pairs are unrelated.
- **V3 Stack and queue:** Nested quotation blocks are tracked on a stack while parsing a document so properly cited text is excluded; document pairs awaiting comparison sit in a batch queue.
- **V4 Tree and heap:** Each submission is parsed into a binary tree of sections/paragraphs and walked with a traversal for sectional analysis; submission pairs are kept in a heap by similarity score so the most suspicious pair is shown to the instructor first.
- **V5 Graph and BFS/DFS:** Submissions are nodes, above-threshold similarity links are edges; BFS finds the shortest copying chain between two suspicious submissions, DFS finds the whole cluster of submissions copying from each other.
- **V6 Search and hashing:** A shingled word-sequence (n-gram) fingerprint maps to the list of documents containing it in a hash table; sorted submission IDs support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm connects all flagged submissions into a copying network with minimum total dissimilarity; cycle detection reveals a closed loop within a cheating ring.
- **F3 Sorting:** Submission pairs are sorted by similarity score, shared n-gram count and submission date with three algorithms, comparing running times.
- **F4 BST and AVL:** All pairs' similarity scores are kept in an AVL tree; "pairs with similarity between 70% and 90%" is answered on this tree.
- **F5 String algorithms:** KMP searches the document body for exactly copied phrases; near-duplicate sentences disguised by word substitutions are measured with edit distance/LCS.
- **F6 Trie and disjoint sets:** All submissions' n-gram fingerprints are kept in a trie for fast matching; mutually similar submissions are grouped into connected clusters with union-find.
- **F7 File organisation:** Raw fingerprint records are kept in a sequential file, submission metadata in a direct-access file keyed by submission ID (collision resolution with progressive overflow).
- **F8 B+ tree index:** A secondary B+ tree index keyed by course section on the submission file speeds up "all submissions of section X."
- **F9 Extendible hashing / external sort:** Millions of n-gram fingerprints accumulated over a semester, too large for memory, are sorted by hash value with an external merge sort before comparison.

Extension: A visual similarity-network export that lets instructors spot clusters at a glance.

065 — E-Book Reader Index and Bookmark Engine



Summary: A console application that manages reading position, bookmarks and notes, and searches page text, across a synthetic library of about 200 e-books totaling 40,000 pages.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A book's pages are kept in reading order in a doubly linked list; jumping between saved bookmarks in sequence is done with a circular list.
- **V2 Sparse matrix:** In the chapter by keyword table, only occurrence counts for words that actually appear in a chapter are stored, so the table is sparse.
- **V3 Stack and queue:** "Back" navigation through visited pages is implemented with a stack; pending highlight/note-sync operations wait in a queue.
- **V4 Tree and heap:** The table of contents is built as a binary tree of chapters/sections and browsed with a traversal; notes and bookmarks are kept in a heap by recency so the most recent appears first in a "continue reading" list.
- **V5 Graph and BFS/DFS:** Pages are nodes, cross-references (footnotes, "see chapter 5," hyperlinked terms) are edges; BFS finds the shortest reference-hop path between two topics, DFS finds every page reachable through a footnote chain.
- **V6 Search and hashing:** An index term maps to the list of pages it appears on in a hash table; sorted page numbers support binary search for a bookmark lookup.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sorting of chapter prerequisite links suggests a valid reading order; cycle detection catches a circular cross-reference loop.
- **F3 Sorting:** Bookmarks are sorted by page number, creation date and tag with three algorithms, comparing running times.
- **F4 BST and AVL:** Bookmarks/notes are kept in an AVL tree by page number; "all notes between page 100 and 150" is answered on this tree.
- **F5 String algorithms:** KMP searches the full book text for a word or phrase a user types; edit distance suggests the closest index term when a search query is mistyped.
- **F6 Trie and disjoint sets:** Index terms are autocompleted with a trie; chapters sharing the same topic are grouped into "reading units" with union-find.
- **F7 File organisation:** The book's page text is kept in a sequential file in reading order, bookmark records in a direct-access file keyed by bookmark ID (collision resolution with linear quotient).
- **F8 B+ tree index:** A secondary B+ tree index keyed by chapter number on the page file speeds up jumping to a chapter.
- **F9 Extendible hashing / external sort:** The personal library file, which keeps growing as the user adds more books over time, is organized with extendible hashing.

Extension: A reading-progress dashboard tracking pages-per-day trends.

066 — Subtitle Sync and Search Tool



Summary: A console application that fixes timing, searches by speaker and compares languages across roughly 150 episode/movie subtitle files, working over tens of thousands of cues in total.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A subtitle file's cues (start/end timestamp and text) are kept in playback order in a doubly linked list; switching between language tracks attached to the same video uses a circular list.
- **V2 Sparse matrix:** In the scene by speaker table, only the line count for a speaker who actually appears in that scene is stored, so the table is sparse.
- **V3 Stack and queue:** Successive timing-shift adjustments are undone with a stack; cues waiting for a batch export sit in a queue.
- **V4 Tree and heap:** The video's scene/chapter breakdown is built as a binary tree and walked with a traversal to browse cues in structured form; overlapping cues are kept in a heap by conflict severity so the worst overlap is fixed first.
- **V5 Graph and BFS/DFS:** Cues are nodes, "same speaker continues" links are edges; BFS finds the shortest chain between two cues in a conversation, DFS finds every cue in one continuous speaker's turn.
- **V6 Search and hashing:** A speaker label maps to the list of cues they speak in a hash table; cues sorted by timestamp support binary search to jump to a given time.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm computes the minimum total re-timing cost needed to fix a chain of overlapping cues; cycle detection catches a malformed circular cue reference.
- **F3 Sorting:** Cues are sorted by start time, duration and speaker with three algorithms, comparing running times.
- **F4 BST and AVL:** Cues are kept in an AVL tree by timestamp; "all cues between 00:12:00 and 00:15:00" is answered on this tree for a scene export.
- **F5 String algorithms:** KMP searches cue text for a spoken phrase a user types; different-language versions of the same content are aligned by matching similarly-sized cues with edit distance.
- **F6 Trie and disjoint sets:** Speaker names/tags are autocompleted with a trie; continuous dialogue scenes are grouped into connected cue clusters with union-find.
- **F7 File organisation:** Exported cues are kept in a sequential file in playback order, cue records in a direct-access file keyed by cue ID (collision resolution with progressive overflow).
- **F8 B+ tree index:** A secondary B+ tree index keyed by speaker label on the cue file speeds up "show all lines from speaker X."
- **F9 Extendible hashing / external sort:** Cues gathered from an entire season of episodes, too large for memory, are sorted by timestamp with an external merge sort before being merged into one archive.

Extension: Automatic detection and flagging of cues that exceed a maximum reading speed, for accessibility compliance.

067 — C Mini Programming Language Interpreter



Summary: A console interpreter that executes, line by line, a small scripting language with variable assignment, arithmetic and loops/conditionals; sample scripts a few hundred lines long generate thousands of executed steps per run.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Tokens produced from a source file are kept in order in a doubly linked list; cycling through previous REPL commands with the up/down arrows uses a circular list.
- **V2 Sparse matrix:** In the scope-level by variable-name table, a value is stored only for variables actually defined at that scope, so the table is sparse.
- **V3 Stack and queue:** Nested arithmetic expressions and function calls are evaluated on a stack, checking parenthesis matching; pending statements are buffered in a queue for execution.
- **V4 Tree and heap:** An expression's abstract syntax tree is built as a binary tree and evaluated with a postorder traversal; the built-in sort() function's internal implementation uses heap sort.
- **V5 Graph and BFS/DFS:** Functions are nodes, call relationships are edges; BFS finds the shortest call chain from main to a target function, DFS finds every function reachable from an entry point for dead-code analysis.
- **V6 Search and hashing:** Variable name maps to its value/type record (the symbol table) in a hash table; the lexer's keyword scan uses binary search over a sorted array of built-in names.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sorting of the variable/function dependency graph determines a safe evaluation order; cycle detection catches illegal circular self-definitions and infinite mutual recursion.
- **F3 Sorting:** User-defined function names, variable declarations and error messages are sorted by line number with three algorithms, comparing running times, to produce an ordered program listing.
- **F4 BST and AVL:** A scope's symbol-table entries are kept in an AVL tree by variable name; "all variables named between 'a' and 'm'" is answered in a debugger scope dump.
- **F5 String algorithms:** The REPL's "find" command searches source code for a token/identifier pattern with KMP; edit distance suggests the closest valid keyword when one is mistyped (e.g. "pirnt" to "print").
- **F6 Trie and disjoint sets:** All reserved keywords and built-in function names are kept in a trie for fast lexer lookup and REPL autocomplete; variables unified through an alias/reference construct are grouped into equivalence classes with union-find.
- **F7 File organisation:** An execution trace of interpreted statements is kept in a sequential file in run order, debugger variable snapshots in a direct-access file keyed by snapshot ID (collision resolution with Brent's method).
- **F8 B+ tree index:** A secondary B+ tree index keyed by function name on the trace-log file speeds up "all calls to function X."
- **F9 Extendible hashing / external sort:** A saved-script library file, which keeps growing as a user names and saves hundreds of scripts over a long REPL session, is organized with extendible hashing.

Extension: A step-through visual debugger that shows the parse tree and call stack live.

068 — CSV Dataset Query Engine



Summary: A console query engine that loads several CSV files and runs SQL-like filter, sort and join queries; it works on synthetic datasets of up to two million rows.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each loaded table's rows are kept in insertion order in a doubly linked list; switching quickly between several datasets open in the same session uses a circular list.
- **V2 Sparse matrix:** In the row by column table, only filled cells are stored since many values are missing, so most cells are empty.
- **V3 Stack and queue:** Nested boolean expressions in a WHERE clause are evaluated on a stack, checking parentheses and precedence; query results are buffered in a queue for batch export.
- **V4 Tree and heap:** A query's filter expression is built as a binary tree and each row is evaluated against the WHERE clause by walking it; a "top-K rows by column value" query uses a heap instead of a full sort.
- **V5 Graph and BFS/DFS:** Tables are nodes, foreign-key relationships between CSV files are edges; BFS finds the shortest join path between two tables, DFS finds every table reachable through joins from a given root table.
- **V6 Search and hashing:** A column's key value maps to its row record in a hash table for fast equality lookups and joins; a sorted column array supports binary search for range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sorting determines table order for a multi-table join query plan; cycle detection catches a circular foreign-key reference between CSV files.
- **F3 Sorting:** Query results are sorted on up to three chosen columns with three algorithms, comparing running times on large result sets.
- **F4 BST and AVL:** An indexed column's values are kept in an AVL tree; "all rows with price between 100 and 500" is answered efficiently on this tree.
- **F5 String algorithms:** A LIKE-query pattern is searched in a text column with KMP; misspelled category values are fuzzy-matched with edit distance during GROUP BY normalization.
- **F6 Trie and disjoint sets:** Column and table names are autocompleted with a trie; rows are grouped into equivalence classes with union-find for GROUP BY and duplicate detection.
- **F7 File organisation:** Raw imported CSV rows are kept in a sequential file in original order, an index keyed by the primary-key column in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** A secondary B+ tree index built on a frequently filtered column speeds up range queries such as "WHERE date between X and Y."
- **F9 Extendible hashing / external sort:** A multi-million-row CSV dataset too large for memory is sorted with an external merge sort before a merge-join with another table.

Extension: A query-plan visualizer showing which index or algorithm was chosen for each query.

069 — .* Simple Regular Expression Engine



Summary: A console engine that parses a simple regular-expression pattern a user types, compiles it into a finite-state machine, and runs it against sample strings; a long testing session works with hundreds of patterns and thousands of sample strings.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Tokens (literals, operators) produced while parsing a pattern are kept in order in a doubly linked list; a history of the last N strings tested against a pattern is kept in a circular list.
- **V2 Sparse matrix:** In the DFA state by input-symbol transition table, only defined transitions are stored, since an undefined transition implicitly means rejection, so most cells are empty.
- **V3 Stack and queue:** Nested groups and operator precedence in a pattern are parsed on a stack and converted to postfix (shunting-yard); the active set of NFA states during a Thompson-construction simulation is held in a queue.
- **V4 Tree and heap:** A pattern's syntax tree (concatenation, alternation and star nodes) is built as a binary tree and walked for Thompson's construction; the most frequently used patterns are ranked with heap sort to drive a cache-eviction policy.
- **V5 Graph and BFS/DFS:** NFA/DFA states are nodes, symbol- or epsilon-labeled transitions are edges; BFS finds the shortest string accepted by the automaton, DFS finds every state reachable from the start state through epsilon-closure.
- **V6 Search and hashing:** A compiled pattern string maps to its already-built DFA in a hash table to avoid recompiling; a sorted array of DFA state IDs supports binary search during simulation.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** DFA minimization merges equivalent states the way finding connected components would; cycle detection in the NFA graph flags patterns with unbounded repetition (Kleene-star loops) for a safety warning.
- **F3 Sorting:** A batch of test strings is sorted by match result, match length and pattern compile time with three algorithms, comparing running times.
- **F4 BST and AVL:** Compiled DFAs are cached in an AVL tree keyed by pattern string; "all cached patterns starting with 'a'" is listed with a range query.
- **F5 String algorithms:** For literal-heavy patterns, candidate substrings are pre-filtered with KMP before full NFA simulation; when a pattern fails to compile due to a typo, edit distance suggests the closest valid pattern.
- **F6 Trie and disjoint sets:** All previously compiled pattern strings are kept in a trie for fast prefix lookup; DFA minimization merges equivalent states into unified partitions with union-find.
- **F7 File organisation:** Every match attempt (pattern, input, result) is logged in a sequential file, DFA state-table cache entries in a direct-access file keyed by a state-set hash (collision resolution with linear quotient).
- **F8 B+ tree index:** A secondary B+ tree index keyed by pattern ID on the match-log file speeds up "all match attempts for pattern X."
- **F9 Extendible hashing / external sort:** A named-pattern library file, which keeps growing as a user saves patterns under names over a long testing session, is organized with extendible hashing.

Extension: A tool that visualizes the NFA/DFA state diagram built from a given pattern.

070 — Email Archive Thread Indexer



Summary: A console application that splits tens of thousands of emails from a multi-year mailbox export into searchable conversation threads, producing thousands of separate threads.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A thread's messages are kept in chronological order in a doubly linked list; cycling between recently viewed threads uses a circular list.
- **V2 Sparse matrix:** In the sender by recipient table, a count is stored only for pairs that actually exchanged messages, so the table is sparse since most pairs never communicated.
- **V3 Stack and queue:** Nested quoted-reply blocks are tracked on a stack while parsing a message body so new text is separated from quoted history; incoming messages wait in a queue for batch indexing.
- **V4 Tree and heap:** A thread's reply tree (a message and its replies) is built as a binary tree and shown with a traversal in structured form; threads are kept in a heap by an "unread plus recency" score so the most important one shows first.
- **V5 Graph and BFS/DFS:** Messages are nodes, In-Reply-To and reference headers are edges; BFS finds the shortest reply distance between two messages, DFS finds the whole reply subtree reachable from a root message.
- **V6 Search and hashing:** Message ID maps to its message record in a hash table to speed up In-Reply-To resolution; messages sorted by date support binary search for a date-range lookup.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** When timestamps are missing or ambiguous, topological sorting reconstructs a valid chronological order from reply dependencies; cycle detection catches a malformed circular reference header.
- **F3 Sorting:** Threads are sorted by last-activity date, participant count and message count with three algorithms, comparing running times.
- **F4 BST and AVL:** Message records are kept in an AVL tree by date; "all messages between two dates" is answered on this tree for an export feature.
- **F5 String algorithms:** KMP searches subjects and bodies for a keyword a user types; near-identical subject lines ("Re:" variants, typos) are linked into the same thread with edit distance.
- **F6 Trie and disjoint sets:** Sender names/addresses are autocompleted with a trie; messages sharing a subject/reference chain are merged into one thread as connected components with union-find.
- **F7 File organisation:** Raw imported messages are kept in a sequential file in arrival order, message records in a direct-access file keyed by message ID (collision resolution with Brent's method).
- **F8 B+ tree index:** A secondary B+ tree index keyed by sender address on the message file speeds up "all messages from sender X."
- **F9 Extendible hashing / external sort:** Millions of messages in a multi-year mailbox archive, too large for memory, are sorted chronologically with an external merge sort before thread reconstruction.

Extension: A duplicate-attachment detector that flags identical files sent across multiple threads.

071 — News Headline Clustering and Search



Summary: A console application for a news desk that groups incoming headlines from many outlets into clusters covering the same event, so a reader can quickly find the story they are after. It works on about 5,000 synthetic headlines gathered from 50 sources, with clusters updated as new headlines arrive.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each source's headlines are kept in a doubly linked list in chronological order; a rotating breaking-news ticker is driven by a circular list.
- **V2 Sparse matrix:** In the category $\times$ source table, only cells where that source actually published in that category are filled, holding the headline count for the pair.
- **V3 Stack and queue:** An editor's edits while reviewing a cluster are undone with a stack; incoming headlines wait to be processed in a queue.
- **V4 Tree and heap:** Headlines within a cluster are kept in a binary tree with in-order traversal for an alphabetic listing; the most-discussed headlines are found via a heap ranked by trending score.
- **V5 Graph and BFS/DFS:** Headlines are nodes, shared-keyword similarity above a threshold is an edge; BFS finds every headline within N similarity hops of a query headline (its cluster), while DFS finds the connected region covering every follow-up story tied to one event.
- **V6 Search and hashing:** Headline id maps to headline record in a hash table; a date-sorted headline array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest "similarity distance" path between two headlines, i.e. how many shared sources or keywords link them; cycle detection catches circular correction-and-follow-up chains that reference each other.
- **F3 Sorting:** Headlines are sorted by date, relevance score, and source name with three different algorithms, comparing running times over about 5,000 records.
- **F4 BST and AVL:** Headlines are kept in an AVL tree keyed by publish timestamp; the query "first headline after 14:20" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches headline text for the user's query; on a typo, edit distance suggests the closest matching keyword.
- **F6 Trie and disjoint sets:** Tags and keywords are autocompleted with a trie; union-find merges headlines into the same story cluster as similarity edges are added.
- **F7 File organisation:** The raw headline archive is kept in a sequential file, headline records in a direct-access file hashed by id, with collision resolution by progressive overflow.
- **F8 B+ tree index:** The headline archive file keeps a secondary B+ tree index keyed by category, speeding up category browsing.
- **F9 Extendible hashing / external sort:** A year's worth of headlines is too large for memory, so it is sorted by date with an external merge sort.

Extension: Auto-tagging headlines with sentiment scores to track trending topics over time.

072 — 🗨️ Proverbs and Idioms Search Engine



Summary: A console application for a language teacher or researcher who has forgotten the meaning of a proverb or idiom and wants to find it and compare its regional variants. It works over about 3,000 synthetic proverbs and idioms tagged with 15 regions.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Proverbs under each theme are kept in a doubly linked list; the "proverb of the day" feature cycles through entries with a circular list.
- **V2 Sparse matrix:** In the theme × region table, only cells where that region has proverbs on that theme are filled, holding the count of matching proverbs.
- **V3 Stack and queue:** Nested parenthetical explanations in a dictionary entry are parsed with a stack; proverbs a user adds to their study list wait to be reviewed in a queue.
- **V4 Tree and heap:** Proverbs are kept in a binary tree by theme with in-order traversal for an alphabetic listing; the most-searched proverbs are found via a heap for a top-10 list.
- **V5 Graph and BFS/DFS:** Proverbs are nodes, a shared-meaning or synonym relation is an edge; BFS finds every proverb within N meaning-steps of a query proverb, while DFS finds the connected region covering all dialect variants of one saying across regions.
- **V6 Search and hashing:** A keyword maps to a proverb record in a hash table; an alphabetically sorted proverb array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Connected components group regional variants of the same underlying saying into one proverb family; Dijkstra finds the shortest "meaning-hop" path between two seemingly unrelated proverbs.
- **F3 Sorting:** Proverbs are sorted by length, alphabetically, and by search popularity with three algorithms, comparing running times over about 3,000 records.
- **F4 BST and AVL:** Proverbs are kept in an AVL tree keyed by first word; the range query "all proverbs starting with D through F" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a partially typed proverb phrase; edit distance matches a misremembered proverb to the closest real one.
- **F6 Trie and disjoint sets:** Proverb opening words are autocompleted with a trie; union-find groups proverbs that are regional variants of the same underlying saying.
- **F7 File organisation:** The imported proverb collection is kept in a sequential file, proverb records in a direct-access file hashed by id, with collision resolution by the linear quotient method.
- **F8 B+ tree index:** The proverb file keeps a secondary B+ tree index keyed by theme, speeding up theme-based browsing.
- **F9 Extendible hashing / external sort:** The growing proverb database, as students keep contributing entries, is managed with extendible hashing.

Extension: Crowd-submitted regional variants going through a moderation queue before being added.

073 — Tag Cloud and Keyword Extractor



Summary: A console application for a content editor that pulls key concepts out of uploaded texts and presents a tag list sized by popularity. It works over about 8,000 synthetic keywords extracted from 500 documents.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Keywords extracted from a document are kept in a doubly linked list ordered by frequency; the tag cloud's rotating display order is driven by a circular list.
- **V2 Sparse matrix:** In the document $\times$ keyword table, only cells where a keyword actually occurs in that document are filled, holding the term's occurrence count.
- **V3 Stack and queue:** A user's manual tag edits are undone with a stack; uploaded documents wait in a queue to be processed for extraction.
- **V4 Tree and heap:** Keywords are kept in a binary tree with in-order traversal for an alphabetic tag listing; the most frequent keywords are found via a heap used to size the tag cloud.
- **V5 Graph and BFS/DFS:** Keywords are nodes, co-occurrence in the same document is an edge; BFS finds every keyword within N hops of a query keyword, while DFS finds the connected cluster of keywords forming one topic.
- **V6 Search and hashing:** A keyword maps to a keyword record in a hash table; an alphabetically sorted keyword array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree (Prim) builds a minimal keyword-relation network for visualization; Dijkstra computes the weighted "semantic distance" between two keywords from their co-occurrence strength.
- **F3 Sorting:** Keywords are sorted by frequency, alphabetically, and by document count with three algorithms, comparing running times over about 8,000 records.
- **F4 BST and AVL:** Keywords are kept in an AVL tree keyed by frequency rank; the query "next most frequent keyword after X" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches raw text for stop-word patterns during extraction; edit distance merges near-duplicate keyword variants, such as plurals or typos, into one tag.
- **F6 Trie and disjoint sets:** Tags a user adds manually are autocompleted with a trie; union-find merges synonymous keyword groups into a single tag.
- **F7 File organisation:** Uploaded raw documents are kept in a sequential file, keyword index records in a direct-access file hashed by keyword, with collision resolution by Brent's method.
- **F8 B+ tree index:** The keyword file keeps a secondary B+ tree index keyed by document id, for fast retrieval of all tags for a document.
- **F9 Extendible hashing / external sort:** A huge keyword-occurrence log too large for memory is sorted with an external merge sort before the final index is built.

Extension: Extending extraction to multiple languages with per-language stop-word lists.

074 — File System Scanner and Duplicate Finder



Summary: A console application that helps a system administrator find wasted disk space by locating duplicate files. It scans a directory tree of about 200,000 synthetic file records and groups duplicates by content hash.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Files within a directory are kept in a doubly linked list during the scan, for easy insertion and removal; recently scanned folders are tracked with a circular list.
- **V2 Sparse matrix:** In the folder × file-extension table, only cells where that folder actually contains files of that extension are filled, holding the count.
- **V3 Stack and queue:** An explicit stack drives recursion-free depth-first descent through the directory tree, with backtracking; pending folders wait in a queue for a breadth-first, level-by-level scan.
- **V4 Tree and heap:** Part of the folder hierarchy is represented as a binary tree with in-order traversal for an alphabetic file listing; the largest files are surfaced via a heap-based priority queue for a cleanup report.
- **V5 Graph and BFS/DFS:** Directories are nodes, subdirectory and symlink relations are edges; BFS finds every folder within N levels of the root, while DFS finds all files reachable once a symlink cycle is entered.
- **V6 Search and hashing:** A content hash maps to a list of file records in a hash table for duplicate detection; a size-sorted array supports binary search for files near a target size.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection catches symlink loops during the scan; connected components group files sharing the same inode, i.e. hard links, together.
- **F3 Sorting:** Files are sorted by size, modified date, and path with three algorithms, comparing running times over about 200,000 scanned entries.
- **F4 BST and AVL:** Files are kept in an AVL tree keyed by size; the range query "smallest file larger than X" is answered on the balanced tree for cleanup suggestions.
- **F5 String algorithms:** Boyer-Moore searches for a filename pattern the user types; edit distance flags likely renamed duplicate filenames.
- **F6 Trie and disjoint sets:** Folder and file path prefixes are autocompleted with a trie; union-find groups files into duplicate sets as matching hashes are found.
- **F7 File organisation:** The full scan session is logged to a sequential file, hash-index records to a direct-access file hashed by content hash, with collision resolution by progressive overflow.
- **F8 B+ tree index:** The file index keeps a secondary B+ tree index keyed by file path, for fast path-prefix browsing.
- **F9 Extendible hashing / external sort:** A multi-million-entry scan log too large for memory is sorted by size with an external merge sort.

Extension: Scheduled background rescans with incremental change detection.

075 — Command Shell History and Completion



Summary: A console application that lets a developer search commands they have typed before in a terminal and reuse them, with autocomplete suggestions while typing. It works over about 50,000 synthetic history entries and 500 aliases.

Midterm — C (weeks 1–6):

- **V1 Linked list:** History entries are kept in a doubly linked list for up/down-arrow navigation; a memory-efficient XOR-linked list of visited working directories supports back/forward navigation, similar to a browser's history.
- **V2 Sparse matrix:** In the command × working-directory table, only cells where that command was actually run in that directory are filled, holding the run count.
- **V3 Stack and queue:** Editing a multi-command line containing pipes and subshells is undone with a stack; commands queued for batch scripting wait in a queue to run.
- **V4 Tree and heap:** Unique commands are kept in a binary tree with in-order traversal for an alphabetic help listing; the most-used commands are found via a heap for a quick-access menu.
- **V5 Graph and BFS/DFS:** Commands are nodes, a "commonly run right after" relation is an edge; BFS suggests command chains within N steps of a given command, while DFS finds every command reachable through a recorded macro or alias chain.
- **V6 Search and hashing:** An alias name maps to a full command record in a hash table; an alphabetically sorted command-name array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders a script's interdependent commands, such as build steps, correctly before execution; Dijkstra finds the command sequence that reaches a target state with the fewest keystrokes.
- **F3 Sorting:** History entries are sorted by frequency, recency, and alphabetically with three algorithms, comparing running times over about 50,000 entries.
- **F4 BST and AVL:** History entries are kept in an AVL tree keyed by timestamp; the range query "commands run after time T" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches history for a partially typed command; edit distance turns a mistyped command into a "did you mean" suggestion.
- **F6 Trie and disjoint sets:** Command names and flags are tab-completed with a trie; union-find groups known aliases into equivalence classes of the same command.
- **F7 File organisation:** The append-only history log is kept in a sequential file, alias records in a direct-access file hashed by name, with collision resolution by the linear quotient method.
- **F8 B+ tree index:** The history file keeps a secondary B+ tree index keyed by working directory, for "history filtered by folder" queries.
- **F9 Extendible hashing / external sort:** The growing alias and command index, as a user's shell configuration accumulates entries over years, is managed with extendible hashing.

Extension: Syncing history across machines with conflict resolution for merged entries.

076–100 · Science, health and bioinformatics

076 — DNA Sequence Alignment Tool



Summary: A console application for a molecular biology student to compare a DNA sequence in hand against reference sequences and find the closest match along with likely mutation sites. It works over about 10,000 synthetic sequence records, such as G0143.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Sequences loaded into a workspace are kept in a doubly linked list for comparison; a set of reference sequences cycled through during batch alignment is traversed with a circular list.
- **V2 Sparse matrix:** In the sequence-id × position table, only positions that actually differ from the reference, i.e. mutated sites, are filled, holding the base substitution.
- **V3 Stack and queue:** Manual gap insertions or removals during alignment editing are undone with a stack; sequences submitted for batch comparison wait in a queue to be aligned against a reference.
- **V4 Tree and heap:** Sequences are grouped into a binary guide tree by similarity, traversed to list families; candidate alignments in a multiple-alignment search are ranked via a heap by score.
- **V5 Graph and BFS/DFS:** Sequences are nodes, pairs above a similarity threshold are edges; BFS finds every sequence within N similarity hops of a query sequence, while DFS finds the connected cluster forming a putative gene family.
- **V6 Search and hashing:** A sequence id, such as "G0143", maps to a sequence record in a hash table; a length-sorted sequence array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree links all sequences into a minimal similarity-based tree sketch; Dijkstra finds the path with the fewest mutation steps between two sequences via known intermediate variants.
- **F3 Sorting:** Sequences are sorted by length, GC content, and similarity score with three algorithms, comparing running times over about 10,000 synthetic records.
- **F4 BST and AVL:** Sequences are kept in an AVL tree keyed by GC-content percentage; the range query "sequences with 40–45% GC content" is answered on the balanced tree.
- **F5 String algorithms:** KMP/Boyer-Moore searches a sequence for a motif substring; edit distance and LCS compute the actual pairwise alignment score between two sequences.
- **F6 Trie and disjoint sets:** Short substrings, or k-mers, are indexed in a trie for fast motif lookup; union-find clusters sequences into gene families as pairwise similarity edges are added.
- **F7 File organisation:** Imported raw sequence batches are kept in a sequential file, sequence records in a direct-access file hashed by id, with collision resolution by Brent's method.
- **F8 B+ tree index:** The sequence file keeps a secondary B+ tree index keyed by synthetic organism tag, speeding up browsing by organism group.
- **F9 Extendible hashing / external sort:** A genome-scale k-mer table too large for memory is sorted with an external merge sort before the index is built.

Extension: Visualizing a multiple sequence alignment together with a consensus sequence.

077 — ✖ Protein Interaction Network Explorer



Summary: A console application for a researcher to explore which other proteins a given protein interacts with and which functional cluster those interactions belong to. It works over about 6,000 synthetic protein records and their interaction links.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A protein's interaction partners are kept in a doubly linked list, forming an adjacency list; a pathway that forms a feedback loop is traversed with a circular list.
- **V2 Sparse matrix:** In the protein $\times$ protein interaction table, only cells for pairs that actually interact are filled, holding the interaction confidence score.
- **V3 Stack and queue:** A curator's manual removal or addition of an interaction is undone with a stack; newly submitted interactions wait in a queue for validation.
- **V4 Tree and heap:** Proteins are kept in a binary tree by function family with traversal for an alphabetic listing; hub proteins are ranked via a heap-based priority queue by number of interactions for a top-hub report.
- **V5 Graph and BFS/DFS:** Proteins are nodes, interactions are edges; BFS finds every protein within N interaction hops of a query protein, its functional neighborhood, while DFS finds the connected component forming a separate interaction module or complex.
- **V6 Search and hashing:** A protein id maps to a protein record in a hash table; a hub-score-sorted protein array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the strongest-confidence interaction path between two proteins; strongly connected components detect feedback-regulation clusters in a directed subset of the interaction network.
- **F3 Sorting:** Proteins are sorted by interaction count, alphabetically, and by function tag with three algorithms, comparing running times over about 6,000 synthetic records.
- **F4 BST and AVL:** Proteins are kept in an AVL tree keyed by hub score; the query "next protein with hub score above X" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches function-tag or annotation text a researcher enters; edit distance matches a misspelled protein name to the closest known id.
- **F6 Trie and disjoint sets:** Protein names and ids are autocompleted with a trie during search; union-find groups proteins into interaction complexes as edges are added.
- **F7 File organisation:** The interaction-submission history is kept in a sequential file, protein records in a direct-access file hashed by id, with collision resolution by progressive overflow.
- **F8 B+ tree index:** The protein file keeps a secondary B+ tree index keyed by function tag, speeding up browsing by function family.
- **F9 Extendible hashing / external sort:** The growing protein-id index, as new synthetic proteins are continuously added, is managed with extendible hashing.

Extension: Overlaying pathway diagrams that highlight the shortest interaction path between two selected proteins.

078 — 🦠 Epidemic Spread Simulator (Contact Network)



Summary: A console application for a public health analyst to simulate how an epidemic would spread through a synthetic community based on daily contact records. It works over about 15,000 synthetic individuals and their daily contact network.

Midterm — C (weeks 1–6):

- **V1 Linked list:** An individual's daily contacts are kept in a doubly linked list, added and removed as contacts change; a recurring 7-day household contact cycle is represented with a circular list.
- **V2 Sparse matrix:** In the person $\times$ day table, only cells where status actually changed are filled, holding the new infection status recorded that day.
- **V3 Stack and queue:** Rolling back simulation parameters to rerun a scenario is done with a stack; individuals wait in a queue to be tested or processed each simulation day.
- **V4 Tree and heap:** Individuals are kept in a binary tree by household with traversal for a household listing; hospital-admission urgency is ranked via a heap-based priority queue by symptom severity.
- **V5 Graph and BFS/DFS:** Individuals are nodes, contacts are edges; BFS simulates the infection spreading wave by wave, day by day, from patient zero, while DFS finds the whole outbreak component reachable through any chain of contacts.
- **V6 Search and hashing:** A person id maps to an individual record in a hash table; an infection-day-sorted array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Connected components identify isolated clusters that are effectively quarantined; Dijkstra, weighted by contact risk, finds the most likely transmission chain between two cases.
- **F3 Sorting:** Individuals are sorted by infection day, symptom severity, and contact count with three algorithms, comparing running times over about 15,000 synthetic records.
- **F4 BST and AVL:** Individuals are kept in an AVL tree keyed by infection day; the range query "all new cases after day X" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches free-text symptom notes entered by contact tracers for a keyword; edit distance corrects misspelled street or location names logged during interviews.
- **F6 Trie and disjoint sets:** Location names are autocompleted with a trie during contact-tracing entry; union-find builds the core outbreak cluster as contact edges are added day by day.
- **F7 File organisation:** The daily simulation log is kept in a sequential file, individual records in a direct-access file hashed by person id, with collision resolution by the linear quotient method.
- **F8 B+ tree index:** The individual file keeps a secondary B+ tree index keyed by household id, for fast household-based lookup.
- **F9 Extendible hashing / external sort:** The full multi-month contact log, too large for memory, is sorted by day with an external merge sort before generating reports.

Extension: Comparing what-if scenarios with adjustable contact-reduction, or lockdown, parameters.

079 — 🏥 Emergency Room Triage Queue



Summary: A console application for a triage nurse to rank arriving emergency-room patients by urgency and route them to the right room. It works over about 20,000 synthetic patient records and one shift's waiting queue.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Patients currently waiting are kept in a doubly linked list, inserted and removed as they arrive or are treated; the on-call doctor roster rotates through a circular list.
- **V2 Sparse matrix:** In the patient × symptom-checklist table, only cells for symptoms the patient actually reported are filled, holding a severity flag.
- **V3 Stack and queue:** A triage nurse's re-categorization of a patient is undone with a stack; patients at the same severity level wait in an actual FIFO queue in the waiting room.
- **V4 Tree and heap:** Treatment-room assignments are kept in a binary tree with traversal for a room-status listing; triage itself runs on a heap-based priority queue, dequeuing the most severe patient first.
- **V5 Graph and BFS/DFS:** Hospital rooms and stations are nodes, corridors are edges; BFS finds the nearest free bed within N corridor hops of the triage desk, while DFS finds every room reachable during an evacuation-route check.
- **V6 Search and hashing:** A patient id maps to a patient record in a hash table; an arrival-time-sorted array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by corridor distance, routes a patient's stretcher to the nearest available specialist room; max flow computes the maximum patient throughput from the triage desk to treatment rooms under bed-capacity constraints.
- **F3 Sorting:** Patients are sorted by severity score, arrival time, and wait duration with three algorithms, comparing running times over about 20,000 synthetic visit records.
- **F4 BST and AVL:** Patients are kept in an AVL tree keyed by arrival time; the range query "next patient after time T" is answered on the balanced tree for shift-report audits.
- **F5 String algorithms:** KMP searches free-text nurse notes for a symptom keyword; edit distance corrects misspelled medication names entered during intake.
- **F6 Trie and disjoint sets:** Medication and allergy names are autocompleted with a trie during intake; union-find groups patients into the same incident, such as a multi-victim accident, as they are linked during intake.
- **F7 File organisation:** The shift's admission log is kept in a sequential file, patient records in a direct-access file hashed by id, with collision resolution by Brent's method.
- **F8 B+ tree index:** The patient file keeps a secondary B+ tree index keyed by severity level, for fast severity-filtered reporting.
- **F9 Extendible hashing / external sort:** The hospital's continuously growing patient-id index is managed with extendible hashing.

Extension: Showing patients a predicted wait time based on the current state of the queue.

080 — Chemical Reaction Pathway Finder

Summary: A console application for a chemistry student to find the lowest-energy reaction pathway from raw materials in hand to a target compound. It works over about 7,000 synthetic compounds and their reaction links.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The steps of a synthesis pathway are kept in an ordered doubly linked list; a catalytic cycle that regenerates its own catalyst is represented with a circular list.
- **V2 Sparse matrix:** In the compound $\times$ reaction-participation table, only cells for compounds that actually appear in that reaction are filled, holding the stoichiometric coefficient.
- **V3 Stack and queue:** A chemist's edits to a proposed synthesis route, trying an alternate step, are undone with a stack; pathway-search requests wait to be processed in a queue.
- **V4 Tree and heap:** Compounds are kept in a binary tree by chemical family with traversal for an alphabetic catalog listing; candidate reaction steps in a best-first pathway search are ranked via a heap by energy cost.
- **V5 Graph and BFS/DFS:** Compounds are nodes, reactions transforming one into another are edges; BFS finds the fewest-step synthesis pathway from a raw material to a target compound, while DFS finds every compound synthesizable from a given starting set of reagents.
- **V6 Search and hashing:** A compound name maps to a compound record in a hash table; a molecular-weight-sorted array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the lowest-energy-cost synthesis route between two compounds; cycle detection flags unwanted catalytic feedback loops in a proposed reaction network.
- **F3 Sorting:** Compounds are sorted by molecular weight, reaction count, and alphabetically with three algorithms, comparing running times over about 7,000 synthetic records.
- **F4 BST and AVL:** Compounds are kept in an AVL tree keyed by molecular weight; the range query "next compound after weight X" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a compound name a chemist types; edit distance matches a misspelled or misremembered compound name to the closest catalog entry.
- **F6 Trie and disjoint sets:** Compound names are autocompleted with a trie; union-find groups compounds into "synthesizable from the same precursor" sets as reaction edges are added.
- **F7 File organisation:** The reaction-database import log is kept in a sequential file, compound records in a direct-access file hashed by name, with collision resolution by progressive overflow.
- **F8 B+ tree index:** The compound file keeps a secondary B+ tree index keyed by chemical family, speeding up browsing by family.
- **F9 Extendible hashing / external sort:** A large reaction-log dataset too large for memory is sorted by energy cost with an external merge sort for analysis.

Extension: Adding cost-optimized route suggestions that factor in reagent price alongside energy cost.

081 — ☒ Periodic Table and Molecular Formula Parser

Summary: A console application that parses chemical formulas a user types (e.g. $\text{Mg}_3(\text{PO}_4)_2$), looks up each element in a fixed periodic table, and computes molar mass. It runs against the full 118-element table and roughly 5,000 synthetic compound records.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A formula's parsed atom sequence is held in a doubly linked list; ring-shaped organic structures such as benzene are represented with a circular list.
- **V2 Sparse matrix:** In the 118-element by 5,000-compound table, only the cells where an element actually appears (with its atom count) are stored.
- **V3 Stack and queue:** Nested-parenthesis formulas like $\text{Mg}_3(\text{PO}_4)_2$ are parsed with a stack; formulas awaiting batch validation wait in a queue.
- **V4 Tree and heap:** The formula's expression tree is walked with a post-order traversal to compute molar mass; a heap-based leaderboard tracks the heaviest synthesised compounds by priority.
- **V5 Graph and BFS/DFS:** Nodes are elements, edges connect elements that co-occur in the same compound; BFS finds elements bonded to carbon within two steps, DFS finds every element reachable within the same compound family.
- **V6 Search and hashing:** Element symbol maps to element record in a hash table; a molar-mass-sorted array of compounds supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the bonding chain with the fewest intermediate compounds between two elements; cycle detection flags whether a compound contains an aromatic ring.
- **F3 Sorting:** Compounds are sorted by molar mass, atom count and melting point using three different algorithms, and their running times are compared.
- **F4 BST and AVL:** Element records are kept in an AVL tree keyed by atomic number; the query "elements with atomic number between 20 and 30" is answered on the balanced tree.
- **F5 String algorithms:** A substructure pattern the user types, such as "SO4", is searched across compound formulas with KMP; on a typo, edit distance suggests the correct element name.
- **F6 Trie and disjoint sets:** Element and compound names are autocompleted with a trie; union-find groups elements that react together into the same chemical family.
- **F7 File organisation:** The reaction log is kept in a sequential file; compound records live in a direct-access file hashed by formula (progressive overflow for collisions).
- **F8 B+ tree index:** The compound file carries a secondary B+ tree index keyed by molar mass, speeding up queries like "compounds between 50 and 100 g/mol".
- **F9 Extendible hashing / external sort:** The compound database file, which keeps growing as new synthetic entries are added, is scaled up with extendible hashing.

Extension: Adding a module that automatically balances an entered reaction equation stoichiometrically.

082 — Star Catalogue and Constellation Explorer



Summary: A console application that shows a stargazer constellation lines and star positions, built on a catalogue of about 20,000 synthetic stars across 90 constellations.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A constellation's stars are kept in brightness order in a doubly linked list; the circular path traced by stars orbiting the celestial pole is modelled with a circular list.
- **V2 Sparse matrix:** In the right-ascension by declination sky-grid table, only cells containing a star are filled; most cells stay empty.
- **V3 Stack and queue:** A user's added or removed constellation lines are undone with a stack; newly logged observations wait in a classification queue.
- **V4 Tree and heap:** Stars are kept in a binary tree ordered by right ascension and an in-order traversal produces a sky sweep; a heap-based priority queue points the telescope at the brightest star first.
- **V5 Graph and BFS/DFS:** Nodes are stars, edges are the traditional figure-lines of a constellation; BFS finds the fewest-hop chain of lines between two stars, DFS finds every star connected to the same figure.
- **V6 Search and hashing:** Catalogue code maps to star record in a hash table; a magnitude-sorted array supports binary search for "brightest N stars" queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal builds the shortest-total-length stick figure connecting all of a constellation's stars; Dijkstra computes the shortest angular-distance hopping route between two stars.
- **F3 Sorting:** Stars are sorted by magnitude, right ascension and distance in parsecs using three algorithms, and running times are compared.
- **F4 BST and AVL:** Star records are kept in an AVL tree keyed by right ascension; the query "stars with right ascension between X and Y" answers a sky-chart window.
- **F5 String algorithms:** A constellation name the user types is searched with KMP; on a typo like "Orion", edit distance suggests "Orion".
- **F6 Trie and disjoint sets:** Star and constellation names are autocompleted with a trie; union-find groups stars into candidate constellation clusters as proximity links are added.
- **F7 File organisation:** The nightly observation log is kept in a sequential file; star records live in a direct-access file hashed by catalogue code (linear quotient for collisions).
- **F8 B+ tree index:** The star file carries a secondary B+ tree index keyed by magnitude, speeding up "all stars brighter than magnitude X" queries.
- **F9 Extendible hashing / external sort:** Too many observation-session files to fit in memory are merged chronologically by right ascension with an external merge sort.

Extension: Computing rise and set times for a given synthetic observer location and date.

083 — 🌐 Earthquake Records Analysis and Alert System



Summary: A console application that analyses 50,000 synthetic earthquake records from about 300 monitoring stations by magnitude, depth and region, detects aftershock clusters, and raises alerts.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A mainshock's aftershocks are kept in chronological order in a doubly linked list; the ring of stations surrounding a caldera is modelled with a circular list.
- **V2 Sparse matrix:** In the latitude-band by longitude-band table, only cells with recorded events are filled; most regions stay quiet.
- **V3 Stack and queue:** An analyst's manual reclassification of an event is undone with a stack; incoming sensor readings wait to be processed in a queue.
- **V4 Tree and heap:** A severity-classification decision tree is walked to produce an alert-level report; pending alerts sit in a heap-based priority queue ordered by magnitude, most severe first.
- **V5 Graph and BFS/DFS:** Nodes are stations, edges connect station pairs that recorded correlated tremors in the same time window; BFS finds the ring of stations that would rapidly feel an event, DFS finds every station within a contiguous fault zone.
- **V6 Search and hashing:** Station ID maps to station record in a hash table; a magnitude-sorted event array supports binary search for "strongest N events".

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the signal-propagation time from the epicentre station to the farthest station; Kruskal computes the cheapest network wiring connecting all stations to a hub.
- **F3 Sorting:** Events are sorted by magnitude, depth and timestamp using three algorithms, and running times over 50,000 records are compared.
- **F4 BST and AVL:** Event records are kept in an AVL tree keyed by timestamp; the query "events between T1 and T2" answers a report for a specific region.
- **F5 String algorithms:** A region name the user types is searched with KMP; on a typo, edit distance suggests the closest region name.
- **F6 Trie and disjoint sets:** Region names are autocompleted with a trie; union-find groups stations into connected fault zones as correlation links are added.
- **F7 File organisation:** The raw sensor log is kept in a sequential file; event records live in a direct-access file hashed by event ID (Brent's method for collisions).
- **F8 B+ tree index:** The event file carries a secondary B+ tree index keyed by magnitude, speeding up "magnitude above X" alert queries.
- **F9 Extendible hashing / external sort:** Years of accumulated event logs, too large for memory, are merged into chronological order with an external merge sort.

Extension: A real-time-style feed simulation with configurable alert thresholds per region.

084 — ⚡ Storm Track Prediction Grid



Summary: A console application that tracks thousands of synthetic hurricanes across 40 storm seasons on a latitude/longitude grid and predicts a storm's likely path and landfall probability.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A storm's track points are kept in chronological order in a doubly linked list; the sensor buoys ringing the eye wall are modelled with a circular list.
- **V2 Sparse matrix:** In the latitude-band by longitude-band grid, only cells a storm has passed through are filled; most cells never see a storm.
- **V3 Stack and queue:** A forecaster's manual track-point corrections are undone with a stack; incoming buoy and satellite readings wait to be merged into the track in a queue.
- **V4 Tree and heap:** A season's track points are kept in a binary tree ordered by timestamp and an in-order traversal produces the chronological path; evacuation alerts sit in a heap-based priority queue ordered by predicted landfall time, soonest first.
- **V5 Graph and BFS/DFS:** Nodes are grid cells, edges connect cells a storm could move to next given its speed; BFS finds every cell reachable within N hours, DFS finds the contiguous coastal region under threat.
- **V6 Search and hashing:** Storm code maps to storm record in a hash table; a max-wind-speed-sorted array supports binary search for "strongest N storms".

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal computes the minimal-cable buoy sensor network; cycle detection catches a storm looping back on its own track, as in the Fujiwhara effect.
- **F3 Sorting:** Storms are sorted by peak wind speed, minimum pressure and duration using three algorithms, and running times are compared.
- **F4 BST and AVL:** Storm records are kept in an AVL tree keyed by formation date; the query "storms formed between date X and Y" answers a season report.
- **F5 String algorithms:** A storm name the user types is searched with KMP; on a typo, edit distance suggests the closest storm name.
- **F6 Trie and disjoint sets:** Storm names are autocompleted with a trie; union-find groups grid cells into "high-risk zone" clusters as historical tracks are merged in.
- **F7 File organisation:** The raw track-point log is kept in a sequential file; storm summary records live in a direct-access file hashed by storm code (linear quotient for collisions).
- **F8 B+ tree index:** The storm file carries a secondary B+ tree index keyed by peak wind speed, speeding up category-based lookups such as "Category 3 and above".
- **F9 Extendible hashing / external sort:** 40 years of track-point logs, too large for memory, are merged by date with an external merge sort for climatology analysis.

Extension: Showing a probabilistic cone of uncertainty built from several simulated tracks.

085 — Ant Colony Foraging Simulator



Summary: A console application that simulates about 5,000 synthetic ants laying pheromone trails on a grid to find the shortest paths to food sources.

Midterm — C (weeks 1–6):

- **V1 Linked list:** An ant's steps are kept in a doubly linked list for backtracking; the ring of ants that forms around a food source is modelled with a circular list.
- **V2 Sparse matrix:** In the grid-cell by pheromone-strength table, only cells that have received pheromone are filled; most cells stay at zero.
- **V3 Stack and queue:** An ant backing out of a dead-end tunnel is handled with a stack; ants waiting at a narrow tunnel entrance are simulated with a queue.
- **V4 Tree and heap:** A direction-choice decision tree at each junction is walked to log an ant's route choice; a heap-based priority queue ranks food sources by attractiveness so the colony favours the strongest one.
- **V5 Graph and BFS/DFS:** Nodes are grid cells, edges are passable tunnels; BFS finds the fewest-hop path from the nest to a food source, DFS finds the whole explored tunnel region reachable from the nest.
- **V6 Search and hashing:** Ant ID maps to ant record in a hash table; a remaining-quantity-sorted array of food sources supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the preferred foraging route weighted inversely by pheromone strength; Kruskal computes the shortest tunnel network connecting every discovered food source back to the nest.
- **F3 Sorting:** Food sources are sorted by distance from the nest, remaining quantity and discovery time using three algorithms, and running times are compared.
- **F4 BST and AVL:** Trail-segment records are kept in an AVL tree keyed by pheromone strength; the query "segments above strength X" answers a visualisation request.
- **F5 String algorithms:** A direction-command sequence such as "NNESS" typed to manually steer a scout ant is matched against valid patterns with KMP; edit distance compares two ants' foraging paths, encoded as direction strings, to detect converging routes.
- **F6 Trie and disjoint sets:** Nest and colony location names are autocompleted with a trie; union-find groups grid cells into connected tunnel-network components as tunnels are dug and discovered.
- **F7 File organisation:** The simulation-tick log is kept in a sequential file; ant records live in a direct-access file hashed by ant ID (linear quotient for collisions).
- **F8 B+ tree index:** The trail-segment file carries a secondary B+ tree index keyed by pheromone strength, speeding up "strongest trails" queries.
- **F9 Extendible hashing / external sort:** The ant record file, which keeps growing as simulation runs accumulate, is scaled up with extendible hashing.

Extension: Adding multiple competing colonies whose pheromone trails interfere with each other.

086 — 📌 Forest Fire Spread Model



Summary: A console application that simulates fire spreading across a synthetic forest grid of about 100,000 cells based on wind, moisture and terrain, tracking burn progress and firebreak effectiveness.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The fire front's cells are kept in ignition order in a doubly linked list; a circular containment line around a controlled burn is modelled with a circular list.
- **V2 Sparse matrix:** In the grid-row by grid-column table, only cells with a moisture sensor are filled; across a large forest most cells stay unmeasured.
- **V3 Stack and queue:** A firefighter's manual firebreak placement is undone with a stack; crews waiting to be dispatched to active fire fronts sit in a queue.
- **V4 Tree and heap:** A zone risk-classification decision tree is walked to produce a risk report; a heap-based priority queue dispatches crews to the cell with the most urgent spread first.
- **V5 Graph and BFS/DFS:** Nodes are grid cells, edges connect adjacent flammable cells; BFS finds every cell the fire reaches within N time steps, DFS finds the total burnable area reachable from the ignition point.
- **V6 Search and hashing:** Zone ID maps to zone record in a hash table; a risk-score-sorted array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal computes the shortest hose/firebreak network connecting all fire stations; cycle detection verifies that a set of firebreak lines fully encloses a burn area without gaps.
- **F3 Sorting:** Zones are sorted by risk score, fuel density and elevation using three algorithms, and running times over 100,000 records are compared.
- **F4 BST and AVL:** Zone records are kept in an AVL tree keyed by risk score; the query "zones with risk score between X and Y" answers a patrol-route priority request.
- **F5 String algorithms:** A vegetation-type term the user types is searched with KMP; on a typo, edit distance suggests the closest zone name.
- **F6 Trie and disjoint sets:** Zone and station names are autocompleted with a trie; union-find groups burning cells into connected fire-front components as the fire spreads, and firebreaks split components apart.
- **F7 File organisation:** The tick-by-tick burn progression log is kept in a sequential file; zone records live in a direct-access file hashed by zone ID (Brent's method for collisions).
- **F8 B+ tree index:** The zone file carries a secondary B+ tree index keyed by elevation, speeding up terrain-based queries.
- **F9 Extendible hashing / external sort:** Logs from multiple simulation runs, too large for memory, are merged by burn time with an external merge sort for post-event analysis.

Extension: Adding mid-simulation wind-direction changes that redirect the fire front.

087 — Cell Lineage Tree Tracker



Summary: A console application that logs division events for about 20,000 synthetic cells tracked under a simulated microscope, builds a multi-generation lineage tree, and flags abnormal division patterns.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A cell's division events are kept in chronological order in a doubly linked list; the G1-S-G2-M cell-cycle phases are modelled with a circular list.
- **V2 Sparse matrix:** In the cell-ID by observation-time-window table, only windows where that cell was actually imaged are filled.
- **V3 Stack and queue:** A researcher's relabelling of a misidentified cell is undone with a stack; newly imaged cells wait to be classified and added to the tree in a queue.
- **V4 Tree and heap:** The lineage tree, naturally binary since each cell divides into two daughters, is walked level-order to count cells per generation; a heap-based priority queue flags cells for review, most abnormal first.
- **V5 Graph and BFS/DFS:** Nodes are cells, edges are parent-daughter division links; BFS finds every descendant within N generations of a given ancestor, DFS finds a cell's whole descendant subtree, or checks whether two cells share a common lineage.
- **V6 Search and hashing:** Cell ID maps to cell record in a hash table; a generation-sorted array of cells supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders division events from overlapping lineages into a valid chronological sequence; cycle detection catches a data-entry error where a cell is wrongly recorded as descending from its own descendant.
- **F3 Sorting:** Cells are sorted by division time, generation number and abnormality score using three algorithms, and running times over 20,000 records are compared.
- **F4 BST and AVL:** Cell records are kept in an AVL tree keyed by division timestamp; the query "cells that divided between T1 and T2" answers a time-window report.
- **F5 String algorithms:** A marker-protein label the user types is searched with KMP; on a typo, edit distance suggests the closest cell-line name.
- **F6 Trie and disjoint sets:** Cell-line and marker names are autocompleted with a trie; union-find groups cells into connected lineage clusters as parent-child links are added, quickly answering whether two cells are related.
- **F7 File organisation:** The imaging-session log is kept in a sequential file; cell records live in a direct-access file hashed by cell ID (progressive overflow for collisions).
- **F8 B+ tree index:** The cell file carries a secondary B+ tree index keyed by generation number, speeding up "all generation-N cells" queries.
- **F9 Extendible hashing / external sort:** The cell record file, which keeps growing as new imaging sessions add tracked cells, is scaled up with extendible hashing.

Extension: Statistically detecting division-rate anomalies that suggest abnormal proliferation.

088 — Drug Interaction Checker



Summary: A console application that cross-checks a synthetic patient's prescription list against an interaction database of about 3,000 drug records and flags dangerous combinations.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A patient's prescription history of start and stop events is kept in chronological order in a doubly linked list; a recurring weekly dosage schedule is modelled with a circular list.
- **V2 Sparse matrix:** In the drug by drug interaction-severity table, only pairs with a known interaction are filled; most drug pairs have no recorded interaction.
- **V3 Stack and queue:** A pharmacist's manual override of a flagged interaction is undone with a stack; new prescriptions wait to be checked against the interaction database in a queue.
- **V4 Tree and heap:** A drug-class decision tree is walked to produce a classification report; a heap-based priority queue orders flagged interactions by severity so the pharmacist reviews the most dangerous one first.
- **V5 Graph and BFS/DFS:** Nodes are drugs, edges are known interactions; BFS finds every drug indirectly interacting with a given drug within N steps, DFS finds the connected set of interacting drugs across a patient's whole prescription list.
- **V6 Search and hashing:** Drug name maps to drug record in a hash table; a generic-name-sorted array supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components identify tightly interacting drug clusters that need joint review; Dijkstra finds the fewest-intermediary path between two drugs to explain an indirect interaction warning.
- **F3 Sorting:** A patient's prescriptions are sorted by risk score, start date and dosage frequency using three algorithms, and running times are compared.
- **F4 BST and AVL:** Drug records are kept in an AVL tree keyed by drug name; the query "the next drug after a given name" answers a formulary lookup.
- **F5 String algorithms:** A generic or brand drug name the pharmacist types is searched with KMP; edit distance suggests the closest name to prevent confusion between sound-alike drugs.
- **F6 Trie and disjoint sets:** Drug names are autocompleted with a trie as the pharmacist types; union-find groups drugs into connected "interaction families" as new interaction pairs are added to the database.
- **F7 File organisation:** The dispensing-event log is kept in a sequential file; drug records live in a direct-access file hashed by drug code (linear quotient for collisions).
- **F8 B+ tree index:** The prescription file carries a secondary B+ tree index keyed by patient ID, speeding up "all prescriptions for patient X" lookups.
- **F9 Extendible hashing / external sort:** Dispensing logs from multiple pharmacy branches, too large for memory, are merged by date with an external merge sort for a regional audit.

Extension: Suggesting dosage adjustments based on a synthetic patient's age and weight profile.

089 — ECG Signal Peak Detector



Summary: A console application that processes a synthetic ECG recording of about 500,000 samples, detects QRS peaks (heartbeats), computes heart-rate variability, and flags irregular rhythms.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each detected beat, with its timestamp, peak amplitude and RR interval, is appended to a doubly linked list as the signal is scanned; a rolling window of the most recent N beats is kept in a circular list.
- **V2 Sparse matrix:** In the ECG-lead by time-window table, only windows flagged with an anomaly are filled; most lead-window pairs stay normal.
- **V3 Stack and queue:** Incoming raw samples wait to be filtered in a queue; a technician marking and unmarking a misdetected peak is undone with a stack.
- **V4 Tree and heap:** A beat-morphology decision tree (normal, premature, etc.) is walked to tally beat-type counts; a heap-based priority queue ranks close candidate peaks in a detection window by amplitude to pick the true peak.
- **V5 Graph and BFS/DFS:** Nodes are detected beats, edges connect consecutive beats with similar RR intervals; BFS measures how many consecutive beats form one arrhythmia episode, DFS finds every beat across the recording that falls into the same rhythm pattern.
- **V6 Search and hashing:** Recording session ID maps to session record in a hash table; a timestamp-sorted array of beats supports binary search for "the beat nearest time T".

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the fewest-step reclassification path across a rhythm-transition graph to explain a diagnosis; cycle detection catches a repeating, self-reinforcing abnormal (reentrant) rhythm pattern.
- **F3 Sorting:** Detected beats are sorted by RR interval, amplitude and abnormality score using three algorithms, and running times over a long recording are compared.
- **F4 BST and AVL:** Beat records are kept in an AVL tree keyed by timestamp; the query "beats between T1 and T2" answers a request to export a specific episode.
- **F5 String algorithms:** Free-text clinician annotations attached to a recording are searched for terms like "irregular" with KMP; edit distance matches a mistyped patient or session ID, or a beat-type abbreviation, against the canonical dictionary.
- **F6 Trie and disjoint sets:** Annotation labels and rhythm-type names are autocompleted with a trie as a clinician types notes; union-find merges consecutive similar beats into connected arrhythmia-episode clusters.
- **F7 File organisation:** The raw sample stream is kept in a sequential, append-only file; per-recording summary records live in a direct-access file hashed by session ID (progressive overflow for collisions).
- **F8 B+ tree index:** The beat file carries a secondary B+ tree index keyed by abnormality score, speeding up "most abnormal beats" queries across a long recording.
- **F9 Extendible hashing / external sort:** A recording of 500,000-plus samples is too large for memory, so it is processed in chunks and merged chronologically with an external merge sort.

Extension: Adding multi-lead correlation to reduce false-positive peak detection caused by motion artefacts.

090 — Blood Bank Stock and Compatibility Matcher



Summary: A console application that matches about 10,000 synthetic blood units spread across roughly 50 branches to patient requests, based on blood-type compatibility rules and expiry dates.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A unit's status history — collected, tested, stored, issued — is kept in a doubly linked list; units on a rotating cold-storage rack are modelled with a circular list.
- **V2 Sparse matrix:** In the branch by blood-type table, only cells where that branch stocks that type are filled; small branches leave most rare-type cells empty.
- **V3 Stack and queue:** A technician's correction of a mislabelled blood type is undone with a stack; transfusion requests waiting to be matched sit in a queue.
- **V4 Tree and heap:** An ABO/Rh compatibility decision tree is walked to determine which types a patient can receive; a heap-based priority queue orders units by expiry date so the soonest-to-expire unit is issued first.
- **V5 Graph and BFS/DFS:** Nodes are (branch, blood type) pairs, edges connect compatible types within a branch and matching types across branches via transfer routes; BFS finds the nearest branch, by transfer hops, that can fill a rare request, DFS finds every branch reachable through the transfer network.
- **V6 Search and hashing:** Unit barcode maps to unit record in a hash table; an expiry-date-sorted array of units supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the fastest transfer route from a branch with surplus stock to one facing a shortage; Kruskal computes the lowest-cost regional transfer backbone connecting all branches.
- **F3 Sorting:** Units are sorted by expiry date, collection date and donor age using three algorithms, and running times over 10,000 records are compared.
- **F4 BST and AVL:** Unit records are kept in an AVL tree keyed by expiry date; the query "units expiring within the next N days" answers a spoilage-prevention report.
- **F5 String algorithms:** A donor name/ID or branch name staff type is searched with KMP; on a typo, edit distance suggests the closest branch name.
- **F6 Trie and disjoint sets:** Donor and branch names are autocompleted with a trie; union-find groups branches into connected transfer-network regions as routes are added or removed.
- **F7 File organisation:** The donation-event log is kept in a sequential file; unit records live in a direct-access file hashed by barcode (linear quotient for collisions).
- **F8 B+ tree index:** The unit file carries a secondary B+ tree index keyed by blood type, speeding up "all available O-negative units" queries.
- **F9 Extendible hashing / external sort:** The unit record file, which keeps growing as donations accumulate across branches, is scaled up with extendible hashing.

Extension: Adding predictive shortage alerts based on historical seasonal donation and demand patterns.

091 — Neural Connectome Explorer



Summary: A console application that lets a neuroscience researcher explore a synthetic connectome, tracing signal paths and circuits. It works on roughly 5,000 neurons and 60,000 synapses spread across 20 brain regions, including signal delays and recurring circuits.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each neuron's spike train is kept as a doubly linked list ordered by time; neurons in a feedback loop are tracked with a circular list.
- **V2 Sparse matrix:** In the neuron-by-neuron synaptic weight table, only connected cells are stored, each holding a synaptic strength value.
- **V3 Stack and queue:** A stack undoes the researcher's synapse-pruning edits; a queue buffers a spike wave as it propagates through the network.
- **V4 Tree and heap:** A binary tree holds the brain-region hierarchy, traversed to list every neuron under a region; a heap keeps the next neuron to simulate ordered by firing strength.
- **V5 Graph and BFS/DFS:** Neurons are nodes and synapses are edges; BFS finds the fewest-hop signal path from a sensory neuron to a motor neuron, DFS finds every neuron reachable within one circuit.
- **V6 Search and hashing:** Neuron ID maps to neuron record in a hash table; sorted synapse conduction delays support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest signal path weighted by transmission delay; cycle detection finds feedback loops (recurrent circuits) in the network.
- **F3 Sorting:** Neurons are sorted by firing rate, region size, and synapse count using three algorithms whose running times are compared.
- **F4 BST and AVL:** Neurons are kept in an AVL tree keyed by activation threshold; the query "neurons with threshold between X and Y" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a neuron tag the researcher types; edit distance suggests the closest matching region name on a typo.
- **F6 Trie and disjoint sets:** Neuron tags are autocompleted with a trie; union-find groups neurons into connected-circuit clusters.
- **F7 File organisation:** Spike event logs are kept in a sequential file, neuron cards in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** The synapse file keeps a secondary B+ tree index keyed by target neuron ID, speeding up "who connects to this neuron" queries.
- **F9 Extendible hashing / external sort:** Millions of spike-timestamp records, too large for memory, are sorted by timestamp with an external merge sort.

Extension: Real-time, color-coded visualization of activation cascades as they spread.

092 — 🌱 Plant Genetics Crossbreeding Planner



Summary: A console application that helps a plant breeder plan crosses between parent varieties and predict offspring traits. It works on about 800 synthetic plant varieties and 15,000 crossbreeding records, tracking each variety's pedigree and trait scores.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A variety's ancestor lineage is kept as a doubly linked list; varieties in a closed recurrent-selection cycle are tracked with a circular list.
- **V2 Sparse matrix:** In the variety-by-trait table, most varieties lack most rare traits, so only filled cells are stored, each holding an expression score.
- **V3 Stack and queue:** A stack undoes the breeder's crossbreeding-pair edits; a queue holds seed batches waiting to be planted.
- **V4 Tree and heap:** A binary tree holds the pedigree, traversed to compute an inbreeding coefficient; a heap orders the planting queue by expected yield score.
- **V5 Graph and BFS/DFS:** Varieties are nodes and crossbreeding pairs are edges; BFS finds the fewest-generation breeding path that combines two target traits, DFS finds every descendant of a founder variety.
- **V6 Search and hashing:** Variety code maps to variety record in a hash table; sorted maturation-day values support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection prevents inbreeding loops; Dijkstra computes the least-genetic-drift breeding path weighted by genetic distance.
- **F3 Sorting:** Varieties are sorted by yield, disease resistance, and days to maturity using three algorithms whose running times are compared.
- **F4 BST and AVL:** Varieties are kept in an AVL tree keyed by yield score; the query "varieties with yield between X and Y" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a trait description the breeder types; edit distance suggests the closest matching variety name on a typo.
- **F6 Trie and disjoint sets:** Variety names are autocompleted with a trie; union-find groups varieties into breeding-lineage families.
- **F7 File organisation:** Crossbreeding event logs are kept in a sequential file, variety cards in a direct-access file (collision resolution with the linear quotient method).
- **F8 B+ tree index:** The trait file keeps a secondary B+ tree index keyed by trait ID, for "which varieties carry trait X" lookups.
- **F9 Extendible hashing / external sort:** Millions of germination-test records, too large for memory, are sorted by test date with an external merge sort.

Extension: Simulating multi-generation trait-inheritance probabilities using Mendelian rules.

093 — ➡ Aquarium Food Web Simulator



Summary: A console application that models a closed aquarium ecosystem's food web, tracking who eats whom and simulating population shifts. It works on about 150 synthetic species, 600 feeding relationships, and 50 tanks, accounting for oxygen and nutrient balance.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each species' list of prey is kept as a doubly linked list; species in a nutrient-recycling loop are tracked with a circular list.
- **V2 Sparse matrix:** In the tank-by-species population table, most tanks do not hold most species, so only filled cells are stored.
- **V3 Stack and queue:** A stack undoes the aquarist's feeding-schedule edits; a queue manages the order in which tanks receive food.
- **V4 Tree and heap:** A binary tree holds the species classification hierarchy, traversed to list every species at a trophic level; a heap orders emergency interventions by population-decline severity.
- **V5 Graph and BFS/DFS:** Species are nodes and predator-prey links are edges; BFS finds the shortest feeding-chain length from a producer to an apex predator, DFS finds every species reachable within one food-web cluster.
- **V6 Search and hashing:** Species name maps to species record in a hash table; sorted population counts support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds nutrient-recycling loops; topological sort orders trophic levels from producers to apex predators for feeding sequencing.
- **F3 Sorting:** Species are sorted by population, biomass, and oxygen consumption using three algorithms whose running times are compared.
- **F4 BST and AVL:** Species are kept in an AVL tree keyed by population count; the query "species with population between X and Y" (endangered species) is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a species name the aquarist types; edit distance suggests the closest matching name on a typo.
- **F6 Trie and disjoint sets:** Species names are autocompleted with a trie; union-find groups species into connected food-web components per tank.
- **F7 File organisation:** Water-quality logs are kept in a sequential file, species cards in a direct-access file (collision resolution with Brent's method).
- **F8 B+ tree index:** The feeding-event file keeps a secondary B+ tree index keyed by tank ID, for "what happened in tank X" lookups.
- **F9 Extendible hashing / external sort:** Millions of temperature and pH sensor readings, too large for memory, are sorted by timestamp with an external merge sort.

Extension: An early-warning system that predicts population-collapse cascades before they happen.

094 — Genome k-mer Counter



Summary: A console application that scans synthetic DNA sequences, counts k-length substring (k-mer) frequencies, and helps a bioinformatics student spot repeats and contamination patterns. It works on about 500 synthetic sequences of roughly 2,000 bases each, with k configurable from 4 to 12.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A sequence's k-mer occurrence positions are kept as a doubly linked list; a circular plasmid genome's wrap-around is scanned with a circular list.
- **V2 Sparse matrix:** In the sequence-by-k-mer table, most k-mers are absent from most sequences, so only filled cells are stored, each holding an occurrence count.
- **V3 Stack and queue:** A stack undoes nested reverse-complement edits on a sequence; a queue holds batches of sequences waiting to be processed.
- **V4 Tree and heap:** A binary tree holds quality-score segments, in-order traversed to compute per-window GC content; a min-heap keeps the top-K most frequent k-mers for repeat detection.
- **V5 Graph and BFS/DFS:** Nodes are k-mers and edges are (k-1)-letter overlaps (a de Bruijn graph); BFS finds the shortest assembly path between two k-mers, DFS finds the connected components that correspond to separate contigs.
- **V6 Search and hashing:** A hash table maps each k-mer to its occurrence count; sorted k-mer frequencies support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection flags repeat regions that make assembly ambiguous; strongly connected components highlight k-mer clusters from highly repetitive regions.
- **F3 Sorting:** k-mers are sorted by frequency, sequences by length, and by GC content using three algorithms whose running times are compared.
- **F4 BST and AVL:** k-mers are kept in an AVL tree keyed by occurrence count; the query "k-mers with frequency between X and Y" is answered on the balanced tree.
- **F5 String algorithms:** KMP/Boyer-Moore searches a subsequence the student types within the genome; edit distance/LCS aligns two similar sequences to detect mutations.
- **F6 Trie and disjoint sets:** k-mer prefixes are searched and autocompleted with a trie; union-find groups sequences into sample/family clusters by shared k-mer signatures.
- **F7 File organisation:** Raw read records are kept in a sequential file, k-mer records in a direct-access file keyed by hash (collision resolution with progressive overflow).
- **F8 B+ tree index:** The k-mer file keeps a secondary B+ tree index keyed by frequency, speeding up "most frequent k-mers" range queries.
- **F9 Extendible hashing / external sort:** Millions of k-mer occurrence records, too large for memory, are sorted by k-mer value and counted with an external merge sort.

Extension: Automatically assembling contigs by tracing an Eulerian path through the de Bruijn graph.

095 — Organ Transplant Matching Simulator



Summary: A console application that matches synthetic organ donors to waiting recipients by blood type, tissue compatibility, and urgency within a regional transplant network. It works on about 300 recipients, 120 donors, and 40 hospitals, ranking every decision by an urgency score.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A recipient's waiting-status history is kept as a doubly linked list; the hospital notification list is dialed through with a circular list.
- **V2 Sparse matrix:** In the donor-by-recipient compatibility table, most pairs are incompatible, so only filled cells are stored, each holding a compatibility score.
- **V3 Stack and queue:** A stack undoes the coordinator's manual match overrides; a queue manages the waiting list per organ type.
- **V4 Tree and heap:** A binary tree holds the hospital-region hierarchy, traversed to list recipients under a region; a heap orders organ allocation by urgency score.
- **V5 Graph and BFS/DFS:** Donors, recipients, and hospitals are nodes, compatibility and transport links are edges; BFS finds the fewest-hop delivery chain within the organ's viability window, DFS finds every recipient reachable from a donor within one compatibility cluster.
- **V6 Search and hashing:** Recipient ID maps to recipient record in a hash table; sorted urgency scores support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Max flow maximizes the number of matched donor-recipient pairs across the network; Dijkstra computes the fastest delivery route weighted by transport time.
- **F3 Sorting:** Recipients are sorted by urgency, waiting time, and compatibility score using three algorithms whose running times are compared.
- **F4 BST and AVL:** Recipients are kept in an AVL tree keyed by waiting time; the query "next recipient who has waited more than X days" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a clinical note the coordinator types; edit distance suggests the closest matching hospital name on a typo.
- **F6 Trie and disjoint sets:** Hospital names are autocompleted with a trie; union-find groups donors and recipients into blood-type/tissue compatibility clusters.
- **F7 File organisation:** Match event logs are kept in a sequential file, recipient cards in a direct-access file (collision resolution with the linear quotient method).
- **F8 B+ tree index:** The donor file keeps a secondary B+ tree index keyed by blood type, for "find all type-X donors" lookups.
- **F9 Extendible hashing / external sort:** Millions of historical match-log entries, too large for memory, are sorted by date with an external merge sort.

Extension: Scoring predicted graft survival to refine how matches are ranked.

096 — Bacterial Colony Growth Grid



Summary: A console application that simulates bacterial colony growth on a synthetic petri-dish grid, tracking spread, nutrient depletion, and antibiotic zones. It works on a roughly 200-by-200 grid with 5,000 colony seed points across several synthetic bacterial strains.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A colony's growth-front cells are kept as a doubly linked list; a ring-shaped colony forming around an antibiotic disc is tracked with a circular list.
- **V2 Sparse matrix:** In the grid's row-by-column table, most cells are empty, so only occupied cells are stored, each holding a strain ID and density.
- **V3 Stack and queue:** A stack undoes the last simulated growth step; a queue manages the order lab samples are plated.
- **V4 Tree and heap:** A binary tree holds the strain taxonomy, traversed to list every colony in a strain family; a heap orders colonies by growth rate for resource allocation.
- **V5 Graph and BFS/DFS:** Grid cells are nodes and adjacency defines edges; BFS floods outward to find a colony's boundary after N generations, DFS finds all cells belonging to one colony, separating distinct colonies.
- **V6 Search and hashing:** Strain name maps to strain record in a hash table; sorted colony diameters support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest spread path weighted by nutrient resistance; max flow determines the growth ceiling a colony can reach from a nutrient source.
- **F3 Sorting:** Colonies are sorted by diameter, growth rate, and cell count using three algorithms whose running times are compared.
- **F4 BST and AVL:** Colonies are kept in an AVL tree keyed by growth rate; the query "colonies growing between rate X and Y" (anomalous growth) is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a genetic marker sequence the technician types; edit distance suggests the closest matching strain code on a typo.
- **F6 Trie and disjoint sets:** Strain names are autocompleted with a trie; union-find merges cells into one colony when two growth fronts meet.
- **F7 File organisation:** Per-time-step growth measurements are kept in a sequential file, strain cards in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** The colony file keeps a secondary B+ tree index keyed by antibiotic resistance level, for finding resistant colonies.
- **F9 Extendible hashing / external sort:** Millions of per-cell growth measurements across the grid's time steps, too large for memory, are sorted by time step with an external merge sort.

Extension: Modeling the antibiotic gradient's diffusion to predict the shape of the inhibition zone.

097 — 🌊 River Basin Flood Flow Model



Summary: A console application that models a synthetic river basin — main stem, tributaries, and gauge stations — to simulate rainfall runoff and predict downstream flood arrival. It works on about 80 river segments, 40 gauge stations, and 500 synthetic rainfall events.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A segment's upstream tributaries are kept as a doubly linked list; a rejoining braided-delta channel is represented with a circular list.
- **V2 Sparse matrix:** In the gauge-by-time-step water-level table, most cells are unmeasured, so only filled cells are stored, each holding a water level.
- **V3 Stack and queue:** A stack undoes the hydrologist's manual gauge-reading corrections; a queue dispatches flood-warning notifications to downstream towns in order.
- **V4 Tree and heap:** A binary tree holds the basin hierarchy, post-order traversed to sum catchment area up the tree; a heap orders gauge alerts by flood severity for emergency dispatch.
- **V5 Graph and BFS/DFS:** River segments and confluences are nodes, flow connections are edges; BFS finds the fewest-confluence path from a rainfall source to a downstream town, DFS finds every segment in one connected sub-basin.
- **V6 Search and hashing:** Gauge station ID maps to station record in a hash table; sorted historical peak-flow values support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra predicts flood arrival time downstream, weighted by flow travel time; topological sort orders segments from headwaters to river mouth for flow computation.
- **F3 Sorting:** Segments are sorted by flow rate, catchment area, and elevation drop using three algorithms whose running times are compared.
- **F4 BST and AVL:** Gauge readings are kept in an AVL tree keyed by water level; the query "readings between flood stage X and Y" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a station or location name the hydrologist types; edit distance suggests the closest matching station name on a typo.
- **F6 Trie and disjoint sets:** Station names are autocompleted with a trie; union-find groups river segments into connected sub-basins (watershed delineation).
- **F7 File organisation:** Rainfall event logs are kept in a sequential file, gauge station cards in a direct-access file (collision resolution with Brent's method).
- **F8 B+ tree index:** The flow-record file keeps a secondary B+ tree index keyed by timestamp, for fast time-range lookups of historical flow.
- **F9 Extendible hashing / external sort:** Millions of per-minute readings across all gauges, too large for memory, are sorted by timestamp with an external merge sort.

Extension: Feeding forecast-driven rainfall scenarios into multi-day flood predictions.

098 — Species Taxonomy Classification Tree



Summary: A console application that lets a biology student maintain and query a synthetic species-taxonomy tree, classify sample specimens, and compare evolutionary relationships. It works on about 3,000 species records across 200 families in a seven-rank taxonomy tree.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A species' list of historical synonym names is kept as a doubly linked list; a chain of synonyms that redirect to one another is represented with a circular list.
- **V2 Sparse matrix:** In the species-by-trait table, most species lack most rare traits, so only filled cells are stored, each holding a trait measurement.
- **V3 Stack and queue:** A stack undoes the taxonomist's reclassification edits (moving a species to a different genus); a queue holds specimen batches awaiting expert review.
- **V4 Tree and heap:** A binary dichotomous identification key is stored as a binary tree, traversed to identify a specimen step by step down to species; a heap orders specimens awaiting classification by rarity/priority.
- **V5 Graph and BFS/DFS:** Taxa from kingdom to species are nodes, parent-child rank links are edges; BFS lists every species under a family level by level, DFS traces a species' full ancestor lineage up to kingdom.
- **V6 Search and hashing:** Scientific (binomial) name maps to species record in a hash table; sorted discovery years support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders ranks from kingdom down to species for consistent processing; cycle detection verifies that no reclassification has introduced an invalid loop.
- **F3 Sorting:** Species are sorted by population estimate, discovery year, and subspecies count using three algorithms whose running times are compared.
- **F4 BST and AVL:** Species are kept in an AVL tree keyed by population estimate; the query "species with population between X and Y" (endangered species) is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a scientific or common name the student types; edit distance corrects a mistyped Latin binomial name.
- **F6 Trie and disjoint sets:** Scientific names are autocompleted with a trie; union-find groups species into clade clusters when previously separate branches are merged.
- **F7 File organisation:** Field observation logs are kept in a sequential file, species cards in a direct-access file (collision resolution with the linear quotient method).
- **F8 B+ tree index:** The species file keeps a secondary B+ tree index keyed by family name, for "list every species in family X" lookups.
- **F9 Extendible hashing / external sort:** Millions of field-observation records, too large for memory, are sorted by observation date with an external merge sort.

Extension: Cross-referencing DNA barcode similarity to suggest reclassification candidates.

099 — 🚒 Radiation Monitoring Network Map



Summary: A console application that tracks a synthetic network of radiation-monitoring sensors across a region, mapping readings and tracing contamination spread paths. It works on about 500 sensor stations, 10,000 synthetic readings a day, and 25 zones.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A sensor's reading history is kept as a doubly linked list; sensors ringing a facility's perimeter are tracked with a circular list.
- **V2 Sparse matrix:** In the zone-by-time-slot radiation table, most cells are unmeasured, so only filled cells are stored, each holding a dose rate.
- **V3 Stack and queue:** A stack undoes the analyst's manual sensor-calibration edits; a queue dispatches alerts to response teams in the order received.
- **V4 Tree and heap:** A binary tree holds the administrative region hierarchy, traversed to average radiation per region; a heap orders sensors by current dose-rate severity for inspection priority.
- **V5 Graph and BFS/DFS:** Sensors are nodes and proximity/wind-dispersion links are edges; BFS traces the fewest-hop path a contamination plume would take outward from a source sensor, DFS finds every sensor in one connected contaminated region.
- **V6 Search and hashing:** Sensor ID maps to sensor record in a hash table; sorted dose-rate thresholds support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra estimates plume arrival time at a town, weighted by wind-dispersion distance; a minimum-cost network (Prim/Kruskal) connects all sensor stations with the cheapest communication links.
- **F3 Sorting:** Sensors are sorted by dose rate, battery level, and last calibration date using three algorithms whose running times are compared.
- **F4 BST and AVL:** Readings are kept in an AVL tree keyed by dose rate; the query "next reading above safety threshold X" is answered on the balanced tree.
- **F5 String algorithms:** KMP searches a location description the analyst types; edit distance suggests the closest matching zone name on a typo.
- **F6 Trie and disjoint sets:** Sensor and zone names are autocompleted with a trie; union-find merges sensors into connected contaminated clusters as a plume spreads.
- **F7 File organisation:** Daily reading logs are kept in a sequential file, sensor cards in a direct-access file (collision resolution with progressive overflow).
- **F8 B+ tree index:** The reading file keeps a secondary B+ tree index keyed by timestamp, for fast "readings in the last 24 hours" queries.
- **F9 Extendible hashing / external sort:** Millions of per-minute readings across all sensors, too large for memory, are sorted by timestamp with an external merge sort.

Extension: Simulating wind-driven plume dispersion to forecast which towns should evacuate.

100 — f_x Math Expression Tree and Derivative Engine



Summary: A console application that parses user-typed algebraic expressions into expression trees, symbolically differentiates and simplifies them, and evaluates them numerically. It works on a synthetic library of about 1,000 sample expressions and a variable/symbol table.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A parsed expression's token stream is kept as a doubly linked list for lookahead and backtracking; a fixed-size circular list holds recently used expressions as an undo/redo ring.
- **V2 Sparse matrix:** In the expression-by-variable usage table, most expressions use only a few variables, so only filled cells are stored, each holding an exponent or coefficient.
- **V3 Stack and queue:** A classic evaluation stack handles nested parenthesized expressions and undoes edits; a queue holds expressions from a script file waiting to be processed.
- **V4 Tree and heap:** The expression tree itself is a binary tree — in-order traversal prints infix form, post-order traversal evaluates it; a heap-based priority queue applies the highest-priority simplification rule first.
- **V5 Graph and BFS/DFS:** For shared subexpressions, nodes (operators/operands) and dependency edges extend into a DAG; BFS prints the tree level by level, DFS traces every variable-dependency path from root to leaves affecting a node.
- **V6 Search and hashing:** A hash table maps each variable name to its current numeric value (symbol table); sorted predefined constant names support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders the shared-subexpression DAG into a safe evaluation sequence; cycle detection catches invalid circular variable definitions ($x = y+1$, $y = x+1$) before evaluation.
- **F3 Sorting:** Expressions are sorted by node count, evaluation time, and distinct-variable count using three algorithms whose running times are compared.
- **F4 BST and AVL:** Variables are kept in an AVL tree keyed by last-used order in the symbol table; the query "next variable alphabetically after X" is answered on the balanced tree.
- **F5 String algorithms:** KMP/Boyer-Moore searches a subexpression pattern the user types within the expression library; edit distance/LCS matches a mistyped expression to the closest known one.
- **F6 Trie and disjoint sets:** Function and variable names are autocompleted with a trie; union-find groups variables linked by shared equations into constraint clusters.
- **F7 File organisation:** Evaluation history is kept in a sequential file, expression records in a direct-access file (collision resolution with the linear quotient method).
- **F8 B+ tree index:** The expression library file keeps a secondary B+ tree index keyed by complexity score, for finding expressions of similar complexity.
- **F9 Extendible hashing / external sort:** Millions of logged evaluation results, too large for memory, are sorted by timestamp with an external merge sort.

Extension: Extending the differentiation engine to symbolic integration for a subset of expression forms.

101–125 · Networks and computer systems

101 — Network Topology and Packet Routing Simulator

Summary: A console application that models the routers and links of an organisation's network and simulates forwarding packets along the best path. It works on a synthetic topology of about 40 routers and 120 links; link failures are reflected in routing decisions immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each router's neighbour interface list is kept as a doubly linked list; links added or removed update it right away.
- **V2 Sparse matrix:** In the router-pair × link-cost table, only cells for directly connected pairs are stored.
- **V3 Stack and queue:** Incoming packets wait in the interface queue; the path traced by traceroute is kept on a stack so it can be unwound.
- **V4 Tree and heap:** In Dijkstra's computation, the next node to process is chosen from the heap by lowest accumulated cost.
- **V5 Graph and BFS/DFS:** Routers are nodes and links are edges; BFS finds the path with the fewest hops, DFS finds regions cut off after a link failure.
- **V6 Search and hashing:** IP address maps to an interface record in a hash table; binary search runs over the sorted subnet list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the lowest-cost path per packet, Kruskal computes the cheapest tree for backup backbone links.
- **F3 Sorting:** Links are sorted by bandwidth with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The routing table is kept in an AVL tree keyed by subnet prefix; the longest-prefix query is answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a hostname; on a typo, edit distance suggests the closest hostname.
- **F6 Trie and disjoint sets:** A trie of IP prefixes performs longest-prefix matching; union-find groups routers that remain reachable after a link failure.
- **F7 File organisation:** The packet log is a sequential file; router settings sit in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the packet log file.
- **F9 Extendible hashing / external sort:** Millions of daily packet records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Automatically recomputing the packet path from live congestion data.

102 — OS Process Scheduler Simulator



Summary: A console application that assigns processes in an operating-system kernel's ready queue to the CPU according to scheduling policies. It works on a synthetic system with about 30 processes and 4 cores; each process's arrival time, burst time and priority are generated randomly.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The ready queue (with pid, burst time and priority fields) is a doubly linked list; round-robin scheduling is implemented with a circular list.
- **V2 Sparse matrix:** In the core × time-slice utilisation table, only cells where a process actually ran are stored.
- **V3 Stack and queue:** A process's call stack simulates function-call depth; waiting processes are handled in order from an FCFS queue.
- **V4 Tree and heap:** In shortest-job-first scheduling, the next process to run is picked from the heap by smallest burst time.
- **V5 Graph and BFS/DFS:** Processes are nodes and resource-wait relations are edges; BFS finds processes at the same priority level, DFS finds all processes dependent through a resource chain.
- **V6 Search and hashing:** pid maps to a process control block in a hash table; binary search runs over the sorted pid list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the execution order of dependent tasks; Dijkstra computes the shortest total wait time along a chain of dependent tasks.
- **F3 Sorting:** The ready queue is sorted by burst time with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The process priority table is kept in an AVL tree; a "top 5 highest-priority processes" query is answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a process name; edit distance finds similar error lines in the log.
- **F6 Trie and disjoint sets:** Process names are auto-completed with a trie; union-find groups processes sharing the same resource group.
- **F7 File organisation:** The scheduling log is a sequential file; the process table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on pid is kept over the scheduling log file.
- **F9 Extendible hashing / external sort:** Millions of monthly scheduling events do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Simulating inter-core load balancing on a multi-core system.

103 — Memory Allocator (malloc) Simulator



Summary: A console application that manages memory blocks allocated and freed at a program's runtime and compares different allocation strategies. It works on a synthetic 8 KB heap; each allocation request has a randomly generated size.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The free list is kept as a doubly linked list of blocks with address and size fields.
- **V2 Sparse matrix:** In the size-class × time-slice allocation-count table, only cells for allocations that actually happened are stored.
- **V3 Stack and queue:** Allocate/free operations are kept on an undo stack; pending allocation requests are processed in order from a queue.
- **V4 Tree and heap:** The best-fit free block is selected from the heap; a binary tree ranks blocks by size for reporting.
- **V5 Graph and BFS/DFS:** Adjacent free blocks are nodes and adjacency is edges; BFS finds the largest contiguous free region, DFS finds blocks reachable through a pointer chain.
- **V6 Search and hashing:** An allocated block's address maps to block info in a hash table; binary search runs over the sorted free-block sizes.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds circular pointer chains (a sign of a memory leak); topological sort determines a valid order of dependent frees.
- **F3 Sorting:** Free blocks are sorted by size with three different algorithms to compare best-fit and worst-fit strategies.
- **F4 BST and AVL:** Free blocks are kept in an AVL tree keyed by size; the fastest best-fit lookup is answered on the balanced tree.
- **F5 String algorithms:** KMP searches allocation tags (module names); edit distance suggests the closest tag on a typo.
- **F6 Trie and disjoint sets:** Tag names are auto-completed with a trie; union-find merges adjacent free blocks in a buddy system into one block.
- **F7 File organisation:** The allocation log is a sequential file; block metadata sits in a direct-access file (hashed by address).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the allocation log file.
- **F9 Extendible hashing / external sort:** A long-running program's millions of allocation records do not fit in memory, so they are sorted by address with an external merge sort.

Extension: Visualising the fragmentation ratio over time.

104 — Disk Block and File System Simulator

Summary: A console application that manages the blocks, directories and files of a disk, mimicking a simple file system. It works on a synthetic disk of about 500 blocks; creating, deleting and growing files immediately affects block allocation.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A file's blocks (linked allocation) are chained as a doubly linked list; the free-block list is kept as a circular list.
- **V2 Sparse matrix:** In the directory × file-size table, only cells for directories that contain files are stored.
- **V3 Stack and queue:** Directory navigation history (cd/back) is kept on a stack; disk read/write requests are ordered in a queue.
- **V4 Tree and heap:** The directory structure is kept as a hierarchical tree; requests are picked from the heap by nearest block to reduce disk-arm movement.
- **V5 Graph and BFS/DFS:** Directories and symbolic links are nodes and edges; BFS searches all directories for a file by name, DFS recursively computes a directory's total size.
- **V6 Search and hashing:** Filename maps to an inode number in a hash table; binary search runs over the sorted inode list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes an access order that minimises disk-arm movement; cycle detection finds symbolic-link loops.
- **F3 Sorting:** Files in a directory are sorted by size with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Directory entries are kept in an AVL tree keyed by filename.
- **F5 String algorithms:** KMP searches for a filename; edit distance suggests the closest filename on a typo.
- **F6 Trie and disjoint sets:** Filenames are auto-completed with a trie; union-find tracks the blocks belonging to a fragmented file as one group.
- **F7 File organisation:** The file allocation table is a sequential file; the inode table sits in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** A B+ tree secondary index on filename is kept over the inode table.
- **F9 Extendible hashing / external sort:** As the disk grows, the inode table uses extendible hashing; the access log is sorted with an external merge sort.

Extension: A defragmentation simulation that reduces fragmentation.

105 — LRU Cache Simulator



Summary: A console application that caches an application's data accesses and evicts the least-recently-used (LRU) entries to keep a bounded size. It works on a synthetic 200-entry cache; every access is logged as a hit or a miss.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Cache entries are kept as a doubly linked list ordered by recency; each access moves its entry to the front of the list.
- **V2 Sparse matrix:** In the key $\times$ time-window hit/miss table, only accessed cells are stored.
- **V3 Stack and queue:** Recent access history is kept on a stack; the FIFO variant compares eviction candidates through a queue.
- **V4 Tree and heap:** In the LFU variant, the least-used entry is selected from the heap; a binary tree represents cache tiers (L1/L2).
- **V5 Graph and BFS/DFS:** Invalidation dependencies between keys are nodes and edges; BFS finds the invalidation chain, DFS finds deep dependencies.
- **V6 Search and hashing:** Key maps to a cache entry in a hash table; binary search runs over sorted key ranges for range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the invalidation order of dependent keys; cycle detection catches a circular-dependency error.
- **F3 Sorting:** Keys are sorted by access frequency with three different algorithms to produce an LFU report.
- **F4 BST and AVL:** Entries are kept in an AVL tree keyed by expiry time (TTL); expired entries are found quickly on the balanced tree.
- **F5 String algorithms:** KMP searches key patterns like "user:1:profile"; edit distance finds similar keys.
- **F6 Trie and disjoint sets:** A hierarchical key namespace is kept with a trie; union-find groups keys invalidated together.
- **F7 File organisation:** The cache-miss log is a sequential file; a persisted snapshot sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on key is kept over the snapshot file.
- **F9 Extendible hashing / external sort:** Millions of access records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Simulating consistency across distributed cache nodes.

106 — Shared Printer Queue Manager



Summary: A console application that queues documents sent to an office's shared printers and routes them to an available printer. It works with 6 printers and about 300 synthetic jobs a day; printers that jam or run out of paper are disabled immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A printer's pending jobs are kept as a doubly linked list; a circular list spreads jobs round-robin across several printers.
- **V2 Sparse matrix:** In the printer $\times$ hour usage table, only cells with submitted jobs are stored.
- **V3 Stack and queue:** Print jobs wait in arrival order in a queue; cancelling the user's last job is supported by an undo stack.
- **V4 Tree and heap:** Jobs flagged urgent are moved ahead in the heap by priority.
- **V5 Graph and BFS/DFS:** Printers and departments are nodes and edges; BFS finds the nearest available printer for a user.
- **V6 Search and hashing:** Job number maps to a job record in a hash table; binary search runs over the sorted job numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest network time for a job to reach the nearest idle printer; Kruskal plans the print-server backbone at lowest cost.
- **F3 Sorting:** Queued jobs are sorted by priority and page count with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Per-user page quotas are kept in an AVL tree keyed by user number.
- **F5 String algorithms:** KMP searches for a document name; edit distance suggests the closest document name on a typo.
- **F6 Trie and disjoint sets:** File extensions are grouped with a trie; union-find merges jobs submitted in the same session into one batch.
- **F7 File organisation:** The job history is a sequential file; printer status sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on user number is kept over the job history file.
- **F9 Extendible hashing / external sort:** Millions of monthly job records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Tracking printer status on a real-time dashboard.

107 — DNS Resolver and Cache



Summary: A console application that resolves domain-name queries and caches the results, working over a synthetic domain-to-IP database. It holds about 5,000 domain records; each query's time-to-live (TTL) varies by record type.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A domain's several A records are kept as a doubly linked list; a circular list tries backup servers in turn.
- **V2 Sparse matrix:** In the domain × record-type table, only cells for defined records are stored.
- **V3 Stack and queue:** The referral chain followed during recursive resolution is kept on a call stack; pending queries are processed from a queue.
- **V4 Tree and heap:** Records whose TTL has expired are picked from the heap by nearest expiry and evicted from the cache.
- **V5 Graph and BFS/DFS:** Domains are nodes and CNAME redirects are edges; BFS finds the shortest resolution chain, DFS finds a CNAME loop.
- **V6 Search and hashing:** Domain name maps to an IP address in a hash table; binary search runs over the sorted domain list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection catches invalid CNAME loops; Dijkstra computes the fastest-responding server chain.
- **F3 Sorting:** Cached records are sorted by TTL and hit count with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Domain records are kept in an AVL tree; alphabetical range queries are answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a domain name; edit distance offers a "did you mean" suggestion on a typo.
- **F6 Trie and disjoint sets:** The subdomain hierarchy is kept with a trie; union-find groups domains sharing the same nameserver.
- **F7 File organisation:** The query log is a sequential file; the zone file sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on domain name is kept over the zone file.
- **F9 Extendible hashing / external sort:** Millions of daily query records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Detecting anomalous responses that indicate cache poisoning.

108 — Routing Table Longest-Prefix Matcher



Summary: A console application that simulates a router forwarding incoming packets to the correct interface by longest-prefix match on the destination IP address. It works over a synthetic routing table of about 2,000 prefix entries.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Routing entries sharing the same outgoing interface are chained as a doubly linked list.
- **V2 Sparse matrix:** In the prefix-length × interface usage table, only cells for defined prefixes are stored.
- **V3 Stack and queue:** Incoming packets are queued in the interface buffer; branches tried during prefix matching are kept on a backtracking stack.
- **V4 Tree and heap:** Routing updates are ordered in the heap by priority (metric value), so the highest-priority one is applied first.
- **V5 Graph and BFS/DFS:** Routers are nodes and links are edges; BFS finds the path with the fewest hops, DFS checks whether a subnet is still reachable.
- **V6 Search and hashing:** An exact-match route cache is kept in a hash table; binary search over the sorted prefix list is used for verification.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the lowest-cost path metric for a prefix; Kruskal plans backbone links with the cheapest tree.
- **F3 Sorting:** Routing entries are sorted by prefix length with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The routing table is kept in an AVL tree keyed by prefix value.
- **F5 String algorithms:** KMP searches an interface description; edit distance suggests the closest interface name on a typo.
- **F6 Trie and disjoint sets:** A binary trie of IP prefixes performs longest-prefix matching; union-find groups interfaces in the same broadcast domain (VLAN).
- **F7 File organisation:** The routing-update log is a sequential file; a table snapshot sits in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the update log.
- **F9 Extendible hashing / external sort:** Millions of daily update records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Temporarily damping routes that flap frequently.

109 — Package Manager Dependency Resolver



Summary: A console application that resolves a software package's dependencies and determines the correct install order. It works over about 500 synthetic packages and their version dependencies; conflicting version requests are reported immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A package's direct dependencies are kept as a doubly linked list.
- **V2 Sparse matrix:** In the package × version compatibility table, only defined compatibility constraints are stored.
- **V3 Stack and queue:** The install order is kept on a stack so it can be undone; download requests are processed in order from a queue.
- **V4 Tree and heap:** The package with the fewest dependencies is picked from the heap and installed first.
- **V5 Graph and BFS/DFS:** Packages are nodes and dependencies are edges; BFS finds the shortest install chain, DFS finds a circular dependency.
- **V6 Search and hashing:** Package name maps to metadata in a hash table; binary search runs over the sorted package list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the install order; cycle detection reports an invalid circular dependency.
- **F3 Sorting:** Packages are sorted by size and dependency count with three different algorithms and their running times are compared.
- **F4 BST and AVL:** A package's version numbers are kept in an AVL tree; a "newest version above 2.x" query is answered on the balanced tree.
- **F5 String algorithms:** KMP searches for a package name; edit distance offers a "did you mean numpy" suggestion on a typo.
- **F6 Trie and disjoint sets:** Package names are auto-completed with a trie; union-find groups packages in the same dependency set.
- **F7 File organisation:** The install log is a sequential file; package metadata sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on package name is kept over the metadata file.
- **F9 Extendible hashing / external sort:** Millions of daily download records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Scheduling independent packages to download in parallel.

110 — ? Mini Version Control System



Summary: A console application that commits file changes and merges between branches, mimicking a simple version control system. It works over about 300 synthetic commits and 5 branches; each commit links to its parent commit through a pointer.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Commit history is kept as a doubly linked list through parent pointers.
- **V2 Sparse matrix:** In the file × commit change table, only cells for files that changed in that commit are stored.
- **V3 Stack and queue:** Local edits keep an undo/redo stack; commits to be pushed remotely are ordered in a queue.
- **V4 Tree and heap:** Branch structure is kept as a tree; merge conflicts are ordered in the heap by severity.
- **V5 Graph and BFS/DFS:** Commits are nodes and parent relations are edges; BFS finds the common ancestor of two branches, DFS checks a commit's reachability.
- **V6 Search and hashing:** Commit hash maps to a commit object in a hash table; binary search runs over sorted commit timestamps.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort reorders commits for a rebase; cycle detection prevents an invalid history from forming.
- **F3 Sorting:** Commits are sorted by date and author with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Branch names are kept in an AVL tree.
- **F5 String algorithms:** KMP searches a commit message; the difference between two file versions is computed with longest common subsequence (LCS).
- **F6 Trie and disjoint sets:** File paths are kept with a trie; union-find groups files that changed together in the same commit.
- **F7 File organisation:** The commit log is a sequential file; the object store sits in a direct-access file (hashed by commit hash).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the commit log.
- **F9 Extendible hashing / external sort:** A long-lived repository's millions of commit records do not fit in memory, so they are sorted with an external merge sort.

Extension: Browsing commit history through a visual branch graph.

111 — Key-Value Database Engine



Summary: A console application that simulates a simple key-value database engine, complete with a write-ahead log, a memtable and flushed segments on disk. It works over about 10,000 synthetic keys; every write is applied to the log first and then to the memtable.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Write-ahead log (WAL) records are chained as a doubly linked list.
- **V2 Sparse matrix:** In the field $\times$ key presence table, only cells for keys where that field is defined are stored.
- **V3 Stack and queue:** Changes made during a transaction are kept on a stack so they can be rolled back; pending writes are ordered in a log queue.
- **V4 Tree and heap:** During compaction, the smallest segment is selected from the heap and merged first.
- **V5 Graph and BFS/DFS:** Reference relations between keys are nodes and edges; BFS finds all records linked to a key, DFS finds a deep reference chain.
- **V6 Search and hashing:** Key maps to value in a hash table; binary search runs over a sorted key range.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds circular references; topological sort determines the order in which dependent writes are applied.
- **F3 Sorting:** For a range scan, keys are sorted with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The memtable is kept as an AVL tree keyed by key.
- **F5 String algorithms:** KMP searches a key prefix; edit distance finds similar keys.
- **F6 Trie and disjoint sets:** A hierarchical key namespace is kept with a trie; union-find groups keys that fall into the same shard.
- **F7 File organisation:** The write-ahead log is a sequential file; the flushed table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on the value field is kept over the flushed table.
- **F9 Extendible hashing / external sort:** A growing hash index uses extendible hashing; segments are merged during compaction with an external merge sort.

Extension: Simulating data replication across multiple nodes.

112 — ☆ Web Crawler and Link Graph

Summary: A console application that crawls a set of websites, turns the links between pages into a graph, and tracks visited pages. It works over about 3,000 synthetic pages and the links between them; each crawl round updates the visited list.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The frontier of URLs to visit is kept as a doubly linked list; pages belonging to one domain are also collected in a separate linked list.
- **V2 Sparse matrix:** In the page × page link matrix, only cells where a link actually exists are stored.
- **V3 Stack and queue:** A frontier queue drives breadth-first crawling, and a frontier stack drives depth-first crawling.
- **V4 Tree and heap:** Pages with high crawl priority (many inbound links) are picked from the heap first.
- **V5 Graph and BFS/DFS:** Pages are nodes and links are edges; BFS gives the crawl order, DFS finds deep link chains.
- **V6 Search and hashing:** The set of visited URLs is kept in a hash table; binary search runs over the sorted domain list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components find clusters of pages that link to each other; Dijkstra computes the click distance between two pages.
- **F3 Sorting:** Pages are sorted by inbound link count with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Page counts per domain are kept in an AVL tree.
- **F5 String algorithms:** KMP searches a page title; edit distance finds near-identical titles to detect duplicates.
- **F6 Trie and disjoint sets:** URL paths are kept with a trie; union-find groups pages that are linked together into connected components.
- **F7 File organisation:** The crawl log is a sequential file; the page cache sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on URL is kept over the page cache.
- **F9 Extendible hashing / external sort:** Millions of daily crawl records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Incremental re-crawling that prioritises changed pages.

113 — Distributed File Chunk Placement Simulator

Summary: A console application that splits a large file into chunks, distributes them across multiple nodes and makes replication decisions. It works over about 200 synthetic nodes with 3 copies per file; re-replication starts as soon as a node fails.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A file's chunks are kept in order as a doubly linked list.
- **V2 Sparse matrix:** In the node × chunk placement table, only cells for chunks a node actually holds are stored.
- **V3 Stack and queue:** Replication tasks are ordered in a queue; a failed placement is kept on an undo stack.
- **V4 Tree and heap:** The target node for a new chunk is picked from the heap by the most free space.
- **V5 Graph and BFS/DFS:** Nodes are graph nodes and network links are edges; BFS finds the nearest replica, DFS checks for a network partition.
- **V6 Search and hashing:** Chunk id maps to a list of nodes in a hash table; binary search runs over the sorted node ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim plans inter-node replication traffic with the lowest-cost tree; Dijkstra finds the node that can replicate a chunk fastest.
- **F3 Sorting:** Nodes are sorted by free space and load with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The chunk table is kept in an AVL tree keyed by chunk id.
- **F5 String algorithms:** KMP searches a file path; edit distance suggests the closest path on a typo.
- **F6 Trie and disjoint sets:** File paths are kept with a trie; union-find groups nodes belonging to the same rack or cluster.
- **F7 File organisation:** The placement log is a sequential file; chunk metadata sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on file id is kept over the chunk metadata.
- **F9 Extendible hashing / external sort:** As the cluster grows, the chunk table uses extendible hashing.

Extension: Automatic rebalancing of copies after a node fails.

114 — Load Balancer Simulator

Summary: A console application that spreads incoming requests across a server pool and monitors server health. It works over about 20 synthetic servers under a load of 500 requests per second; unresponsive servers are removed from the pool immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The server pool is kept as a circular linked list; the next request is assigned to the next server round-robin.
- **V2 Sparse matrix:** In the server $\times$ time-slice request-count table, only cells with received requests are stored.
- **V3 Stack and queue:** Incoming requests wait in a queue and are dispatched in order.
- **V4 Tree and heap:** The server with the fewest active connections is picked from the heap for a new request.
- **V5 Graph and BFS/DFS:** Servers and data centres are nodes and edges; BFS finds the nearest healthy server.
- **V6 Search and hashing:** Session id maps to the assigned server in a hash table (sticky sessions); binary search runs over the sorted server ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra routes requests to the fastest server by latency; Kruskal plans the backup link network at the lowest cost.
- **F3 Sorting:** Servers are sorted by load and response time with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The server health table is kept in an AVL tree keyed by response time.
- **F5 String algorithms:** KMP searches request paths in routing rules; edit distance suggests the closest path on a typo.
- **F6 Trie and disjoint sets:** URL paths are mapped to routing rules with a trie; union-find groups servers into the same pool.
- **F7 File organisation:** The access log is a sequential file; server settings sit in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the access log.
- **F9 Extendible hashing / external sort:** Millions of daily access records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Simulating auto-scaling by adding servers under load.

115 — Message Queue Broker Simulator



Summary: A console application that simulates a message-queue broker, queuing messages producers send by topic and delivering them to consumers. It works over about 50 topics and 1,000 synthetic messages per second; each consumer group tracks its own offset.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each topic's message queue is kept as a doubly linked list.
- **V2 Sparse matrix:** In the topic × consumer subscription table, only subscribed cells are stored.
- **V3 Stack and queue:** Messages wait in FIFO order in the topic queue; messages that cannot be processed move to a dead-letter stack.
- **V4 Tree and heap:** Priority messages are processed from the heap first.
- **V5 Graph and BFS/DFS:** Topic-consumer subscriptions are nodes and edges; BFS finds all consumers affected by a failure on a topic.
- **V6 Search and hashing:** Message id maps to a message in a hash table; binary search runs over sorted offsets.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders the stages of the message-processing pipeline; cycle detection catches a circular topic-subscription error.
- **F3 Sorting:** Pending messages are sorted by priority and timestamp with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Consumer offsets per partition are kept in an AVL tree.
- **F5 String algorithms:** KMP searches for a topic name; edit distance suggests the closest topic name on a typo.
- **F6 Trie and disjoint sets:** Hierarchical topic names are kept with a trie; union-find groups consumers in the same consumer group.
- **F7 File organisation:** The message log is a sequential file; the consumer offset table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on offset is kept over the message log.
- **F9 Extendible hashing / external sort:** Millions of daily message records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Simulating exactly-once delivery guarantees.

116 — Cron Job Scheduler



Summary: A console application that triggers recurring tasks (backups, report delivery) on time at set intervals. It works over about 100 synthetic scheduled jobs; each job's next run time is continuously updated.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Defined jobs are kept as a doubly linked list; jobs that repeat daily are cycled through with a circular list.
- **V2 Sparse matrix:** In the job × hour-of-day schedule table, only cells for jobs scheduled at that hour are stored.
- **V3 Stack and queue:** Jobs whose run time has arrived are processed from a queue; failed jobs are pushed onto a retry stack.
- **V4 Tree and heap:** The next job to run is selected from the heap by the nearest run time.
- **V5 Graph and BFS/DFS:** Job dependencies are nodes and edges; BFS finds the dependent job chain, DFS finds deep dependencies.
- **V6 Search and hashing:** Job name maps to a job record in a hash table; binary search runs over the sorted next-run times.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the run order of dependent jobs; cycle detection catches a circular-dependency error.
- **F3 Sorting:** Jobs are sorted by next run time with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The job schedule is kept in an AVL tree keyed by run time.
- **F5 String algorithms:** KMP searches for a job name; edit distance suggests the closest job name on a typo.
- **F6 Trie and disjoint sets:** Job names are auto-completed with a trie; union-find groups jobs that share the same resource lock.
- **F7 File organisation:** The execution log is a sequential file; job definitions sit in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the execution log.
- **F9 Extendible hashing / external sort:** Millions of monthly execution records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Leader election that guarantees only one node runs a job in a distributed setup.

117 — Logic Circuit Simulator



Summary: A console application that simulates, step by step, how signals propagate through a circuit built from basic logic gates (AND, OR, NOT). It works over a synthetic circuit of about 80 gates and their connections; outputs are recomputed as input signals change.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The other gates a gate connects to (the netlist) are kept as a doubly linked list; gates with a feedback loop are flagged with a circular list.
- **V2 Sparse matrix:** In the gate × signal connectivity table, only cells that are actually connected are stored.
- **V3 Stack and queue:** Signal propagation is processed through an event queue; circuit edits are kept on an undo stack.
- **V4 Tree and heap:** A boolean expression is kept as a parse tree; events are ordered in the heap by simulation time.
- **V5 Graph and BFS/DFS:** Gates are nodes and connections are edges; BFS propagates signal levels layer by layer, DFS finds a feedback loop.
- **V6 Search and hashing:** Gate id maps to a gate object in a hash table; binary search runs over the sorted signal names.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the evaluation order of gates; cycle detection catches an invalid feedback loop.
- **F3 Sorting:** Gates are sorted by evaluation order and fan-out with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Signal names are kept in an AVL tree.
- **F5 String algorithms:** KMP searches a gate label; edit distance suggests the closest label on a typo.
- **F6 Trie and disjoint sets:** Signal names are kept with a trie; union-find tracks the wires belonging to the same connected net.
- **F7 File organisation:** The simulation trace is a sequential file; the netlist sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the simulation trace.
- **F9 Extendible hashing / external sort:** A long simulation's millions of trace records do not fit in memory, so they are sorted with an external merge sort.

Extension: Visualising signal levels over time as a waveform.

118 — Deadlock Detector



Summary: A console application that tracks the resources processes hold and wait for, and checks whether a cycle has formed among them. It works over about 25 processes and 15 synthetic resources; every resource request updates the wait-for graph.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of resources a process is waiting for is kept as a doubly linked list.
- **V2 Sparse matrix:** In the process $\times$ resource allocation table, only cells for resources actually held or requested are stored.
- **V3 Stack and queue:** Resource requests are processed in order from a queue; the DFS scan that searches for a cycle keeps the path it has taken on a stack.
- **V4 Tree and heap:** When resolving a deadlock, the victim process is picked from the heap by the lowest priority.
- **V5 Graph and BFS/DFS:** Processes and resources are nodes and wait relations are edges; DFS searches the wait-for graph for a cycle, BFS finds every process affected by a deadlock.
- **V6 Search and hashing:** Process id maps to the list of resources it holds in a hash table; binary search runs over the sorted resource ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds the cycle that indicates a deadlock directly; strongly connected components group clusters of mutually waiting processes.
- **F3 Sorting:** For victim selection, processes are sorted by wait time and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The resource allocation table is kept in an AVL tree keyed by resource id.
- **F5 String algorithms:** KMP searches for a process name; edit distance suggests the closest process name on a typo.
- **F6 Trie and disjoint sets:** Process names are auto-completed with a trie; union-find gathers processes in the same deadlock cluster into one group.
- **F7 File organisation:** The detection log is a sequential file; the resource table sits in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the detection log.
- **F9 Extendible hashing / external sort:** A long monitoring session's millions of detection records do not fit in memory, so they are sorted with an external merge sort.

Extension: Automatically terminating and recovering the victim process once a deadlock is detected.

119 — Website Sitemap and Broken Link Finder



Summary: A console application that extracts a website's page hierarchy and checks its links to report broken ones. It works over about 1,500 synthetic pages; each link check records a status code.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A section's page list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the page $\times$ page link matrix, only cells where a link actually exists are stored.
- **V3 Stack and queue:** A breadth-first crawl uses a URL queue, and checking deep links uses a URL stack.
- **V4 Tree and heap:** The site hierarchy is kept as a tree; the page least recently checked is picked from the heap first.
- **V5 Graph and BFS/DFS:** Pages are nodes and links are edges; BFS gives the crawl order, DFS finds orphan pages and broken chains.
- **V6 Search and hashing:** URL maps to status (working/broken) in a hash table; binary search runs over the sorted URL list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds circular redirects; strongly connected components group pages that link to each other.
- **F3 Sorting:** Broken links are sorted by page and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The page hierarchy is kept in an AVL tree.
- **F5 String algorithms:** KMP searches URL/path text; edit distance suggests the closest URL on a 404 error.
- **F6 Trie and disjoint sets:** URL paths are mapped to the sitemap tree with a trie; union-find groups pages that are linked together.
- **F7 File organisation:** Crawl results form a sequential log file; page status sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on URL is kept over the status file.
- **F9 Extendible hashing / external sort:** A large site's millions of crawl records do not fit in memory, so they are sorted with an external merge sort.

Extension: Scheduled re-checking with change detection.

120 — Wireless Channel Assignment (Graph Colouring)



Summary: A console application that assigns channels to wireless access points in a building so neighbours do not interfere with each other. It works over about 60 synthetic access points; two points whose signal areas overlap are never given the same channel.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The neighbours an access point interferes with are kept as a doubly linked list.
- **V2 Sparse matrix:** In the access-point × access-point interference/overlap table, only cells for overlapping pairs are stored.
- **V3 Stack and queue:** Channel assignment is tried through a backtracking stack; access points newly joining the network are processed in order from a queue.
- **V4 Tree and heap:** The access point with the most interference is picked from the heap first and assigned a channel before the rest.
- **V5 Graph and BFS/DFS:** Access points are nodes and interference is edges; BFS/DFS find connected interference clusters (components).
- **V6 Search and hashing:** MAC address maps to an access-point record in a hash table; binary search runs over the sorted channel numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim connects the wired backbone between access points at the lowest cost; cycle detection finds unnecessary redundant-link loops.
- **F3 Sorting:** Access points are sorted by interference degree with three different algorithms to compare the resulting greedy channel-assignment order.
- **F4 BST and AVL:** Channel assignments are kept in an AVL tree keyed by access-point id.
- **F5 String algorithms:** KMP searches a network name (SSID); edit distance suggests the closest SSID on a typo.
- **F6 Trie and disjoint sets:** SSID prefixes are kept with a trie; union-find gathers access points in the same interference component into one group.
- **F7 File organisation:** The signal-scan log is a sequential file; access-point settings sit in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the scan log.
- **F9 Extendible hashing / external sort:** Millions of daily scan records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Dynamically reassigning channels as new access points join.

121 — Network Packet Capture Analyser



Summary: A console application that groups packets captured on a network interface into flows and reports traffic patterns. It works over about 50,000 synthetic packets; each packet is assigned to a flow by source/destination address and port.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Captured packets are kept as a doubly linked list in arrival order.
- **V2 Sparse matrix:** In the source-IP × destination-IP traffic-volume table, only cells for pairs that actually communicated are stored.
- **V3 Stack and queue:** Packets are examined in order from a processing queue; resolving protocol layers (Ethernet-IP-TCP) is simulated with a stack.
- **V4 Tree and heap:** The top-talker flows (most bytes sent) are picked from the heap by priority.
- **V5 Graph and BFS/DFS:** Hosts are nodes and communications are edges; BFS finds a conversation chain, DFS finds clusters of heavy communication.
- **V6 Search and hashing:** The 5-tuple (source, destination, port) maps to a flow record in a hash table; binary search runs over the sorted timestamps.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components find clusters of hosts that talk heavily to each other; Dijkstra estimates hop distance from the TTL value.
- **F3 Sorting:** Flows are sorted by byte count and duration with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The flow table is kept in an AVL tree keyed by flow id.
- **F5 String algorithms:** KMP searches packet payloads for known signatures; edit distance finds similar signatures.
- **F6 Trie and disjoint sets:** Protocol and port names are kept with a trie; union-find groups heavily communicating hosts into one cluster.
- **F7 File organisation:** The capture file is a sequential file; a flow index sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the capture file.
- **F9 Extendible hashing / external sort:** A long capture session's millions of packets do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Turning anomalous traffic patterns into real-time alerts.

122 — Spreadsheet Cell Formula Engine



Summary: A console application that mimics a spreadsheet engine by parsing formulas typed into cells and automatically recalculating dependent cells. It works over a synthetic 50-row by 20-column sheet; when a cell changes, every cell that depends on it is updated.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of other cells that depend on a cell is kept as a doubly linked list.
- **V2 Sparse matrix:** In the row $\times$ column cell-value table, only filled cells are stored; most cells are empty.
- **V3 Stack and queue:** Postfix formula evaluation is done with a stack; the recalculation order is processed through a queue.
- **V4 Tree and heap:** A formula is kept as a parse tree; recalculation priorities are ordered in a heap.
- **V5 Graph and BFS/DFS:** Cells are nodes and formula dependencies are edges; BFS propagates a recalculation to dependent cells, DFS finds a circular reference.
- **V6 Search and hashing:** A cell address like "B12" maps to a cell object in a hash table; binary search runs over the sorted cell addresses.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the recalculation order of cells; cycle detection catches a circular-reference error.
- **F3 Sorting:** Rows are sorted by a column value with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Named ranges are kept in an AVL tree.
- **F5 String algorithms:** KMP searches formula text for a function name; edit distance offers a "did you mean SUM" suggestion on a typo.
- **F6 Trie and disjoint sets:** Function names are auto-completed with a trie; union-find groups cells in the same dependency chain.
- **F7 File organisation:** The change log is a sequential file; cell data sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on cell address is kept over the sheet data.
- **F9 Extendible hashing / external sort:** A large sheet's millions of change records do not fit in memory, so they are sorted with an external merge sort.

Extension: Conflict resolution for several users editing the same sheet at once.

123 — File Manager with Tree View and Search



Summary: A console application that browses a file system in a tree view and supports searching and copying files. It works over about 10,000 synthetic files and folders; every navigation step is kept in history.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A folder's file list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the folder × file-type count table, only cells with files of that type are stored.
- **V3 Stack and queue:** Back/forward navigation is supported with a stack; bulk copy operations are processed in order from a queue.
- **V4 Tree and heap:** The folder structure is kept as a tree; the largest-files list is pulled from the heap by size.
- **V5 Graph and BFS/DFS:** Folders and symbolic links are nodes and edges; BFS does a shallow search, DFS does a full recursive search.
- **V6 Search and hashing:** Filename maps to a file record in a hash table; binary search runs over the sorted filenames.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds symbolic-link loops; Dijkstra computes the relative path between two folders.
- **F3 Sorting:** Files are sorted by name, size and date with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Folder contents are kept in an AVL tree keyed by name.
- **F5 String algorithms:** KMP searches for a filename; edit distance lists the closest names for a fuzzy search.
- **F6 Trie and disjoint sets:** Filename prefixes are auto-completed with a trie; union-find groups files in the same hard-link group.
- **F7 File organisation:** The access log is a sequential file; file metadata sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on name is kept over the file metadata.
- **F9 Extendible hashing / external sort:** Millions of daily access records do not fit in memory, so they are sorted with an external merge sort.

Extension: Finding duplicate files by content hash.

124 — API REST API Rate Limiter Simulator



Summary: A console application that rate-limits requests to an API per client and rejects requests that exceed the limit. It works over about 200 synthetic clients under 2,000 requests per second; each client's remaining quota is updated continuously.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A client's request timestamps within the sliding window are kept as a doubly linked list.
- **V2 Sparse matrix:** In the client × endpoint request-count table, only cells with sent requests are stored.
- **V3 Stack and queue:** Requests wait in a token/leaky-bucket queue; throttled requests are pushed onto a retry stack.
- **V4 Tree and heap:** The client whose quota resets soonest is picked from the heap.
- **V5 Graph and BFS/DFS:** Client-endpoint access relations are nodes and edges; BFS finds clients sending requests in the same pattern.
- **V6 Search and hashing:** Client id (API key) maps to counter info in a hash table; binary search over sorted request timestamps answers window queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components cluster clients coming from the same IP block; cycle detection flags chained calls that keep triggering each other.
- **F3 Sorting:** Clients are sorted by request count with three different algorithms to report the top offenders.
- **F4 BST and AVL:** Client quota is kept in an AVL tree keyed by remaining requests.
- **F5 String algorithms:** KMP searches endpoint paths; edit distance suggests the closest path on a typo.
- **F6 Trie and disjoint sets:** API paths are matched with a trie; union-find groups clients sharing the same API key.
- **F7 File organisation:** The access log is a sequential file; client quota sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the access log.
- **F9 Extendible hashing / external sort:** Millions of daily access records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Dynamically adjusting limits based on server load.

125 — Compiler Symbol Table and Scope Manager



Summary: A console application that mimics a simple compiler front-end component, tracking identifiers (variables, functions) in a language's source code by scope. It works over about 400 lines of synthetic source code; entering a block opens a new scope.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A scope's identifiers are kept as a doubly linked list.
- **V2 Sparse matrix:** In the scope × variable-type usage table, only cells that are defined are stored.
- **V3 Stack and queue:** A scope is pushed/popped on a stack when a block is entered/left; tokens are processed in order from a queue.
- **V4 Tree and heap:** The source code is kept as an abstract syntax tree (AST); register-allocation priorities are ordered in a heap.
- **V5 Graph and BFS/DFS:** Functions are nodes and calls are edges; BFS/DFS find dead code that is never called.
- **V6 Search and hashing:** Identifier name maps to a symbol record in a hash table; binary search runs over the sorted identifier list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the dependency order of compilation units; cycle detection catches a circular-import error.
- **F3 Sorting:** Symbols are sorted by scope depth and type with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Each scope's symbol table is kept as an AVL tree.
- **F5 String algorithms:** KMP searches for an identifier name; edit distance produces a "did you mean" error on a typo.
- **F6 Trie and disjoint sets:** Identifier prefixes are auto-completed with a trie; union-find groups types merged during type inference.
- **F7 File organisation:** The compile log is a sequential file; a symbol-table snapshot sits in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on identifier name is kept over the snapshot.
- **F9 Extendible hashing / external sort:** A large project's millions of compile-log records do not fit in memory, so they are sorted with an external merge sort.

Extension: An incremental compilation cache that skips recompiling unchanged files.

126–150 · Logistics, manufacturing and commerce

126 — Warehouse Shelf Placement and Picking Route



Summary: A console application that places products on shelves in a warehouse and suggests the shortest route to a picker fulfilling an order. It works over about 3,000 synthetic products in a 40-aisle warehouse; the route is recomputed immediately when stock runs out.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A shelf's product list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the shelf × product-quantity table, only cells for products actually on that shelf are stored.
- **V3 Stack and queue:** Picking tasks are processed in order from a queue; placement changes are kept on an undo stack.
- **V4 Tree and heap:** Urgent orders are picked from the heap by priority first.
- **V5 Graph and BFS/DFS:** Aisles are nodes and passages are edges; BFS finds the shortest picking path, DFS finds a route that sweeps the whole inventory.
- **V6 Search and hashing:** SKU maps to a shelf location in a hash table; binary search runs over the sorted SKU list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest picking route; Prim minimises aisle-layout cost.
- **F3 Sorting:** The pick list is sorted by shelf order and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Shelf locations are kept in an AVL tree keyed by SKU.
- **F5 String algorithms:** KMP searches for a product name; edit distance suggests the closest product name on a typo.
- **F6 Trie and disjoint sets:** SKU prefixes are kept with a trie; union-find groups shelves in the same zone.
- **F7 File organisation:** The picking log is a sequential file; shelf inventory sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on SKU is kept over the inventory file.
- **F9 Extendible hashing / external sort:** Millions of daily picking records do not fit in memory, so they are sorted with an external merge sort.

Extension: Coordinating routes for several pickers so they never collide.

127 — 🔄 Production Line Workstation Balancer

Summary: A console application that assigns tasks in an assembly line to workstations so each station's working time is balanced. It works over about 60 tasks on a synthetic 10-station line; precedence relations between tasks are respected.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The tasks assigned to a station are kept in order as a doubly linked list.
- **V2 Sparse matrix:** In the station × task-time table, only cells for tasks assigned to that station are stored.
- **V3 Stack and queue:** Each station's task queue is processed in order; defective parts are sent to a rework stack.
- **V4 Tree and heap:** The longest-duration task is picked from the heap first to balance the line.
- **V5 Graph and BFS/DFS:** Tasks are nodes and precedence relations are edges; BFS/DFS verify precedence constraints.
- **V6 Search and hashing:** Task id maps to a task record in a hash table; binary search runs over the sorted task durations.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the precedence order of tasks; cycle detection catches an invalid circular-precedence error.
- **F3 Sorting:** Tasks are sorted by duration and precedence weight with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Station loads are kept in an AVL tree keyed by station id.
- **F5 String algorithms:** KMP searches for a task name; edit distance suggests the closest task name on a typo.
- **F6 Trie and disjoint sets:** Task names are kept with a trie; union-find groups tasks in the same precedence cluster.
- **F7 File organisation:** The production log is a sequential file; station settings sit in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the production log.
- **F9 Extendible hashing / external sort:** Millions of monthly production records do not fit in memory, so they are sorted with an external merge sort.

Extension: Real-time bottleneck detection and rebalancing of the line.

128 — Factory Work Order Prioritiser



Summary: A console application that orders work orders arriving at a factory by due date and severity and assigns them to machines. It works over about 150 synthetic work orders and 12 machines; the queue is reordered immediately when an urgent order arrives.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Pending work orders are kept as a doubly linked list.
- **V2 Sparse matrix:** In the machine × work-order hours table, only cells for hours allocated to that machine are stored.
- **V3 Stack and queue:** Work orders are processed in order from a queue; urgent orders wait on a priority stack.
- **V4 Tree and heap:** The next order to process is picked from the heap by due date.
- **V5 Graph and BFS/DFS:** Raw-material dependencies are nodes and edges; BFS/DFS find all sub-materials an order requires.
- **V6 Search and hashing:** Order number maps to an order record in a hash table; binary search runs over the sorted due dates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the order of multi-stage production orders; Dijkstra estimates the critical delivery chain.
- **F3 Sorting:** Orders are sorted by due date and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The machine schedule is kept in an AVL tree keyed by machine id.
- **F5 String algorithms:** KMP searches for a product name; edit distance suggests the closest product name on a typo.
- **F6 Trie and disjoint sets:** Product codes are kept with a trie; union-find groups orders sharing the same machine batch.
- **F7 File organisation:** The production log is a sequential file; the order table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on due date is kept over the order table.
- **F9 Extendible hashing / external sort:** Millions of monthly production records do not fit in memory, so they are sorted with an external merge sort.

Extension: Scheduling that is sensitive to predictive-maintenance forecasts.

129 — 🛒 E-Commerce Cart and Recommendation Engine



Summary: A console application that simulates cart management and a "customers who bought this also bought" recommendation engine for a shopping site. It works over about 1,000 synthetic customers and 500 products; every purchase updates recommendation scores.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Cart contents are kept as a doubly linked list of added and removed items.
- **V2 Sparse matrix:** In the customer × product purchase matrix, only cells for pairs that actually purchased are stored.
- **V3 Stack and queue:** Recently viewed products are kept on a stack; carts moving to checkout are processed in order from a queue.
- **V4 Tree and heap:** The most recommended products are picked from the heap by score.
- **V5 Graph and BFS/DFS:** Products are nodes and co-purchase relations are edges; BFS/DFS walk the "customers who bought this also bought" chain.
- **V6 Search and hashing:** Product id maps to a product record in a hash table; binary search runs over the sorted price list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the co-purchase distance between two products; strongly connected components find clusters of frequently co-purchased products.
- **F3 Sorting:** Products are sorted by price and rating with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The product catalogue is kept in an AVL tree keyed by price.
- **F5 String algorithms:** KMP searches for a product name in the search bar; typo-tolerant search uses edit distance.
- **F6 Trie and disjoint sets:** Product names are auto-completed with a trie; union-find groups products in the same recommendation cluster.
- **F7 File organisation:** The order log is a sequential file; the product catalogue sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on product id is kept over the catalogue.
- **F9 Extendible hashing / external sort:** Millions of daily order records do not fit in memory, so they are sorted with an external merge sort.

Extension: Updating personalised recommendations in real time.

130 — Price Comparison and Product Search

Summary: A console application that compares the same product's price across different stores and shows the user the cheapest option. It works over about 800 synthetic products and 20 stores; price updates are reflected immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A product's price history is kept as a doubly linked list.
- **V2 Sparse matrix:** In the store $\times$ product price table, only cells for products that store actually sells are stored.
- **V3 Stack and queue:** Search history is kept on a stack; price-check requests are processed in order from a queue.
- **V4 Tree and heap:** The lowest-priced options are listed by pulling them from the heap in order.
- **V5 Graph and BFS/DFS:** The category hierarchy is nodes and edges; BFS browses subcategories, DFS scans the whole category tree.
- **V6 Search and hashing:** Product id maps to per-store price records in a hash table; binary search over the sorted price list answers range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders category-tree processing; cycle detection catches an invalid circular category link.
- **F3 Sorting:** Search results are sorted by price and rating with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Products are kept in an AVL tree keyed by price; price-range queries are answered quickly on the balanced tree.
- **F5 String algorithms:** KMP searches for a product name; edit distance lists the closest products for a fuzzy search.
- **F6 Trie and disjoint sets:** Product names are auto-completed with a trie; union-find groups products in the same model or variant group.
- **F7 File organisation:** The price-update log is a sequential file; the product catalogue sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on price is kept over the catalogue.
- **F9 Extendible hashing / external sort:** Millions of daily price records do not fit in memory, so they are sorted with an external merge sort.

Extension: An alert system that notifies users when a price drops.

131 — Supermarket Checkout Queue Simulator



Summary: A console application that simulates checkout queues and barcode scanning in a supermarket and suggests the shortest queue. It works over about 10 registers and 600 synthetic customers per hour; the suggested register changes with the number of items in a customer's basket.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A register's waiting-customer list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the register × hour transaction-count table, only cells with transactions are stored.
- **V3 Stack and queue:** The checkout queue holds customers in order; voiding the last item on a receipt is done with an undo stack.
- **V4 Tree and heap:** The shortest queue is picked from the heap to route a customer to the express lane.
- **V5 Graph and BFS/DFS:** Store aisles are nodes and passages are edges; BFS finds the path from an aisle to the nearest free register.
- **V6 Search and hashing:** Barcode maps to a product price in a hash table; binary search runs over the sorted barcode list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest time for a customer to reach the nearest free register; cycle detection finds a dead-end loop in the aisle layout.
- **F3 Sorting:** Receipts are sorted by total amount and hour with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The product price list is kept in an AVL tree keyed by barcode.
- **F5 String algorithms:** KMP searches for a product name during a price check; edit distance suggests the closest product on a typo.
- **F6 Trie and disjoint sets:** Product names are kept with a trie; union-find groups registers under the same staff shift.
- **F7 File organisation:** The transaction log is a sequential file; the product price table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the transaction log.
- **F9 Extendible hashing / external sort:** Millions of end-of-day transaction records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Automatically suggesting an extra register based on predicted queue length.

132 — Stock Exchange Order Book Matching Engine



Summary: A console application that simulates a stock-exchange order book, matching buy and sell orders by price-time priority. It works over about 20 synthetic stocks and 200 orders per second; every match is recorded as a trade.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Orders at the same price level are kept in arrival order as a doubly linked list.
- **V2 Sparse matrix:** In the symbol $\times$ price-level volume table, only cells with orders are stored.
- **V3 Stack and queue:** Orders at a price level wait in a queue; a cancelled order is removed with an undo stack.
- **V4 Tree and heap:** The best bid/ask price is picked from the heap instantly.
- **V5 Graph and BFS/DFS:** A trade chain is nodes and edges; BFS traces an order's matching chain, DFS finds suspicious circular (wash) trades.
- **V6 Search and hashing:** Order id maps to an order record in a hash table; binary search over the sorted price levels applies price-time priority.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection flags suspected wash trades; topological sort determines the settlement dependency order.
- **F3 Sorting:** Orders are sorted by price and time priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The order book is kept in an AVL tree keyed by price.
- **F5 String algorithms:** KMP searches for a ticker symbol; edit distance suggests the closest symbol on a typo.
- **F6 Trie and disjoint sets:** Ticker symbols are kept with a trie; union-find merges related orders into one trade batch.
- **F7 File organisation:** The trade log is a sequential file; the order table sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the trade log.
- **F9 Extendible hashing / external sort:** Millions of trade records for end-of-day settlement do not fit in memory, so they are sorted with an external merge sort.

Extension: Visualising market depth in real time.

133 — Bank Branch Queue and Teller Simulator



Summary: A console application that simulates the customer queue and teller transactions at a bank branch. It works over 5 tellers and about 400 synthetic customers a day; each transaction type has a different average duration.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The waiting-customer list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the teller × hour transaction-count table, only cells with transactions are stored.
- **V3 Stack and queue:** Customers wait in a queue by ticket number; a mistyped transaction is corrected with an undo stack.
- **V4 Tree and heap:** Priority (VIP) customers are moved ahead from the heap.
- **V5 Graph and BFS/DFS:** The branch network is nodes and edges; BFS finds the nearest branch with a free teller.
- **V6 Search and hashing:** Account number maps to an account record in a hash table; binary search runs over the sorted account numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra routes a customer to the nearest suitable branch; Kruskal plans the ATM network cabling at the lowest cost.
- **F3 Sorting:** Transactions are sorted by amount and hour with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Accounts are kept in an AVL tree keyed by account number.
- **F5 String algorithms:** KMP searches transaction description text; edit distance finds similar descriptions.
- **F6 Trie and disjoint sets:** Account-number prefixes (branch codes) are grouped with a trie; union-find keeps joint accounts in one group.
- **F7 File organisation:** The transaction log is a sequential file; the account table sits in a direct-access file (hashed by account number).
- **F8 B+ tree index:** A B+ tree secondary index on account number is kept over the account table.
- **F9 Extendible hashing / external sort:** Millions of end-of-day transaction records do not fit in memory, so they are sorted with an external merge sort.

Extension: Turning unusual transaction patterns into fraud alerts.

134 — 🚚 Truck Loading (Bin Packing) Planner



Summary: A console application that packs differently sized packages into a truck's cargo bed to leave the least wasted space (bin packing). It works over about 300 synthetic packages and a single truck bed; each package's size and weight are randomly generated.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of packages to load is kept as a doubly linked list.
- **V2 Sparse matrix:** In the cargo-slot × package-size occupancy table, only cells for occupied slots are stored.
- **V3 Stack and queue:** The loading order is kept on a stack (last loaded, first unloaded); pending packages are ordered in a queue.
- **V4 Tree and heap:** For the largest-first placement heuristic, the largest package is picked from the heap by size.
- **V5 Graph and BFS/DFS:** Delivery stops are nodes and edges; BFS/DFS check whether a stop ordering is feasible.
- **V6 Search and hashing:** Package id maps to a package record in a hash table; binary search runs over the sorted package weights.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest delivery route; Kruskal plans the depot network at the lowest cost.
- **F3 Sorting:** Packages are sorted by size and weight with three different algorithms to compare placement heuristics.
- **F4 BST and AVL:** Packages are kept in an AVL tree keyed by volume.
- **F5 String algorithms:** KMP searches a destination address; edit distance suggests the closest address on a typo.
- **F6 Trie and disjoint sets:** Destination postcodes are kept with a trie; union-find groups packages in the same delivery zone.
- **F7 File organisation:** The loading manifest is a sequential file; the package table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on destination is kept over the manifest.
- **F9 Extendible hashing / external sort:** Millions of monthly shipment records do not fit in memory, so they are sorted with an external merge sort.

Extension: Balancing a load across several trucks.

135 — Parcel Hub Sorting Simulator



Summary: A console application that scans parcels arriving at a parcel hub on a conveyor belt and routes each one to the correct chute by destination zone. It works over about 5,000 synthetic parcels; misrouted parcels are flagged separately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The parcel order on the conveyor belt is kept as a doubly linked list.
- **V2 Sparse matrix:** In the chute × hour parcel-count table, only cells with parcels passing through are stored.
- **V3 Stack and queue:** Each chute's parcel queue is drained in order; misrouted parcels move to a manual-recheck stack.
- **V4 Tree and heap:** Priority (express) parcels are picked from the heap and handled first.
- **V5 Graph and BFS/DFS:** The hub network is nodes and edges; BFS finds the shortest route between two hubs, DFS lists all possible routes.
- **V6 Search and hashing:** Tracking number maps to a parcel record in a hash table; binary search runs over the sorted tracking numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest route between hubs; Prim plans the hub-network backbone at the lowest cost.
- **F3 Sorting:** Parcels are sorted by destination zone and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Parcels are kept in an AVL tree keyed by tracking number.
- **F5 String algorithms:** KMP searches an address label; edit distance fixes an unreadable address.
- **F6 Trie and disjoint sets:** Postcode prefixes are kept with a trie to speed up sorting by postcode; union-find groups parcels in the same delivery batch.
- **F7 File organisation:** The scan log is a sequential file; the parcel table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the scan log.
- **F9 Extendible hashing / external sort:** Millions of daily scan records do not fit in memory, so they are sorted by time with an external merge sort.

Extension: Detecting misrouted parcels in real time and rerouting them.

136 — 🚚 Food Delivery Courier Assignment



Summary: A console application that assigns orders arriving from restaurants to the best available courier and suggests a delivery route. It works over about 80 synthetic couriers and 300 orders per hour; the courier's location is taken into account for every assignment.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The orders assigned to a courier are kept as a doubly linked list.
- **V2 Sparse matrix:** In the courier × zone availability table, only cells for couriers working that zone are stored.
- **V3 Stack and queue:** The order queue is processed in arrival order; a cancelled delivery is removed with an undo stack.
- **V4 Tree and heap:** The courier nearest the restaurant is picked from the heap for the assignment.
- **V5 Graph and BFS/DFS:** The city road network is nodes and edges; BFS finds the shortest route by hop count, DFS finds full coverage of a zone.
- **V6 Search and hashing:** Order number maps to an order record in a hash table; binary search runs over the sorted courier ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest delivery route; strongly connected components find delivery zones that can reach each other.
- **F3 Sorting:** Couriers are sorted by distance and rating with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Couriers are kept in an AVL tree keyed by rating.
- **F5 String algorithms:** KMP searches for a restaurant name; edit distance suggests the closest restaurant name on a typo.
- **F6 Trie and disjoint sets:** Restaurant names are kept with a trie; union-find groups couriers in the same delivery zone.
- **F7 File organisation:** The delivery log is a sequential file; the courier table sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the delivery log.
- **F9 Extendible hashing / external sort:** Millions of daily delivery records do not fit in memory, so they are sorted with an external merge sort.

Extension: Recomputing estimated arrival time in real time from traffic data.

137 — Barcode Stock Count and Index



Summary: A console application that scans a store's products by barcode, compares the count to recorded stock, and reports discrepancies. It works over about 4,000 synthetic products; every scan updates the live stock count.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A shelf's or bin's product list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the warehouse-zone × product-count table, only cells for products found in that zone are stored.
- **V3 Stack and queue:** Scans are processed in order from a queue; products with a discrepancy are pushed to a recount stack.
- **V4 Tree and heap:** Products running low on stock are moved ahead from the heap for an alert.
- **V5 Graph and BFS/DFS:** The category hierarchy is nodes and edges; BFS/DFS compute a rolled-up stock summary for parent categories.
- **V6 Search and hashing:** Barcode maps to a product record in a hash table; binary search runs over the sorted barcode list.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the category roll-up order; cycle detection catches an invalid category loop.
- **F3 Sorting:** Products are sorted by stock level and discrepancy size with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Products are kept in an AVL tree keyed by barcode.
- **F5 String algorithms:** KMP searches a product description; edit distance suggests the closest description on a typo.
- **F6 Trie and disjoint sets:** Barcode prefixes (manufacturer codes) are grouped with a trie; union-find groups products from the same lot.
- **F7 File organisation:** The scan log is a sequential file; the product table sits in a direct-access file (hashed by barcode).
- **F8 B+ tree index:** A B+ tree secondary index on barcode is kept over the product table.
- **F9 Extendible hashing / external sort:** Millions of daily scan records do not fit in memory, so they are sorted with an external merge sort.

Extension: Scheduling cycle counts by discrepancy risk.

138 — ✕ Spare Parts Bill of Materials Explorer



Summary: A console application that walks a machine's bill of materials (BOM) to show its sub-parts and list what is missing. It works over about 600 synthetic parts in nested assemblies; ordering a top-level assembly computes the full sub-part demand.

Midterm — C (weeks 1–6):

- **V1 Linked list:** An assembly's sub-part list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the assembly $\times$ part-quantity table, only cells for parts used in that assembly are stored.
- **V3 Stack and queue:** Exploding the BOM recursively is done with a stack; orders for missing parts are ordered in a queue.
- **V4 Tree and heap:** The BOM is kept as a hierarchical tree; reorder priority is ordered in the heap by shortage severity.
- **V5 Graph and BFS/DFS:** Part dependencies are nodes and edges; BFS explodes the assembly level by level, DFS walks all sub-assemblies.
- **V6 Search and hashing:** Part number maps to a part record in a hash table; binary search runs over the sorted part numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the build order; cycle detection catches an invalid part that depends on itself.
- **F3 Sorting:** Parts are sorted by cost and quantity with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Parts are kept in an AVL tree keyed by part number.
- **F5 String algorithms:** KMP searches for a part name; edit distance suggests the closest part name on a typo.
- **F6 Trie and disjoint sets:** Part-number prefixes are kept with a trie; union-find groups parts shared across multiple assemblies.
- **F7 File organisation:** The usage log is a sequential file; the part table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on part number is kept over the part table.
- **F9 Extendible hashing / external sort:** A large manufacturing line's millions of usage records do not fit in memory, so they are sorted with an external merge sort.

Extension: Recomputing rolled-up cost when a part's price changes.

139 — 📅 Project Task Dependency and Critical Path



Summary: A console application that schedules a project's tasks along with their dependencies and computes the critical path. It works over about 120 synthetic tasks; the critical path is recomputed whenever a task's duration changes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Tasks are kept as a doubly linked list.
- **V2 Sparse matrix:** In the task × resource-allocation table, only cells with an assigned resource are stored.
- **V3 Stack and queue:** Tasks are ordered in a scheduling queue; plan changes are supported by an undo stack.
- **V4 Tree and heap:** The task with the least slack time is picked from the heap first.
- **V5 Graph and BFS/DFS:** Tasks are nodes and dependencies are edges; BFS schedules level by level, DFS finds deep dependency chains.
- **V6 Search and hashing:** Task id maps to a task record in a hash table; binary search runs over the sorted end dates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines task order; the critical path is found with a longest-path computation.
- **F3 Sorting:** Tasks are sorted by end date and duration with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Tasks are kept in an AVL tree keyed by start date.
- **F5 String algorithms:** KMP searches for a task name; edit distance suggests the closest task name on a typo.
- **F6 Trie and disjoint sets:** Task names are kept with a trie; union-find groups tasks in the same work package.
- **F7 File organisation:** The progress log is a sequential file; the task table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on end date is kept over the task table.
- **F9 Extendible hashing / external sort:** A large programme's millions of progress records do not fit in memory, so they are sorted with an external merge sort.

Extension: Rebalancing the plan by resource load (resource levelling).

140 — \$ Currency Arbitrage Cycle Finder



Summary: A console application that watches exchange rates between currency pairs and finds arbitrage cycles where a chain of conversions returns more than the starting currency. It works over about 15 synthetic currencies and the rate table between them; rates update once a minute.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A currency's rates against other currencies are kept as a doubly linked list.
- **V2 Sparse matrix:** In the currency × currency rate table, only cells for pairs with a direct quote are stored.
- **V3 Stack and queue:** The chain tried while searching for a conversion path is kept on a backtracking stack; rate updates are processed from a queue.
- **V4 Tree and heap:** The conversion with the best rate is picked from the heap.
- **V5 Graph and BFS/DFS:** Currencies are nodes and rates are edges; BFS finds the shortest conversion chain, DFS walks every conversion path.
- **V6 Search and hashing:** Currency pair maps to a rate in a hash table; binary search runs over sorted rates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds the profitable negative cycle (arbitrage); Dijkstra computes the best conversion chain.
- **F3 Sorting:** Currency pairs are sorted by volatility with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Rate history is kept in an AVL tree keyed by timestamp.
- **F5 String algorithms:** KMP searches for a currency code; edit distance suggests the closest code on a typo.
- **F6 Trie and disjoint sets:** Currency codes are kept with a trie; union-find groups currencies pegged to the same base currency.
- **F7 File organisation:** The rate-update log is a sequential file; the rate table sits in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the rate log.
- **F9 Extendible hashing / external sort:** A long monitoring session's millions of rate records do not fit in memory, so they are sorted with an external merge sort.

Extension: Turning a new arbitrage opportunity into a real-time alert as soon as a rate arrives.

141 — 🎫 Concert Hall Seat Map and Ticket Queue



Summary: A console application that manages a concert hall's seat map and puts ticket-purchase requests into a virtual waiting-room queue. It works over about 2,000 synthetic seats across 5 events on sale; every purchase updates seat status immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The waiting list of customers who could not buy a ticket is kept as a doubly linked list.
- **V2 Sparse matrix:** In the event × seat-hold table, only cells for seats currently held are stored.
- **V3 Stack and queue:** Customers in the virtual waiting room are admitted to checkout in order from a queue; a cancelled reservation is removed with an undo stack.
- **V4 Tree and heap:** Fan-club priority customers are called from the waiting-list heap first.
- **V5 Graph and BFS/DFS:** Adjacency between seats is nodes and edges; BFS searches for a block of adjacent free seats for a group booking.
- **V6 Search and hashing:** Seat id maps to a reservation in a hash table; binary search over the sorted seat numbers supports adjacent-seat search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Max flow fairly distributes the limited ticket count across different sales channels; cycle detection catches a double-booking loop.
- **F3 Sorting:** The waiting list is sorted by priority and time with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Reservations are kept in an AVL tree keyed by seat id.
- **F5 String algorithms:** KMP searches for an event name; edit distance suggests the closest event name on a typo.
- **F6 Trie and disjoint sets:** Event names are auto-completed with a trie; union-find groups adjacent seat blocks for group sales.
- **F7 File organisation:** The sales log is a sequential file; the seat map sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the sales log.
- **F9 Extendible hashing / external sort:** A popular event's millions of sales records do not fit in memory, so they are sorted with an external merge sort.

Extension: Dynamic pricing based on demand.

142 — 🚚 Vending Machine Restock Route Planner



Summary: A console application that monitors the stock levels of vending machines spread across a city and suggests the most efficient visiting route for the restock crew. It works over about 120 synthetic machines; every sale decrements a machine's stock counter immediately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A machine's product slots are kept as a doubly linked list.
- **V2 Sparse matrix:** In the machine × product stock-level table, only cells for products that machine sells are stored.
- **V3 Stack and queue:** Restock tasks are processed in order from a queue; low-stock alerts are pushed onto an alert stack.
- **V4 Tree and heap:** The machine with the lowest stock is picked from the heap first and added to the restock list.
- **V5 Graph and BFS/DFS:** Machine locations are nodes and edges; BFS/DFS plan the restock route.
- **V6 Search and hashing:** Machine id maps to status in a hash table; binary search runs over the sorted machine ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest restock route; Kruskal plans the visiting network at the lowest cost.
- **F3 Sorting:** Machines are sorted by stock urgency and distance with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Machines are kept in an AVL tree keyed by machine id.
- **F5 String algorithms:** KMP searches for a product name; edit distance suggests the closest product name on a typo.
- **F6 Trie and disjoint sets:** Product codes are kept with a trie; union-find groups machines in the same restock-route cluster.
- **F7 File organisation:** The sales log is a sequential file; the machine stock table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on machine id is kept over the stock table.
- **F9 Extendible hashing / external sort:** Millions of daily sales records do not fit in memory, so they are sorted with an external merge sort.

Extension: Predictive restocking based on sales rate.

143 — Store Chain Stock Transfer Network



Summary: A console application that suggests transferring stock between branches of a store chain, from where a product is oversupplied to where it is short. It works over about 40 synthetic branches and 600 products; when a branch runs out, the nearest source branch is searched for.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Pending transfer requests are kept as a doubly linked list.
- **V2 Sparse matrix:** In the branch × product stock-level table, only cells for products present at that branch are stored.
- **V3 Stack and queue:** Transfer requests are processed in order from a queue; a cancelled transfer is removed with an undo stack.
- **V4 Tree and heap:** The transfer with the highest stockout risk is processed from the heap first.
- **V5 Graph and BFS/DFS:** The branch network is nodes and edges; BFS finds the nearest branch with stock, DFS scans the whole network's stock status.
- **V6 Search and hashing:** Product id maps to per-branch stock in a hash table; binary search runs over the sorted branch ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the route to the nearest source branch; Kruskal plans the distribution network at the lowest cost.
- **F3 Sorting:** Transfer requests are sorted by urgency and distance with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Branches are kept in an AVL tree keyed by branch id.
- **F5 String algorithms:** KMP searches for a product name; edit distance suggests the closest product name on a typo.
- **F6 Trie and disjoint sets:** Product codes are kept with a trie; union-find groups branches in the same regional distribution cluster.
- **F7 File organisation:** The transfer log is a sequential file; the stock table sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on product id is kept over the stock table.
- **F9 Extendible hashing / external sort:** Millions of daily transfer records do not fit in memory, so they are sorted with an external merge sort.

Extension: An automatic rebalancing-suggestion engine.

144 — 🌸 Florist Cold Room Shelf-Life Manager



Summary: A console application that tracks flower batches in a florist's cold room by expiry date and fulfils orders from the oldest batch first. It works over about 300 synthetic batches; each batch is defined by a flower type, arrival date and shelf life.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A flower type's batches are kept in arrival order as a doubly linked list.
- **V2 Sparse matrix:** In the shelf × flower-type quantity table, only cells for types present on that shelf are stored.
- **V3 Stack and queue:** Order fulfilment is processed in order from a queue; expired batches move to a discard stack.
- **V4 Tree and heap:** The next batch to expire is picked from the heap by nearest expiry date.
- **V5 Graph and BFS/DFS:** The supplier delivery network is nodes and edges; BFS/DFS walk a flower type's supply chain.
- **V6 Search and hashing:** Batch id maps to a batch record in a hash table; binary search runs over the sorted expiry dates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest supplier delivery route; topological sort determines the order of bouquet-assembly steps.
- **F3 Sorting:** Batches are sorted by expiry date and quantity with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Batches are kept in an AVL tree keyed by expiry date.
- **F5 String algorithms:** KMP searches for a flower-type name; edit distance suggests the closest type name on a typo.
- **F6 Trie and disjoint sets:** Flower type and colour names are kept with a trie; union-find groups batches belonging to the same bouquet order.
- **F7 File organisation:** The waste log is a sequential file; the batch table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on expiry date is kept over the batch table.
- **F9 Extendible hashing / external sort:** Millions of yearly sales and waste records do not fit in memory, so they are sorted with an external merge sort.

Extension: Automatic discount pricing as expiry approaches.

145 — Supply Chain Disruption Impact Analyser



Summary: A console application that computes, through the supply chain, which products and customers a disruption at a supplier will affect. It works over about 80 synthetic suppliers and 300 components; when a supplier goes offline, every affected product is listed.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A component's supplier list is kept as a doubly linked list.
- **V2 Sparse matrix:** In the supplier × component supply-capacity table, only cells for components that supplier provides are stored.
- **V3 Stack and queue:** Disruption events are processed in order from a queue; recovery actions are tracked on an undo stack.
- **V4 Tree and heap:** The most critical disruption is handled from the heap first by impact size.
- **V5 Graph and BFS/DFS:** The supply chain is nodes and edges; BFS finds the downstream products a disruption affects, DFS finds the full impact chain.
- **V6 Search and hashing:** Component id maps to a supplier list in a hash table; binary search runs over the sorted supplier ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components find clusters of mutually dependent suppliers; Dijkstra computes an alternate supply route.
- **F3 Sorting:** Disruptions are sorted by impact size with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Components are kept in an AVL tree keyed by component id.
- **F5 String algorithms:** KMP searches a supplier or component name; edit distance suggests the closest name on a typo.
- **F6 Trie and disjoint sets:** Component codes are kept with a trie; union-find gathers components affected by the same disrupted supplier into one impact cluster.
- **F7 File organisation:** The disruption log is a sequential file; the supplier table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on component id is kept over the supplier table.
- **F9 Extendible hashing / external sort:** A long monitoring session's millions of disruption records do not fit in memory, so they are sorted with an external merge sort.

Extension: A what-if simulation for switching to an alternate supplier.

146 — Machine Maintenance Scheduler



Summary: A console application that tracks a factory's machines by their periodic maintenance dates and suggests a priority order for the maintenance crew. It works over about 90 synthetic machines; one machine's failure can also affect the maintenance priority of others.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A machine's maintenance history is kept as a doubly linked list.
- **V2 Sparse matrix:** In the machine × month maintenance-hours table, only cells for months with maintenance are stored.
- **V3 Stack and queue:** Maintenance requests are processed in order from a queue; completed maintenance is kept on a history stack.
- **V4 Tree and heap:** The machine due for maintenance soonest is picked from the heap.
- **V5 Graph and BFS/DFS:** Production-line dependency between machines is nodes and edges; BFS/DFS find the line section affected if a machine stops.
- **V6 Search and hashing:** Machine id maps to a maintenance record in a hash table; binary search runs over the sorted next-due dates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the order of multi-stage maintenance dependencies; cycle detection catches an invalid circular dependency.
- **F3 Sorting:** Machines are sorted by urgency and downtime cost with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The maintenance schedule is kept in an AVL tree keyed by due date.
- **F5 String algorithms:** KMP searches for a machine name; edit distance suggests the closest machine name on a typo.
- **F6 Trie and disjoint sets:** Machine model codes are kept with a trie; union-find groups machines on the same maintenance technician's route.
- **F7 File organisation:** The maintenance log is a sequential file; the machine table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on due date is kept over the machine table.
- **F9 Extendible hashing / external sort:** A long monitoring session's millions of maintenance records do not fit in memory, so they are sorted with an external merge sort.

Extension: Predictive maintenance forecasting from sensor trend data.

147 — 3D Printer Job Queue and Layer Slicer



Summary: A console application that slices model files sent to a 3D printer into layers and queues the resulting jobs. It works over about 200 synthetic jobs averaging 150 layers per model; each layer's infill pattern is computed separately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A job's layers are kept in order as a doubly linked list.
- **V2 Sparse matrix:** In the layer $\times$ pixel occupancy table, only cells matching the model's cross-section on that layer are stored.
- **V3 Stack and queue:** The printer job queue is processed in order; slicer-setting changes are kept on an undo stack.
- **V4 Tree and heap:** Jobs with a near deadline are picked from the heap first for printing.
- **V5 Graph and BFS/DFS:** Model-surface connectivity is nodes and edges; BFS/DFS find regions that need support structures.
- **V6 Search and hashing:** Job id maps to a job record in a hash table; binary search runs over the sorted job ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra minimises the print head's travel distance within a layer; cycle detection finds invalid (non-manifold) model geometry.
- **F3 Sorting:** Jobs are sorted by estimated time and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** The printer queue is kept in an AVL tree keyed by priority.
- **F5 String algorithms:** KMP searches for a model filename; edit distance suggests the closest filename on a typo.
- **F6 Trie and disjoint sets:** Material and colour codes are kept with a trie; union-find gathers a layer's connected surface islands into one group.
- **F7 File organisation:** The print log is a sequential file; the job table sits in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the print log.
- **F9 Extendible hashing / external sort:** A busy printer farm's millions of print records do not fit in memory, so they are sorted with an external merge sort.

Extension: Balancing workload across several printers.

148 — Promotion and Discount Rule Engine

Summary: A console application that applies a shopping site's promo codes and discount rules to a cart, managing which rule runs in which order. It works over about 150 synthetic promotion rules; some promotions cannot be combined with each other.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The list of active promotions is kept as a doubly linked list.
- **V2 Sparse matrix:** In the product × promo-code applicability table, only cells for products that promotion applies to are stored.
- **V3 Stack and queue:** Discounts applied to a cart are pushed onto a stack in order (stacked discounts); rule evaluation order is processed through a queue.
- **V4 Tree and heap:** Rule evaluation is kept as a decision tree; the highest discount is picked from the heap by priority.
- **V5 Graph and BFS/DFS:** Mutual-exclusion relations between rules (some promotions cannot be used together) are nodes and edges; BFS/DFS find conflicting rules.
- **V6 Search and hashing:** Promo code maps to a rule record in a hash table; binary search runs over the sorted promo codes.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the evaluation order of rules; cycle detection catches a contradictory mutual-exclusion rule.
- **F3 Sorting:** Promotions are sorted by discount value and priority with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Promotions are kept in an AVL tree keyed by start date.
- **F5 String algorithms:** KMP searches for a promo code; edit distance suggests the closest code on a typo.
- **F6 Trie and disjoint sets:** Promo-code prefixes are kept with a trie; union-find groups mutually exclusive promotions into one conflict cluster.
- **F7 File organisation:** The redemption log is a sequential file; the rule table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on timestamp is kept over the redemption log.
- **F9 Extendible hashing / external sort:** A large campaign's millions of redemption records do not fit in memory, so they are sorted with an external merge sort.

Extension: Measuring discount-variant impact on sales through A/B test analysis.

149 — Invoice Reconciliation Matcher



Summary: A console application that simulates an accounting reconciliation tool, comparing incoming invoices line by line against purchase orders and flagging mismatches. It works over about 5,000 synthetic invoices; each line item's amount and quantity are compared separately.

Midterm — C (weeks 1–6):

- **V1 Linked list:** An invoice's line items are kept as a doubly linked list.
- **V2 Sparse matrix:** In the invoice × purchase-order line-match table, only cells where a match was attempted are stored.
- **V3 Stack and queue:** Reconciliation checks are processed in order from a queue; a bad match is corrected with an undo stack.
- **V4 Tree and heap:** Invoices with the largest discrepancy are picked from the heap first for review.
- **V5 Graph and BFS/DFS:** The invoice-order-payment reference chain is nodes and edges; BFS/DFS trace an invoice's full matching chain.
- **V6 Search and hashing:** Invoice number maps to an invoice record in a hash table; binary search runs over the sorted invoice numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Cycle detection finds an invalid self-referencing invoice loop; topological sort determines the multi-step approval order.
- **F3 Sorting:** Invoices are sorted by discrepancy size and date with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Invoices are kept in an AVL tree keyed by invoice number.
- **F5 String algorithms:** KMP searches supplier/product descriptions; edit distance matches similar line-item descriptions across the two documents.
- **F6 Trie and disjoint sets:** Invoice-number prefixes are kept with a trie; union-find groups invoices in the same payment batch.
- **F7 File organisation:** The reconciliation log is a sequential file; the invoice table sits in a direct-access file (collisions resolved by chaining).
- **F8 B+ tree index:** A B+ tree secondary index on date is kept over the invoice table.
- **F9 Extendible hashing / external sort:** Millions of yearly invoice records do not fit in memory, so they are sorted with an external merge sort.

Extension: Automatically flagging recurring mismatches as anomalies.

150 — Port Container Stacking Planner



Summary: A console application that plans which block and tier to stack containers in on a port yard, trying to minimise crane movement. It works over about 2,500 synthetic containers; every container has a ship departure date.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The container list for a row is kept as a doubly linked list.
- **V2 Sparse matrix:** In the block × tier occupancy table, only cells with a container are stored.
- **V3 Stack and queue:** Containers in a block are kept as a stack (the top one comes down first); crane tasks are processed in order from a queue.
- **V4 Tree and heap:** The container with the nearest departure date is picked from the heap first for unloading.
- **V5 Graph and BFS/DFS:** The yard layout is nodes and edges; BFS finds the shortest crane travel path, DFS sweeps the whole yard to find a container.
- **V6 Search and hashing:** Container id maps to a location in a hash table; binary search runs over the sorted container ids.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra minimises crane travel distance; Kruskal plans the yard rail/crane network at the lowest cost.
- **F3 Sorting:** Containers are sorted by departure date and weight with three different algorithms and their running times are compared.
- **F4 BST and AVL:** Containers are kept in an AVL tree keyed by departure date.
- **F5 String algorithms:** KMP searches a container id or manifest; edit distance suggests the closest id on a typo.
- **F6 Trie and disjoint sets:** Container-id prefixes (shipping-line codes) are grouped with a trie; union-find gathers containers for the same vessel into one loading batch.
- **F7 File organisation:** The movement log is a sequential file; the container table sits in a direct-access file (collisions resolved by open addressing).
- **F8 B+ tree index:** A B+ tree secondary index on departure date is kept over the container table.
- **F9 Extendible hashing / external sort:** A busy port's millions of movement records do not fit in memory, so they are sorted with an external merge sort.

Extension: Scheduling crane moves to minimise unnecessary reshuffling.

151–175 · Media, social networks and culture

151 — Social Network Friend Recommender

Summary: A console application that suggests new connections to users based on mutual friend counts, shared interests and interaction frequency. It runs on a synthetic network of about 2,000 users and 15,000 friendship links; blocked users and accounts with privacy settings turned off are automatically excluded from suggestions.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each user's friend list is kept as a doubly linked list for adding and removing friends; the "last 10 profiles viewed" list is a circular list that drops the oldest entry automatically.
- **V2 Sparse matrix:** In the 2,000×2,000 user interaction matrix (likes and comments), only pairs that actually interacted are stored — roughly 2% of cells are filled.
- **V3 Stack and queue:** Sent friend requests can be undone through a stack; pending approval requests are processed FIFO through a queue.
- **V4 Tree and heap:** Interest categories (sports, music, technology and their sub-branches) are organised in a binary tree; recommendation candidates sit in a heap keyed by mutual-friend count so the top 10 matches can be pulled instantly.
- **V5 Graph and BFS/DFS:** Users are nodes and follow relationships are directed edges in an adjacency list; BFS builds the "friend of a friend" candidate set, and DFS finds connected components that reveal isolated user clusters.
- **V6 Search and hashing:** Username maps to user record in a hash table with chaining; the sorted array of user IDs supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes a weighted "how many hops away" distance based on interaction frequency; Kosaraju's algorithm finds strongly connected components to surface tightly-knit friend groups.
- **F3 Sorting:** About 5,000 recommendation candidates are sorted by mutual-friend count using insertion sort, quicksort and merge sort, with running times compared.
- **F4 BST and AVL:** Users are kept in a BST by join date; an AVL tree keyed by username, rebalanced through rotations, supports fast lookups.
- **F5 String algorithms:** Username search uses KMP; a misspelled friend-name query returns the three closest matches via edit distance.
- **F6 Trie and disjoint sets:** Usernames are autocompleted with a trie; union-find merges users into community (friend-group) clusters based on friendship links.
- **F7 File organisation:** User profile records live in a sequential file, and login logs live in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The user file carries a B+ tree secondary index on the email field.
- **F9 Extendible hashing / external sort:** The growing user table is managed with extendible hashing, doubling the directory to absorb growth without overflow.

Extension: Recommendation scoring could be extended with a machine-learning model based on interest similarity.

152 — Music Playlist and Shuffle Engine



Summary: A desktop application that builds listening sessions from a user's personal song library and computes a balanced, non-repetitive shuffle order. It works with a synthetic library of about 5,000 tracks and playlists of up to 200 songs; user ratings feed the recommendation engine.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A playlist is a doubly linked list for moving forward and backward between tracks; "continuous repeat" mode wraps from the last song back to the first through a circular list.
- **V2 Sparse matrix:** In the song $\times$ user rating matrix (1–5 stars), each user rates only a small slice of the library, so only the ratings actually given are stored.
- **V3 Stack and queue:** "Go back to the previous track" pops a stack of recently played songs; songs the user queues up next play out FIFO from a queue.
- **V4 Tree and heap:** Songs are categorised by genre and sub-genre (rock, pop, jazz) in a binary tree; a heap keyed by play count keeps the "top 10 most played" list instantly available.
- **V5 Graph and BFS/DFS:** "Similar style" links between songs form a graph via adjacency lists; BFS starting from one track finds radio-mode suggestions, and DFS explores the whole connected genre cluster.
- **V6 Search and hashing:** Song title maps to song record in a hash table with open addressing; songs sorted by release year support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm builds a minimum spanning tree over tempo/key differences to give the smoothest shuffle order; Dijkstra computes the shortest transition chain from a calm track to an energetic one.
- **F3 Sorting:** Songs are sorted by play count using selection sort, quicksort and heapsort, and their running times are compared.
- **F4 BST and AVL:** Songs are kept in a BST by duration; an AVL tree keyed by artist name, rebalanced with rotations, supports fast lookups.
- **F5 String algorithms:** Lyric search uses Boyer-Moore; similar-title suggestions are found via the longest common subsequence (LCS).
- **F6 Trie and disjoint sets:** Artist names are autocompleted with a trie; union-find groups songs played back-to-back in the same session into "played together" clusters.
- **F7 File organisation:** Song metadata lives in a sequential file, and play-history entries live in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The song file carries a B+ tree secondary index on artist ID.
- **F9 Extendible hashing / external sort:** As the 5,000-track library grows, extendible hashing manages the song-ID directory.

Extension: Automatic mood-based playlist generation could be added based on a user's listening habits.

153 — 🎬 Movie Recommendation and Similarity Engine



Summary: An application that recommends new titles from a synthetic catalogue of 3,000 films rated by 800 users, drawing on a viewer's past ratings as well as shared-cast and shared-director connections between films. Recommendations combine rating similarity with structural links between titles.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Films waiting in a user's watch queue are kept in a doubly linked list; browsing forward and backward through viewing history uses a memory-efficient XOR linked list.
- **V2 Sparse matrix:** In the 800×3,000 user-film rating matrix, each user rates on average only about 3% of the catalogue, so only the ratings given are stored.
- **V3 Stack and queue:** Filter choices (genre, year, runtime) can be undone through a stack; films added to the watch list are processed FIFO through a queue.
- **V4 Tree and heap:** Films are organised by genre and sub-genre (action > thriller > espionage) in a binary tree; recommendation candidates sit in a heap keyed by similarity score so the top 10 matches can be pulled.
- **V5 Graph and BFS/DFS:** Films are nodes and shared-cast/shared-director links are edges in an adjacency list; BFS finds how many hops separate two films through an actor chain, and DFS explores a director's whole connected filmography.
- **V6 Search and hashing:** Film title maps to film record in a hash table with chaining; films sorted by release year support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the closest recommendation chain based on similarity weights; cycle detection in the actor-film graph flags teams that keep working together (closed loops).
- **F3 Sorting:** Film recommendations are sorted by similarity score using insertion sort, quicksort and merge sort, with running times compared.
- **F4 BST and AVL:** Films are kept in a BST by average user rating; an AVL tree keyed by title supports fast lookups.
- **F5 String algorithms:** Title search uses Boyer-Moore; a misspelled query returns the closest title via edit distance.
- **F6 Trie and disjoint sets:** Film and actor names are autocompleted with a trie; union-find merges films connected through a shared-cast or shared-director chain into a single "cinematic universe" cluster.
- **F7 File organisation:** Film records live in a sequential file, and the user rating log lives in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The film file carries a B+ tree secondary index on genre.
- **F9 Extendible hashing / external sort:** When the growing rating log no longer fits in memory, external merge sort orders it by user ID.

Extension: Recommendations could be enriched with content-based filtering using the similarity of film synopsis texts.

154 — Image Flood Fill and Layer Editor



Summary: A simple image editor that performs multi-layer painting and flood-fill operations on 512×512-pixel canvases. Every action taken with the brush, fill-bucket and layer tools can be undone, and a canvas is assumed empty until the user paints on it.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Layers are kept in a doubly linked list to support moving them forward, backward and toggling visibility; the last 12 colours used are kept as a palette in a circular list.
- **V2 Sparse matrix:** On the 512×512-pixel canvas, unpainted pixels count as empty; only painted pixels (colour, layer) are stored in the sparse matrix.
- **V3 Stack and queue:** Brush strokes are undone and redone through a stack; batch export requests (PNG/JPEG) are processed in order through a queue.
- **V4 Tree and heap:** Layer groups (layers nested inside folders) form a binary tree, with pre-order traversal giving the render order; pending filter jobs (blur, sharpen) sit in a heap keyed by processing time so the shortest job runs first.
- **V5 Graph and BFS/DFS:** The pixel grid is modelled as a 4-directional adjacency graph; flood fill uses BFS to find and recolour the connected same-colour region, and DFS extracts that region's boundary (contour) pixels.
- **V6 Search and hashing:** Colour values (RGB codes) map to a usage counter in a hash table with open addressing; the sorted list of layer IDs supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A shortest-path algorithm weighted by pixel-intensity difference computes the boundary for the "intelligent scissors" selection tool; cycle detection blocks circular references among layer masks.
- **F3 Sorting:** Layers are sorted by opacity, pixel count and creation date using three algorithms, and their running times are compared.
- **F4 BST and AVL:** Undo-history actions are kept in a BST by timestamp; an AVL tree keyed by layer name supports fast lookups.
- **F5 String algorithms:** Layer-name search uses KMP; similar filename suggestions use edit distance.
- **F6 Trie and disjoint sets:** Filter and effect names are autocompleted with a trie; union-find merges connected pixel regions touched during flood fill and counts the resulting connected regions ("islands").
- **F7 File organisation:** Project layer metadata lives in a sequential file, and the pixel-block cache lives in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The project file carries a B+ tree secondary index on layer ID.
- **F9 Extendible hashing / external sort:** When high-resolution pixel blocks no longer fit in memory, external merge sort writes them to disk in layer order.

Extension: Real-time multi-user layer editing could be added.

155 — Video Editing Timeline



Summary: A desktop editing application that combines a short video project's clip, audio and effect layers into a single timeline and works out the export order. A typical project has around 50 clips across several video and audio tracks; render jobs are queued and processed in the background.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Clips on the timeline are kept in a doubly linked list to support drag-and-drop reordering; looping audio segments repeat continuously through a circular list.
- **V2 Sparse matrix:** In the clip × effect (transition, filter, colour correction) application matrix, most clips have no effects applied, so only the effects that are actually applied are stored.
- **V3 Stack and queue:** Editing actions (cut, move, delete) can be undone through a stack; export (render) requests are processed in order through a queue.
- **V4 Tree and heap:** Clip groups (scene > shot > clip) form a binary tree; pending render jobs sit in a heap keyed by clip duration so the shortest clip renders first.
- **V5 Graph and BFS/DFS:** Transition and effect dependencies between clips form a graph via adjacency lists; BFS determines a render order that processes dependent clips first, and DFS finds every clip affected when one clip changes.
- **V6 Search and hashing:** Clip name maps to clip record in a hash table with chaining; clips sorted by timecode support binary search for jumping to a given second.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders rendering so an effect that depends on a previous one always runs after it; a shortest-path algorithm finds the lowest-cost route through the export pipeline (raw video → effects → encoding).
- **F3 Sorting:** Clips are sorted by timeline start time, duration and file size using three algorithms, with running times compared.
- **F4 BST and AVL:** Clips are kept in a BST by timecode; an AVL tree keyed by clip name across the whole project supports fast lookups.
- **F5 String algorithms:** Subtitle keyword search uses Boyer-Moore; similar clip-name suggestions use edit distance.
- **F6 Trie and disjoint sets:** Tags are autocompleted with a trie; union-find groups clips that are close together on the timeline and share a tag into the same scene cluster.
- **F7 File organisation:** The project timeline lives in a sequential file, and the video-frame cache lives in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The clip file carries a B+ tree secondary index on scene number.
- **F9 Extendible hashing / external sort:** When the raw frame list no longer fits in memory, external merge sort orders it by timecode.

Extension: Collaborative multi-user editing support could be added for simultaneous timeline changes.

156 — # Hashtag Trend Analyser



Summary: An analytics tool that tracks, hour by hour, which hashtags are trending among roughly 10,000 synthetic posts a day. It turns co-occurrence between hashtags into a graph to surface topic clusters and sudden spikes in usage.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each hashtag's timestamped usage history is kept in a doubly linked list; the rolling 24-hour trend window is a circular list where the oldest hour drops off as a new one is added.
- **V2 Sparse matrix:** In the 24-hour × hashtag usage-count density table, most hashtags appear in only a few hours, so only the filled cells are stored.
- **V3 Stack and queue:** Analysis filter choices (date range, region) can be undone through a stack; incoming posts are queued FIFO for processing.
- **V4 Tree and heap:** Hashtags are organised by category (sports, politics, entertainment) in a binary tree; hashtags sit in a heap keyed by usage count so the current "top 10 trending" list is available instantly.
- **V5 Graph and BFS/DFS:** Hashtags that appear together in the same post are connected by a graph edge; BFS finds the tags most closely related to a given one, and DFS explores a whole connected topic cluster.
- **V6 Search and hashing:** Hashtag name maps to a usage-counter record in a hash table with open addressing; the alphabetically sorted hashtag list supports binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm builds a minimum spanning tree from co-occurrence frequency to give a skeleton topic map; cycle detection flags circular reply chains ($A \rightarrow B \rightarrow A$) in hashtag discussions.
- **F3 Sorting:** Hashtags are sorted by hourly growth rate using selection sort, quicksort and heapsort, with running times compared.
- **F4 BST and AVL:** Hashtags are kept in a BST by first-seen time; an AVL tree keyed by hashtag name supports fast lookups.
- **F5 String algorithms:** In-post hashtag search uses KMP; edit distance merges near-identical spelling variants of the same hashtag.
- **F6 Trie and disjoint sets:** Hashtags are autocompleted with a trie; union-find merges hashtags that co-occur into the same topic cluster.
- **F7 File organisation:** Hourly hashtag counters live in a sequential file, and user post records live in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The post file carries a B+ tree secondary index on the hashtag field.
- **F9 Extendible hashing / external sort:** When the 10,000-plus daily posts no longer fit in memory, external merge sort orders them by timestamp.

Extension: Regional trend comparison across cities could be added.

157 — 🎧 Podcast Episode Archive and Search



Summary: A podcast application that archives over 8,000 episodes from 200 shows and lets listeners find episodes by topic, guest or keyword. Shared-guest connections between episodes drive both browsing and recommendations.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each show's episodes are kept in a doubly linked list in broadcast order (previous/next episode); episodes queued up for continuous listening wrap around through a circular list.
- **V2 Sparse matrix:** In the show $\times$ listener completion-rate matrix, each listener finishes only a handful of episodes from any given show, so only the listener-episode pairs that occurred are stored.
- **V3 Stack and queue:** 15-second rewind/fast-forward actions are undone through a stack; download requests for offline listening are processed in order through a queue.
- **V4 Tree and heap:** Episodes are organised by category and sub-category (science > space, science > biology) in a binary tree; newly released episodes sit in a heap keyed by publish date so the 10 newest can be listed instantly.
- **V5 Graph and BFS/DFS:** Episodes sharing a guest are connected by a graph edge; BFS reaches every episode featuring a given guest in the fewest steps, and DFS explores a show's whole connected guest network.
- **V6 Search and hashing:** Episode title maps to episode record in a hash table with chaining; episodes sorted by publish date support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes a weighted "recommended listening order" route based on episode length and topic similarity; topological sort guarantees multi-part mini-series episodes (part 1 before part 2) play in the correct order.
- **F3 Sorting:** Episodes are sorted by listen count, duration and publish date using three algorithms, with running times compared.
- **F4 BST and AVL:** Episodes are kept in a BST by duration; an AVL tree keyed by show name supports fast lookups.
- **F5 String algorithms:** Transcript keyword search uses KMP; similar show-name suggestions use edit distance.
- **F6 Trie and disjoint sets:** Show and guest names are autocompleted with a trie; union-find merges shows connected through a shared guest into the same network cluster.
- **F7 File organisation:** Episode metadata lives in a sequential file, and listening history lives in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The episode file carries a B+ tree secondary index on show ID.
- **F9 Extendible hashing / external sort:** As the archive of 8,000-plus episodes grows, extendible hashing manages the episode-ID directory.

Extension: Full-text search could be enriched with automatic transcript generation.

158 — Photo Album Duplicate and Similarity Finder



Summary: A desktop tool that finds near-identical shots and exact duplicates in personal photo albums of up to 6,000 pictures and suggests a clean-up to the user. Similar photos from burst shooting are grouped automatically.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Photos in the album are kept in a doubly linked list ordered by capture date for chronological browsing; stepping forward and backward through a large slideshow uses a memory-efficient XOR linked list.
- **V2 Sparse matrix:** In the 6,000×6,000 photo-to-photo similarity matrix, only pairs above the similarity threshold are computed and stored; most pairs are never compared and stay empty.
- **V3 Stack and queue:** Duplicate-deletion actions can be undone through a stack (restoring an accidentally deleted photo); newly added photos are queued FIFO for scanning.
- **V4 Tree and heap:** Photos are organised in a binary tree of folders and sub-folders (holidays > 2024 > beach); duplicate candidates sit in a heap keyed by similarity score so the 10 most similar pairs surface first.
- **V5 Graph and BFS/DFS:** Photos above the similarity threshold are connected by a graph edge; BFS starting from one photo finds its whole near-duplicate set, and DFS groups connected components representing different shots of the same scene.
- **V6 Search and hashing:** File name maps to photo record in a hash table with chaining; photos sorted by capture date support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm builds a minimum spanning tree from similarity-weighted edges to pick a representative photo for each duplicate group; cycle detection flags circular duplicate chains (A marked as a copy of B, B of C, and C of A).
- **F3 Sorting:** Photos are sorted by file size, resolution and similarity score using three algorithms, with running times compared.
- **F4 BST and AVL:** Photos are kept in a BST by capture date; an AVL tree keyed by file name supports fast lookups.
- **F5 String algorithms:** File-name search uses KMP; matching near-identical file names (such as IMG_001 and IMG_001_copy) uses the longest common subsequence (LCS).
- **F6 Trie and disjoint sets:** Location and person tags are autocompleted with a trie; union-find merges mutually similar photos into a single duplicate group.
- **F7 File organisation:** Photo metadata lives in a sequential file, and the thumbnail cache lives in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The photo file carries a B+ tree secondary index on folder ID.
- **F9 Extendible hashing / external sort:** When the 6,000-plus-photo album no longer fits in memory, external merge sort orders it by capture date.

Extension: Automatic grouping of photos featuring the same person could be added using face recognition.

**159 — Vector Drawing Layer Tree**

Summary: A drawing application for editing vector illustrations made of nested layers and groups, aligning shapes and exporting the result. A typical project contains around 300 shapes spread across a few dozen layers and groups.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Shapes within a layer are kept in a doubly linked list ordered by stacking (z) order, to support sending shapes forward and backward; stepping forward and backward through view (zoom/pan) history uses an XOR linked list.
- **V2 Sparse matrix:** In the shape × attribute (fill colour, stroke, shadow, gradient) matrix, most shapes have only one or two attributes set, so only the attributes actually assigned are stored.
- **V3 Stack and queue:** Drawing actions (add, move, rotate a shape) can be undone through a stack; batch export requests (SVG/PNG) are processed in order through a queue.
- **V4 Tree and heap:** Layers and groups form a nested binary tree (folder > group > shape), with pre-order traversal giving the render order; alignment operations sit in a heap keyed by shape size so the largest shape is aligned first.
- **V5 Graph and BFS/DFS:** Shapes belonging to the same group are connected by a graph edge; BFS finds the nearest linked shapes in the same group when one is moved, and DFS explores every nested sub-group inside a group.
- **V6 Search and hashing:** Shape ID maps to a shape-attribute record in a hash table with chaining; shapes sorted by layer depth support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort keeps layer dependencies correct at render time (when one shape acts as a mask for another, the mask renders first); a shortest-path algorithm computes the shortest drawing route between two shapes for the connector tool.
- **F3 Sorting:** Shapes are sorted by area, creation order and layer depth using three algorithms, with running times compared.
- **F4 BST and AVL:** Shapes are kept in a BST by x-coordinate (for alignment queries); an AVL tree keyed by layer name supports fast lookups.
- **F5 String algorithms:** Layer/group name search uses KMP; similar-name suggestions use edit distance.
- **F6 Trie and disjoint sets:** Style preset names are autocompleted with a trie; union-find merges grouped shapes into a single set so the whole group moves together when dragged.
- **F7 File organisation:** Layer metadata lives in a sequential file, and shape geometry data lives in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The drawing file carries a B+ tree secondary index on shape ID.
- **F9 Extendible hashing / external sort:** In large multi-layer drawings, extendible hashing grows the shape-ID table without overflow.

Extension: Real-time collaborative drawing support with simultaneous layer editing could be added.

160 — Music Score and Chord Progression Analyser



Summary: A music-theory application that analyses chord progressions in a synthetic archive of 1,000 songs, recognises familiar patterns and recommends songs with similar progressions. A user can enter a chord sequence and query which songs contain it.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A song's note sequence is kept in a doubly linked list for playing forward and backward; a repeating chorus section loops through a circular list.
- **V2 Sparse matrix:** In the chord $\times$ measure matrix, each measure usually holds a single chord, so most cells are empty and only chord changes are stored.
- **V3 Stack and queue:** Note entry and editing actions are undone through a stack; pending note events (note-on/note-off) during MIDI playback are queued in order.
- **V4 Tree and heap:** Song structure (intro > verse > chorus > bridge) forms a binary tree, with in-order traversal giving the playback order; chord usage frequency sits in a heap so the 5 most-used chords can be listed.
- **V5 Graph and BFS/DFS:** Chords are nodes, with edges weighted by how often one chord follows another; BFS starting from a chord finds the most common transitions, and DFS explores every possible progression chain.
- **V6 Search and hashing:** Chord name (Cmaj7, Am, and so on) maps to a fingering record in a hash table with open addressing; notes sorted by measure number support binary search.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra finds the smoothest (least jarring) transition path between two chords; cycle detection flags recurring progression patterns, such as I–V–vi–IV, as loops.
- **F3 Sorting:** Songs are sorted by tempo (BPM), chord complexity and duration using three algorithms, with running times compared.
- **F4 BST and AVL:** Songs are kept in a BST by key (tonic note); an AVL tree keyed by title supports fast lookups.
- **F5 String algorithms:** Chord-sequence search (e.g., a "C-G-Am-F" pattern) uses KMP; detecting similar progressions uses edit distance.
- **F6 Trie and disjoint sets:** Chord names are autocompleted with a trie; union-find merges songs sharing the same chord pattern into a "pattern family" cluster.
- **F7 File organisation:** Song note records live in a sequential file, and the chord-pattern cache lives in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The song file carries a B+ tree secondary index on key.
- **F9 Extendible hashing / external sort:** When the 1,000-plus-song note archive no longer fits in memory, external merge sort orders it by tempo.

Extension: Automatic chord recognition could be added to transcribe note sequences from audio recordings.

161 — Forum Discussion Thread Tree



Summary: A console application that runs a university club's internal discussion board, letting members reply to threads, moderators review reports, and the system surface trending topics. It works on a synthetic dataset of about 500 users, 3000 threads and 40000 posts.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Replies within a thread are kept in a doubly linked list ordered by time; the homepage's featured-threads card is rotated through a circular list.
- **V2 Sparse matrix:** In the 500-user × 300-popular-thread "has replied" matrix, only cells where a user actually posted a reply are stored.
- **V3 Stack and queue:** Deleting one's own post is undoable through a stack; incoming moderator reports wait their turn for review in a queue.
- **V4 Tree and heap:** Each thread's replies are kept in a binary tree limited to two visible child replies, rendered in a nested view with an in-order traversal; reports sit in a heap keyed by urgency score so the most urgent one reaches a moderator first.
- **V5 Graph and BFS/DFS:** "See also" cross-links between threads are graph edges; BFS finds the shortest link chain between two threads, and DFS finds every thread reachable from a given tag cluster.
- **V6 Search and hashing:** A hash table maps username to profile record; binary search over threads sorted by creation date finds the first thread after a given timestamp.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders pinned "read this first" threads correctly; cycle detection catches an invalid merge where a thread ends up marked as a duplicate of itself through a chain of redirects.
- **F3 Sorting:** The thread list is sorted by reply count, last-activity time and upvotes using three algorithms, with timings compared on 5000 threads.
- **F4 BST and AVL:** Threads are kept in a BST keyed by thread ID; the "active now" sidebar keeps threads in an AVL tree keyed by last-activity time.
- **F5 String algorithms:** A banned-word scan of post bodies uses KMP; near-duplicate thread titles submitted as new posts are flagged using edit distance.
- **F6 Trie and disjoint sets:** Tag autocomplete in the tag box uses a trie; duplicate threads reported by users are merged into one cluster with union-find.
- **F7 File organisation:** The post archive is kept in a sequential file in date order; user profiles are kept in a direct-access file keyed by user ID, with progressive overflow for collision resolution.
- **F8 B+ tree index:** A B+ tree index on the timestamp field over the post archive speeds up "all posts this quarter" queries.
- **F9 Extendible hashing / external sort:** A 2-million-row post log too large for memory is sorted by thread ID using external merge sort for the monthly analytics export.

Extension: Sending a live notification when a followed thread receives a new reply.

162 — 📺 TV Broadcast Schedule Planner



Summary: A desktop tool that plans a local broadcaster's daily and weekly programme schedule, ad breaks and the airing order of multi-part series. It works on a synthetic weekly schedule of 20 channels, 800 programmes and thousands of episodes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each channel's daily line-up is kept as a doubly linked list of programmes, so a delay can be absorbed by shifting slots forward or back; a music channel's repeating 24-hour block is stored as a circular list.
- **V2 Sparse matrix:** In the 20-channel × 672-quarter-hour-slot weekly grid, only cells with a programme actually assigned are stored; empty air time is not.
- **V3 Stack and queue:** Drag-and-drop schedule edits are undoable through a stack; ad breaks inserted while a broadcast is live are processed in order from a queue.
- **V4 Tree and heap:** The genre catalogue (Drama > Crime > Legal Drama, and so on) is kept in a binary tree traversed to populate the browse tabs; breaking-news interrupt requests sit in a heap keyed by urgency, so the most urgent one preempts the schedule first.
- **V5 Graph and BFS/DFS:** Programmes are nodes, with an edge for "viewers of this typically watch that next"; BFS finds every programme within two hops of a hit show for a themed marathon night, and DFS traces the full viewing-order dependency chain for a multi-part special.
- **V6 Search and hashing:** A hash table maps programme code to programme record; binary search over programmes sorted by start time finds what is airing at a given instant.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort resolves "part 2 must air after part 1" constraints across a multi-part mini-series; a max-flow computation over an advertiser × time-slot × channel-capacity network finds the maximum number of ad bookings that can be honoured given slot capacity.
- **F3 Sorting:** The weekly programme list is sorted by rating, slot revenue and duration using three algorithms, with timings compared on 800 programmes.
- **F4 BST and AVL:** The programme catalogue is kept in a BST keyed by programme ID; the "on now / up next" query is answered from programmes kept in an AVL tree keyed by air time.
- **F5 String algorithms:** A content-advisory scan of programme descriptions uses KMP; a viewer's misspelled programme title is corrected using edit distance.
- **F6 Trie and disjoint sets:** Actor and host name autocomplete in the search bar uses a trie; a franchise's spin-offs and specials are merged into one "production family" cluster with union-find.
- **F7 File organisation:** The as-aired log is kept in a sequential file in chronological order; programme master records are kept in a direct-access file keyed by programme code, with linear quotient for collision resolution.
- **F8 B+ tree index:** A B+ tree index on the genre field over the programme file speeds up "show me all comedies" queries.
- **F9 Extendible hashing / external sort:** The viewer-ratings log, which doubles in size each ratings season, is reorganised with extendible hashing as it grows.

Extension: Automatically rescheduling affected programmes when a live sports broadcast runs over time.

163 — Influencer Network Reach Analyser



Summary: An analysis tool that helps a marketing agency decide which social accounts to partner with before a campaign launches. It estimates reach and engagement over a synthetic network of 5000 accounts and 50000 follow relationships between them.

Midterm — C (weeks 1–6):

- **V1 Linked list:** An account's post history is stored as an XOR linked list that combines the forward and backward pointers into one field to save memory during infinite scroll; each account also keeps its latest 200 posts in a plain doubly linked list.
- **V2 Sparse matrix:** In the 2000-account $\times$ 150-campaign-hashtag participation matrix, only cells for accounts that actually used a given hashtag are filled in.
- **V3 Stack and queue:** Removing an account from the shortlist is undoable through a stack; accounts sent a partnership request wait in an approval queue.
- **V4 Tree and heap:** Audience age/region classification is kept in a binary decision tree traversed to build reports; accounts sit in a heap keyed by a computed reach score, and a heap sort produces the top-50 leaderboard.
- **V5 Graph and BFS/DFS:** In the follow graph, nodes are accounts and edges are "follows"; BFS computes how many hops separate a brand account from an influencer to estimate campaign spread depth, and DFS finds every account in one niche community cluster.
- **V6 Search and hashing:** A hash table maps username to profile statistics in $O(1)$; binary search over accounts sorted by follower count finds an account's percentile rank.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm, over a graph weighted by inverse engagement rate, finds the strongest share chain from the brand to the target audience; strongly connected components detect accounts that consistently like and comment on each other's posts (engagement pods).
- **F3 Sorting:** The account list is sorted by reach, engagement rate and follower growth using three algorithms, with timings compared on 5000 accounts.
- **F4 BST and AVL:** The account directory is kept in a BST keyed by account ID; the live "trending now" ranking keeps accounts in an AVL tree keyed by engagement rate.
- **F5 String algorithms:** Campaign hashtag and brand mentions in post captions are scanned for using KMP; suspected impersonator handles are flagged by comparing them to real ones with edit distance.
- **F6 Trie and disjoint sets:** Hashtag autocomplete in the search bar uses a trie; accounts linked by mutual-engagement edges are merged into engagement-pod clusters with union-find.
- **F7 File organisation:** The daily engagement log is kept in a sequential file; account profiles are kept in a direct-access file keyed by account ID, with progressive overflow for collision resolution.
- **F8 B+ tree index:** A B+ tree index on the follower-count field over the account file speeds up "accounts between 10k and 50k followers" range queries.
- **F9 Extendible hashing / external sort:** A multi-million-row engagement event log too large for memory is sorted by timestamp using external merge sort for trend analysis.

Extension: Adding a reach-decay model that reduces a post's weight as it ages.

164 — Museum Exhibit Route and Artwork Index



Summary: A kiosk application that recommends the most efficient walking route through a museum's galleries while also managing the artwork catalogue behind it. It works on a synthetic collection of 40 rooms and 1200 artworks, accounting for room occupancy and rooms closed for restoration.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A visitor's audio-guide stops are kept in a doubly linked list (step forward/back, insert a detour); the looping tour through a rotunda gallery is stored as a circular list that wraps back to the first stop.
- **V2 Sparse matrix:** In the 40-room × 12-hour-of-day visitor-density table, only cells with an actual occupancy measurement are stored.
- **V3 Stack and queue:** Stepping back in the augmented-reality guide is undoable through a stack; ticket scanning at the entrance is simulated with a queue that caps how many visitors a room can hold at once.
- **V4 Tree and heap:** Artwork classification (period > movement > sub-style) is kept in a binary tree traversed to browse the catalogue; restoration requests sit in a heap keyed by urgency, so the most fragile piece is handled first.
- **V5 Graph and BFS/DFS:** Rooms are nodes and corridors are edges; BFS finds the shortest walking route between two exhibits, and DFS builds a "see everything" route covering every room reachable from the entrance.
- **V6 Search and hashing:** A hash table maps accession number to artwork record; binary search over artworks sorted by year jumps straight to a given era.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree (Kruskal) computes the layout connecting all rooms at the lowest total corridor cost for a new wing; Dijkstra's algorithm, over a graph weighted by walking distance and crowding, recommends the fastest, least crowded tour.
- **F3 Sorting:** The artwork catalogue is sorted by acquisition year, estimated value and popularity score using three algorithms, with timings compared on 1200 artworks.
- **F4 BST and AVL:** Artworks are kept in a BST keyed by accession number; the alphabetical artist index keeps artworks in an AVL tree keyed by artist name.
- **F5 String algorithms:** A keyword search over exhibit description plaques uses KMP; a visitor's misspelled artist name is corrected using edit distance.
- **F6 Trie and disjoint sets:** Artist- and title-name autocomplete on the kiosk uses a trie; artworks donated together as a single bequest are merged into one collection cluster with union-find.
- **F7 File organisation:** The chronological acquisition ledger is kept in a sequential file; artwork master records are kept in a direct-access file keyed by accession number, with Brent's method for collision resolution.
- **F8 B+ tree index:** A B+ tree index on artist name over the artwork file speeds up "all works by this artist" queries.
- **F9 Extendible hashing / external sort:** The digitised-artwork metadata file, which keeps growing as the collection expands each year, is reorganised with extendible hashing as it grows.

Extension: Rerouting the tour instantly when a room closes for restoration.

165 — Theatre Script Scene and Role Analyser



Summary: A desktop tool that helps a theatre company break a play's text into scenes and track each character's lines and stage time during rehearsal. It works on a synthetic archive covering 25 plays in the repertoire, with 400 scenes and their cast lists in total.

Midterm — C (weeks 1–6):

- **V1 Linked list:** A play's scenes are kept in a doubly linked list in performance order (jump to the next/previous scene); a rehearsal drill that repeats a chosen set of scenes is stored as a circular list.
- **V2 Sparse matrix:** In the 40-character × 60-scene "on stage" matrix for one play, only cells where a character actually appears in a scene are filled in.
- **V3 Stack and queue:** A director's blocking-note edits are undoable through a stack; during a live rehearsal, a queue drives the cue-call system that tells each actor when to speak next.
- **V4 Tree and heap:** The act/scene hierarchy (Act > Scene > Sub-beat) is kept in a binary tree traversed to generate the printed programme; scenes with the most reported line errors sit in a heap, so the most troubled scene is rehearsed first.
- **V5 Graph and BFS/DFS:** Characters are nodes and "appears in the same scene as" is an edge; BFS finds the shortest chain of shared scenes between two characters for a doubling-casting decision, and DFS finds every character connected within one subplot.
- **V6 Search and hashing:** A hash table maps character name to role record (lines, entrances); binary search over scenes sorted by page number jumps straight to a given page.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort orders flashback dependencies where "scene B assumes the reveal from scene A already happened"; cycle detection catches a script continuity error where such dependencies loop back on themselves.
- **F3 Sorting:** The cast list is sorted by total line count, stage time and number of scenes using three algorithms, with timings compared on a play with about 600 lines of dialogue.
- **F4 BST and AVL:** Scenes are kept in a BST keyed by scene number; the alphabetical cast list keeps characters in an AVL tree keyed by name, rebalanced as characters are added or renamed during rewrites.
- **F5 String algorithms:** A character's cue line or a prop keyword is searched for in the full script using KMP; changed lines between a draft and the final script are aligned using edit distance/LCS.
- **F6 Trie and disjoint sets:** Character- and prop-name autocomplete uses a trie; minor roles that one actor doubles up across are merged into one "same actor plays" cluster with union-find.
- **F7 File organisation:** The script text is kept scene by scene, in performance order, in a sequential file; character records are kept in a direct-access file keyed by character ID, with progressive overflow for collision resolution.
- **F8 B+ tree index:** A B+ tree index on page number over the script file supports fast "go to page" lookups during a technical rehearsal.
- **F9 Extendible hashing / external sort:** The archive of every line from all 25 plays is too large for memory, so it is sorted by character name using external merge sort to build a company-wide "who played what" report.

Extension: Automatically flagging every scene affected when a line of dialogue is cut during an edit.

166 — Animation Frame Sequencer



Summary: An in-house studio tool that manages the frame-by-frame timeline, layers and render order of a short animated film. It works on a synthetic project with 8 scenes and 3000 frames in total, played back at 12 frames per second.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The timeline itself is a doubly linked list of frames (scrub forward/back, insert or delete a frame mid-sequence); a repeating block such as a 12-frame walk cycle is stored as a circular list.
- **V2 Sparse matrix:** In the 20-layer × 500-frame "keyframe present" matrix, only cells where a keyframe was actually drawn are filled in — the frames in between are interpolated.
- **V3 Stack and queue:** Draw, delete and move operations are supported by an undo/redo stack; frames sent to the render farm are processed in submission order from a queue.
- **V4 Tree and heap:** The film > scene > shot hierarchy is kept in a binary tree traversed to generate the shot list; render jobs sit in a heap keyed by deadline urgency, and a heap sort produces the render order for the night's batch.
- **V5 Graph and BFS/DFS:** In the layer-dependency graph, a character layer references a background layer and an effect layer references a character layer; BFS finds every layer affected within N hops when a base layer changes, and DFS walks the full dependency order before a frame is composited.
- **V6 Search and hashing:** A hash table maps frame ID to frame data and thumbnail for instant seeking; binary search over frames sorted by timestamp finds the frame nearest a given playback time.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort determines the correct compositing order of layers given their dependencies; cycle detection catches an invalid circular reference where layer A depends on layer B, which depends back on layer A.
- **F3 Sorting:** The frame list is sorted by render time, file size and layer count using three algorithms, with timings compared on 3000 frames.
- **F4 BST and AVL:** Frames are kept in a BST keyed by frame number; the onion-skin preview's "nearest keyframe" query is answered from keyframes kept in an AVL tree keyed by timestamp.
- **F5 String algorithms:** Frame and layer tags are searched for a keyword (such as finding every frame tagged "explosion") using KMP; a mistyped layer name in the asset browser is fuzzy-matched using edit distance.
- **F6 Trie and disjoint sets:** Layer-name and tag autocomplete uses a trie; consecutive frames belonging to the same camera shot are merged into one "shot" cluster with union-find for batch export.
- **F7 File organisation:** The exported frame sequence is kept in playback order in a sequential file; the frame metadata cache is kept in a direct-access file keyed by frame ID, with linear quotient for collision resolution.
- **F8 B+ tree index:** A B+ tree index on scene/shot number over the frame file speeds up "export all frames of shot 12" queries.
- **F9 Extendible hashing / external sort:** Once the project reaches thousands of high-resolution frames, its frame-metadata log no longer fits in memory and is sorted by frame number with external merge sort before the final render assembly.

Extension: Suggesting automatic in-between frames by interpolating neighbouring keyframes.

167 — 📻 Radio Song Request Queue



Summary: A studio application that manages a local radio station's listener song requests, the DJ's live playlist and rotation rules. It works on a synthetic dataset of about 200 requests a day drawn from a 5000-song library.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The live playlist itself is a doubly linked list of songs, so the DJ can move a song earlier or later or jump back to a previous one; the fixed genre block that repeats every hour is stored as a circular list.
- **V2 Sparse matrix:** In the day-of-week $\times$ hour-of-day $\times$ genre play-count table, only cells for genres actually played in that hour are filled in.
- **V3 Stack and queue:** A DJ's playlist edits, such as pulling a song out, are undoable through a stack; listener-submitted song requests wait their turn in a queue until they go on air.
- **V4 Tree and heap:** The genre/sub-genre catalogue is kept in a binary tree traversed to browse it; pending requests sit in a heap keyed by listener votes, so the most-voted request airs next, and a heap sort builds tonight's top-10 countdown.
- **V5 Graph and BFS/DFS:** Songs are nodes, with an edge for "commonly played back to back"; BFS finds every song within two transitions of a hit song for an auto-generated mix, and DFS walks similar-mood edges to build a complete themed playlist chain.
- **V6 Search and hashing:** A hash table maps song title to library record (artist, duration, play count) in $O(1)$; binary search over songs sorted by release year jumps straight to a given decade.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm, over a graph weighted by tempo/key mismatch, computes the smoothest transition path between two songs in a DJ set; cycle detection catches an invalid playlist loop where a scheduled block ends up referencing itself.
- **F3 Sorting:** The song library is sorted by play count, request count and duration using three algorithms, with timings compared on 5000 songs.
- **F4 BST and AVL:** The library index is kept in a BST keyed by song ID; the live request-queue view keeps requests in an AVL tree keyed by timestamp.
- **F5 String algorithms:** A partial song title texted in by a listener is searched for using KMP; a misspelled artist or song name on the request form is corrected using edit distance.
- **F6 Trie and disjoint sets:** Song-title and artist autocomplete in the request app uses a trie; songs by the same artist or album are merged into one cluster with union-find for building an artist-spotlight block.
- **F7 File organisation:** The as-aired log is kept in chronological order in a sequential file for royalty reporting; the library master record is kept in a direct-access file keyed by song ID, with progressive overflow for collision resolution.
- **F8 B+ tree index:** A B+ tree index on artist name over the song library file speeds up "play everything by this artist" queries.
- **F9 Extendible hashing / external sort:** The song library file, which keeps growing as new releases are added each week, is reorganised with extendible hashing as it grows.

Extension: Auto-balancing rotation so the same artist never repeats within a two-hour window.

168 — 📱 Emoji Keyboard Search and Favourites



Summary: The emoji-picker module of a mobile keyboard app, offering category browsing, keyword search, recently used and favourites. It works on a library of 1800 emoji and a usage history of thousands of taps from 1000 synthetic users.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The "recently used" strip is a doubly linked list where the newest emoji is added at the head and the oldest is evicted from the tail once it is full; long-pressing an emoji cycles through its five skin-tone variants via a circular list.
- **V2 Sparse matrix:** In the 1000-user × 1800-emoji tap-count matrix, only cells for the small personal subset each user actually taps are filled in.
- **V3 Stack and queue:** Removing a favourite is undoable through a stack; animation requests for newly typed emoji reactions wait their turn in a queue before they are drawn on screen.
- **V4 Tree and heap:** The category hierarchy (Smileys > People > Animals, and so on) is kept in a binary tree traversed to populate the picker tabs; emoji sit in a heap keyed by tap frequency, and a heap sort produces the "Frequently Used" row each session.
- **V5 Graph and BFS/DFS:** Emoji are nodes, with an edge for "used in the same message"; BFS finds every emoji within two hops of a given one for "you might also want" suggestions, and DFS explores a full theme cluster, such as every food emoji reachable from one seed.
- **V6 Search and hashing:** A hash table maps a keyword or tag to the list of matching emoji for instant search-as-you-type; binary search over emoji sorted by Unicode code point jumps to a given code range.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree connects all category tabs with the shortest total swipe distance for the layout; strongly connected components detect emoji that are almost always used together, such as a flag-plus-celebration combo, for a "combo pack" suggestion feature.
- **F3 Sorting:** The emoji list is sorted by tap frequency, recency and alphabetical tag using three algorithms, with timings compared on 1800 emoji.
- **F4 BST and AVL:** The master emoji index is kept in a BST keyed by Unicode code point; the live "trending" ranking keeps emoji in an AVL tree keyed by tap count, rebalanced after every tap.
- **F5 String algorithms:** Typing "cel" matching the tag "celebration" is handled with KMP; a misspelled search term is fuzzy-matched against tags using edit distance.
- **F6 Trie and disjoint sets:** Tag and keyword autocomplete as the user types in the search bar uses a trie; emoji from co-occurrence data are merged into combo clusters with union-find for the combo-pack suggestion feature.
- **F7 File organisation:** The full tap-event log is kept in chronological order in a sequential file; the master emoji record is kept in a direct-access file keyed by code point, with Brent's method for collision resolution.
- **F8 B+ tree index:** A B+ tree index on the category field over the emoji file speeds up "show all Animals" queries.
- **F9 Extendible hashing / external sort:** A multi-million-row tap-event log too large for memory is sorted by user ID using external merge sort to rebuild each user's personalised ranking overnight.

Extension: Syncing favourites across devices with a lightweight conflict-merge rule.

169 — Photo Geotag Map Clusterer



Summary: A desktop application that automatically groups a user's geotagged photo library into trip and day clusters and displays them on a map. It works on a synthetic archive of 10000 photos, each carrying latitude, longitude and a timestamp.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Photos in one trip album are kept in a doubly linked list ordered by timestamp (scroll forward/back through the album, insert a photo mid-album); a trip's GPS breadcrumb trail, which can run to tens of thousands of points, is walked in both directions with an XOR linked list that packs the two pointers into a single field to save memory.
- **V2 Sparse matrix:** In the 30-trip-day $\times$ 50 $\times$ 50 latitude/longitude grid-cell photo-count table, only cells where a photo was actually taken are filled in.
- **V3 Stack and queue:** Editing actions such as moving a photo to an album or deleting it are undoable through a stack; background thumbnail generation for newly imported photos is processed in order from a queue.
- **V4 Tree and heap:** The album hierarchy (Year > Trip > Day) is kept in a binary tree traversed to build the gallery view; photos sit in a heap keyed by a computed highlight score (sharpness, faces, rarity), and a heap sort produces the auto-generated "Best of Trip" reel.
- **V5 Graph and BFS/DFS:** Candidate location clusters are nodes, with an edge between clusters within walking distance of each other; BFS groups nearby clusters into a single "place" within N hops, and DFS merges every cluster in one connected city visit.
- **V6 Search and hashing:** A hash table maps a geohash cell key to the list of photos in that cell for fast spatial lookup; binary search over photos sorted by timestamp jumps straight to a given date.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** A minimum spanning tree (Kruskal) computes the shortest total-distance "trip route" connecting all visited clusters for the map overlay; Dijkstra's algorithm computes the actual walking distance between two consecutive photo locations for a "you walked X km today" statistic.
- **F3 Sorting:** Photos are sorted by timestamp, GPS accuracy and highlight score using three algorithms, with timings compared on 10000 photos.
- **F4 BST and AVL:** The master index is kept in a BST keyed by photo ID; the fast date-range scrubber keeps photos in an AVL tree keyed by timestamp, rebalanced as new photos are imported.
- **F5 String algorithms:** A keyword search over photo captions and auto-tags uses KMP; small differences between reverse-geocoded place names across imports are fuzzy-matched using edit distance.
- **F6 Trie and disjoint sets:** Place-name and tag autocomplete in the search bar uses a trie; nearby geo-clusters are merged into a single "place" with union-find as new photos fill the gap between them.
- **F7 File organisation:** The import log is kept in chronological order in a sequential file; photo metadata (GPS, timestamp, tags) is kept in a direct-access file keyed by photo ID, with progressive overflow for collision resolution.
- **F8 B+ tree index:** A B+ tree index on geohash cell over the photo file speeds up "all photos near this point" range queries.
- **F9 Extendible hashing / external sort:** The photo-metadata file, which keeps growing past hundreds of thousands of entries, is reorganised with extendible hashing as the library expands.

Extension: Suggesting an automatic trip title from the dominant cluster's place name and date range.

170 — Video Platform Watch History and Recommender



Summary: A backend module for a video-streaming app that logs watch history and produces a simple recommendation list. It works on a dataset of 3000 videos, 800 synthetic users and tens of thousands of watch events.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each user's watch history is kept in a doubly linked list in chronological order (resume playback, step back to a previous video for the "continue watching" row); an autoplaying music-video playlist that repeats indefinitely is stored as a circular list.
- **V2 Sparse matrix:** In the 800-user $\times$ 3000-video watch-time matrix, only cells for videos a user actually watched are filled in; since each user only watches a small slice of the catalogue, the matrix is mostly empty.
- **V3 Stack and queue:** Removing an entry from watch history is undoable through a stack; newly uploaded videos are transcoded in submission order from a queue.
- **V4 Tree and heap:** The genre/category hierarchy (Movies > Action > Sub-genre) is kept in a binary tree traversed to populate the browse menu; videos sit in a heap keyed by a trending score based on views in the last 24 hours, and a heap sort produces the daily "Trending Now" row.
- **V5 Graph and BFS/DFS:** Videos are nodes, with an edge for "watched by the same user in one session"; BFS finds every video within two hops of the one currently playing for "Up Next" suggestions, and DFS walks a connected viewing cluster to find a niche fan community's shared taste.
- **V6 Search and hashing:** A hash table maps video ID to metadata record for instant playback lookup; binary search over videos sorted by upload date jumps straight to a release window.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra's algorithm, over a graph weighted by inverse co-watch strength, finds the closest recommendation chain from a video a user just finished to an under-watched one, as a discovery boost; strongly connected components detect binge clusters, such as all episodes of a series that are consistently watched together.
- **F3 Sorting:** Videos are sorted by view count, average watch percentage and upload date using three algorithms, with timings compared on 3000 videos.
- **F4 BST and AVL:** The video catalogue is kept in a BST keyed by video ID; the live "Trending Now" ranking keeps videos in an AVL tree keyed by trending score, rebalanced as view counts update.
- **F5 String algorithms:** A keyword search over video titles and descriptions uses KMP; changed lines between auto-generated subtitles and a user-submitted correction are aligned using edit distance/LCS.
- **F6 Trie and disjoint sets:** Title and channel-name autocomplete in the search bar uses a trie; episodes referencing the same show ID are merged into one series/season group with union-find.
- **F7 File organisation:** The chronological watch-event log is kept in a sequential file; the video metadata master record is kept in a direct-access file keyed by video ID, with linear quotient for collision resolution.
- **F8 B+ tree index:** A B+ tree index on the upload-date field over the video file speeds up "everything released this month" range queries.
- **F9 Extendible hashing / external sort:** A multi-million-row watch-event log too large for memory is sorted by user ID using external merge sort to rebuild each user's recommendation profile overnight.

Extension: Adding a simple content-based fallback recommendation for a brand-new user with no watch history yet.

171 — Interactive Branching Story Engine



Summary: A console application where the reader makes a choice at every scene and reaches a different ending depending on the path taken. It runs on a synthetic story of about 150 scenes and 40 distinct endings; the reader's past choices are kept for the whole session.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The scenes a reader has visited are kept in a doubly linked list, enabling "back/forward" navigation; the pages of a time-loop chapter are chained in a circular list that repeats endlessly.
- **V2 Sparse matrix:** In the 150-scene by 40-choice-label table, only the choices that are actually defined for a given scene are stored.
- **V3 Stack and queue:** The reader's last choice is undone with a stack; side events the author schedules for later chapters wait in a queue until it's their turn to trigger.
- **V4 Tree and heap:** The main story branch is kept as a binary tree and an in-order traversal builds the table-of-contents preview; discovered endings sit in a heap ranked by rarity score, and heap sort produces the "10 rarest endings" list.
- **V5 Graph and BFS/DFS:** With scenes as nodes and choices as edges, BFS finds the route from the opening scene to a target ending using the fewest choices, and DFS surfaces scenes that no choice can ever reach.
- **V6 Search and hashing:** Player-typed commands like "save" and "back" are mapped to their handlers in a hash table (chaining for collisions); ending codes are looked up with binary search over a sorted array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort verifies that scenes gated behind an earlier-chapter flag only unlock in the right order; cycle detection catches unintended infinite loops that slipped in outside the designed loop chapter.
- **F3 Sorting:** Ending statistics gathered from 5,000 simulated playthroughs are sorted with three different algorithms and their running times compared.
- **F4 BST and AVL:** Scene records first sit in a binary search tree; because authoring involves frequent inserts and deletes, they move into an AVL tree that rebalances to keep lookups fast.
- **F5 String algorithms:** KMP scans scene text for a banned word; when a reader mistypes a chapter title, edit distance suggests the closest real title.
- **F6 Trie and disjoint sets:** Player commands are autocompleted with a trie as they're typed; union-find groups scenes that arrive from different branches but merge at the same point into one story arc.
- **F7 File organisation:** Scene text is stored in a sequential file in chapter order, while scene records live in a direct-access file (collisions resolved by progressive overflow) for instant lookup.
- **F8 B+ tree index:** A B+ tree secondary index on the scene file, keyed by character-name tag, speeds up "every scene this character appears in" queries.
- **F9 Extendible hashing / external sort:** Because authors keep adding new scenes, the growing scene file is managed with extendible hashing so it never needs a full reorganisation.

Extension: Add a choice-analytics panel that shows which branches readers pick least often.

**172 — Community Detection Tool**

Summary: An analysis tool that groups the users of a social network into tightly connected friend clusters based on how they interact. It works on a synthetic network of about 5,000 users and 30,000 interactions; the resulting communities are reported by size and density.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each user's interaction history — up to 300 likes, comments and messages — is kept in a doubly linked list ordered by time; a user's closest circle is displayed as a rotating carousel backed by a circular list.
- **V2 Sparse matrix:** In the 5,000-by-5,000 pairwise interaction matrix, only the roughly 30,000 pairs who actually interacted are stored.
- **V3 Stack and queue:** An analyst's manual merge or split of two communities is undone with a stack; interactions flagged as suspicious wait in a moderation queue for review.
- **V4 Tree and heap:** The merge order of hierarchical clustering is kept as a binary tree; at each step, a heap picks the pair of communities with the highest density gain to merge next, and heap sort ranks the final communities by size.
- **V5 Graph and BFS/DFS:** With users as nodes and interactions as edges, BFS finds the degree of separation between two users, and DFS uncovers isolated user groups disconnected from the main network.
- **V6 Search and hashing:** A username maps to its user record through a hash table (chaining for collisions); administrative reports look up user IDs with binary search over a sorted array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm builds a minimum-weight backbone for visualizing the network; strongly connected components reveal cores of users who all follow or engage with one another mutually.
- **F3 Sorting:** Communities are sorted by size and density score with three algorithms, and running times are compared over the full 5,000-user dataset.
- **F4 BST and AVL:** User records are kept in an AVL tree keyed by user ID; rebalancing rotations kick in as batches of synthetic new users are imported.
- **F5 String algorithms:** KMP searches user bios for a keyword; a misspelled username search is matched to the closest real one with edit distance.
- **F6 Trie and disjoint sets:** Usernames are autocompleted with a trie; every pair above a similarity threshold is merged with union-find, building up communities incrementally.
- **F7 File organisation:** The interaction log is stored in a sequential file in time order, while user profiles live in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on the interaction file, keyed by community ID, speeds up "all interactions inside community X" queries.
- **F9 Extendible hashing / external sort:** The user file grows with every new batch of synthetic accounts, so it is managed with extendible hashing to avoid full reorganisation.

Extension: Track how communities evolve across a sequence of network snapshots taken over time.

173 — Karaoke Lyrics Synchroniser



Summary: A desktop tool that aligns lyric text with the timestamps of an audio recording to produce scrolling on-screen captions. It works on a catalogue of about 300 synthetic songs, each with lines timed to the second.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each song's line-and-timestamp nodes are kept in a doubly linked list to support scrubbing forward and backward during playback; a rehearsal "loop" mode repeats a chosen verse endlessly using a circular list.
- **V2 Sparse matrix:** In the song-by-syllable emphasis table, only syllables actually marked with vibrato or a held note are stored.
- **V3 Stack and queue:** An editor's manual timestamp corrections are undone with a stack; singers signed up for karaoke night wait their turn in a queue.
- **V4 Tree and heap:** The catalogue is kept in a binary tree ordered by song title so an in-order traversal produces an alphabetical listing; the most-requested songs sit in a heap and heap sort reveals the top 10 at set breaks.
- **V5 Graph and BFS/DFS:** With songs linked by genre and tempo similarity, BFS finds songs within two hops of a favourite, and DFS explores an entire genre cluster to build a themed playlist.
- **V6 Search and hashing:** A song title maps to its record through a hash table (open addressing for collisions); songs are looked up by release year with binary search over a sorted array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm builds a minimal transition backbone connecting each song to its closest neighbour in the similarity graph; Dijkstra finds the smoothest tempo-transition path between two chosen songs.
- **F3 Sorting:** The catalogue, expanded to 20,000 entries, is sorted by title, duration and popularity with three algorithms and the running times compared.
- **F4 BST and AVL:** An AVL tree keyed by song ID keeps lookups balanced as new catalogue batches are continuously imported.
- **F5 String algorithms:** KMP finds every song containing a given lyric line for a "name that tune" game; a phonetically misspelled song request is matched to the closest title with edit distance.
- **F6 Trie and disjoint sets:** Song and artist names are autocompleted with a trie; union-find groups different recorded covers of the same underlying song into one cover group.
- **F7 File organisation:** Timestamped lyrics are stored per song in a sequential file, while song metadata lives in a direct-access file (collisions resolved by progressive overflow) for instant lookup during requests.
- **F8 B+ tree index:** A B+ tree secondary index on the song file, keyed by artist name, speeds up "every song by this artist" queries.
- **F9 Extendible hashing / external sort:** When the request log built up over months of karaoke nights no longer fits in memory, an external merge sort by popularity compiles the monthly top-songs report.

Extension: Add live pitch-detection scoring from the microphone input alongside the scrolling lyrics.

174 — Magazine Page Layout and Article Index



Summary: A layout desk application that manages where articles and ads sit on each page of a magazine issue and builds the topic index at the back. It works on a synthetic archive of roughly 120 pages, 12 issues a year, and about 80 articles per issue.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The pages within an issue are kept in a doubly linked list so a late-arriving ad can be inserted mid-issue; a fixed rotation of recurring columnists cycles from issue to issue using a circular list.
- **V2 Sparse matrix:** In the 80-article by 40-topic-tag table, only the tags actually assigned to an article are stored.
- **V3 Stack and queue:** Layout edits are undone with a stack; articles awaiting editorial approval wait in a review queue.
- **V4 Tree and heap:** The table of contents is kept as a binary tree by section, and an in-order traversal produces the printed order; pending articles sit in a heap ranked by deadline, and heap sort schedules them for the next issue.
- **V5 Graph and BFS/DFS:** With articles as nodes and "see also" cross-references as edges, BFS finds the shortest reference chain between two articles, and DFS uncovers reference cycles where article A eventually points back to itself.
- **V6 Search and hashing:** An index keyword maps to its list of article IDs through a hash table (chaining for collisions); jumping to a specific page uses binary search over sorted page numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Topological sort ensures a main article is laid out before the sidebars and pull-quotes that depend on it; cycle detection catches invalid "continued on page..." loops that circle back on themselves.
- **F3 Sorting:** The roughly 2,000 articles in the multi-year archive are sorted by date, author or page count with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by article ID keeps lookups balanced as the archive keeps growing with every new issue.
- **F5 String algorithms:** KMP scans full article text to check whether the same phrase appears in another article; a misspelled author-name search is matched to the closest archive entry with edit distance.
- **F6 Trie and disjoint sets:** Index terms are autocompleted with a trie; union-find merges a multi-part series (part 1, 2, 3) scattered across issues into a single series cluster.
- **F7 File organisation:** Article bodies are stored in a sequential file in issue order, while article metadata lives in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on the article file, keyed by author name, speeds up "every article by this author" queries.
- **F9 Extendible hashing / external sort:** The metadata file keeps growing with every new issue, so it is managed with extendible hashing rather than a full reorganisation each time.

Extension: Add an automatic layout suggestion that balances page white-space using estimated article length.

175 — TV Series Episode and Character Network



Summary: A viewer's companion tool that tracks a series' seasons, episodes and characters, and visualises how the characters relate to one another. It works on a synthetic show spanning about 10 seasons, 220 episodes and 150 characters.

Midterm — C (weeks 1–6):

- **V1 Linked list:** The episodes of a season are kept in a doubly linked list in air order, so a special episode can be inserted between two regular ones; a season built around a time loop has its episodes modelled with a circular list.
- **V2 Sparse matrix:** In the 220-episode by 150-character appearance table, only the episodes a character actually appears in are stored.
- **V3 Stack and queue:** Manual edits to a character's timeline are undone with a stack; a viewer's "watch next" list is managed as a queue.
- **V4 Tree and heap:** The season-arc-episode hierarchy is kept as a binary tree that produces a chronological viewing order; rated episodes sit in a heap, and heap sort produces the "top 10 episodes" countdown.
- **V5 Graph and BFS/DFS:** With characters as nodes and their interactions as edges, BFS finds the degree of separation between two characters, and DFS uncovers every character reachable from a given one within a storyline cluster.
- **V6 Search and hashing:** A character name maps to its record through a hash table (chaining for collisions); "jump to episode" uses binary search over sorted episode numbers.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Strongly connected components find cores of characters who are all mutually linked, such as a friend group or a rival faction; Dijkstra, weighted by shared scene count, finds the strongest connection path between two characters.
- **F3 Sorting:** The full episode log, including reruns, is sorted by air date, rating or runtime with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by episode number keeps lookups balanced when out-of-sequence flashback or special episodes are inserted.
- **F5 String algorithms:** KMP finds a quoted line of dialogue across every episode transcript; a fan's misremembered quote is matched to the actual transcript line with edit distance.
- **F6 Trie and disjoint sets:** Character names are autocompleted with a trie; union-find merges characters who share scenes into "storyline cliques" that update as the series progresses.
- **F7 File organisation:** Episode transcripts are stored in a sequential file in air order, while character records live in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** A B+ tree secondary index on the episode file, keyed by director name, speeds up "every episode this director made" queries.
- **F9 Extendible hashing / external sort:** When the character-appearance log across every season and rerun no longer fits in memory, an external merge sort by air date compiles the full chronological viewing guide.

Extension: Build a spoiler-safe recommendation feature that only uses relationship data up to the episode the viewer has actually watched.

176–200 · City, environment, disasters and agriculture

176 — 🏠 Disaster Evacuation Route Planner



Summary: A crisis-management tool that routes households in a flooding city district to the nearest open shelter. It works on a synthetic district of about 500 road segments, 80 neighbourhood blocks and 15 shelters; road closures are factored in immediately as they're reported.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each shelter's list of registered households is kept in a doubly linked list, with check-ins and check-outs processed from either end; shelter staff shifts rotate continuously through a circular list.
- **V2 Sparse matrix:** In the block-by-hour water-level table, only cells with an actual sensor reading are stored.
- **V3 Stack and queue:** An operator's manual road-closure overrides are undone with a stack; households at a shelter's check-in desk wait in a queue.
- **V4 Tree and heap:** Shelters are organised by region in a binary tree; evacuation requests sit in a heap prioritised by urgency (medical, elderly, general), and heap sort dispatches rescue teams accordingly.
- **V5 Graph and BFS/DFS:** With intersections as nodes and roads as edges, BFS finds the route from a block to the nearest open shelter with the fewest turns, and DFS uncovers blocks left fully isolated after a set of closures.
- **V6 Search and hashing:** A block ID maps to its population and assigned-shelter record through a hash table (open addressing for collisions); the nearest shelter with remaining capacity is found with binary search over a sorted capacity array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by water depth, computes the fastest evacuation route; Prim proposes a minimal road backbone that keeps every block connected to at least one shelter.
- **F3 Sorting:** Blocks are sorted by risk score and shelters by remaining capacity with three algorithms, and running times are compared over the full sensor log.
- **F4 BST and AVL:** An AVL tree keyed by shelter ID keeps lookups balanced as shelters open and close dynamically during the event.
- **F5 String algorithms:** KMP scans incoming SMS and radio message text for keywords like "water" or "trapped"; a misspelled street name typed by an operator is matched to the correct one with edit distance.
- **F6 Trie and disjoint sets:** Street names are autocompleted with a trie during address entry; union-find recomputes which blocks remain mutually reachable after every road closure, instantly spotting isolated pockets.
- **F7 File organisation:** Sensor readings are stored in a sequential file in time order, while household registration records live in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on the household file, keyed by shelter ID, speeds up "everyone currently at shelter X" queries.
- **F9 Extendible hashing / external sort:** When the citywide sensor log built up over the event no longer fits in memory, an external merge sort by timestamp reconstructs the full flood timeline.

Extension: Re-route evacuees live as new road-closure reports come in during the event.

177 — 🚒 Fire Station Coverage Analyser



Summary: A planning tool that checks whether a city's fire stations meet their target response times and highlights coverage gaps. It works on a synthetic road network of about 20 stations and 300 city blocks.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each station's fleet of trucks is kept in a doubly linked list, with vehicles entering or leaving maintenance added and removed from either end; the A-B-C shift rotation for crews cycles continuously through a circular list.
- **V2 Sparse matrix:** In the 300-block by 12-incident-type historical count table, only cells where an incident actually occurred are stored.
- **V3 Stack and queue:** A dispatcher's manual dispatch overrides are undone with a stack; incoming emergency calls during a busy period wait in a dispatch queue.
- **V4 Tree and heap:** Stations are organised by district in a binary tree; simultaneous incidents sit in a heap prioritised by severity, and heap sort ranks stations by response-time performance for the monthly report.
- **V5 Graph and BFS/DFS:** With blocks as nodes and roads as edges, BFS finds the route from a station to an incident block with the fewest segments, and DFS uncovers blocks that no station can reach within the target response window.
- **V6 Search and hashing:** A block ID maps to its nearest-station record through a hash table (chaining for collisions); the monthly report looks up stations by response time with binary search over a sorted array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by travel time, determines which station can respond to a given incident fastest; Kruskal's algorithm suggests the cheapest new road link that closes the largest coverage gap.
- **F3 Sorting:** A historical log of thousands of synthetic incidents is sorted by response time, severity or date with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by incident ID keeps lookups balanced as new incidents keep getting logged.
- **F5 String algorithms:** KMP scans incident report text for hazard keywords like "gas leak" or "chemical spill"; a misspelled street name in a call transcript is matched to the nearest known address with edit distance.
- **F6 Trie and disjoint sets:** Street and address names are autocompleted with a trie during call intake; union-find groups blocks reachable from the same station within the target time into a coverage cluster, recomputed whenever a station goes offline.
- **F7 File organisation:** Incident records are stored in a sequential file in chronological order, while station and truck records live in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on the incident file, keyed by block ID, speeds up "every incident at this block" queries.
- **F9 Extendible hashing / external sort:** When the citywide incident log spanning many years no longer fits in memory, an external merge sort by date compiles a decade-long trend report.

Extension: Simulate a station going temporarily offline and recompute the resulting coverage gaps live.

178 — 🏠 City Water Network Leak Locator



Summary: A maintenance tool that pinpoints likely leak locations in a city's water pipe network from patterns of pressure drop. It works on a synthetic network of about 1,000 pipe segments, 600 junctions and 200 pressure sensors.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each pipe segment's maintenance history is kept in a doubly linked list, with new repair records insertable at either end; a ring-shaped distribution loop in the network is modelled as a circular list of junctions.
- **V2 Sparse matrix:** In the junction-by-hour pressure-reading table, only cells where a sensor exists and actually reported are stored.
- **V3 Stack and queue:** An operator's valve-state changes made while investigating a leak are undone with a stack; pending repair work orders wait in a queue for the next available crew.
- **V4 Tree and heap:** The network is organised by district in a binary tree; suspected leak sites sit in a heap ranked by estimated water-loss rate, and heap sort produces the priority list for the maintenance team.
- **V5 Graph and BFS/DFS:** With junctions as nodes and pipes as edges, BFS finds which junctions lose supply when a given valve is closed, and DFS traces the flow path upstream from a low-pressure sensor to isolate the leaking segment.
- **V6 Search and hashing:** A junction ID maps to its sensor ID and elevation record through a hash table (chaining for collisions); maintenance scheduling by age uses binary search over a sorted array of installation dates.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by pipe length and diameter, finds the fastest reroute when a segment must be shut for repair; Prim proposes the minimum-length backbone of pipes needed to supply every junction in a new district design.
- **F3 Sorting:** Pressure-drop events in the large sensor log are sorted by magnitude or date with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by sensor ID keeps lookups balanced as sensors are added network-wide over time.
- **F5 String algorithms:** KMP scans a field technician's free-text notes for keywords like "leak" or "crack"; a misspelled junction or street name typed by a technician is matched to the correct network record with edit distance.
- **F6 Trie and disjoint sets:** Junction and street names are autocompleted with a trie during technician lookup; union-find recomputes which junctions stay hydraulically connected as "pressure zones" every time a valve changes state.
- **F7 File organisation:** Sensor pressure readings are stored in a sequential file in time order, while pipe segment records live in a direct-access file (collisions resolved by progressive overflow).
- **F8 B+ tree index:** A B+ tree secondary index on the pipe file, keyed by installation year, speeds up "every pipe older than N years" maintenance queries.
- **F9 Extendible hashing / external sort:** When the citywide sensor log built up over years no longer fits in memory, an external merge sort by timestamp reconstructs the full pressure history for leak forensics.

Extension: Add a live leak-probability score that updates as new sensor readings stream in.

179 — Power Grid Fault Isolator



Summary: A grid operations tool that isolates faults in a city's power distribution network and restores customers through an alternate feeder. It works on a synthetic grid of about 50 substations, 800 feeder segments and 5,000 customer nodes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each substation's connected feeder segments are kept in a doubly linked list, with segments added or decommissioned from either end; a ring feeder topology used for redundancy is modelled as a circular list of switch points.
- **V2 Sparse matrix:** In the feeder-by-hour load-reading table, only cells with an actual recorded reading are stored.
- **V3 Stack and queue:** A grid operator's manual switch-state changes made while investigating a fault are undone with a stack; customer outage reports wait in a queue to be triaged.
- **V4 Tree and heap:** Substations are organised by region in a binary tree; simultaneous fault reports sit in a heap ranked by number of affected customers, and heap sort prioritises restoration tasks.
- **V5 Graph and BFS/DFS:** With substations and switches as nodes and feeder segments as edges, BFS finds which customer nodes lose power when a given segment faults, and DFS traces the fault back to its source switch along the energised path.
- **V6 Search and hashing:** A customer node ID maps to its address and connected-feeder record through a hash table (open addressing for collisions); dispatch lookup uses binary search over a sorted array of substation IDs.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by feeder capacity and length, finds the best reroute to restore power to an isolated section through an alternate feeder; cycle detection identifies the backup loops in the ring topology, distinguishing sections with a backup path from radial dead ends.
- **F3 Sorting:** A large historical outage log is sorted by duration, number of customers affected, or date with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by feeder segment ID keeps lookups balanced as new segments are added during grid expansion.
- **F5 String algorithms:** KMP scans an operator's free-text incident log for keywords like "overload" or "short circuit"; a misspelled customer address given during an outage call is matched to the nearest known address with edit distance.
- **F6 Trie and disjoint sets:** Substation and feeder names are autocompleted with a trie during operator lookup; union-find recomputes the set of de-energised "islands" every time a switch opens or closes.
- **F7 File organisation:** Outage event records are stored in a sequential file in chronological order, while customer account records live in a direct-access file (collisions resolved by linear quotient).
- **F8 B+ tree index:** A B+ tree secondary index on the customer file, keyed by feeder ID, speeds up "every customer on this feeder" restoration-order queries.
- **F9 Extendible hashing / external sort:** When the multi-year outage and load-reading log no longer fits in memory, an external merge sort by timestamp compiles the annual reliability report.

Extension: Simulate cascading faults across several feeders and validate that the restoration plan still holds.

180 — ♻️ Recycling Collection Route Planner



Summary: A logistics tool that builds truck routes for collecting a city's recycling bins in the most efficient order based on fill level. It works on a synthetic fleet covering about 40 collection zones, 600 bins and 8 trucks.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each truck's list of stops is kept in a doubly linked list, so a last-minute bin stop can be inserted or removed at either end; a fixed downtown loop route is modelled with a circular list.
- **V2 Sparse matrix:** In the zone-by-material-type (paper, glass, plastic, metal) fill-level table, only readings from sensor-equipped bins are stored.
- **V3 Stack and queue:** A dispatcher's manual reordering of stops is undone with a stack; overflow alerts from bins wait in a queue until the next available truck picks them up.
- **V4 Tree and heap:** Zones are organised by district in a binary tree; overflow alerts sit in a heap ranked by urgency, and heap sort ranks all zones by average fill level for the weekly schedule.
- **V5 Graph and BFS/DFS:** With intersections as nodes and streets as edges, BFS finds the route between two bins with the fewest turns, and DFS builds a complete tour that visits every bin in a zone.
- **V6 Search and hashing:** A bin ID maps to its location, material type and capacity record through a hash table (chaining for collisions); "which bins need attention" is answered with binary search over a sorted fill-level array.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra, weighted by travel time, computes the fastest inter-zone travel for a truck moving between assigned zones; Kruskal's algorithm builds a minimal road backbone connecting every bin cluster for a new depot-placement study.
- **F3 Sorting:** Daily collection records from the fleet's history are sorted by weight collected, route duration or fuel used with three algorithms, and running times are compared.
- **F4 BST and AVL:** An AVL tree keyed by bin ID keeps lookups balanced as new bins are added citywide.
- **F5 String algorithms:** KMP scans a driver's free-text trip notes for keywords like "blocked" or "unreachable"; a misspelled street name entered during route planning is matched to the correct street record with edit distance.
- **F6 Trie and disjoint sets:** Street and zone names are autocompleted with a trie during route entry; union-find merges nearby bins that can be serviced in a single pass without returning to the depot into "single-trip clusters."
- **F7 File organisation:** Daily collection logs are stored in a sequential file in date order, while bin records live in a direct-access file (collisions resolved by Brent's method).
- **F8 B+ tree index:** A B+ tree secondary index on the bin file, keyed by zone ID, speeds up "every bin in this zone" route-generation queries.
- **F9 Extendible hashing / external sort:** When the citywide multi-year collection log no longer fits in memory, an external merge sort by date compiles the annual recycling-volume report.

Extension: Re-route a truck mid-shift when a bin's sensor unexpectedly reports a much higher fill level than planned.

181 — Field Irrigation Planner (Grid)



Summary: A console application that divides a farm's land into a grid of rows and columns and computes valve-opening order and pipe routing based on each plot's moisture deficit. It works on a synthetic 40-by-60 grid of 2,400 plots, with 300 valves fed from a single reservoir.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each pipe lateral is a doubly linked list of segments carrying diameter and flow rate; the nightly rotating drip-valve group is walked as a circular list.
- **V2 Sparse matrix:** In the plot grid, only cells needing water this week store a moisture-deficit value; unwatered plots are left empty.
- **V3 Stack and queue:** An operator's mistaken valve setting is undone with a stack; field workers' irrigation requests wait in a queue until the pump has capacity.
- **V4 Tree and heap:** Plots are held in a binary tree by farm > field > block > plot and traversed for reports; a heap-based priority queue routes the driest zone to the pump first.
- **V5 Graph and BFS/DFS:** Pipe junctions are nodes and segments are edges in an adjacency list; BFS finds the shortest water path from the reservoir to a plot, DFS finds the pipe branch cut off by a leak.
- **V6 Search and hashing:** Plot number maps to a plot record (crop type, last watered) in a hash table; binary search over sorted valve open-times finds the first irrigation after a given hour.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm computes the cheapest pipe expansion to new plots; Dijkstra finds the route with least pressure loss to the farthest plot.
- **F3 Sorting:** 2,400 plots are ranked by moisture deficit with insertion sort, quicksort and merge sort, and the three runtimes are compared.
- **F4 BST and AVL:** Crop varieties are kept in a binary search tree by name for catalog lookup; plot moisture-deficit values sit in an AVL tree that rebalances on every reading, answering "most overdue plot" instantly.
- **F5 String algorithms:** A disease keyword in field-inspection notes is found with KMP; a mistyped crop-variety name is matched to the closest catalog entry by edit distance.
- **F6 Trie and disjoint sets:** Crop-variety names autocomplete during planting entry via a trie; when a new pipe connection is added, the plots it links are merged into the same irrigation zone with union-find.
- **F7 File organisation:** Irrigation logs (date, plot, volume) sit in a sequential file; plot master records sit in a direct-access file using progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on the irrigation log, keyed by date, answers "all irrigation between two dates" range queries.
- **F9 Extendible hashing / external sort:** The season's millions of irrigation records are too large for memory, so they are ordered by plot number with an external merge sort.

Extension: Adding weather-forecast ingestion that automatically postpones the schedule ahead of predicted rainfall.

182 — 🐾 Animal Migration Route Tracker



Summary: A field application for a conservation park that merges GPS stopover data from tagged migratory birds to report their routes and resting sites. It works on roughly 500 tagged birds and 5,000 sighting records over a synthetic season; tags with low battery are flagged to the ranger automatically.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each bird's route history is a doubly linked list of stopover points carrying a timestamp and coordinates; flocks that migrate round-trip are stored as a circular list.
- **V2 Sparse matrix:** In the month $\times$ stopover-region sighting-count matrix, regions a species never visited that month are left empty; only observed cells are stored.
- **V3 Stack and queue:** A ranger's fix to a bad GPS reading is undone with a stack; field sighting reports wait in a queue until they are validated.
- **V4 Tree and heap:** Species are held in a binary tree by species > genus > family and traversed for species reports; a heap-based priority queue surfaces the tag with the lowest battery to the ranger first.
- **V5 Graph and BFS/DFS:** Stopover points are nodes and observed transitions are edges in an adjacency list; BFS finds the fewest-hop route between two stopovers, DFS finds sanctuaries left unreachable after a route change.
- **V6 Search and hashing:** Tag ID maps to an animal record in a hash table; binary search over sorted sighting dates finds the first sighting after a given date.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest migration corridor between breeding and wintering grounds; cycle detection identifies round-trip migration loops in the stopover graph.
- **F3 Sorting:** 5,000 sightings are ranked by date with selection sort, quicksort and heap sort, and the three runtimes are compared.
- **F4 BST and AVL:** Species names sit in a binary search tree for the ranger's field-guide lookup; tag battery levels sit in an AVL tree that rebalances on every reading, answering "lowest 10 batteries" instantly.
- **F5 String algorithms:** A species name in a ranger's free-text note is found with KMP; a misspelled species name is matched to the closest master-list entry by edit distance.
- **F6 Trie and disjoint sets:** Species names autocomplete on the ranger's tablet via a trie; once a transition edge is confirmed, the stopovers it connects are merged into the same migration corridor with union-find.
- **F7 File organisation:** Raw GPS pings sit in a sequential file; animal master records sit in a direct-access file using linear quotient.
- **F8 B+ tree index:** A B+ tree secondary index on the sightings file, keyed by species name, answers "all elk sightings in March" range queries.
- **F9 Extendible hashing / external sort:** As new birds are tagged each season, the growing tag file is managed with extendible hashing, doubling its directory as needed.

Extension: Adding a predictive model that forecasts next season's route from climate-trend data.

183 — 🍵 Tea Plantation Harvest Scheduler



Summary: An operations application that routes harvest crews to tea-garden blocks by leaf ripeness and logs the resulting weighing tickets at the collection station. It works on a synthetic estate of 60 blocks across 800 rows, producing roughly 2,000 weighing tickets a day over one season.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each garden row is a doubly linked list of bushes carrying a bush number and ripeness score; the crews' weekly block-rotation schedule is walked as a circular list.
- **V2 Sparse matrix:** In the block × week ripeness-alert matrix, only blocks crossing the ripeness threshold that week are marked; most cells stay empty.
- **V3 Stack and queue:** The last crew reassignment is undone with a stack; weighing tickets at the collection station wait in a queue until they are recorded.
- **V4 Tree and heap:** Blocks are held in a binary tree by estate > sector > block and traversed for reports; a heap-based priority queue decides which block a crew harvests next by ripeness urgency.
- **V5 Graph and BFS/DFS:** Footpaths between blocks form an adjacency list; BFS finds the shortest walking path between two blocks, DFS finds every block reachable from the collection center.
- **V6 Search and hashing:** Worker ID maps to a worker record (name tag, daily quota) in a hash table; binary search over sorted ticket weights finds the median for quality control.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm computes the shortest new footpath network to unconnected blocks; cycle detection checks that a proposed footpath does not create a redundant loop.
- **F3 Sorting:** Roughly 2,000 daily tickets are ranked by weight with insertion sort, merge sort and quicksort, and the three runtimes are compared.
- **F4 BST and AVL:** Tea-grade codes (OP, BOP, FBOP) sit in a binary search tree for fast lookup during ticket entry; block ripeness scores sit in an AVL tree that rebalances daily, answering "next block to harvest".
- **F5 String algorithms:** A worker's name in the daily roster is found with Boyer-Moore; this season's harvest-schedule text is aligned against last season's with LCS to flag recurring patterns.
- **F6 Trie and disjoint sets:** Tea-grade names are looked up quickly via a trie; blocks that finish harvesting the same day are merged into one completed zone with union-find.
- **F7 File organisation:** Daily weighing tickets sit in a sequential file; worker master records sit in a direct-access file using Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on the ticket file, keyed by block number, answers "all tickets for blocks 14 through 22" range queries.
- **F9 Extendible hashing / external sort:** The season's accumulated tickets are too large for memory for payroll processing, so they are ordered by date with an external merge sort.

Extension: Adding weather-based yield prediction that automatically adjusts next week's harvest priority.

184 — Coastal Erosion Monitoring Grid



Summary: A coastal-management application that divides a shoreline into cross-shore survey transects and tracks land loss from monthly field measurements. It works on a synthetic 30-kilometre coastline with 300 transects and a five-year archive of monthly readings.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each transect's elevation profile is a chronological doubly linked list of survey points carrying a date and elevation; buoys anchoring survey equipment in a bay are stored as a circular list.
- **V2 Sparse matrix:** In the transect × month erosion-rate matrix, only cells crossing the alert threshold are stored; most transects stay stable most months, leaving cells empty.
- **V3 Stack and queue:** A mistyped elevation reading is corrected and undone with a stack; field-survey requests wait in a queue until a survey team is available.
- **V4 Tree and heap:** Transects are held in a binary tree by region > sub-region > transect and traversed for reports; a heap-based priority queue sends the most severely eroded transect to urgent sea-wall inspection first.
- **V5 Graph and BFS/DFS:** Adjacent transects along the coastline form an adjacency list; BFS finds the nearest safe transect for evacuation from a flooded one, DFS finds the contiguous erosion-affected stretch of coastline.
- **V6 Search and hashing:** Transect number maps to a transect record (GPS location, baseline elevation) in a hash table; binary search over sorted annual loss values finds the median erosion rate.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest access route for emergency crews along the coastal road; topological sort orders a planned sea-wall project's phases (foundation, wall, drainage) by dependency.
- **F3 Sorting:** 300 transects are ranked by erosion rate with quicksort, merge sort and heap sort, and the three runtimes are compared.
- **F4 BST and AVL:** Transect names sit in a binary search tree for the survey team's quick lookup; cumulative loss values sit in an AVL tree that rebalances after each survey, answering "top 10 most eroded transects".
- **F5 String algorithms:** A keyword like "wall breach" in a technician's field note is found with KMP; inconsistent transect names entered by different survey teams are matched by edit distance.
- **F6 Trie and disjoint sets:** Hazard-tag keywords autocomplete during note tagging via a trie; adjacent transects that cross the danger threshold together are grouped into the same critical erosion zone with union-find.
- **F7 File organisation:** Raw survey readings sit in a sequential file; transect master records sit in a direct-access file using progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on the survey file, keyed by date, answers "all readings between two storms" range queries.
- **F9 Extendible hashing / external sort:** As new monitoring points are added along the coast each year, the growing transect file is managed with extendible hashing, doubling its directory as needed.

Extension: Adding drone-imagery comparison that automatically flags new erosion hot spots between surveys.

185 — 🌲 Forest Inventory Tree Census Index



Summary: A field application for a forestry service that records species, diameter and growth data for tagged trees across sample plots and estimates timber volume. It works on roughly 12,000 tagged trees across 150 synthetic sample plots.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each plot's tagged trees form a doubly linked list, in tagging order, of records carrying species and trunk diameter; a ranger's looped patrol trail is stored as a circular list.
- **V2 Sparse matrix:** In the plot × species count matrix, each plot records only the handful of species actually present; most of the possible species per plot are left empty.
- **V3 Stack and queue:** A field crew's mistaken tagging entry is undone with a stack; tree-health inspection requests wait in a queue for the technician.
- **V4 Tree and heap:** Trees are held in a binary tree by family > genus > species and traversed for species reports; a heap-based priority queue routes the highest disease-risk tree to urgent treatment first.
- **V5 Graph and BFS/DFS:** Forest trails connecting plots form an adjacency list; BFS finds the shortest trail from the ranger station to a plot, DFS finds every plot still reachable after a trail closure.
- **V6 Search and hashing:** Tree tag number maps to a tree record in a hash table; binary search over sorted diameter values finds trees within a percentile for harvest planning.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm computes the shortest new trail network linking isolated plots; Dijkstra finds the shortest firebreak-clearing route in an emergency.
- **F3 Sorting:** 12,000 tree records are ranked by trunk diameter with insertion sort, quicksort and merge sort, and the three runtimes are compared.
- **F4 BST and AVL:** Species Latin names sit in a binary search tree for the botanist's quick lookup; estimated tree ages sit in an AVL tree that rebalances as new measurements arrive, answering "oldest N trees".
- **F5 String algorithms:** A species Latin name in a botanist's note is found with KMP; a misspelled common name is matched to the closest master-list entry by edit distance.
- **F6 Trie and disjoint sets:** Species names autocomplete on the field tablet via a trie; adjacent plots confirmed to share canopy cover are merged into a single forest stand with union-find.
- **F7 File organisation:** Growth-measurement records sit in a sequential file; tree master records sit in a direct-access file using linear quotient.
- **F8 B+ tree index:** A B+ tree secondary index on the tree file, keyed by species, answers "all Scots pine records" range queries.
- **F9 Extendible hashing / external sort:** A decade of growth-measurement archives is too large for memory, so it is ordered by measurement date with an external merge sort for trend analysis.

Extension: Adding a carbon-stock estimate computed from each tree's measured volume for climate reporting.

186 — 🏠 Urban Planning Zoning Parcel Index



Summary: A municipal application that classifies land parcels by zoning code and tracks zoning amendments and permit applications. It works on roughly 20,000 parcels in a synthetic district and also keeps boundary relationships between neighboring parcels.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each parcel's zoning-change history is a chronological doubly linked list of records carrying a date and old/new code; boundary parcels along the district's ring road are stored as a circular list.
- **V2 Sparse matrix:** In the block $\times$ land-use-type matrix, most blocks hold only one or two use types, so only occupied cells are stored.
- **V3 Stack and queue:** A planner's last rezoning is undone with a stack; pending permit applications wait in a queue for committee review.
- **V4 Tree and heap:** Parcels are held in a binary tree by district $>$ neighborhood $>$ block $>$ parcel and traversed for reports; a heap-based priority queue surfaces the application closest to its statutory deadline first.
- **V5 Graph and BFS/DFS:** Parcels are nodes and shared boundaries are edges in an adjacency list; BFS finds every parcel within N blocks of a proposed rezoning for notification, DFS finds the full contiguous zone of one land-use type.
- **V6 Search and hashing:** Parcel number maps to a parcel record (owner code, zoning status, area) in a hash table; binary search over sorted parcel areas finds parcels within a size range.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm computes the cheapest utility line to new development parcels; topological sort orders permit-approval steps (survey, permit, construction) by dependency.
- **F3 Sorting:** 20,000 parcels are ranked by area with selection sort, quicksort and merge sort, and the three runtimes are compared.
- **F4 BST and AVL:** Zoning-code definitions sit in a binary search tree for quick lookup during permit review; parcel assessed values sit in an AVL tree that rebalances after each reassessment, answering property-tax percentile queries.
- **F5 String algorithms:** A street name in the address field is found with KMP; inconsistently spelled street names from digitized paper records are matched by edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete in the address form via a trie; two parcels legally merged are combined into one parcel with union-find.
- **F7 File organisation:** Permit-application records sit in a sequential file; parcel master records sit in a direct-access file using Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on the parcel file, keyed by zoning code, answers "all commercial-zoned parcels" range queries.
- **F9 Extendible hashing / external sort:** As the district annexes new land, the growing parcel file is managed with extendible hashing, splitting its directory as needed.

Extension: Adding automatic conflict detection between a proposed rezoning and neighboring parcels' existing variances.

187 — Real Estate Map Range Search



Summary: An application that lets a buyer search listings within a map bounding box and a chosen price range, and manages saved searches and viewing appointments. It works on roughly 15,000 synthetic listings scattered across a city grid.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each listing's price history is a doubly linked list of date/amount records; an agent's Saturday open-house circuit is stored as a circular list.
- **V2 Sparse matrix:** In the city grid's latitude-band × longitude-band cells, only cells with active listings are stored; most cells stay empty on the map's heat display.
- **V3 Stack and queue:** A buyer's last filter change is undone with a stack via the back button; pending viewing-appointment requests wait in a queue for agent confirmation.
- **V4 Tree and heap:** Listings are held in a binary tree by neighborhood > street and traversed for browsing; a heap-based priority queue surfaces the lead with the longest gap since last contact first.
- **V5 Graph and BFS/DFS:** Street intersections are nodes and streets are edges in an adjacency list; BFS finds listings within N intersections of a point, DFS finds every listing reachable within a school-district road network.
- **V6 Search and hashing:** Listing number maps to a listing record in a hash table; binary search over the grid's sorted price array answers "between \$200,000 and \$350,000" range queries.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest driving distance from a listing to the nearest school or transit stop; Prim's algorithm finds the cheapest access road to an isolated new subdivision's parcels.
- **F3 Sorting:** 15,000 listings are ranked by price with quicksort, merge sort and heap sort, and the three runtimes are compared.
- **F4 BST and AVL:** Neighborhood names sit in a binary search tree for the browse menu; listing prices sit in an AVL tree that rebalances as listings are added or sold, answering "under \$300,000" instantly.
- **F5 String algorithms:** Keywords like "balcony" or "garage" in a listing description are found with Boyer-Moore; a misspelled neighborhood name typed into the search box is matched by edit distance.
- **F6 Trie and disjoint sets:** Neighborhood and street names autocomplete in the search box via a trie; as school-boundary data is merged, listings are grouped into the same catchment cluster with union-find.
- **F7 File organisation:** Viewing-appointment records sit in a sequential file; listing master records sit in a direct-access file using progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on the listing file, keyed by price, efficiently answers "price between X and Y" range queries.
- **F9 Extendible hashing / external sort:** As new listings are added citywide every day, the growing listing file is managed with extendible hashing, splitting its directory as needed.

Extension: Adding a recommendation engine that ranks listings by similarity to a buyer's saved-search history.

188 — Air Quality Sensor Network Analyser



Summary: An environmental application that collects readings from fixed air-quality stations across a city and reports threshold breaches and sensors needing maintenance. It works on 250 synthetic stations with a year-long archive of readings taken every ten minutes.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each station's reading history is a doubly linked list of timestamped values for trend plotting; sensors along a ring-road monitoring loop are stored as a circular list for the maintenance van's route.
- **V2 Sparse matrix:** In the hour $\times$ station threshold-exceedance matrix, only hours crossing the pollutant threshold are marked; most stations stay within limits, leaving cells empty.
- **V3 Stack and queue:** A technician's last manual calibration override is undone with a stack; sensor-malfunction alerts wait in a queue for a technician to be dispatched.
- **V4 Tree and heap:** Stations are held in a binary tree by city > district > station and traversed for reports; a heap-based priority queue surfaces the station with the current worst reading for an urgent alert first.
- **V5 Graph and BFS/DFS:** Stations are linked by the road network in an adjacency list; BFS finds the nearest clean station to redirect pedestrians during a spike, DFS finds the contiguous area affected by a spreading pollution plume.
- **V6 Search and hashing:** Sensor number maps to a station record (location, last calibration date) in a hash table; binary search over sorted PM2.5 readings finds a given percentile for the daily report.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the lowest-exposure walking route across the city, avoiding high-pollution edges; strongly connected components identifies station clusters that mutually influence each other's readings through wind-driven transfer.
- **F3 Sorting:** 250 stations are ranked by average air-quality index with insertion sort, quicksort and heap sort, and the three runtimes are compared.
- **F4 BST and AVL:** Pollutant codes sit in a binary search tree for quick lookup on the technician's dashboard; live station index values sit in an AVL tree that rebalances on every reading cycle, answering "worst 10 stations right now".
- **F5 String algorithms:** A fault code in a technician's maintenance log is found with KMP; a mistyped sensor model name is matched to the closest master-equipment entry by edit distance.
- **F6 Trie and disjoint sets:** Pollutant codes and keywords autocomplete during log tagging via a trie; adjacent stations crossing the threshold together are grouped into the same pollution zone with union-find.
- **F7 File organisation:** Raw ten-minute readings sit in a sequential file; station master records sit in a direct-access file using linear quotient.
- **F8 B+ tree index:** A B+ tree secondary index on the readings file, keyed by timestamp, answers "all readings during a smog event" range queries.
- **F9 Extendible hashing / external sort:** A year's millions of readings are too large for memory, so the annual compliance report orders them by station number with an external merge sort.

Extension: Adding predictive alerts that forecast tomorrow's pollution spike from today's wind and traffic patterns.

189 — ⚡ Solar Panel Layout Planner



Summary: An installation-design application that lays out solar panels on a rooftop or field grid and computes shading and cable routing. It works on roughly 600 panels placed across a synthetic 50-by-80 rooftop grid.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each electrical string is a doubly linked list of series-connected panels (panel number, voltage) that a technician follows to trace a fault; inverters wired in a redundant ring bus are stored as a circular list.
- **V2 Sparse matrix:** In the rooftop's mounting grid, only cells that hold a panel are stored; vents and chimneys leave most cells empty.
- **V3 Stack and queue:** The last panel placement or removal during layout design is undone with a stack; pending shading-analysis jobs wait in a queue for the simulation engine.
- **V4 Tree and heap:** Panels are held in a binary tree by roof section > row > panel and traversed for maintenance reports; a heap-based priority queue surfaces the most underperforming panel for inspection first.
- **V5 Graph and BFS/DFS:** The DC wiring network has junction boxes as nodes and cable runs as edges in an adjacency list; BFS finds the shortest cable path from a panel to the inverter, DFS finds the panels isolated when a junction box fails.
- **V6 Search and hashing:** Panel serial number maps to a panel record in a hash table; binary search over sorted daily-yield values finds the panel at a given performance percentile.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Prim's algorithm computes the shortest cable network connecting every string to the inverter; Dijkstra finds the lowest-resistance-loss route for the longest string.
- **F3 Sorting:** 600 panels are ranked by daily yield with selection sort, quicksort and merge sort, and the three runtimes are compared.
- **F4 BST and AVL:** Panel model names sit in a binary search tree for the manufacturer-catalog lookup; daily energy yields sit in an AVL tree that rebalances nightly, answering the "bottom 20 underperforming panels" alert.
- **F5 String algorithms:** A fault code in a technician's inspection log is found with KMP; a mistyped panel model name is matched to the closest catalog entry by edit distance.
- **F6 Trie and disjoint sets:** Panel model names autocomplete on the install form via a trie; as wiring connections are added during design, the panels they link are grouped into the same electrical string with union-find.
- **F7 File organisation:** Hourly yield logs sit in a sequential file; panel master records sit in a direct-access file using Brent's method.
- **F8 B+ tree index:** A B+ tree secondary index on the yield log, keyed by date, answers "yield between two dates" range queries for maintenance billing.
- **F9 Extendible hashing / external sort:** As the installation grows across new rooftops each season, the growing panel master file is managed with extendible hashing, splitting its directory as needed.

Extension: Adding automatic shading simulation that re-optimizes the layout as nearby trees grow taller each year.

190 — 🚧 Wind Farm Cable Network Planner



Summary: An engineering application that plans the lowest-cost underground cable network connecting a wind farm's turbines to the substation and tracks turbine faults. It works on a synthetic farm of 80 turbines.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each cable route between turbines is a doubly linked list of trench segments carrying length and depth; turbines arranged in a ring layout around a central platform are stored as a circular list.
- **V2 Sparse matrix:** In the turbine $\times$ month maintenance-event matrix, only months with a logged fault or service call are stored; most cells stay empty.
- **V3 Stack and queue:** A planning engineer's last manual edit to a cable route is undone with a stack; pending maintenance tickets wait in a queue for the service crew.
- **V4 Tree and heap:** Turbines are held in a binary tree by farm > cluster > turbine and traversed for reports; a heap-based priority queue routes the highest-severity fault to crew dispatch first.
- **V5 Graph and BFS/DFS:** Turbines and the substation are nodes and candidate cable routes are edges in an adjacency list; BFS finds the fewest-hop cable path from a turbine to the substation, DFS finds the turbines cut off when a cable segment fails.
- **V6 Search and hashing:** Turbine number maps to a turbine record (model, hub height, commissioning date) in a hash table; binary search over sorted cable-length values finds segments within a length range for cost estimation.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Kruskal's algorithm computes the minimum-length cable network connecting all 80 turbines to the substation; Dijkstra finds the lowest electrical-loss route for the farthest turbine's string.
- **F3 Sorting:** 80 turbines are ranked by power output with quicksort, merge sort and heap sort, and the three runtimes are compared.
- **F4 BST and AVL:** Turbine model names sit in a binary search tree for the equipment-catalog lookup; cumulative downtime sits in an AVL tree that rebalances after each maintenance log entry, answering "least reliable 10 turbines".
- **F5 String algorithms:** A fault code in the SCADA alarm log is found with Boyer-Moore; a mistyped turbine tag number is matched to the closest master-asset entry by edit distance.
- **F6 Trie and disjoint sets:** Fault-code keywords autocomplete during technician log tagging via a trie; as cable connections are finalized during layout design, the turbines they link are grouped into the same electrical string with union-find.
- **F7 File organisation:** SCADA alarm records sit in a sequential file; turbine master records sit in a direct-access file using progressive overflow.
- **F8 B+ tree index:** A B+ tree secondary index on the alarm log, keyed by fault code, answers "all gearbox faults last quarter" range queries.
- **F9 Extendible hashing / external sort:** A year of SCADA telemetry is too large for memory, so the annual performance audit orders it by timestamp with an external merge sort.

Extension: Adding predictive-maintenance scoring that flags a turbine likely to fail before its next scheduled service.

191 — Avalanche and Landslide Risk Map



Summary: A console application that maps avalanche and landslide risk from slope, snow-depth and soil-saturation readings across a mountain range, helping a rescue coordinator prioritise the most dangerous valleys. It works on a synthetic terrain grid of 300×300 cells with about 40 sensor stations.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each station keeps its last 30 days of soil-saturation readings in a circular linked list, so the oldest reading is overwritten automatically.
- **V2 Sparse matrix:** In the 300×300 terrain grid, only cells with a sensor or a field observer's report store a risk score; the rest stay empty.
- **V3 Stack and queue:** An operator's mistaken risk-zone marking is undone with a stack; evacuation alerts to villages are processed in order from a queue.
- **V4 Tree and heap:** Risk zones sit in a binary tree keyed by elevation band, root being the whole mountain, so an inorder traversal yields an elevation-ordered report; the most dangerous zone is pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** The road and trail network is modelled as a graph; BFS finds the safe shelter reachable in the fewest stops, DFS finds every settlement lying below a landslide path.
- **V6 Search and hashing:** Station ID maps to a station record in a hash table with chaining; binary search runs over the sorted risk thresholds.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the fastest evacuation route in minutes; Prim's algorithm derives the cheapest communication backbone linking all sensor stations.
- **F3 Sorting:** All cells of the 300×300 grid are sorted by risk score with insertion sort, quicksort and merge sort, and their running times compared.
- **F4 BST and AVL:** Station records sit in an AVL tree keyed by risk score, so a query for "every zone above score 80" is answered in the balanced tree.
- **F5 String algorithms:** An observer's typed village name is searched with KMP; a misspelled name is corrected to the nearest match with edit distance.
- **F6 Trie and disjoint sets:** Village names autocomplete through a trie; union-find merges neighbouring high-risk cells into a single hazard zone.
- **F7 File organisation:** Daily readings sit in a sequential file, station cards in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The readings file carries a B+ tree secondary index on date, speeding up "readings from the last 24 hours" queries.
- **F9 Extendible hashing / external sort:** The station registry file, which keeps growing as new stations are added over the years, is managed with extendible hashing.

Extension: Automatically recomputing the hourly risk score from live satellite rainfall data.

192 — 📍 Emergency Assembly Area Assigner



Summary: A console application that assigns city buildings to the nearest suitable emergency assembly area after an earthquake, tracks each area's occupancy, and logs check-in and check-out events. It works on synthetic data covering about 800 assembly areas and 12,000 buildings.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each area keeps its registered family groups in a doubly linked list; supply trucks are routed through the areas in a fixed order via a circular linked list.
- **V2 Sparse matrix:** In the building × hazard-type matrix (gas leak, structural damage), only flagged cells are stored.
- **V3 Stack and queue:** A coordinator's mistaken capacity change is undone with a stack; people waiting to check in at an area's gate are held in a queue.
- **V4 Tree and heap:** Assembly areas sit in a binary tree following the district–neighbourhood hierarchy, traversed to build a district report; the areas closest to capacity are pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** In the city graph of intersections and streets, BFS finds the nearest area to a given building, DFS finds every building cut off after a road closure.
- **V6 Search and hashing:** Household ID maps to its assigned-area record in a hash table with open addressing; binary search runs over areas sorted by remaining capacity.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest walking time from a building to its assigned area; Kruskal's algorithm derives the network with the least total cable length connecting the areas to backup generators.
- **F3 Sorting:** The 800 assembly areas are sorted by remaining capacity with selection sort, quicksort and heapsort, and their running times compared.
- **F4 BST and AVL:** Buildings sit in an AVL tree keyed by building ID, rebalancing as new buildings are added so that assigned-area lookups stay fast.
- **F5 String algorithms:** A call-centre operator's typed address is searched with KMP; a misspelled address is matched to the nearest one with edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete through a trie; union-find groups buildings that remain mutually reachable after a road closure into one cluster.
- **F7 File organisation:** Check-in events sit in a sequential file, household records in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The building file carries a B+ tree secondary index on district code.
- **F9 Extendible hashing / external sort:** The household registry file, which keeps growing as the population increases, is managed with extendible hashing.

Extension: Automatically suggesting a redirect to a neighbouring area once occupancy passes 90%.

193 — 📁 Post-Earthquake Rubble Search Prioritiser



Summary: A console application that logs survival-probability signals (dog alerts, heat sensors, sound detection) from collapsed-building sites and directs rescue teams to the site with the highest chance of survivors. It works on synthetic data covering about 150 active sites and 30 teams.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each site's search log (team entry, dog alert, drilling progress) sits in a doubly linked list; on the handheld logging device, browsing the log forward and backward uses a memory-efficient XOR linked list instead.
- **V2 Sparse matrix:** In a building's floor-plan grid, only cells with a detected signal or a hazard are stored; cleared cells hold nothing.
- **V3 Stack and queue:** A zone mistakenly marked "cleared" is undone with a stack; teams awaiting their next assignment sit in a dispatch queue.
- **V4 Tree and heap:** The building's structure sits in a binary tree following the section–room hierarchy, traversed to build the search checklist; the site with the highest survival-probability score is pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** In a partially collapsed building's room-adjacency graph, BFS finds the shortest path from the entrance to a room with a detected signal, DFS sweeps every reachable room for a full search.
- **V6 Search and hashing:** Site ID maps to a site record in a hash table with chaining; binary search runs over sites sorted by priority score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest dispatch time from a team's base to a site, accounting for rubble-lengthened routes; cycle detection flags unsafe, looping collapse paths inside a structure so they are never recommended.
- **F3 Sorting:** The 150 active sites are sorted by survival-probability score with insertion sort, quicksort and heapsort, and their running times compared.
- **F4 BST and AVL:** Sites sit in an AVL tree keyed by priority score, so a query for "the top 10 priority sites" reflects the latest signals in the balanced tree.
- **F5 String algorithms:** A missing person's name coming in over the radio is searched with KMP; a misheard name is matched to the nearest one with edit distance.
- **F6 Trie and disjoint sets:** Missing-person names are looked up quickly through a trie; union-find merges neighbouring collapsed cells into a single search zone.
- **F7 File organisation:** The search log sits in a sequential file, site records in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The search-log file carries a B+ tree secondary index on timestamp, speeding up "events from the last hour" queries.
- **F9 Extendible hashing / external sort:** The city-wide daily search log, once too large for memory, is sorted by priority score with an external merge sort.

Extension: Live-tracking each team's GPS position to automatically suggest the nearest free team.

194 — Municipal Complaint Queue Prioritiser



Summary: A console application that collects citizen reports — potholes, missed waste collection, noise — and routes each one to the right department by urgency. It works on synthetic data covering about 2,000 complaints a month across 120 neighbourhoods.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each complaint category keeps its complaints in a doubly linked list; field crews are assigned tasks in turn through a circular linked list.
- **V2 Sparse matrix:** In the neighbourhood × complaint-type matrix, only cells with an actual reported count are stored.
- **V3 Stack and queue:** A wrongly closed complaint is reopened by undoing it with a stack; new complaints arriving at a department wait their turn in a FIFO queue.
- **V4 Tree and heap:** The district–neighbourhood–street hierarchy sits in a binary tree, traversed to build a regional report; the most urgent complaints are pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** In the street graph, BFS finds the nearest free crew, DFS finds every complaint along a given street for a batch repair run.
- **V6 Search and hashing:** Complaint ID maps to its record in a hash table with open addressing; binary search runs over complaints sorted by urgency score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes a crew's shortest route to a complaint site; topological sort orders dependent repair jobs, such as fixing a water main before repaving the street above it.
- **F3 Sorting:** The month's 2,000 complaints are sorted by urgency score with insertion sort, quicksort and merge sort, and their running times compared.
- **F4 BST and AVL:** Complaints sit in an AVL tree keyed by urgency score, rebalancing as new complaints arrive so a "top 20 most urgent" query stays fast.
- **F5 String algorithms:** Complaint text is searched for keywords such as "burst pipe" with KMP to auto-categorise it; a misspelled street name is corrected with edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete through a trie; union-find merges nearby complaints within the same block into a single work order.
- **F7 File organisation:** Complaint submissions sit in a sequential file, citizen records in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The complaint file carries a B+ tree secondary index on district code.
- **F9 Extendible hashing / external sort:** The year's complaint log, once too large for memory, is sorted by date with an external merge sort to build the annual report.

Extension: Automatically flagging a recurring complaint type in one neighbourhood as an infrastructure-investment suggestion.

195 — Bridge and Tunnel Maintenance Prioritiser



Summary: A console application that tracks inspection records and sensor data for a region's bridges and tunnels and recommends the riskiest structures to a maintenance crew. It works on about 300 structures with a decade of synthetic inspection history.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each structure's inspection history sits in a chronological doubly linked list; maintenance crews rotate between regions through a circular linked list.
- **V2 Sparse matrix:** In the structure × sensor-type matrix (strain, vibration, corrosion), only cells for an installed sensor are stored.
- **V3 Stack and queue:** A mistaken edit to the maintenance schedule is undone with a stack; structures waiting for inspection pile up in a queue.
- **V4 Tree and heap:** A structure's components (deck → girders → supports) sit in a binary tree, traversed to build the inspection checklist; the highest-risk structures are pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** In the road-network graph, closing a bridge triggers BFS to find the shortest detour, and DFS to find every downstream route affected by a tunnel closure.
- **V6 Search and hashing:** Structure ID maps to its record in a hash table with chaining; binary search runs over structures sorted by risk score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest detour during a closure; Prim's algorithm derives the least-cable sensor-monitoring backbone linking every structure to the control centre.
- **F3 Sorting:** The 300 structures are sorted by risk score with insertion sort, quicksort and heapsort, and their running times compared.
- **F4 BST and AVL:** Structures sit in an AVL tree keyed by last-inspection date, so a query for "structures overdue by 2 years" is answered in the balanced tree.
- **F5 String algorithms:** An inspector's free-text note is searched for defect keywords such as "crack" with KMP; a misspelled structure name is corrected with edit distance.
- **F6 Trie and disjoint sets:** Structure codes autocomplete through a trie; union-find groups structures along the same corridor into a single maintenance run.
- **F7 File organisation:** The inspection log sits in a sequential file, structure records in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The structure file carries a B+ tree secondary index on risk score.
- **F9 Extendible hashing / external sort:** The structure registry file, which keeps growing as new bridges and tunnels are built, is managed with extendible hashing.

Extension: Sending an instant alert to the maintenance crew whenever sensor data shows a sudden strain spike.

196 — Park Accessibility Analyser



Summary: A console application that measures how well neighbourhoods can walk to a park and flags green-space-poor areas for investment. It works on a synthetic city of about 120 neighbourhoods and 250 parks.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each park's amenities (bench, playground equipment, exercise station) sit in a doubly linked list; a maintenance crew visits parks in a weekly rotation through a circular linked list.
- **V2 Sparse matrix:** In the neighbourhood × amenity-type matrix, only cells for a park that actually has that amenity are stored.
- **V3 Stack and queue:** A planner's mistaken edit to a park boundary is undone with a stack; maintenance requests for a given park wait their turn in a queue.
- **V4 Tree and heap:** The district–neighbourhood–block hierarchy sits in a binary tree, traversed to build a coverage report; the neighbourhoods with the least green space per resident are pulled first from a heap-based priority queue for investment.
- **V5 Graph and BFS/DFS:** In the pedestrian-street graph, BFS finds the park reachable in the fewest steps from a neighbourhood, DFS finds every neighbourhood reachable on foot from a given park.
- **V6 Search and hashing:** Park ID maps to its record in a hash table with open addressing; binary search runs over parks sorted by distance value.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest walking distance from each neighbourhood's centre to its nearest park; Kruskal's algorithm derives the cheapest new pathway network linking isolated parks to the street grid.
- **F3 Sorting:** The 120 neighbourhoods are sorted by green space per resident with selection sort, quicksort and merge sort, and their running times compared.
- **F4 BST and AVL:** Parks sit in an AVL tree keyed by area size, so a query for "parks larger than 5,000 m²" is answered in the balanced tree.
- **F5 String algorithms:** A resident's typed park name is searched with KMP; a misspelled name is matched to the nearest one with edit distance.
- **F6 Trie and disjoint sets:** Park names autocomplete through a trie; union-find merges parks connected by a path into a single green corridor.
- **F7 File organisation:** The maintenance-visit log sits in a sequential file, park records in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The park file carries a B+ tree secondary index on neighbourhood code.
- **F9 Extendible hashing / external sort:** The city's yearly pedestrian-survey data, once too large for memory, is sorted by walking distance with an external merge sort.

Extension: Recomputing the access score of every affected neighbourhood as soon as a new park's location is entered.

197 — Road Accident Black-Spot Analyser



Summary: A console application that analyses accident records across road segments to identify the most dangerous "black spots" and recommends priority interventions to a traffic engineer. It works on about 4,000 road segments with a decade of synthetic accident logs.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each segment's accident records sit in a chronological doubly linked list; on the report viewer, browsing the timeline forward and backward uses a memory-efficient XOR linked list instead.
- **V2 Sparse matrix:** In the intersection × time-of-day accident-count matrix, only cells with at least one recorded accident are stored.
- **V3 Stack and queue:** A wrong black-spot classification is undone with a stack; segments awaiting engineering review pile up in a queue.
- **V4 Tree and heap:** The highway–avenue–street hierarchy sits in a binary tree, traversed to build a report; the segments with the highest severity score are pulled first from a heap-based priority queue for intervention.
- **V5 Graph and BFS/DFS:** In the road-network graph, BFS finds the shortest route that avoids a given black spot, DFS finds every upstream segment that feeds traffic into it.
- **V6 Search and hashing:** Segment ID maps to its record in a hash table with chaining; binary search runs over segments sorted by severity score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the safest route weighted by accident risk; a strongly-connected-components analysis (Kosaraju/Tarjan) reveals the tightly looped cluster of one-way streets around a black spot that keeps recirculating traffic through it.
- **F3 Sorting:** The 4,000 road segments are sorted by severity score with insertion sort, quicksort and heapsort, and their running times compared.
- **F4 BST and AVL:** Segments sit in an AVL tree keyed by yearly accident count, so a query for "segments with more than 10 accidents a year" is answered in the balanced tree.
- **F5 String algorithms:** An accident report's text is searched for cause keywords such as "speeding" with KMP; a misspelled street name is corrected with edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete through a trie; union-find merges neighbouring black-spot segments into a single corridor-level intervention zone.
- **F7 File organisation:** Accident reports sit in a sequential file, segment records in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The segment file carries a B+ tree secondary index on severity score.
- **F9 Extendible hashing / external sort:** The decade-long accident log, once too large for memory, is sorted by date with an external merge sort.

Extension: Correlating the log with weather data to automatically raise the risk score on rainy days.

198 — 📌 Flood Warning Siren Coverage Planner



Summary: A console application that maps how well settlements in a river basin are covered by flood warning sirens and recommends where to install new ones to close the gaps. It works on a synthetic basin of about 90 sirens and 200 settlements.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each siren's test and activation log sits in a doubly linked list; a technician visits sirens in a weekly rotation through a circular linked list.
- **V2 Sparse matrix:** In the basin grid, only cells with a water-level gauge store a reading; the rest stay empty.
- **V3 Stack and queue:** A planner's mistaken siren placement is undone with a stack; siren installation work orders wait for a crew in a FIFO queue.
- **V4 Tree and heap:** The basin–sub-basin–settlement hierarchy sits in a binary tree, traversed to build a coverage report; the settlements with the highest flood-risk score are pulled first from a heap-based priority queue for a new siren.
- **V5 Graph and BFS/DFS:** In the settlement-road graph, BFS finds every settlement within audible range of the control centre in the fewest hops, DFS finds every settlement downstream of a flood path in the river-connected graph.
- **V6 Search and hashing:** Siren ID maps to its record in a hash table with open addressing; binary search runs over settlements sorted by risk score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest signal-relay path from the control centre to each siren; Kruskal's algorithm derives the least-cable network connecting all sirens to the centre.
- **F3 Sorting:** The 200 settlements are sorted by flood-risk score with selection sort, quicksort and merge sort, and their running times compared.
- **F4 BST and AVL:** Sirens sit in an AVL tree keyed by coverage radius, so a query for "settlements covered by less than 500 metres" is answered in the balanced tree.
- **F5 String algorithms:** A planner's typed settlement name is searched with KMP; a misspelled name is matched to the nearest one with edit distance.
- **F6 Trie and disjoint sets:** Settlement names autocomplete through a trie; union-find groups settlements within the same siren's audible range into one coverage cluster.
- **F7 File organisation:** Siren test records sit in a sequential file, siren cards in a direct-access file with collision resolution by progressive overflow.
- **F8 B+ tree index:** The coverage file carries a B+ tree secondary index on settlement code.
- **F9 Extendible hashing / external sort:** The water-level sensor log, growing over the years and eventually too large for memory, is sorted by timestamp with an external merge sort.

Extension: Integrating rainfall forecast data to automatically trigger a siren once a threshold is crossed.

199 — City Cycle Lane Network Designer



Summary: A console application that examines a city's cycle lane network at the street-segment level, flagging safety and connectivity gaps and proposing new lane segments. It works on a synthetic city of about 1,500 street segments.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each route's waypoints sit in an ordered doubly linked list; the mobile app's route cache keeps the waypoint sequence in a memory-efficient XOR linked list to save space.
- **V2 Sparse matrix:** In the street segment $\times$ time-of-day cyclist-count matrix, only measured cells are stored.
- **V3 Stack and queue:** A planner's route edit is undone with a stack; citizen-submitted route suggestions wait their turn in a review queue.
- **V4 Tree and heap:** The district hierarchy sits in a binary tree, traversed to build a coverage report; the segments with the highest "missing link" priority score are pulled first from a heap-based priority queue.
- **V5 Graph and BFS/DFS:** In the street graph, where intersections are nodes and cycle-laned streets are edges, BFS finds the shortest fully-laned route between two points, DFS finds every segment reachable via existing lanes from a starting point.
- **V6 Search and hashing:** Segment ID maps to its record in a hash table with chaining; binary search runs over segments sorted by safety score.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the safest route weighted by a segment's safety score; Kruskal's algorithm derives the cheapest set of new lanes connecting otherwise disconnected cycling islands into one network.
- **F3 Sorting:** The 1,500 segments are sorted by daily cyclist count with insertion sort, quicksort and heapsort, and their running times compared.
- **F4 BST and AVL:** Segments sit in an AVL tree keyed by safety score, so a query for "segments below the safety threshold" is answered in the balanced tree.
- **F5 String algorithms:** A citizen's typed street name on the suggestion form is searched with KMP; a misspelled name is matched to the nearest one with edit distance.
- **F6 Trie and disjoint sets:** Street names autocomplete through a trie; union-find merges connected cycle-lane components to detect disconnected islands.
- **F7 File organisation:** Citizen suggestions sit in a sequential file, segment records in a direct-access file with collision resolution by linear quotient.
- **F8 B+ tree index:** The segment file carries a B+ tree secondary index on safety score.
- **F9 Extendible hashing / external sort:** The year's cyclist-count sensor log, once too large for memory, is sorted by segment ID with an external merge sort.

Extension: Recomputing a suggested route in real time to avoid segments flagged as slippery or risky in current weather.

200 — Mountain Trail Difficulty Map



Summary: A console application that computes the difficulty rating of hiking trails across a mountain range from their elevation profile and recommends a suitable route to a hiker. It works on a synthetic trail network of about 80 trails and 500 checkpoints.

Midterm — C (weeks 1–6):

- **V1 Linked list:** Each trail's checkpoints (elevation, GPS marker) sit in an ordered doubly linked list; a circuit trail that loops back to the trailhead is modelled with a circular linked list.
- **V2 Sparse matrix:** In a 400×400-cell mountain grid, only cells containing a checkpoint are stored; the rest of the terrain holds nothing.
- **V3 Stack and queue:** The last waypoint a hiker adds while building a custom route is undone with a stack; hikers checking in at a trailhead wait for the ranger briefing in a queue.
- **V4 Tree and heap:** Trail segments sit in a binary tree keyed by elevation band, traversed to build the elevation profile; the trails with the highest post-storm damage score are pulled first from a heap-based priority queue for inspection.
- **V5 Graph and BFS/DFS:** In the junction graph, where checkpoints are nodes and trail segments are edges, BFS finds the route to the summit with the fewest segments, DFS sweeps every trail reachable from a trailhead to map the network and find dead ends.
- **V6 Search and hashing:** Trail ID maps to its record in a hash table with open addressing; binary search runs over a trail's sorted elevation profile.

Final — Java (weeks 9–14):

- **F2 Graph algorithms:** Dijkstra computes the shortest-time route weighted by segment hiking time and elevation gain; Prim's algorithm derives the new marker network with the least total installation distance connecting all checkpoints.
- **F3 Sorting:** The 80 trails are sorted by difficulty score with selection sort, quicksort and merge sort, and their running times compared.
- **F4 BST and AVL:** Checkpoints sit in an AVL tree keyed by elevation, so a query for "checkpoints between 1,500 and 2,000 metres" is answered in the balanced tree.
- **F5 String algorithms:** A hiker's typed trail name in the app is searched with KMP; a misspelled name is matched to the nearest peak name with edit distance.
- **F6 Trie and disjoint sets:** Trail and peak names autocomplete through a trie; union-find merges connected trail segments to flag isolated clusters that need a new connector path.
- **F7 File organisation:** Hiker check-in and check-out events sit in a sequential file, trail records in a direct-access file with collision resolution by Brent's method.
- **F8 B+ tree index:** The trail file carries a B+ tree secondary index on difficulty score.
- **F9 Extendible hashing / external sort:** A season's worth of hiker GPS logs, once too large for memory, is sorted by timestamp with an external merge sort.

Extension: Automatically raising the difficulty score of affected trails when a severe weather alert is issued.